CUDA Kernel 面试背题笔记

本书整理自 LeetCUDA 项目，系统汇编了 CUDA 面试中
最高频的 GPU Kernel 实现，涵盖从 Warp/Block 原语、
Elementwise、Softmax、GEMM、FlashAttention 等
面试核心知识体系。

每个 Kernel 配备详细的中文面试要点注释（WHY + HOW），
并以递进式优化路线（naive → tiling → vectorize →
tensor core → warp specialized）展示 GPU 性能调优思维。

30 个 Kernel · 9 个 Phase · 全中文注释

项目主页：`https://github.com/xlite-dev/LeetCUDA`

更新日期：2026 年 7 月 17 日

```
// ================================================================================
// notes-v2.cu — CUDA Kernel 面试背题笔记
// ================================================================================
//
// 整理自 LeetCUDA 项目（https://github.com/xlite-dev/LeetCUDA），涵盖：
//   - 面试高频 CUDA kernel 的完整实现（~30 个 kernel）
//   - 每类 kernel 附带详细的面试要点注释（WHY + HOW）
//   - 优化技术的递进式讲解（naive → tiling → vectorize → tensor core → ws）
//   - BLAS 语义：N=col-major(Normal), T=row-major(Transposed)
//
// 10 个 Phase 覆盖：
//   Phase 0 — 面试框架速查（GPU 架构 / Memory Hierarchy / Roofline / 优化清单）
//   Phase 1 — 基础原语：Warp Reduce / Block Reduce / Dot Product（含 broadcast 增强版）
//   Phase 2 — Elementwise：ReLU / Elementwise Add / Histogram（基础 + float4 向量化 + atomic）
//   Phase 3 — Softmax：naive → safe → online + RMS/Layer Norm
//   Phase 4 — RoPE：旋转位置编码（Llama 风格 theta=10000）
//   Phase 5 — Mat Transpose：基础版 + BCF merge_write 最佳版（Bank Conflict专题）
//   Phase 6 — GEMV：SGEMV K32/K128/K16（warp-per-row）
//   Phase 7 — GEMM ★：SGEMM → HGEMM → MMA m16n8k16(TN布局) → WGMMA m64n128k16
//   Phase 8 — FlashAttention split_q（FA-2, 含 online softmax + P@V 寄存器复用）
//
// ================================================================================
// Phase 0: 面试框架速查（纯注释，面试开场必备的基础知识）
// ================================================================================

// ---- GPU 架构速查 ----
//
// SM (Streaming Multiprocessor) 内部结构：
//   - Warp Scheduler ×4：每 SM 4 个 warp scheduler，每个每周期可发射 1 条指令
//   - Register File：每 SM 65536 × 32-bit (4 bytes) = 256KB
//   - Shared Memory / L1：可配置，最大 shared memory ~228KB (Hopper)
//   - Tensor Cores：Hopper 每 SM 4 个；Blackwell 数量随型号/定义不同，建议以官方 ISV guide 为准
//   - Warp = 32 threads：最小调度单元，SIMT 执行模型
//
// Memory Hierarchy 带宽数量级（H100 参考）：
//   HBM3:         ~3.35 TB/s（理论），实际 ~2.5-3.0 TB/s
//   L2 Cache:     ~12 TB/s（50MB，跨 SM 共享）
//   L1/SMEM:      ~19 TB/s（每 SM ~228KB）
//   Register:     ~0 延迟，~100+ TB/s 等效带宽
//
// 关键瓶颈判断：
//   Memory-bound：AI (Arithmetic Intensity) < 机器 FLOPS/带宽 比值
//   Compute-bound：AI 足够大，受限于计算吞吐
//   Latency-bound：线程不够多，无法隐藏内存延迟
//
// Occupancy 公式：
//   occupancy = active_warps / max_warps_per_SM
//   受三类资源分别取下限：每线程寄存器数 → threads/SM；每 block shared memory
//   → blocks/SM；block 大小 → blocks/SM

// ---- 常见优化手段速查清单 ----
//
// 1. Coalesced Memory Access（合并访问）
//    - 同一 warp 的线程访问连续的 128B 对齐地址 → 1 次内存事务
//    - 否则产生多次事务（最坏 32 次）
//
// 2. Tiling（分块）
//    - 将数据从 HBM 分块加载到 shared memory 复用，减少 HBM 访问
//    - GEMM: Block Tile (BM×BN) + K Tile (BK)
//
// 3. Vectorized Memory Access（向量化）
//    - 使用 float4/half2 等向量类型，减少 load/store 指令数
```

```
//    - float4 = 128-bit，单条指令加载 16 bytes
//
// 4. Thread Tile（寄存器分块）
//    - 每个线程计算多个输出元素（TM×TN），提高计算密度
//    - 减少线程总数，降低同步开销
//
// 5. Bank Conflict Avoidance
//    - Shared memory 有 32 banks × 4 bytes
//    - 同 warp 多线程访问同一 bank 的不同地址 → bank conflict → 串行化
//    - 解决方案：PAD（在每行末尾加 1 个元素打破对齐）
//
// 6. Pipeline / Double Buffering（流水线）
//    - cp.async 异步拷贝下一批数据时同时做当前批的计算
//    - Stage 数 = 2/3/4，权衡 shared memory 占用和延迟隐藏
//
// 7. Tensor Core（MMA / WGMMA）
//    - MMA m16n8k16 (Ampere): warp 级指令，单 warp 完成 16×8×16 的矩阵乘
//    - WGMMA m64n128k16 (Hopper): warpgroup 级指令（128 threads），异步执行
//
// 8. Warp Specialization（Hopper+）
//    - Producer warpgroup 做 TMA 数据搬运，Consumer warpgroup 做计算
//    - 通过 cuda::barrier 同步，完全解耦数据搬运和计算
//
// 9. TMA (Tensor Memory Accelerator, Hopper+)
//    - 硬件 DMA 引擎，支持 1D~5D 寻址，低寄存器开销
//    - 配合 cp.async.bulk 实现异步数据搬运

// ---- Roofline 分析公式 ----
//
// AI (Arithmetic Intensity) = FLOPs / Bytes_transferred
//
// GEMM (M=N=K=4096):
//   FLOPs = 2 × M × N × K = 2 × 4096³ ≈ 137 GFLOPS
//   Bytes = (M×K + K×N + M×N) × sizeof(float) ≈ 200 MB
//   AI    ≈ 137G / 200M ≈ 685 FLOPS/Byte → compute-bound（远超 H100 ridge point:
//   FP16 TC ≈ 295:1, FP32 ≈ 20:1）
//
// GEMV (M=4096, K=4096):
//   Bytes = (M×K + K + M) × sizeof(float) ≈ 67 MB
//   AI    ≈ 33M / 67M ≈ 0.5 FLOPS/Byte → severely memory-bound
//
// Softmax (N=4096): AI ≈ (5×N) / (2×N×4) = 5/8 ≈ 0.625 FLOPS/Byte → memory-bound

// ============================================================================
// Phase 1: 头文件 + 宏定义 + 基础原语（Warp Reduce / Block Reduce）
// ============================================================================
// 面试要点：
//   - warp_reduce: 用 __shfl_xor_sync 做蝶形归约，O(logN) 步，无需 shared
//   memory
//   - block_reduce: 两级归约（warp → shared memory → warp0 broadcast），
//     注意最后必须 broadcast 回所有线程（__shfl_sync），否则只有 warp0 知道结果
//   - 为什么不用 __shfl_down_sync? xor 模式所有线程做相同工作量，更均衡

#include <algorithm>
#include <cstring>
#include <cuda_fp16.h>
#include <cuda_runtime.h>
#include <float.h>
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#include <cublas_v2.h>
#include <cuda.h>
```

```cpp
#include <cuda/barrier>
#include <cuda/ptx>

#if defined(NOTES_V2_ENABLE_CUDNN)
#pragma nv_diag_suppress 128
#include <cudnn_frontend.h>
#pragma nv_diag_default 128
namespace fe = cudnn_frontend;
#endif

#if defined(NOTES_V2_ENABLE_TMA_MMA_WS) && CUDART_VERSION < 13000
#error "NOTES_V2_ENABLE_TMA_MMA_WS requires CUDA Toolkit 13.0 or newer"
#endif

#define INT4(value) (reinterpret_cast<int4 *>(&(value))[0])
#define FLOAT4(value) (reinterpret_cast<float4 *>(&(value))[0])
#define HALF2(value) (reinterpret_cast<half2 *>(&(value))[0])

static constexpr int kWarpSize = 32;

// ============================================================================
// Phase 1a: Warp Reduce (warp 内归约，纯寄存器操作，无需 shared memory)
// ============================================================================

// Warp Reduce Sum — generic (used by both FP32 and FP16 contexts)
// 使用 __shfl_xor_sync 做蝶形归约 (butterfly reduction)
// 复杂度 O(logN)，N=32 时仅需 5 步
// 模板参数: T=数据类型, kWarpWidth=segment width (默认 32)
// kWarpWidth 会作为 __shfl_xor_sync 的第 4 个实参 width，限制 shuffle 在同一 segment 内
// 当 kWarpWidth < 32 (如 FA 中 kWarpWidth=4) 时，只有同 segment 的 lane 参与通信
//   蝶形归约示意 (以 warpSize=8 为例，实际 warpSize=32 有 5 次迭代)：
//
//   初始: 每个 lane 持有自己的值 v0..v7
//   lane:  0    1    2    3    4    5    6    7
//   val:  v0   v1   v2   v3   v4   v5   v6   v7
//
//   mask=4 (第1次迭代，lane i 与 lane i^4 交换并累加):
//          ┌─────────────────┐
//   对:   (0,4) (1,5) (2,6) (3,7)
//
//   lane:  0    1    2    3    4    5    6    7
//   val: v0+v4 v1+v5 v2+v6 v3+v7 v4+v0 v5+v1 v6+v2 v7+v3
//
//   mask=2 (第2次迭代，lane i 与 lane i^2 交换并累加):
//          ┌───────┐  ┌───────┐  ┌───────┐  ┌───────┐
//   对:   (0,2) (1,3) (4,6) (5,7)
//       ─── 前4个一组 ───   ─── 后4个一组 ───
//
//   lane:  0    1    2    3    4    5    6    7  (每lane持有前一轮2个值的和再加本轮配对)
//   val: Σ{0,2,4,6} Σ{1,3,5,7} Σ{0,2,4,6} Σ{1,3,5,7} Σ{0,2,4,6} Σ{1,3,5,7} Σ{0,2,4,6} Σ{1,3,5,7}
//        = v0+v2+v4+v6 ... (逐步归约)
//
//   mask=1 (第3次迭代，lane i 与 lane i^1 交换并累加):
//          ┌─┐ ┌─┐ ┌─┐ ┌─┐ ┌─┐ ┌─┐ ┌─┐ ┌─┐
//   对:   (0,1)(2,3)(4,5)(6,7)
//
//   lane:  0    1    2    3    4    5    6    7
//   val:  Σall Σall Σall Σall Σall Σall Σall Σall  ← 所有 lane 拥有全归约结果!
//
//   mask=0: 循环终止，归约完成。
//
//   关键性质:
//   - XOR 配对是对称的: lane i 的配对对象是 lane i^mask，而 (i^mask)^mask = i
```

```cpp
//   - 每轮每个 lane 只和恰好 1 个其他 lane 通信（一对一，无冲突）
//   - 每轮信息传递距离减半：16→8→4→2→1（距离减半，信息翻倍）
//   - O(log₂ N) 步完成，无需 shared memory，纯寄存器操作
//   - __shfl_xor_sync 第四个参数 kWarpWidth 限制 segment width：
//     当 kWarpWidth=4 时，只有同 segment(4个一组)内的 lane 参与 shuffle
template <const int kWarpWidth = kWarpSize, typename T = float>
__device__ __forceinline__ T warp_reduce_sum(T val) {
#pragma unroll
  for (int mask = kWarpWidth >> 1; mask >= 1; mask >>= 1) {
    val += __shfl_xor_sync(0xffffffff, val, mask, kWarpWidth);
  }
  return val;
}

// Warp Reduce Max — generic
template <const int kWarpWidth = kWarpSize, typename T = float>
__device__ __forceinline__ T warp_reduce_max(T val) {
#pragma unroll
  for (int mask = kWarpWidth >> 1; mask >= 1; mask >>= 1) {
    val = max(val, __shfl_xor_sync(0xffffffff, val, mask, kWarpWidth));
  }
  return val;
}

// ============================================================================
// Phase 1b: Block Reduce（block 内归约，两级：warp → shared memory → warp reduce）
// ============================================================================

// Block Reduce Sum — FP32（增强版，带 broadcast）
// 两级归约流程：
//   1. 每个 warp 内做 warp_reduce_sum → 得到每 warp 的一个值
//   2. warp leader (lane=0) 写入 shared memory
//   3. syncthreads 后，lane 0~kNumWarps-1 读取 shared memory
//   4. 所有 warp 内再做一次 warp_reduce<kNumWarps> → 得到最终结果
//   5. __shfl_sync broadcast 到所有线程（关键！否则每个warp只有lane<kNumWarps 知道结果）
template <const int kNumThreads = 256>
__device__ float block_reduce_sum(float val) {
  constexpr int kNumWarps = (kNumThreads + kWarpSize - 1) / kWarpSize;
  int warp = threadIdx.x / kWarpSize;
  int lane = threadIdx.x % kWarpSize;
  __shared__ float shared[kNumWarps];

  float value = warp_reduce_sum<kWarpSize>(val);
  if (lane == 0)
    shared[warp] = value;
  __syncthreads();
  value = (lane < kNumWarps) ? shared[lane] : 0.0f;
  value = warp_reduce_sum<kNumWarps>(value);
  // 关键：broadcast 结果到所有线程，后续用 result
  // 做除法等操作时所有线程都能拿到
  value = __shfl_sync(0xffffffff, value, 0, 32);
  return value;
}

// Block Reduce Max — FP32（增强版，带 broadcast）
template <const int kNumThreads = 256>
__device__ float block_reduce_max(float val) {
  constexpr int kNumWarps = (kNumThreads + kWarpSize - 1) / kWarpSize;
  int warp = threadIdx.x / kWarpSize;
  int lane = threadIdx.x % kWarpSize;
  __shared__ float shared[kNumWarps];

  float value = warp_reduce_max<kWarpSize>(val);
```

```cpp
  if (lane == 0)
    shared[warp] = value;
  __syncthreads();
  value = (lane < kNumWarps) ? shared[lane] : -FLT_MAX;
  value = warp_reduce_max<kNumWarps>(value);
  value = __shfl_sync(0xffffffff, value, 0, 32);
  return value;
}

// ---- Block Reduce Sum All: y = sum(a[0..N-1]) ----
// 多 block 各自做 warp→smem→warp0 reduce，然后 atomicAdd 到全局 y
// 跨 block 求和的常见模式，适合 N 较大时使用；
// Grid:  ((N + 255) / 256, 1, 1)
// Block: (256, 1, 1)
// source: LeetCUDA/kernels/reduce/block_all_reduce.cu
template <const int kNumThreads = 256>
__global__ void block_reduce_all(float *a, float *y, int N) {
  int tid = threadIdx.x;
  int idx = blockIdx.x * kNumThreads + tid;
  constexpr int kNumWarps = (kNumThreads + kWarpSize - 1) / kWarpSize;
  __shared__ float shared[kNumWarps];

  float val = (idx < N) ? a[idx] : 0.0f;
  int warp = tid / kWarpSize;
  int lane = tid % kWarpSize;

  val = warp_reduce_sum<kWarpSize>(val);
  if (lane == 0)
    shared[warp] = val;
  __syncthreads();

  val = (lane < kNumWarps) ? shared[lane] : 0.0f;
  if (warp == 0)
    val = warp_reduce_sum<kNumWarps>(val);
  if (tid == 0) // tid == 0, not lane 0. 只有 block 内的一个线程负责写回全局结果，避免重复累加
    atomicAdd(y, val);
}

// ---- Dot Product: y = sum(a[i] * b[i]) ----
// 核心模式: elementwise 乘法 → block reduce → atomicAdd 全局累加
// Grid:  ((N + 255) / 256, 1, 1)
// Block: (256, 1, 1)
// source: LeetCUDA/kernels/dot-product/dot_product.cu
template <const int kNumThreads = 256>
__global__ void dot(float *a, float *b, float *y, int N) {
  int tid = threadIdx.x;
  int idx = blockIdx.x * kNumThreads + tid;
  constexpr int kNumWarps = (kNumThreads + kWarpSize - 1) / kWarpSize;
  __shared__ float shared[kNumWarps];

  float prod = (idx < N) ? a[idx] * b[idx] : 0.0f;
  int warp = tid / kWarpSize;
  int lane = tid % kWarpSize;

  prod = warp_reduce_sum<kWarpSize>(prod);
  if (lane == 0)
    shared[warp] = prod;
  __syncthreads();

  prod = (lane < kNumWarps) ? shared[lane] : 0.0f;
  if (warp == 0) // 只需要 warp 0 的线程继续 reduce 即可
    prod = warp_reduce_sum<kNumWarps>(prod);
  if (tid == 0) // tid == 0, not lane 0. 只有 block 内的一个线程负责写回全局结果，避免重复累加
```

```
    atomicAdd(y, prod);
}

// Dot Product + float4
// Grid:  ((N + 255) / 256, 1, 1), 每个block处理256元素，float4向量化后每个线程处理4元素
// Block: (64, 1, 1)，256/4=64
// 注意：该版本默认输入地址满足 float4 对齐；最适合 N 按 4 对齐的场景
// source: LeetCUDA/kernels/dot-product/dot_product.cu
template <const int kNumThreads = 256 / 4>
__global__ void dot_vec4(float *a, float *b, float *y, int N) {
  int tid = threadIdx.x;
  int idx = (blockIdx.x * kNumThreads + tid) * 4;
  constexpr int kNumWarps = (kNumThreads + kWarpSize - 1) / kWarpSize;
  __shared__ float shared[kNumWarps];

  float4 reg_a = FLOAT4(a[idx]);
  float4 reg_b = FLOAT4(b[idx]);
  float prod = (idx < N) ? (reg_a.x * reg_b.x + reg_a.y * reg_b.y +
                            reg_a.z * reg_b.z + reg_a.w * reg_b.w)
                         : 0.0f;
  int warp = tid / kWarpSize;
  int lane = tid % kWarpSize;

  prod = warp_reduce_sum<kWarpSize>(prod);
  if (lane == 0)
    shared[warp] = prod;
  __syncthreads();

  prod = (lane < kNumWarps) ? shared[lane] : 0.0f;
  if (warp == 0)
    prod = warp_reduce_sum<kNumWarps>(prod);
  if (tid == 0)
    atomicAdd(y, prod);
}


// ============================================================================
// Phase 2: Elementwise Ops（逐元素操作，演示 coalesced access + vectorize）
// ============================================================================
// 面试要点：
//   - 逐元素操作是最简单的 kernel，核心考点是 memory coalescing
//   - float4 向量化可将内存事务数减为 1/4，大幅提升 bandwidth utilization
//   - grid/block 维度设计：grid(N/threads), block(threads)，一维即可

// ---- ReLU: y = max(0, x) ----
// Grid:  ((N + 255) / 256, 1, 1)
// Block: (256, 1, 1)
// source: LeetCUDA/kernels/relu/relu.cu
__global__ void relu(float *x, float *y, int N) {
  int idx = blockIdx.x * blockDim.x + threadIdx.x;
  if (idx < N)
    y[idx] = fmaxf(0.0f, x[idx]);
}

// ReLU + float4 向量化：每个线程处理 4 个元素，减少 75% 的 load/store 指令
// block(64)×4(float4)=256 元素/block，与基础版吞吐相同
// Grid:  ((N + 255) / 256, 1, 1)
// Block: (64, 1, 1)
// 注意：该版本默认地址满足 float4 对齐；最适合 N 按 4 对齐的场景
// source: LeetCUDA/kernels/relu/relu.cu
__global__ void relu_vec4(float *x, float *y, int N) {
  int idx = (blockIdx.x * blockDim.x + threadIdx.x) * 4;
  if (idx < N) {
```

```
    float4 reg_x = FLOAT4(x[idx]); // 单条 128-bit load
    float4 reg_y;
    reg_y.x = fmaxf(0.0f, reg_x.x);
    reg_y.y = fmaxf(0.0f, reg_x.y);
    reg_y.z = fmaxf(0.0f, reg_x.z);
    reg_y.w = fmaxf(0.0f, reg_x.w);
    FLOAT4(y[idx]) = reg_y; // 单条 128-bit store
  }
}

// ---- Elementwise Add: c = a + b ----
// Grid:  ((N + 255) / 256, 1, 1)
// Block: (256, 1, 1)
// source: LeetCUDA/kernels/elementwise/elementwise.cu
__global__ void elementwise_add(float *a, float *b, float *c, int N) {
  int idx = blockIdx.x * blockDim.x + threadIdx.x;
  if (idx < N)
    c[idx] = a[idx] + b[idx];
}

// Elementwise Add + float4 向量化
// Grid:  ((N + 255) / 256, 1, 1)
// Block: (64, 1, 1), block(64)×4(float4)=256 元素/block
// 注意: 主路径要求 4 元素对齐; 尾部不足 4 个元素时回退到标量处理
// source: LeetCUDA/kernels/elementwise/elementwise.cu
__global__ void elementwise_add_vec4(float *a, float *b, float *c, int N) {
  int idx = 4 * (blockIdx.x * blockDim.x + threadIdx.x);
  if ((idx + 3) < N) {
    float4 reg_a = FLOAT4(a[idx]);
    float4 reg_b = FLOAT4(b[idx]);
    float4 reg_c;
    reg_c.x = reg_a.x + reg_b.x;
    reg_c.y = reg_a.y + reg_b.y;
    reg_c.z = reg_a.z + reg_b.z;
    reg_c.w = reg_a.w + reg_b.w;
    FLOAT4(c[idx]) = reg_c;
  } else if (idx < N) {
    for (int i = 0; (idx + i) < N; i++) {
      c[idx + i] = a[idx + i] + b[idx + i];
    }
  }
}

// ---- Histogram: y[a[i]]++ ----
// 演示 atomicAdd 的用法: 多个线程可能同时更新同一个 bin
// Grid:  ((N + 255) / 256, 1, 1)
// Block: (256, 1, 1)
// source: LeetCUDA/kernels/histogram/histogram.cu
__global__ void histogram(int *a, int *y, int N) {
  int idx = blockIdx.x * blockDim.x + threadIdx.x;
  if (idx < N)
    atomicAdd(&(y[a[idx]]), 1);
}

// ---- Merge Attn States: 合并 Split-KV Attention 的部分结果 ----
// 实现 FlashInfer 论文 (arXiv 2501.01005) Section 2.2 的 attention 合并逻辑。
//
// 面试要点:
//   - 应用场景: Split-KV attention 将序列沿 K 维分段计算 attention,
//     每段产出部分结果 (O_i, LSE_i), 本 kernel 将两段按指数权重加权合并。
//   - 数学公式 (5 步推导):
//     Step 1 — max 归一化 (数值稳定, 与 safe softmax 同理):
//              L_max = max(LSE_1, LSE_2)
```

```
//     Step 2 — 还原指数权重（un-normalized）：
//              w_1 = exp(LSE_1 - L_max),  w_2 = exp(LSE_2 - L_max)
//     Step 3 — 归一化为混合比例：
//              alpha = w_1 / (w_1 + w_2),  beta = w_2 / (w_1 + w_2)
//              满足 alpha + beta = 1
//     Step 4 — 逐元素加权合并：
//              O = alpha * O_1 + beta * O_2
//   - LSE 布局 [num_heads, num_tokens]（与 FlashAttention 惯例一致，
//     lse[head_idx][token_idx]）
//   - Output 布局 [num_tokens, num_heads, head_size]（token 维在最外，
//     展平为 [T0H0, T0H1, ..., T1H0, ...]）
//   - inf LSE → -inf：空 attention 段（causal mask 等导致全部 score
//     为 -inf）的 LSE 可能为 +inf，替换后 exp(-inf - L_max) = 0，
//     该段权重退化为 0
//   - 128-bit 向量化：uint4 = 4 × float，每线程处理 4 个输出元素，
//     pack load → FMA → pack store 一条流水线
//   - AI ≈ 3 FLOP / 20 bytes ≈ 0.15 FLOPS/Byte → severely memory-bound
//
// 线程到数据的 3D 映射（以 num_tokens=2, num_heads=2, head_size=8 为例）：
//   kPackSize = 16 / sizeof(float) = 4（每线程 4 个 float = 128-bit）
//   threads_per_head = head_size / 4 = 2
//   total_threads = num_tokens * num_heads * threads_per_head = 8
//
//   global_idx:       0      1      2      3      4      5      6      7
//   token_head_idx:   0      0      1      1      2      2      3      3
//        （第几个 (token,head) 对，行优先）
//   pack_idx:         0      1      0      1      0      1      0      1
//        （该 (token,head) 对内第几个 pack）
//   token_idx:        0      0      0      0      1      1      1      1
//        （token_head_idx / num_heads，token 变化慢）
//   head_idx:         0      0      1      1      0      0      1      1
//        （token_head_idx % num_heads，head 变化快）
//   pack_offset:      0      4      0      4      0      4      0      4
//        （pack_idx * 4，该 pack 在 head_size 中的起始元素偏移）
//   head_offset:      0      0      8      8     16     16     24     24
//        （token_idx * num_heads * head_size + head_idx * head_size，
//        该 (token,head) 对在 output 展平数组中的起始偏移）
//
// Grid:  ((total_threads + 127) / 128, 1, 1)
// Block: (128, 1, 1)
// source: LeetCUDA/kernels/openai-triton/merge-attn-states/cuda_merge_attn_states.cu
__global__ void merge_attn_states(
    float *output,              // [num_tokens, num_heads, head_size]
    const float *prefix_output, // [num_tokens, num_heads, head_size]
    const float *prefix_lse,    // [num_heads, num_tokens]
    const float *suffix_output, // [num_tokens, num_heads, head_size]
    const float *suffix_lse,    // [num_heads, num_tokens]
    int num_tokens, int num_heads, int head_size) {

  constexpr int kNumThreads = 128;
  constexpr int kPackSize = 16 / sizeof(float); // 4 floats = 128-bit
  using pack_t = uint4;

  // 每个 (token, head) 对需要 threads_per_head 个线程覆盖 head_size 个元素
  const int threads_per_head = head_size / kPackSize;
  // 实际有效总线程数，超过这个数的线程会被忽略，block是按照kNumThreads=128
  // 来分配的，那么最后一个block的线程数可能会超过实际需要的线程数
  const int total_threads = num_tokens * num_heads * threads_per_head;

  const int global_idx = blockIdx.x * kNumThreads + threadIdx.x;
  if (global_idx >= total_threads)
    return;

```

```
  // global_idx → (token_head_idx, pack_idx) → (token_idx, head_idx, pack_offset)
  // token_head_idx: 第几个 (token, head) 对，行优先展平
  const int token_head_idx = global_idx / threads_per_head;
  // pack_idx: 该 (token, head) 对内第几个 pack，0 ~ threads_per_head-1
  const int pack_idx = global_idx % threads_per_head;

  // token_head_idx 分解为 (token_idx, head_idx)
  const int token_idx = token_head_idx / num_heads; // token 变化慢（外维）
  const int head_idx = token_head_idx % num_heads;  // head 变化快（内维）

  // pack_offset: 该 pack 覆盖的元素在 head_size 中的起始偏移（0, 4, 8, ...）
  const int pack_offset = pack_idx * kPackSize;
  // head_offset: 该 (token, head) 对在 output 展平一维数组中的起始偏移
  const int head_offset = token_idx * num_heads * head_size + head_idx * head_size;

  // 定位到当前 (token, head) 的输出段起始
  const float *prefix_head = prefix_output + head_offset;
  const float *suffix_head = suffix_output + head_offset;
  float *output_head = output + head_offset;

  // LSE 布局 [num_heads, num_tokens]: lse[head_idx][token_idx]
  float p_lse = prefix_lse[head_idx * num_tokens + token_idx];
  float s_lse = suffix_lse[head_idx * num_tokens + token_idx];

  // inf → -inf: 空 attention 段 LSE 可能为 +inf，替换后权重退化为 0
  p_lse = isinf(p_lse) ? -INFINITY : p_lse;
  s_lse = isinf(s_lse) ? -INFINITY : s_lse;

  // Step 1: max 归一化（与 safe softmax 同理，防 exp 溢出）
  const float max_lse = fmaxf(p_lse, s_lse);
  p_lse -= max_lse;
  s_lse -= max_lse;

  // Step 2-3: 指数还原 → 混合比例 alpha, beta
  const float p_se = expf(p_lse);
  const float s_se = expf(s_lse);
  const float out_se = p_se + s_se;
  const float p_scale = p_se / out_se; // alpha = w_1 / (w_1 + w_2)
  const float s_scale = s_se / out_se; // beta  = w_2 / (w_1 + w_2)

  // Step 4: 逐元素加权合并 O = alpha * O_1 + beta * O_2
  // 128-bit 向量化: uint4 load → per-element FMA → uint4 store
  if (pack_offset < head_size) {
    pack_t p_pack =
        reinterpret_cast<const pack_t *>(prefix_head)[pack_idx];
    pack_t s_pack =
        reinterpret_cast<const pack_t *>(suffix_head)[pack_idx];
    pack_t o_pack;

#pragma unroll
    for (int i = 0; i < kPackSize; ++i) {
      const float p_v = reinterpret_cast<const float *>(&p_pack)[i];
      const float s_v = reinterpret_cast<const float *>(&s_pack)[i];
      const float o_v = p_v * p_scale + (s_v * s_scale); // FMA
      reinterpret_cast<float *>(&o_pack)[i] = o_v;
    }

    reinterpret_cast<pack_t *>(output_head)[pack_idx] = o_pack;
  }
}

// ================================================================================
// Phase 3: Reduce 类 Ops — Softmax / RMS Norm / Layer Norm
```

```cpp
// ================================================================================
// 面试要点:
//   - Softmax 三种实现递进: naive (溢出) → safe (2-pass, max 减法) →
//   online (1-pass, 增量更新)
//   - Online Softmax 是 FlashAttention 的数学基础
//   - RMS Norm vs Layer Norm: RMS 只需 1 次 reduce, Layer Norm 需要 2 次 (mean
//   + variance)
//   - Per-token 设计: 一个 block 处理一个 token, 无需跨 block 同步

// ================================================================================
// Phase 3a: Softmax — 三级递进 (面试核心考点)
// ================================================================================
// 面试常问: 「Softmax 有哪些实现方式? 各有什么优缺点? 」
// 回答线索: naive(溢出) → safe → online

// ---- Level 1: 基础 Softmax (per-token, 无 max 减法, 数值不稳定) ----
// grid(S*h/h, h), block(h), 一个 block 处理一个 token
// 问题: x 值很大时 exp(x) 溢出为 inf
template <const int kNumThreads = 256>
// Grid:  (S, 1, 1), S=batch*seq_len, DISPATCH_SOFTMAX_F32_PER_TOKEN_KERNEL
// Block: (H, 1, 1), 由外层 dispatch 选择 H=32/64/128/256/512/1024, 一个 block 处理一个 token
// source: LeetCUDA/kernels/softmax/softmax.cu
__global__ void softmax_per_token(float *x, float *y, int N) {
  const int tid = threadIdx.x;
  const int idx = blockIdx.x * blockDim.x + tid;

  float exp_val = (idx < N) ? expf(x[idx]) : 0.0f;
  float exp_sum = block_reduce_sum<kNumThreads>(exp_val);
  if (idx < N)
    y[idx] = exp_val / exp_sum;
}

// ---- Level 2: Safe Softmax (2-pass: 先 max 再 exp, 数值稳定) ----
// 面试重点: 为什么 Softmax 需要 Safe?
//   - expf 溢出阈值约 88.7: exp(88)≈1.65e38 (接近 float32 上限 3.4e38) , exp(89)≈4.5e38 已溢出为 inf
//   - 减去 max 后: exp(x - max) ≤ exp(0) = 1.0, 永不超过 1
//   - 数学等价性: softmax(x) = softmax(x - c) 对任意常数 c 成立
//   - 代价: 2 次 block reduce (先 max, 再 sum) , 但仍 O(N/B) 高效
// Grid:  (S, 1, 1)
// Block: (H, 1, 1), 由外层 dispatch 选择 H=32/64/128/256/512/1024
// source: LeetCUDA/kernels/softmax/softmax.cu
template <const int kNumThreads = 256>
__global__ void safe_softmax_per_token(float *x, float *y, int N) {
  const int tid = threadIdx.x;
  const int idx = blockIdx.x * blockDim.x + tid;

  // Pass 1: block reduce max — 找最大值
  float val = (idx < N) ? x[idx] : -FLT_MAX;
  float max_val = block_reduce_max<kNumThreads>(val);

  // Pass 2: exp(x - max) → block reduce sum
  float exp_val = (idx < N) ? expf(x[idx] - max_val) : 0.0f;
  float exp_sum = block_reduce_sum<kNumThreads>(exp_val);

  if (idx < N)
    y[idx] = exp_val / exp_sum;
}

// ---- Level 3: Online Safe Softmax (FlashAttention 的数学基础) ----
// 面试重点:
// 两种使用场景 (公式不同, 但结果等价) :
//   1) 单元素增量更新 (online update, 处理新元素 x_i):
//      m_new = max(m_old, x_i)
```

```
//      d_new = d_old * exp(m_old - m_new) + exp(x_i - m_new)
//   2) 二元合并 (binary merge，合并两个部分累加器，warp/block reduce 使用)：
//      m = max(m1, m2) safe softmax用的max值
//      d = d1*exp(m1-m) + d2*exp(m2-m) softmax用的分母值
//      当 m1≥m2 时退化为 d1 + d2*exp(m2-m1) (d_bigger + d_smaller*exp(m_smaller - m_bigger))
// 算法来源: "Online normalizer calculation for softmax" (arXiv:1805.02867)
// Warp Reduce for Online Softmax — binary merge of two partial (m,d) accumulators.
//
// 不同于单元素增量更新公式 (m_new = max(m_old, x_i); d_new = d_old*exp(m_old-m_new) + exp(x_i-m_new))，
// warp reduce 中每次合并的是两个**已各自归一化到各自 max 的部分累加器**：
//   (m1,d1): max 和 Σexp(x_j - m1)，覆盖集合 S1
//   (m2,d2): max 和 Σexp(x_k - m2)，覆盖集合 S2
//
// 合并公式（对称，不依赖"新/旧"概念）：
//   m = max(m1, m2)
//   d = d1*exp(m1-m) + d2*exp(m2-m)   ← m1≥m2 时退化为 d1 + d2*exp(m2-m1)
//
// 该操作满足结合律 → XOR butterfly 任意归约顺序均保证全局正确结果。
struct __align__(8) MD {
  float m; // running max
  float d; // running denominator (sum of exp(x - max))
};

template <const int kWarpWidth = kWarpSize>
__device__ __forceinline__ MD warp_reduce_md(MD md1) {
#pragma unroll
  for (int mask = kWarpWidth >> 1; mask >= 1; mask >>= 1) {
    MD md2;
    md2.m = __shfl_xor_sync(0xffffffff, md1.m, mask, kWarpWidth);
    md2.d = __shfl_xor_sync(0xffffffff, md1.d, mask, kWarpWidth);

    MD b_m = (md1.m > md2.m) ? md1 : md2; // max
    MD s_m = (md1.m > md2.m) ? md2 : md1;

    // b_m.d 无需 rescale（其基准 max 已是全局 max），
    // s_m.d 需 rescale: exp(s_m.m - b_m.m)
    md1.d = b_m.d + s_m.d * __expf(s_m.m - b_m.m);
    md1.m = b_m.m;
  }
  return md1;
}

// 注意：这里默认一个 block 处理一个 token；边界线程的 d=0 只参与归约，不会写回 y
// Grid:  (S, 1, 1)
// Block: (H, 1, 1)，由外层 dispatch 选择 H=32/64/128/256/512/1024
// source: LeetCUDA/kernels/softmax/softmax.cu
template <const int kNumThreads = 256>
__global__ void online_safe_softmax_per_token(const float *x, float *y, int N) {
  int tid = threadIdx.x;
  int idx = blockIdx.x * kNumThreads + threadIdx.x;
  const int kNumWarps = (kNumThreads + kWarpSize - 1) / kWarpSize;
  __shared__ MD shared[kNumWarps];

  float val = (idx < N) ? x[idx] : -FLT_MAX;
  int warp = tid / kWarpSize;
  int lane = tid % kWarpSize;

  // 初始化：每个线程持有一个 (max, denom) 对
  MD md;
  md.m = idx < N ? val : -FLT_MAX;
  md.d = idx < N ? 1.0f : 0.0f;

  // 第一级规约：warp_reduce_md 在归约中自动更新 m 和 d
```

```
  md = warp_reduce_md<kWarpSize>(md);

  if (lane == 0)
    shared[warp] = md;
  __syncthreads();

  // 第二级归约：每个 warp 结果再做一次 warp_reduce_md（复用 block_reduce 模式）
  md = lane < kNumWarps ? shared[lane] : MD{-FLT_MAX, 0.0f};
  md = warp_reduce_md<kWarpSize>(md); // 用kWarpSize确保每个lane都能拿到最终结果

  // 用全局 max 和 denom 做最终 softmax
  float d_inv = __fdividef(1.0f, md.d);
  // 边界线程即使看到 d=0 的填充值，也不会走到写回路径
  if (idx < N) {
    y[idx] = __expf(val - md.m) * d_inv;
  }
}

// ============================================================================
// Phase 3b: RMS Normalization (1-pass reduce)
// ============================================================================
// 面试要点:
//   - RMS Norm: y = (x / rms(x)) * g, 1/rms(x) = rsqrt(mean(x²))
//   - 只需 1 次 block reduce (sum of squares)，比 Layer Norm 少 1 次同步
//   - Llama 系列使用 RMS Norm
//   - grid(N, K/K), block(K): 一行一个 block
// Grid:  (N, 1, 1), N=batch*seq_len, 每行一个 block
// Block: (128, 1, 1), kNumThreads=K=128 (K>128 时调整模板参数)
// source: LeetCUDA/kernels/rms-norm/rms_norm.cu
template <const int kNumThreads = 128>
__global__ void rms_norm(float *x, float *y, float g, int N, int K) {
  int tid = threadIdx.x;
  int idx = blockIdx.x * blockDim.x + threadIdx.x;
  const float epsilon = 1e-5f;

  __shared__ float s_variance;
  float value = (idx < N * K) ? x[idx] : 0.0f;
  float variance = value * value;
  variance = block_reduce_sum<kNumThreads>(variance);
  if (tid == 0)
    s_variance = rsqrtf(variance / (float)K + epsilon); // 1/rms(x)
  __syncthreads();
  if (idx < N * K)
    y[idx] = (value * s_variance) * g;
}

// RMS Norm + float4
// Grid:  (N, 1, 1)
// Block: (32, 1, 1), 128/4=32; 对应一行 K 元素按 4 个一组交给 32 个线程处理
// 注意：该版本默认 K 按 4 对齐，且输入/输出地址满足 float4 对齐
// source: LeetCUDA/kernels/rms-norm/rms_norm.cu
template <const int kNumThreads = 128 / 4>
__global__ void rms_norm_vec4(float *x, float *y, float g, int N, int K) {
  int tid = threadIdx.x;
  int idx = (blockIdx.x * blockDim.x + threadIdx.x) * 4;
  const float epsilon = 1e-5f;

  __shared__ float s_variance;
  float4 reg_x = FLOAT4(x[idx]);
  float variance = (idx < N * K) ? (reg_x.x * reg_x.x + reg_x.y * reg_x.y +
                                    reg_x.z * reg_x.z + reg_x.w * reg_x.w)
                                 : 0.0f;
  variance = block_reduce_sum<kNumThreads>(variance);
```

```
  if (tid == 0)
    s_variance = rsqrtf(variance / (float)K + epsilon);
  __syncthreads();
  float4 reg_y;
  reg_y.x = reg_x.x * s_variance * g;
  reg_y.y = reg_x.y * s_variance * g;
  reg_y.z = reg_x.z * s_variance * g;
  reg_y.w = reg_x.w * s_variance * g;
  if (idx < N * K)
    FLOAT4(y[idx]) = reg_y;
}

// ==========================================================================
// Phase 3c: Layer Normalization (2-pass reduce)
// ==========================================================================
// 面试要点:
//   - Layer Norm: y = ((x - mean) / std) * g + b, std = sqrt(variance), variance = mean((x - mean)²)
//   - 需要 2 次 block reduce: 先 mean (sum/K) , 再 variance (sum((x-mean)²)/K)
//   - 两次 __syncthreads 必须到位, 否则 s_mean 未对所有线程可见就计算 variance
// Grid:  (N, 1, 1), 一行一个 block
// Block: (128, 1, 1), kNumThreads=K=128
// source: LeetCUDA/kernels/layer-norm/layer_norm.cu
template <const int kNumThreads = 128>
__global__ void layer_norm(float *x, float *y, float g, float b, int N, int K) {
  int tid = threadIdx.x;
  int idx = blockIdx.x * blockDim.x + threadIdx.x;
  const float epsilon = 1e-5f;

  __shared__ float s_mean;
  __shared__ float s_variance;
  float value = (idx < N * K) ? x[idx] : 0.0f;

  // Pass 1: compute mean
  float sum = block_reduce_sum<kNumThreads>(value);
  if (tid == 0)
    s_mean = sum / (float)K;
  __syncthreads(); // 必须等待 s_mean 对所有线程可见

  // Pass 2: compute variance = (x - mean)²
  float variance = (value - s_mean) * (value - s_mean);
  variance = block_reduce_sum<kNumThreads>(variance);
  if (tid == 0)
    s_variance = rsqrtf(variance / (float)K + epsilon); // 1/std
  __syncthreads(); // 必须等待 s_variance 对所有线程可见

  if (idx < N * K)
    y[idx] = ((value - s_mean) * s_variance) * g + b;
}

// Layer Norm + float4
// Grid:  (N, 1, 1)
// Block: (32, 1, 1), 128/4=32; 对应一行 K 元素按 4 个一组交给 32 个线程处理
// 注意: 该版本默认 K 按 4 对齐, 且输入/输出地址满足 float4 对齐
// source: LeetCUDA/kernels/layer-norm/layer_norm.cu
template <const int kNumThreads = 128 / 4>
__global__ void layer_norm_vec4(float *x, float *y, float g, float b, int N, int K) {
  int tid = threadIdx.x;
  int idx = (blockIdx.x * blockDim.x + threadIdx.x) * 4;
  const float epsilon = 1e-5f;

  __shared__ float s_mean;
  __shared__ float s_variance;
  float4 reg_x = FLOAT4(x[idx]);
```

```cpp
  float value = (idx < N * K) ? (reg_x.x + reg_x.y + reg_x.z + reg_x.w) : 0.0f;

  float sum = block_reduce_sum<kNumThreads>(value);
  if (tid == 0)
    s_mean = sum / (float)K;
  __syncthreads();

  float4 reg_x_hat;
  reg_x_hat.x = reg_x.x - s_mean;
  reg_x_hat.y = reg_x.y - s_mean;
  reg_x_hat.z = reg_x.z - s_mean;
  reg_x_hat.w = reg_x.w - s_mean;
  float variance = reg_x_hat.x * reg_x_hat.x + reg_x_hat.y * reg_x_hat.y +
                   reg_x_hat.z * reg_x_hat.z + reg_x_hat.w * reg_x_hat.w;
  variance = block_reduce_sum<kNumThreads>(variance);
  if (tid == 0)
    s_variance = rsqrtf(variance / (float)K + epsilon);
  __syncthreads();

  float4 reg_y;
  reg_y.x = reg_x_hat.x * s_variance * g + b;
  reg_y.y = reg_x_hat.y * s_variance * g + b;
  reg_y.z = reg_x_hat.z * s_variance * g + b;
  reg_y.w = reg_x_hat.w * s_variance * g + b;
  if (idx < N * K)
    FLOAT4(y[idx]) = reg_y;
}

// ============================================================================
// Phase 4: RoPE — 旋转位置编码 (Rotary Position Embedding)
// ============================================================================
// 面试要点:
//   - RoPE 数学公式: 对每对相邻维度做 2D 旋转
//     [x1']   [cos(θ)  -sin(θ)] [x1]
//     [x2'] = [sin(θ)   cos(θ)] [x2]
//   - θ_i = 1 / (theta^(2i/d)), theta=10000.0f (Llama 风格)
//   - token_pos = idx / N: token 在序列中的位置
//   - token_idx = idx % N: token 内的维度对索引
//   - 输入 [seq_len, hidden_size], 输出同形状

// Grid:  ((seq_len * N + 255) / 256, 1, 1)
// Block: (256, 1, 1)
// source: LeetCUDA/kernels/rope/rope.cu
__global__ void rope(float *x, float *out, int seq_len, int N) {
  int idx = blockIdx.x * blockDim.x + threadIdx.x;
  float x1 = x[idx * 2];
  float x2 = x[idx * 2 + 1];
  int token_pos = idx / N; // 序列位置
  int token_idx = idx % N; // 维度对索引

  // 频率计算: θ_i = 1 / (10000^(2i/d))
  float exp_v = 1.0f / powf(10000.0f, 2 * token_idx / (N * 2.0f));
  float sin_v = sinf(token_pos * exp_v);
  float cos_v = cosf(token_pos * exp_v);

  // 2D 旋转
  float out1 = x1 * cos_v - x2 * sin_v;
  float out2 = x1 * sin_v + x2 * cos_v;
  out[idx * 2] = out1;
  out[idx * 2 + 1] = out2;
}

// ============================================================================
```

```cpp
// Phase 5: Mat Transpose — 矩阵转置（Bank Conflict 专题）
// ============================================================================
// 面试要点（Bank Conflict 专题）：
//   - Shared memory 有 32 个 bank，每个 bank 4 bytes（32-bit）
//   - 同一 warp 的多个线程访问同一 bank 的不同地址 → bank conflict
//   - n-way bank conflict: n 个线程冲突 → 访问串行化为 n 次
//   - 解决方案：PAD（在每行末尾加 1 个元素，打破地址对齐）
//
// 转置的四步演进：
//   naive: 非合并写入（列优先写）→ 每个 warp 产生 32 次内存事务
//   shared: 写入 smem（行优先）→ 从 smem 读取（列优先）→ 合并写入 gmem
//   BCF:   smem 布局 [kWarpSize_S*4][kWarpSize_S+PAD] = [64][17]，PAD=1 加在第二维消除 bank conflict
//   merge_write: 进一步将 4 次 separate store 合并为 1 次 float4 store
// 注：本文件仅实现 Level 1(naive) 与 Level 4(BCF+merge_write)，Level 2/3 省略

// 每个线程处理 1 个元素，block(16,16)
// 读：x 按行优先访问，warp 的 32 线程访问 32 个连续地址 → 合并读取 ✓
// 写：y 按列优先写入，32 线程跨 row 行分散 → 非合并写入 ✗（32 次内存事务）
// Grid:  ((col + 15) / 16, (row + 15) / 16, 1)，每线程 1 元素
// Block: (16, 16, 1)
// source: LeetCUDA/kernels/mat-transpose/mat_transpose.cu
__global__ void mat_transpose(float *x, float *y, const int row, const int col) {
  const int c = blockIdx.x * blockDim.x + threadIdx.x;  // col in x
  const int r = blockIdx.y * blockDim.y + threadIdx.y;  // row in x
  if (r < row && c < col) {
    // x[r][c] → y[c][r]; x stride=col, y (transposed) stride=row
    y[c * row + r] = x[r * col + c];
  }
}

// Grid:  ((col + 15) / 16, (row + 63) / 64, 1)，每线程 4 元素(float4)
// Block: (16, 16, 1)
// 注意：该版本默认按 float4 打包写回；最适合 row 能按 4 对齐的场景
// source: LeetCUDA/kernels/mat-transpose/mat_transpose.cu
__global__ void mat_transpose_padded(
    float *x, float *y, const int row, const int col) {
  const int tx = threadIdx.x;
  const int ty = threadIdx.y;

  constexpr int TILE = 16;
  constexpr int PAD = 1;
  // Bank conflict fix: 每行多 1 个元素，打破 32-bank 对齐
  __shared__ float tile[TILE * 4][TILE + PAD]; // 64x16

  // x 空间坐标；每线程覆盖 4 行 (actual row = x_r * 4)
  const int x_c = blockIdx.x * TILE + tx;
  const int x_r = blockIdx.y * TILE + ty;

  if (x_r * 4 < row && x_c < col) {
    // Step 1: 4 rows × 1 col → smem (row-major);
#pragma unroll
    for (int i = 0; i < 4; ++i) {
      tile[ty * 4 + i][tx] = x[(x_r * 4 + i) * col + x_c];
    }
    __syncthreads();

    // Step 2: Transposed read from smem
    float4 reg_trans;
    reg_trans.x = tile[tx * 4 + 0][ty];
    reg_trans.y = tile[tx * 4 + 1][ty];
    reg_trans.z = tile[tx * 4 + 2][ty];
    reg_trans.w = tile[tx * 4 + 3][ty];

```

```
    // y 空间坐标: y_r = x 的列, y_c = x 的行
    const int y_r = blockIdx.x * TILE + ty;          // = x col
    const int y_c = (blockIdx.y * TILE + tx) * 4;    // = x row

    // Coalesced write: y[y_r][y_c .. y_c+3]
    FLOAT4(y[y_r * row + y_c]) = reg_trans;
  }
}

// ============================================================================
// Phase 6: GEMV — 矩阵向量乘 (M or N = 1, 纯 memory-bound 算子, warp-per-row 策略)
// ============================================================================
// 面试要点:
//   - GEMV 是典型的 memory-bound 算子: AI ≈ O(1), 瓶颈在内存带宽
//   - 核心策略: warp-per-row (每个 warp 负责矩阵的一行)
//   - 不同 K 值对应不同分块策略:
//     K=32 倍数: 一个 warp 的 32 线程恰好覆盖 K 维
//     K=128 倍数: 每个线程用 float4 处理 4 元素, warp 覆盖 128
//     K=16 < 32: 一个 warp 用不满, 用 kRowPerWarp=2 让每个 warp 处理 2 行
//
// a: M×K, x: K×1, y: M×1, 计算: y = a * x; N = 1

// ---- SGEMV K32: 基础 warp-per-row ----
// 设计: block(32, 4), blockDim.x=kWarpSize=32 (K 需为 32 倍数时一轮覆盖, 否则内层循环 kNumWarps 次)
// grid(M/4), 每个 warp 负责一行
// K 为 32 的倍数时, warp 的 32 个线程恰好覆盖 K 维
// Grid:  ((M + 3) / 4, 1, 1), 每 block 处理 4 行
// Block: (32, 4, 1), 每行 1 warp
// 注意: 该版本最适合 K 按 32 对齐; 当 K 更小时通常切到 K16 这类专用分支
// source: LeetCUDA/kernels/sgemv/sgemv.cu
__global__ void sgemv_k32(float *a, float *x, float *y, int M, int K) {
  int tx = threadIdx.x; // 0~31
  int ty = threadIdx.y; // 0~3
  int lane = tx % kWarpSize; // 0~31
  int m = blockIdx.x * blockDim.y + ty; // 全局行号
  if (m < M) {
    float sum = 0.0f;
    // 沿 K 维的迭代数 = ceil(K/32), 每个 warp 要累加完整的K, 那么
    // 每个thread就要负责累加NUM_ITERS个元素, NUM_ITERS = ceil(K/32)
    const int NUM_ITERS = (K + kWarpSize - 1) / kWarpSize;
#pragma unroll
    for (int w = 0; w < NUM_ITERS; ++w) {
      // 假设K是32的整倍数, m * K 本行的起始地址, x: Kx1
      int k = w * kWarpSize + lane;
      sum += a[m * K + k] * x[k];
    }
    sum = warp_reduce_sum<kWarpSize>(sum);
    // 每个 warp 处理一行, lane 0 写回结果
    if (lane == 0)
      y[m] = sum;
  }
}

// ---- SGEMV K128: float4 向量化 ----
// 每个线程处理 4 个元素(float4), 一个 warp 覆盖 128 个元素
// Grid:  ((M + 3) / 4, 1, 1)
// Block: (32, 4, 1)
// 注意: 该版本最适合 K 按 128 对齐, 且 x/a 的地址满足 float4 对齐
// source: LeetCUDA/kernels/sgemv/sgemv.cu
__global__ void sgemv_k128(float *a, float *x, float *y, int M, int K) {
  int tx = threadIdx.x; // 0~31
  int ty = threadIdx.y; // 0~3
  int lane = tx % kWarpSize; // 0~31
```

```cpp
  int m = blockDim.y * blockIdx.x + ty;

  if (m < M) {
    float sum = 0.0f;
    // 沿 K 维的迭代数 = ceil(K/128)，每个 warp 每轮用 float4 覆盖 128 个 K 元素
    const int NUM_ITERS = (((K + kWarpSize - 1) / kWarpSize) + 4 - 1) / 4;
#pragma unroll
    for (int w = 0; w < NUM_ITERS; ++w) {
      int k = (w * kWarpSize + lane) * 4;
      float4 reg_x = FLOAT4(x[k]);
      float4 reg_a = FLOAT4(a[m * K + k]);
      sum += (reg_a.x * reg_x.x + reg_a.y * reg_x.y + reg_a.z * reg_x.z +
              reg_a.w * reg_x.w);
    }
    sum = warp_reduce_sum<kWarpSize>(sum);
    if (lane == 0)
      y[m] = sum;
  }
}

// ---- SGEMV K16: K < WarpSize, kRowPerWarp=2 ----
// 面试亮点：K=16 < 32，一个 warp 可以处理多行
// kRowPerWarp=2，kNumLanePerRow=16，前 16 个 lane 处理 row0，后 16 个 lane 处理 row1
// Grid:  ((M + 7) / 8, 1, 1)，NUM_ROWS=8
// Block: (32, 4, 1)
// 注意：这一版是面向 K=16 的专用写法；kRowPerWarp=2 时一个 warp 同时处理 2 行
// source: LeetCUDA/kernels/sgemv/sgemv.cu
template <const int kRowPerWarp = 2>
__global__ void sgemv_k16(float *A, float *x, float *y, int M, int K) {
  constexpr int kNumLanePerRow = (kWarpSize + kRowPerWarp - 1) / kRowPerWarp; // 16
  int tx = threadIdx.x;
  int ty = threadIdx.y;
  int lane = tx % kWarpSize;
  int k = lane % kNumLanePerRow; // row0: 0~15, t0~t15; row1: 0~15, t16~t31
  int m = (blockDim.y * blockIdx.x + ty) * kRowPerWarp + lane / kNumLanePerRow;
  if (m < M) {
    float sum = A[m * K + k] * x[k];
    // 按照kNumLanePerRow=16，分2组各自做 warp reduce sum，k==0的lane写回结果
    sum = warp_reduce_sum<kNumLanePerRow>(sum);
    // 注意：判断条件是 k == 0，不是 lane == 0!
    if (k == 0)
      y[m] = sum;
  }
}

// ========================================================================
// Phase 7: GEMM — 矩阵矩阵乘（GPU 最重要的算子，面试核心考点）
// ========================================================================
// 面试要点（GEMM 优化五层金字塔）：
//   Level 1 — Tiling（分块 + shared memory）：将数据从 HBM 搬到 SMEM 复用
//   Level 2 — Thread Tile（寄存器分块）：每个线程计算 TM×TN
//   个元素，提高计算密度 Level 3 — Vectorize（向量化访存）：float4/half2，减少
//   load/store 指令数 Level 4 — Tensor Core（MMA
//   m16n8k16）：硬件矩阵乘单元，warp 级指令 Level 5 — Warp Specialization +
//   TMA（WGMMA m64n128k16）：Hopper 异步执行
//
// 计算密度递进：
//   Level 1: AI ≈ B_K / (2×sizeof) ≈ 32/8 = 4 → 仍是 memory-bound
//   Level 2: AI ≈ TM×TN×B_K / (2×sizeof) ≈ 8×8×8/8 = 64 → compute-bound
//   Level 4: Tensor Core 提供硬件加速的 256 FMA/cycle/warp → 大幅提升吞吐

// ========================================================================
// Phase 7a: SGEMM（非 Tensor Core 路径）
```

```cpp
// ================================================================================

// ---- Level 1: SGEMM — Block Tile 32×32 + K Tile 32 ----
// 最基础的 tiling 实现，演示 shared memory 的核心用法
// C = A x B, C[M, N] = A[M, K] x B[K, N]
// BM=BN=32, BK=32, block(32, 32)，一个线程计算 c 的一个元素
// Grid:  ((N + 31) / 32, (M + 31) / 32, 1)
// Block: (32, 32, 1), 1024 线程
// source: LeetCUDA/kernels/sgemm/sgemm.cu
__global__ void sgemm(float *a, float *b, float *c, int M, int N, int K) {
  constexpr int BM = 32; // vec 版: 32x4 = 128
  constexpr int BN = 32; // vec 版: 32x4 = 128
  constexpr int BK = 32;
  __shared__ float s_a[BM][BK], s_b[BK][BN]; //  32x32x4=4KB smem, float = 4 bytes

  int bx = blockIdx.x;
  int by = blockIdx.y;
  int tx = threadIdx.x;
  int tid = threadIdx.y * blockDim.x + tx;

  // 线程到 smem 的映射: 32×32 线程，每个线程加载 a 和 b 各 1 个元素
  // 技巧: 一般来说 “/” 表示线程不是连续排布的，"%" 表示线程是连续排布的
  // 因此，在需要考虑连续访问的维度使用 “%”，比如，连续的线程访问列方向连续的元素
  // A[M, K], M的stride=K, K的stride=1 → 线程连续访问 K 维度 → 用 %,
  // 线程不连续访问 M 维度 → 用 /;
  int load_smem_a_m = tid / 32; // row 0~31 由 32 线程加载;
  int load_smem_a_k = tid % 32; // col 0~31 由 32 线程加载;
  int load_smem_b_k = tid / 32; // row 0~31 由 32 线程加载;
  int load_smem_b_n = tid % 32; // col 0~31 由 32 线程加载;
  int load_gmem_a_m = by * BM + load_smem_a_m; // gmem row;
  int load_gmem_b_n = bx * BN + load_smem_b_n; // gmem col;

  float sum = 0.f; // 遍历完整的K, slice K;
  // 这里不用pragma unroll，因为K不是编译器常量，编译器无法展开循环
  for (int bk = 0; bk < (K + BK - 1) / BK; ++bk) {
    int load_gmem_a_k = bk * BK + load_smem_a_k; // A [M, K]
    int load_gmem_a_addr = load_gmem_a_m * K + load_gmem_a_k;
    s_a[load_smem_a_m][load_smem_a_k] = a[load_gmem_a_addr];
    int load_gmem_b_k = bk * BK + load_smem_b_k; // B [K, N]
    int load_gmem_b_addr = load_gmem_b_k * N + load_gmem_b_n;
    s_b[load_smem_b_k][load_smem_b_n] = b[load_gmem_b_addr];
    __syncthreads(); // 确保整个 smem tile 加载完毕

#pragma unroll
    for (int k = 0; k < BK; ++k) {
      int comp_smem_a_m = load_smem_a_m; // vec 版: 0~127, 0, 1, 2, ... (连续)
      int comp_smem_b_n = load_smem_b_n; // vec 版: 0~127, 0, 4, 8, ... (间隔)
      sum += s_a[comp_smem_a_m][k] * s_b[k][comp_smem_b_n];
    }
    __syncthreads(); // 确保 smem 不会在下一轮加载时被覆盖
  }
  int store_gmem_c_m = load_gmem_a_m; // vec 版: 0~127, 0, 1, 2, ... (连续) [128x128]
  int store_gmem_c_n = load_gmem_b_n; // vec 版: 0~127, 0, 4, 8, ... (间隔)
  int store_gmem_c_addr = store_gmem_c_m * N + store_gmem_c_n;
  c[store_gmem_c_addr] = sum; // C [M, N] = A[M, K] x B[K, N]
}

// ---- Level 1+: SGEMM Vec4 — Block Tile 128×128 + K Tile 32 + Thread Tile 4×4 ----
// 在 Level 1 基础上引入两层优化:
//   1) float4 向量化加载: A/B 各用 1 条 128-bit load 取代 4 条 32-bit load
//   2) Thread Tile 4×4: 每线程计算 16 个 C 元素，提升计算/访存比 (AI 从
//   BK/2≈16 提升到 TM*TN*BK/2≈256)，减少线程总数带来的同步开销
// C = A x B, C[M, N] = A[M, K] x B[K, N], A/B 均 row-major
```

```cpp
// BM=BN=128, BK=32, block(32, 32)=1024 线程，每线程负责 4×4=16 个 C 元素
//   1024 × 16 = 16384 = 128 × 128 ✓
//
// 线程到 4×4 tile 的映射（与加载映射解耦，独立计算更清晰）：
//   m_tile = tid / 32 (0~31)，每 tile 4 行 → 行 [m_tile*4, m_tile*4+3]，覆盖 0~127
//   n_tile = tid % 32 (0~31)，每 tile 4 列 → 列 [n_tile*4, n_tile*4+3]，覆盖 0~127
//
// 加载映射（每线程 4 个元素，float4）：
//   A[128][32]: a_m = tid/8 (8 线程/行), a_k = (tid%8)*4 (4 列/线程) → 8×4=32 列 ✓
//   B[32][128]: b_k = tid/32 (32 线程/行), b_n = (tid%32)*4 (4 列/线程) → 32×4=128 列 ✓
//   row-major 下 A[m][k..k+3] 与 B[k][n..n+3] 均连续 → float4 load 合法
//
// ⚠ Bank Conflict 提示（面试加分点）：
//   s_b[32][128] 上 warp 内 32 线程按 stride=4 访问（tid%32 决定列 0,4,8,...,124）
//   → 每 4 个线程落同一 bank 不同地址 → 4-way bank conflict。生产代码可用
//   s_b[BK][BN+1] PAD 打散，这里保持最简布局便于讲解。
//
// Grid:  ((N + 127) / 128, (M + 127) / 128, 1)
// Block: (32, 32, 1), 1024 线程
// 假设: M/N 为 128 的倍数，K 为 32 的倍数（与 Level 1 naive 版一致的边界约定）
// source: LeetCUDA/kernels/sgemm/sgemm.cu (vec4 variant)
__global__ void sgemm_vec4(float *a, float *b, float *c, int M, int N, int K) {
  constexpr int BM = 128;
  constexpr int BN = 128;
  constexpr int BK = 32;
  __shared__ float s_a[BM][BK]; // 128*32*4 = 16KB, float = 4 bytes
  __shared__ float s_b[BK][BN]; // 32*128*4 = 16KB

  int bx = blockIdx.x;
  int by = blockIdx.y;
  int tx = threadIdx.x;
  int tid = threadIdx.y * blockDim.x + tx; // 0~1023

  // 加载 A: 每线程加载 s_a[a_m][a_k..a_k+3] 共 4 个元素
  int load_smem_a_m = tid / 8;        // 0~127, 8 线程/行
  int load_smem_a_k = (tid % 8) * 4;  // 0,4,...,28
  // 加载 B: 每线程加载 s_b[b_k][b_n..b_n+3] 共 4 个元素
  int load_smem_b_k = tid / 32;       // 0~31, 32 线程/行
  int load_smem_b_n = (tid % 32) * 4; // 0,4,...,124

  int load_gmem_a_m = by * BM + load_smem_a_m;
  int load_gmem_b_n = bx * BN + load_smem_b_n;

  // 4×4 Thread Tile 基址（独立于加载映射），这里compute索引的计算逻辑要和
  // load索引的计算逻辑分开，load/compute是可以独立索引的，理解这点很重要。
  // 目标C Tile为[BM,BN]=[128x128], 有32x32线程，则每个线程处理4x4 tile
  // 那么，就可以不重不漏地覆盖[32x4,32x4]=[128x128]的大小
  int comp_smem_a_m_base = (tid / 32) * 4; // 0,4,8,...,124
  int comp_smem_b_n_base = (tid % 32) * 4; // 0,4,8,...,124

  float sum[4][4] = {0.f};
  for (int bk = 0; bk < (K + BK - 1) / BK; ++bk) {
    int load_gmem_a_k = bk * BK + load_smem_a_k; // A [M, K]
    int load_gmem_a_addr = load_gmem_a_m * K + load_gmem_a_k;
    FLOAT4(s_a[load_smem_a_m][load_smem_a_k]) = FLOAT4(a[load_gmem_a_addr]); // s_a [BM,BK]
    int load_gmem_b_k = bk * BK + load_smem_b_k; // B [K, N]
    int load_gmem_b_addr = load_gmem_b_k * N + load_gmem_b_n;
    FLOAT4(s_b[load_smem_b_k][load_smem_b_n]) = FLOAT4(b[load_gmem_b_addr]); // s_b [BK,BN]
    __syncthreads();

#pragma unroll
    for (int k = 0; k < BK; ++k) {
      // 每次迭代加载 4 个 A 元素 + 4 个 B 元素，再做 4×4=16 次 FMA
```

```
      float a_vals[4] = {s_a[comp_smem_a_m_base + 0][k],
                         s_a[comp_smem_a_m_base + 1][k],
                         s_a[comp_smem_a_m_base + 2][k],
                         s_a[comp_smem_a_m_base + 3][k]};
      float b_vals[4] = {s_b[k][comp_smem_b_n_base + 0],
                         s_b[k][comp_smem_b_n_base + 1],
                         s_b[k][comp_smem_b_n_base + 2],
                         s_b[k][comp_smem_b_n_base + 3]};
#pragma unroll
      for (int i = 0; i < 4; ++i) {
#pragma unroll
        for (int j = 0; j < 4; ++j) {
          sum[i][j] += a_vals[i] * b_vals[j];
        }
      }
    }
    __syncthreads();
  }

  // 存储 4×4: 每行 4 个元素连续 → 可用 float4 store (要求 N 为 4 的倍数以保证对齐)
  int store_gmem_c_m = by * BM + comp_smem_a_m_base;
  int store_gmem_c_n = bx * BN + comp_smem_b_n_base;
#pragma unroll
  for (int i = 0; i < 4; ++i) {
    int store_gmem_c_addr = (store_gmem_c_m + i) * N + store_gmem_c_n;
    float4 reg_c;
    reg_c.x = sum[i][0];
    reg_c.y = sum[i][1];
    reg_c.z = sum[i][2];
    reg_c.w = sum[i][3];
    FLOAT4(c[store_gmem_c_addr]) = reg_c;
  }
}

// ============================================================================
// Phase 7b: HGEMM — Tensor Core 路径 (MMA m16n8k16 + WGMMA m64n128k16)
// ============================================================================
// 面试要点:
//   - MMA (Matrix Multiply-Accumulate): Ampere+ 的 Tensor Core 指令
//   - m16n8k16 含义: M=16, N=8, K=16 的矩阵乘, 结果 [16×8] = [16×16]×[16×8]
//   - ldmatrix: 从 shared memory 加载 16×16 的矩阵片段到寄存器 (4 条 32-bit
//   寄存器)
//   - Multistage Pipeline: s2/s3/s4 个 stage, 用 cp.async 异步加载下一批数据
//   - Block Swizzle: 在 grid 维度做 swizzle, 改善 L2 cache locality
//
// ★ TN 布局详解 (面试高频考点):
//   TN 命名约定: 来自 BLAS 的 op(A) × op(B) 语义
//     BLAS 源自 Fortran, 默认列优先 (column-major) 存储
//     N = Normal (列优先, BLAS 原生格式)
//     T = Transposed (行优先, 相对 BLAS 来说是"转置过的")
//     第一个字母 → A 的 op, 第二个字母 → B 的 op
//     所以 TN 表示: A 是行优先 (相对 BLAS=Transposed), B 是列优先 (相对
//     BLAS=Normal)  即: C = op(A) × op(B) = A^T × B? 不对!  在 row-major
//     视角下: TN = A row-major [M×K], B^T row-major [N×K] (等价于 B col-major [K×N])  在 cuBLAS
//     调用中: cublasGemmEx(..., CUBLAS_OP_T, CUBLAS_OP_N, ...)
//       T on A: BLAS 把 row-major 的 A 视为 A^T, 传 T 表示"转置回去"
//       N on B: B 已经是 BLAS 原生的 col-major, 无需转置
//
//   记忆口诀: TN = A行(T) B列(N), 第一字母 A 第二字母 B
//     T = row-major (行优先, 对 BLAS 来说是 transposed)
//     N = col-major (列优先, BLAS native = normal)
//
//   LeetCUDA _nn 布局对比 (A/B 均 row-major 自然存储) : C[M×N] = A[M×K] × B[K×N]
```

```
//     - A: row-major [M, K], B: row-major [K, N]
//     - 按 N=col-major/T=row-major 约定，二者 BLAS 视角均为 T → cuBLAS 等效 (T,T)
//     - 问题: ldmatrix 默认加载 col-major，B 是 row-major 需要 .trans
//
//   TN 布局: C[M×N] = A[M×K] × B[K×N]，B 以 B^T=[N×K] row-major 存储（A 行优先，B 列优先）
//     - A [M×K]: row-major → 全局索引 A[m*K + k]，smem s_a[BM][BK]
//     - B^T [N×K]: row-major → 全局索引 B[n*K+k] 即访问原 B 元素 (k,n)（⚠ 内维连续的是 K）
//     - 优势: B^T 已是 row-major，ldmatrix 无需 .trans，天然匹配 MMA row.col
//   MMA 指令: mma.sync.aligned.m16n8k16.row.col
//     - row.col: A 输入 row-major，B 输入 col-major → 与 TN 布局天然匹配
//
// WGMMA (Warp Group MMA, Hopper+):
//   - warpgroup 级指令（128 threads = 4 warps），异步执行
//   - m64n128k16: 一次处理 64×128×16 的矩阵乘（总 tile 量 131072，是 MMA m16n8k16 的 64 倍）
//   - Warp Specialization: Producer(128 threads) 做 TMA 搬运, Consumer(128
//   threads) 做计算
//   - TMA: 硬件 DMA，~零寄存器开销，支持 1D~5D 寻址

// ============================================================================
// Phase 7b-1: MMA PTX 宏定义
// ============================================================================

// ---- gmem → smem: cp.async ----
// cp.async.commit_group / wait_group / wait_all 语义（PTX ISA §9.7.9.25.3）：
//   - commit_group: 将此前所有未提交的 cp.async 归入一个新的 async-group（per-thread）。
//   - wait_group N: 阻塞直到最多 N 个 async-group 尚未完成（即 pending ≤ N）。
//     N=0 → 等所有 group 完成。与 wgmma.wait_group 语义一致。
//   - wait_all: 等价于 commit_group + wait_group 0。
#define CP_ASYNC_COMMIT_GROUP() asm volatile("cp.async.commit_group;\n" ::)
#define CP_ASYNC_WAIT_ALL() asm volatile("cp.async.wait_all;\n" ::)
#define CP_ASYNC_WAIT_GROUP(n)                                                 \
  asm volatile("cp.async.wait_group %0;\n" ::"n"(n))

// 注意: cg 只支持 16 bytes，ca 支持 4/8/16 bytes
#define CP_ASYNC_CG(dst, src, bytes)                                           \
  asm volatile(                                                                \
      "cp.async.cg.shared.global.L2::128B [%0], [%1], %2;\n" ::"r"(dst),       \
      "l"(src), "n"(bytes))

// ---- ldmatrix: smem → register（Tensor Core 专用）----
// ldmatrix.sync.aligned.xN.m8n8.shared.b16
// 每次加载 8×8 的 half 矩阵片段到 1/2/4 条 32-bit 寄存器
// aligned: 要求 128-bit 对齐, trans:  转置加载
#define LDMATRIX_X4(R0, R1, R2, R3, addr)                                      \
  asm volatile(                                                                \
      "ldmatrix.sync.aligned.x4.m8n8.shared.b16 {%0, %1, %2, %3}, [%4];\n"     \
      : "=r"(R0), "=r"(R1), "=r"(R2), "=r"(R3)                                 \
      : "r"(addr))

#define LDMATRIX_X2(R0, R1, addr)                                              \
  asm volatile("ldmatrix.sync.aligned.x2.m8n8.shared.b16 {%0, %1}, [%2];\n"    \
               : "=r"(R0), "=r"(R1)                                            \
               : "r"(addr))

// ldmatrix.x2.trans: 转置加载（用于 NN 布局中需要 col-major B 矩阵的场景）
// FA 中 V[Bc,d] 为 row-major，但 P@V 的 MMA 需要 col-major 的 B → 使用 trans
#define LDMATRIX_X2_T(R0, R1, addr)                                            \
  asm volatile(                                                                \
      "ldmatrix.sync.aligned.x2.trans.m8n8.shared.b16 {%0, %1}, [%2];\n"       \
      : "=r"(R0), "=r"(R1)                                                     \
      : "r"(addr))

// ---- mma.sync.aligned.m16n8k16.row.col.f16.f16.f16.f16 ----
```

```
// m16n8k16: M=16, N=8, K=16 (Ampere Tensor Core 的基本 tile)
// row.col: A 是 row-major, B 是 col-major
// f16.f16.f16.f16: A/B 是 f16, C/D 是 f16 (f32 累加版本用 f32.f16.f16.f32)
// 2 个输出寄存器 (RD0, RD1), 4 个 A 寄存器 + 2 个 B 寄存器
// C 矩阵大小 16×8=128 元素 = 128 个 half; 32 线程分担, 每线程 4 half = 2 个 uint32
#define HMMA16816(RD0, RD1, RA0, RA1, RA2, RA3, RB0, RB1, RC0, RC1)           \
  asm volatile(                                                               \
      "mma.sync.aligned.m16n8k16.row.col.f16.f16.f16.f16 {%0, %1}, {%2, %3, "  \
      "%4, %5}, {%6, %7}, {%8, %9};\n"                                        \
      : "=r"(RD0), "=r"(RD1)                                                  \
      : "r"(RA0), "r"(RA1), "r"(RA2), "r"(RA3), "r"(RB0), "r"(RB1), "r"(RC0),  \
        "r"(RC1))

// ---- MMA 辅助函数 ----
// div_ceil: 整数除法向上取整
#define HOST_DEVICE_INLINE __device__ __host__ inline
HOST_DEVICE_INLINE int div_ceil(int a, int b) {
  return (a % b != 0) ? (a / b + 1) : (a / b);
}

// ============================================================================
// Phase 7b-2: HGEMM MMA — m16n8k16 + multistage pipeline + TN 布局 (统一循环版)
// ============================================================================
// 面试重点 — Tile Hierarchy:
//   MMA Atom:         m16n8k16 (1 条 MMA 指令处理的最小 tile)
//   MMA Tile (more warps):  2×4=8 个 MMA atom → [2×16, 8×4]=[32,32]
//   VAL Tile (more values): 4×4=16 expand → [32×4, 32×4]=[128,128]
//   实际: kMmaTileM=2, kMmaTileN=4, kValTileM=4, kValTileN=4
//         → BM=16×2×4=128, BN=8×4×4=128, Warps=2×4=8, Threads=8×32=256
//
// ★ TN 布局 (T=A 行优先, N=B 列优先):
//   - A[M][K]: row-major → 全局索引 A[m*K + k], smem s_a[BM][BK]
//   - B[K][N]: col-major (等价于 B^T[N][K] row-major) → 全局索引 B[n*K+k]
//   - ldmatrix A: 用 x4 (非转置), A row-major 原生匹配
//   - ldmatrix B: 用 x2 (非转置), B^T row-major 逐行加载 = B 的列, 天然匹配 MMA row.col
//   - MMA 指令: mma.sync.aligned.m16n8k16.row.col → 天然匹配 TN 布局
//
// ★ 统一循环设计 (k 从 0 开始, 消除尾端重复代码):
//   - k 从 0 开始: 每次迭代 k 直接对应 tile k, sel = k % kStages (零偏移)
//   - 加载语义为"预取未来": 迭代 k 加载 tile (k+kStages-1) 供后续使用
//   - cp.async 条件化: 仅当 k+kStages-1 < NUM_K_TILES 时加载
//   - WAIT_GROUP 自适应: 满载期用 kStages-2, 尾部排空用 0
//
// ★ Block Swizzle: 把 C 的逻辑 tile 布局改写为紧凑的二维发射窗口, 增加 A/B tile 的 L2 reuse 机会。
//   C tile 坐标为 (by, bx), by 沿 M 方向, bx 沿 N 方向; 一个 bx 读取同一块 B^T[bx*BN:(bx+1)*BN, :]
//    (TN 布局中 B^T[N][K] 为 row-major), 一个 by 读取同一块 A[by*BM:(by+1)*BM, :]。
//   下图只画 4 个逻辑上相邻的 CTA; SM0~SM3 仅表示可能的发射序列, 不是硬件 SM 绑定或调度保证。
//
//   No Swizzle: grid = (tiles_n, tiles_m, 1), blockIdx.x = bx。
//   C-Matrix (同一 by 行; CTA 先在很宽的 N 方向范围内展开):
//
//       N / bx →  0          1          2          3        ...
//     M  ↓      ┌──────────┬──────────┬──────────┬──────────┐
//     by = 0    │  SM0     │  SM1     │  SM2     │  SM3     │  ...
//               │ A 0,B0   │ A 0,B1   │ A 0,B2   │ A 0,B3   │
//               └──────────┴──────────┴──────────┴──────────┘
//
//   同一 by 的 CTA 复用 A[by], 但分别读取 B^T 0、B^T 1 等不同 B tile。只有 scheduler
//   推进到下一条 by 行、再次遇到相同 bx 时, 才有机会复用 B; 当 tiles_n 很大时, 这段时间内
//   L2 更容易被其他 B tile 挤占。
//
//   Thread Block Swizzle: 先把 N 切成 S 个连续窗口; 一个 z slice 只含 grid_x 个 bx。
//   下图用 grid_x=2 展示一个窄窗口, scheduler 更早进入下一条 by 行:
```

```
//
//       N / local x →  0          1
//     M  ↓          ┌──────────┬──────────┐
//     by = 0        │  SM0     │  SM1     │  → A 0; B^T 0, B^T 1
//                   ├──────────┼──────────┤
//     by = 1        │  SM2     │  SM3     │  → A 1; B^T 0, B^T 1
//                   └──────────┴──────────┘
//
//   对于这个 2x2 窗口：同一行横向复用 A（SM0/SM1、SM2/SM3），同一列纵向复用 B
//    （SM0/SM2、SM1/SM3）。真实 grid_x 不必等于 2，但缩小它会缩短再次访问相同 bx 的距离。
//   此优化只提高 scheduler 在相近时间执行这些 CTA、命中 L2 的机会，不保证 CTA 的 z 顺序、
//   缓存驻留或 L2 hit。
//
//   Launch（kBlockSwizzle=false，当前默认路径）：
//     grid = (ceil(N / BN), ceil(M / BM), 1)，blockIdx.x 直接就是 N-tile 编号 bx。
//   Launch（kBlockSwizzle=true，需由 launch 端显式构造 3D grid）：
//     tiles_n = ceil(N / BN), S = ceil(N / 2048)
//     grid = (ceil(tiles_n / S), ceil(M / BM), S)
//     bx = blockIdx.z * grid.x + blockIdx.x，col_start = bx * BN。
//   例如 N=8192、BN=128：tiles_n=64，S=4，grid.x=16；z=0/1/2/3 分别覆盖
//   bx=0..15/16..31/32..47/48..63，即 columns [0,2048)/[2048,4096)/[4096,6144)/[6144,8192)。
//   grid.x*grid.z 可能产生 bx>=tiles_n 的冗余 CTA。当前 kernel 仍要求 M/N/K 分别按 BM/BN/BK
//   对齐；ceil 只保证逻辑 tile 编号不遗漏，并不使部分尾 tile 变为安全。
// Block: (256, 1, 1), 8 warps
// source: LeetCUDA/kernels/hgemm/mma/basic/hgemm_mma_stage_tn.cu
// ============================================================================
template <const int kMmaM = 16,             // MMA atom M dim (m16n8k16)
          const int kMmaN = 8,              // MMA atom N dim
          const int kMmaK = 16,             // MMA atom K dim, also BK tile = kMmaK
          const int kMmaTileM = 2,          // warps along M, 2 → warp tile M = 32
          const int kMmaTileN = 4,          // warps along N, 4 → warp tile N = 32
          const int kValTileM = 4,          // value-repeat along M, BM = 16*2*4 = 128
          const int kValTileN = 4,          // value-repeat along N, BN = 8*4*4 = 128
          const int kStages = 3,            // cp.async pipeline depth
          const int kBlockSwizzle = 0>      // 1 enables 3D grid swizzle for L2 locality
__global__ void __launch_bounds__(256)
    hgemm_mma_stages_tn(half *A, half *B, half *C, int M, int N, int K) {
  static_assert(kBlockSwizzle == 0 || kBlockSwizzle == 1, "kBlockSwizzle must be 0 or 1");
  static_assert(kStages >= 2, "kStages must be >= 2 for cp.async pipeline");
  // Block Swizzle: 0 时 bx=blockIdx.x; 1 时将 (blockIdx.z, blockIdx.x)
  // 线性化为原始 N-tile 编号 bx=z*gridDim.x+x。by 始终是 M-tile 编号。
  const int bx = ((int)kBlockSwizzle) * blockIdx.z * gridDim.x + blockIdx.x;
  const int by = blockIdx.y;
  constexpr int BM = kMmaM * kMmaTileM * kValTileM; // 16*2*4=128
  constexpr int BN = kMmaN * kMmaTileN * kValTileN; // 8*4*4=128
  constexpr int BK = kMmaK;                         // 16

  // Dynamic shared memory: kStages 个 stage 的 A 和 B
  // TN 布局: s_a[BM][BK]=[128][16](A row-major), s_b[BN][BK]=[128][16](B^T
  // row-major，即 B col-major [K×N] 在 smem 中按 B^T[N×K] 存储)
  // 原始实现会按配置决定是否给 A/B 的 K 维加 PAD；尤其 B 在 TN 布局下常见会额外
  // 加 B_PAD 来打散 bank 映射，避免按列访问时出现明显 bank conflict。这里先保留最简 PAD=0 版本。
  extern __shared__ half smem[];
  half *s_a = smem;
  half *s_b = smem + kStages * BM * BK;     // A 和 B 连续存放
  constexpr int s_a_stage_offset = BM * BK; // 128*16
  constexpr int s_b_stage_offset = BN * BK; // 128*16  ⚠ BN(128)×BK(16) = B^T row-major
  const int tid = threadIdx.y * blockDim.x + threadIdx.x;
  const int warp_id = tid / kWarpSize; // 0~7
  const int lane_id = tid % kWarpSize; // 0~31
  // warp_m变化快(0->0,1->1), warp_n变化慢([0,1]->0,[2,3]->1,...), 因此,
  // 这种2x4的MMA(Warp) layout是按照col major的顺序来排列MMA0~MMA7的
  //                          N direction (warp_n)
```

```
  //                          0       1       2       3
  // M direction (warp_m)  0  MMA0    MMA2    MMA4    MMA6
  //                       1  MMA1    MMA3    MMA5    MMA7
  // MMA的排布方式不是唯一的，现在这样排是因为结合MMA/VAL Tile之后，刚好能覆盖整个
  // C Tile[128,128]。只要调整MMA/VAL Tile，这里的排布方式也可以跟着调整。要注意的
  // 是，MMA0-7逻辑上是可以认为是并行执行的，各自的计算结果累计加到对应的C Tile位置上。
  const int warp_m = warp_id % kMmaTileM; // 0,1 (M 方向 2 个 warp) kMmaTileM = 2
  const int warp_n = warp_id / kMmaTileM; // 0,1,2,3 (N 方向 4 个 warp)

  // 线程到 global memory 的映射（用于加载 A 和 B）共 256 个线程
  // TN 布局关键: A[m*K+k] 是 row-major, B^T[n*K+k] 是 row-major (内维连续的是 K)
  int load_smem_a_m = tid / 2;                 // 0~127
  int load_smem_a_k = (tid % 2 == 0) ? 0 : 8;  // 0, 8
  int load_smem_b_n = tid / 2;                 // 0~127 → B^T 的 N 方向 (row-major 的行)
  int load_smem_b_k = (tid % 2 == 0) ? 0 : 8;  // 0, 8  → B^T 的 K 方向 (row-major 的列)
  int load_gmem_a_m = by * BM + load_smem_a_m; // C/A 全局行号 = M 方向的 tile 起始 + 线程偏移
  int load_gmem_b_n = bx * BN + load_smem_b_n; // C/B 全局列号 = N 方向的 tile 起始 + 线程偏移
  if (load_gmem_a_m >= M || load_gmem_b_n >= N)
    return;

  // 8个Warps(MMAs)的排布是M方向2个，N方向4个，则MMA Atom一次性能处理的tile大小是[2*16,4*8]=[32,32].
  // CUDA中每个block能放的线程数是有上限的（warp数量有限），一般为4或者8个warps。为了能处理更大的C tile,
  // 需要把MMA Atom的tile再M方向和N方向各自重复4次，得到[128,128]的C block tile，这就是Value Tile的作用。
  // MMA Tile对应到cutlass cute中的TiledMMA的概念，Value Tile对应到cutlass cute中的PermuteMNK的概念。
  uint32_t RC[kValTileM][kValTileN][2] = {0}; // 初始化为 0

  // CVTA: 一次转换 smem 基地址，避免每次 cp.async 都做转换
  uint32_t smem_a_base_ptr = __cvta_generic_to_shared(s_a);
  uint32_t smem_b_base_ptr = __cvta_generic_to_shared(s_b);

  // 预加载前 (kStages-1) 个 stage
  // TN 布局: A 的 gmem 索引用 m*K+k(row-major), B^T 用 n*K+k(row-major, 即 B col-major [K×N])
#pragma unroll
  for (int k = 0; k < (kStages - 1); ++k) {
    int load_gmem_a_k = k * BK + load_smem_a_k;
    int load_gmem_a_addr = load_gmem_a_m * K + load_gmem_a_k; // A: [m][k]
    uint32_t load_smem_a_ptr = (smem_a_base_ptr +
      (k * s_a_stage_offset + load_smem_a_m * BK + load_smem_a_k) * sizeof(half)
    );
    CP_ASYNC_CG(load_smem_a_ptr, &A[load_gmem_a_addr], 16);

    int load_gmem_b_k = k * BK + load_smem_b_k;
    // B^T: [n][k] row-major (即 B[k][n] col-major) ⚠
    int load_gmem_b_addr = load_gmem_b_n * K + load_gmem_b_k;
    uint32_t load_smem_b_ptr = (smem_b_base_ptr +
      (k * s_b_stage_offset + load_smem_b_n * BK + load_smem_b_k) * sizeof(half)
    );
    CP_ASYNC_CG(load_smem_b_ptr, &B[load_gmem_b_addr], 16);

    CP_ASYNC_COMMIT_GROUP();
  }

  CP_ASYNC_WAIT_GROUP(kStages - 2); // 允许有 kStages-2 个group未完成
  __syncthreads();

  // 统一循环: k 从 0 开始，每次迭代负责 tile k (加载 + 计算合并为单循环)
  const int NUM_K_TILES = div_ceil(K, BK);
  // 此处不用pragma unroll，因为 K 不是编译期常量，因此NUM_K_TILES 也不是编译期常量
  for (int k = 0; k < NUM_K_TILES; ++k) {
    int smem_sel = k % kStages;                     // 计算 tile k 的 stage
    int smem_sel_next = (k + kStages - 1) % kStages; // 预加载目标 stage

    // 条件加载: 预加载 tile (k+kStages-1) 供将来使用。因为在prefetch loop中已经
```

```
    // load了(kStages-1) 个 stage，因此这里是从 tile (k+kStages-1) 开始预加载
    // TN 布局: A 的 gmem 地址用 m*K+k (row-major)，B^T 的 gmem 地址用 n*K+k
    // (row-major，内维连续的是 K)
    if (k + kStages - 1 < NUM_K_TILES) {
      int load_gmem_a_k = (k + kStages - 1) * BK + load_smem_a_k;
      int load_gmem_a_addr = load_gmem_a_m * K + load_gmem_a_k; // A: row-major [m][k]
      int load_gmem_b_k = (k + kStages - 1) * BK + load_smem_b_k;
      int load_gmem_b_addr = load_gmem_b_n * K + load_gmem_b_k; // B^T: row-major [n][k]

      uint32_t load_smem_a_ptr =
          (smem_a_base_ptr + (smem_sel_next * s_a_stage_offset +
                              load_smem_a_m * BK + load_smem_a_k) *
                                 sizeof(half));
      CP_ASYNC_CG(load_smem_a_ptr, &A[load_gmem_a_addr], 16);

      uint32_t load_smem_b_ptr =
          (smem_b_base_ptr + (smem_sel_next * s_b_stage_offset +
                              load_smem_b_n * BK + load_smem_b_k) *
                                 sizeof(half));
      CP_ASYNC_CG(load_smem_b_ptr, &B[load_gmem_b_addr], 16);
      CP_ASYNC_COMMIT_GROUP();
    }

    // ldmatrix: 从 smem_sel 加载 A 和 B 到寄存器
    // TN 布局关键: A 用 x4 (非转置)，因为 A 是 row-major; B 用 x2 (非转置)，smem 中
    // B^T 为 row-major，逐行加载即得 B 的列，天然匹配 col-major B
    uint32_t RA[kValTileM][4]; // [4][4], M方向4次repeat, A regs = 4 uint32_t per MMA
    uint32_t RB[kValTileN][2]; // [4][2], N方向4次repeat, B regs = 2 uint32_t per MMA

    // ldmatrix.x4: 加载 A 的 m16k16 片段 (row-major A，非转置)
#pragma unroll
    for (int i = 0; i < kValTileM; ++i) {
      // {0,1} * (16 * 4) + i * 16 = {0,64} + {0,16,32,48} = {0,16,32,48,64,80,96,112}
      // 其中 warp_m {0,1}; warp_m_0 offsets {0,16,32,48}; warp_m_1 offsets {64,80,96,112}
      int warp_smem_a_m = warp_m * (kMmaM * kValTileM) + i * kMmaM;
      // {0,16,32,...,112} + {0~15} = {0~127}, 按照col-major的顺序访问A的4个8x8 matrix (16x16)
      // ldmatrix.{...}.x4.{...} 需要warp内32个线程都参与，每个线程提供一个有效的，不重叠的addr
      int lane_smem_a_m = warp_smem_a_m + lane_id % 16; // t{0...15}=0~15, t{16...31}=0~15
      int lane_smem_a_k = (lane_id / 16) * 8; // 0, 8, t{0...15}=0, t{16...31}=8
      uint32_t lane_smem_a_ptr =
          (smem_a_base_ptr +
           (smem_sel * s_a_stage_offset + lane_smem_a_m * BK + lane_smem_a_k) *
               sizeof(half));
      LDMATRIX_X4(RA[i][0], RA[i][1], RA[i][2], RA[i][3], lane_smem_a_ptr);
    }

    // ldmatrix.x2: 加载 B 的 k16n8 片段 (非转置)
    // 为什么不用 .trans? 因为 smem 中存的是 B^T row-major [N][K],
    // ldmatrix 逐行加载 B^T 的行 = B 的列，天然给出 col-major B fragment → 直接匹配 MMA row.col
#pragma unroll
    for (int j = 0; j < kValTileN; ++j) {
      // {0,...,3} * (8 * 4) + j * 8 = {0,32,64,96} + {0,8,16,24} = {0,8,...,120}
      // warp_n_0 offsets {0,8,16,24}; warp_n_1 offsets {32,40,48,56}
      // warp_n_2 offsets {64,72,80,88}; warp_n_3 offsets {96,104,112,120}
      int warp_smem_b_n = warp_n * (kMmaN * kValTileN) + j * kMmaN;
      // {0,8,...,120} + {0~7} = {0~127}, 按照row-major的顺序访问B^T的2个8x8 matrix (8x16)
      // ldmatrix.{...}.x2.{...} 需要warp内前16个线程参与，后16个线程传的addr会被忽略
      int lane_smem_b_n = warp_smem_b_n + lane_id % 8; // t{0...7}=0~7, t{8...15}=0~7
      int lane_smem_b_k = ((lane_id / 8) % 2) * 8; // t{0...7}=0, t{8...15}=8
      uint32_t lane_smem_b_ptr =
          (smem_b_base_ptr +
           (smem_sel * s_b_stage_offset + lane_smem_b_n * BK + lane_smem_b_k) *
               sizeof(half));
```

```
      LDMATRIX_X2(RB[j][0], RB[j][1], lane_smem_b_ptr);
    }

    // MMA compute: 每个Warp(MMA) 在M方向重复kValTileM(4)次，在N方向重复kValTileN(4)次
#pragma unroll
    for (int i = 0; i < kValTileM; ++i) {
#pragma unroll
      for (int j = 0; j < kValTileN; ++j) {
        HMMA16816(RC[i][j][0], RC[i][j][1], // C fragment
                  RA[i][0], RA[i][1], RA[i][2], RA[i][3], // A fragment
                  RB[j][0], RB[j][1], // B fragment
                  RC[i][j][0], RC[i][j][1]);
      }
    }

    // 自适应等待：流水线满载期用 kStages-2，尾部排空用 0
    if (k + kStages - 1 < NUM_K_TILES) {
      CP_ASYNC_WAIT_GROUP(kStages - 2);
    } else { // 对于尾部的 k，等待所有剩余的 cp.async 完成
      CP_ASYNC_WAIT_GROUP(0);
    }
    __syncthreads();
  }

  // Epilogue: 寄存器 → global memory（通过 warp shuffle + 128-bit store）
  {
    for (int i = 0; i < kValTileM; ++i) {
      // RC[kValTileM][kValTileN][2] 中  RC[...][...][2] 中保存了2个 uint32
      // 寄存器，每个uint32 寄存器表示两个临近的fp16值：{c0,c1}，然后RC[...][...][0]
      // 和RC[...][...][1]代表的是按照col-major排布的2个8x8子矩阵上（不同物理行，跨8x8）
      // 同一个位置上的元素，也就是，实际代表了2个不同的行的元素，因此要分开RC0, RC1;
      // RC0表示第一行，8个half，可以用4个uint32寄存器来装；同理，RC1表示第二行。
      uint32_t RC0[kValTileN][4]; // 32 bits x 4 = 128 bits = 8 half
      uint32_t RC1[kValTileN][4]; // 32 bits x 4 = 128 bits = 8 half
      // ========================================================================
      // MMA m16n8k16 C fragment — a single 16×8 matrix (registers per warp):
      // Thread t holds 4 half values → RC[0] for rows 0-7, RC[1] for rows 8-15.
      // Mapping: row = t/4, col-pair = t%4 (each uint32 = 2 half values).
      //
      //       cols: c0-1    c2-3    c4-5    c6-7
      //   r0:  T0{c0,c1}   T1{c2,c3}  T2{c4,c5}  T3{c6,c7}  ┐
      //   r1:  T4{c0,c1}   T5{c2,c3}  T6{c4,c5}  T7{c6,c7}  │
      //   r2:  T8{c0,c1}   T9{c2,c3}  T10{c4,c5} T11{c6,c7} | RC[0]
      //   ...                                               |
      //   r7:  T28{c0,c1}  T29{c2,c3} T30{c4,c5} T31{c6,c7} ┘
      //   r8:  T0{c0,c1}   T1{c2,c3}  T2{c4,c5}  T3{c6,c7}  ┐
      //   r9:  T4{c0,c1}   T5{c2,c3}  T6{c4,c5}  T7{c6,c7}  │
      //  r10:  T8{c0,c1}   T9{c2,c3}  T10{c4,c5} T11{c6,c7} | RC[1]
      //   ...                                               |
      //  r15:  T28{c0,c1}  T29{c2,c3} T30{c4,c5} T31{c6,c7} ┘
      //
      // Within a 4-lane group (e.g. T0-T3 for row 0):
      //   lane+0 holds {c0,c1}, lane+1 holds {c2,c3},
      //   lane+2 holds {c4,c5}, lane+3 holds {c6,c7}.
      // shfl 从 lane+1/+2/+3 各收 2 个 half 到 lane+0，4 次 shuffle 凑齐
      // 一行 8 个 half = {c0,c1,c2,c3,c4,c5,c6,c7}，再由 lane%4==0 128-bit store。
      // ========================================================================
#pragma unroll
      for (int j = 0; j < kValTileN; ++j) {
        RC0[j][0] = RC[i][j][0];
        RC1[j][0] = RC[i][j][1];
        RC0[j][1] = __shfl_sync(0xffffffff, RC[i][j][0], lane_id + 1);
        RC0[j][2] = __shfl_sync(0xffffffff, RC[i][j][0], lane_id + 2);
```

```
        RC0[j][3] = __shfl_sync(0xffffffff, RC[i][j][0], lane_id + 3);
        RC1[j][1] = __shfl_sync(0xffffffff, RC[i][j][1], lane_id + 1);
        RC1[j][2] = __shfl_sync(0xffffffff, RC[i][j][1], lane_id + 2);
        RC1[j][3] = __shfl_sync(0xffffffff, RC[i][j][1], lane_id + 3);
      }
      // 每 4 个 lane 中只有 lane 0 做 128-bit store
      // lane_id / 4 → 行号映射（每 4 个连续 lane 负责上8x8矩阵和下8x8矩阵各一行）：
      //   ┌─────────┬──────────┬──────────────┬──────────────┐
      //   | lane_id | lane_id/4 | RC[0] 的行   | RC[1] 的行   |
      //   ├─────────┼──────────┼──────────────┼──────────────┤
      //   | 0~3     | 0         | row 0        | row 8        |
      //   | 4~7     | 1         | row 1        | row 9        |
      //   | 8~11    | 2         | row 2        | row 10       |
      //   | 12~15   | 3         | row 3        | row 11       |
      //   | 16~19   | 4         | row 4        | row 12       |
      //   | 20~23   | 5         | row 5        | row 13       |
      //   | 24~27   | 6         | row 6        | row 14       |
      //   | 28~31   | 7         | row 7        | row 15       |
      //   └─────────┴──────────┴──────────────┴──────────────┘
      if (lane_id % 4 == 0) {
        // {0,1} * (16 * 4) + i * 16 = {0,64} + {0,16,32,48} = {0,16,32,48,64,80,96,112}
        int store_warp_smem_c_m = warp_m * (kMmaM * kValTileM) + i * kMmaM; // smem row
        // 这里用 lane_id / 4 → {0,1,2,3,4,5,6,7}，对应 RC0/RC1 (+8) 的行号，表示2个8x8矩阵的行号
        int store_lane_gmem_c_m = by * BM + store_warp_smem_c_m + lane_id / 4; // gmem row
#pragma unroll
        for (int j = 0; j < kValTileN; ++j) {
          // {0,...,3} * (8 * 4) + j * 8 = {0,32,64,96} + {0,8,16,24} = {0,8,...,120}
          int store_warp_smem_c_n = warp_n * (kMmaN * kValTileN) + j * kMmaN; // smem col
          int store_lane_gmem_c_n = bx * BN + store_warp_smem_c_n; // gmem col
          int store_gmem_c_addr_0 = store_lane_gmem_c_m * N + store_lane_gmem_c_n; // 1-th 8x8 matrix
          int store_gmem_c_addr_1 = (store_lane_gmem_c_m + 8) * N + store_lane_gmem_c_n; // 2-th 8x8 matrix
          // 128-bit store: 一次写入 8 个 half
          *reinterpret_cast<float4 *>(&C[store_gmem_c_addr_0]) =
              *reinterpret_cast<float4 *>(&RC0[j][0]);
          *reinterpret_cast<float4 *>(&C[store_gmem_c_addr_1]) =
              *reinterpret_cast<float4 *>(&RC1[j][0]);
        }
      }
    }
  }
}

// ============================================================================
// Phase 7b-3: HGEMM MMA Swizzle — m16n8k16 + multistage pipeline + TN 布局
// + XOR swizzle 消除 smem bank conflict（统一循环版）
// ============================================================================
// 面试要点（Swizzle 原理 — smem Bank Conflict 消除）：
//   - 问题：ldmatrix 以 8×8 片段（m8n8）从 smem 加载 16×16 的矩阵。同一 warp 中
//     不同 lane 访问的地址在 bank 上产生冲突 → 串行化（most common: 2/4-way）。
//   - XOR swizzle 方案：对列地址做 `((j>>3) ^ (i>>2)) % 2 << 3` 的 XOR 置换，
//     将连续 4 行的 bank 映射打散。每 4 行为一组的 pattern 交替：
//        rows 0~3: 列 0→地址偏移 0, 列 8→地址偏移 8
//        rows 4~7: 列 0→地址偏移 8, 列 8→地址偏移 0（XOR 翻转）
//        rows 8~11: repeat rows 0~3 pattern
//        rows 12~15: repeat rows 4~7 pattern
//   - 效果：原来 n-way 的 bank conflict 降低到 1-way（fully conflict-free）。
//   - 优势：无需 smem PAD（不浪费空间），原理上完全消除特定 pattern 的 bank conflict。
//   - 局限性：要求 kColStride ≤ 16（即 BK ≤ 16），kStep ∈ {4,8}。
//     对 MMA m16n8k16 的 BK=16 而言刚好满足。
//
// 参考：LeetCUDA/kernels/hgemm/mma/swizzle/hgemm_mma_stage_tn_swizzle.cu
// 参考：https://zhuanlan.zhihu.com/p/4746910252
```

```cpp
// ---- Swizzle 辅助函数 ----

// swizzle_permuted_j: 对 smem 列索引 j 做 XOR 置换，消除 bank conflict。
// i: row index; j: col index.
// e.g kColStride = 16, kStep = 8 -> load 8 half as 128 bits memory issue.
// 公式: ((j / kStep) ^ (i / 4)) % (kColStride / kStep) * kStep
// 用位运算展开（kStep=8）：(((j >> 3) ^ (i >> 2)) % (kColStride >> 3)) << 3
// 限制: kColStride ≤ 16（BK ≤ 16），kStep ∈ {4, 8}，kColStride % kStep == 0
// source: LeetCUDA/kernels/hgemm/mma/swizzle/hgemm_mma_stage_tn_swizzle.cu
template <const int kColStride = 16, const int kStep = 8>
static __device__ __forceinline__ int swizzle_permuted_j(int i, int j) {
  // for col_stride > 16, we have to permute it using col major ZigZag order.
  // e.g, A smem logical layout [Br,d]=[Br,64] -> store layout [4][Br][16].
  static_assert(kColStride <= 16, "kColStride must <= 16");
  // swizzle: ((int(j / kStep) ^ int(i / 4)) % int(kColStride / kStep)) * kStep;
  static_assert(kStep == 4 || kStep == 8, "kStep must be 8 or 4.");
  static_assert(kColStride % kStep == 0,
                "kColStride must be multiple of kStep.");
  if constexpr (kStep == 8) {
    // j >> 3: 表示8个half(=16 bytes)数据为一个chunk; i >> 2: 表示4行为一组
    // kColStride >> 3: 按照kStep=8计算chunk idx，kColStride=16 → {0, 1}
    // 最后的 << 3: 将chunk idx转换为实际的列偏移量（8个half/chunk），{0, 8}
    return (((j >> 3) ^ (i >> 2)) % (kColStride >> 3)) << 3;
  } else {
    static_assert(kStep == 4);
    return (((j >> 2) ^ (i >> 2)) % (kColStride >> 2)) << 2;
  }
}

// swizzle_A: A 矩阵专用封装（kMmaAtomK=16, kStep=8）。
// 16 行（一个 MMA atom 的 M 维）内的 swizzle pattern:
// 8个half=16 bytes=128 bits=4 banks (32 bits/bank) 组成一个phase,
// 触发一次合并的memory transaction. 这里的col=16的swizzle，相当于
// TMA中的SWIZZLE_32B pattern.
// -------------------
// -col 0~16, step 8--
// -------------------
// | row 0  | (0, 8) |
// | row 1  | (0, 8) |
// | row 2  | (0, 8) |
// | row 3  | (0, 8) |
// -------------------
// | row 4  | (8, 0) |
// | row 5  | (8, 0) |
// | row 6  | (8, 0) |
// | row 7  | (8, 0) |
// -------------------
// | row 8  | (0, 8) |
// | row 9  | (0, 8) |
// | row 10 | (0, 8) |
// | row 11 | (0, 8) |
// -------------------
// | row 12 | (8, 0) |
// | row 13 | (8, 0) |
// | row 14 | (8, 0) |
// | row 15 | (8, 0) |
// -------------------
// source: LeetCUDA/kernels/hgemm/mma/swizzle/hgemm_mma_stage_tn_swizzle.cu
template <const int kMmaAtomK = 16>
static __device__ __forceinline__ int swizzle_A(int i, int j) {
  return swizzle_permuted_j<kMmaAtomK, 8>(i, j);
}

```

```cpp
// swizzle_B: B 矩阵专用封装（与 A 相同的 pattern）。
// B^T smem 布局 [BN][BK]=[128][16] 在 BK 维做 swizzle，
// pattern 与 A 完全相同（kMmaAtomK=16, kStep=8）。
// source: LeetCUDA/kernels/hgemm/mma/swizzle/hgemm_mma_stage_tn_swizzle.cu
template <const int kMmaAtomK = 16>
static __device__ __forceinline__ int swizzle_B(int i, int j) {
  return swizzle_permuted_j<kMmaAtomK, 8>(i, j);
}

// ============================================================================
// Phase 7b-3: HGEMM MMA — m16n8k16 + multistage pipeline + TN 布局 + XOR swizzle
//              + Register Double Buffering (kValTileK=2, BK=32 统一 tile)
// ============================================================================
// 在 Phase 7b-2 的 smem XOR swizzle 基础上增加寄存器双缓冲:
//   - kValTileK=2: 每个 BK tile 包含 2 个 kMmaK slice (BK = kMmaK * kValTileK = 32)
//   - BK=32 统一 tile: kMmaK=0 和 kMmaK=1 数据连续存放在同一个 BK=32 宽的 smem tile 中
//   - RA[2][kValTileM][4] / RB[2][kValTileN][2]: 双份寄存器乒乓切换
//   - ldmatrix 与 MMA 计算重叠: 加载 k_step=1 的同时用另一组寄存器做 k_step=0 计算
//
// ★ smem 布局:
//   s_a: [stage0][stage1][stage2], 每个 stage = BM × BK = 128 × 32
//   s_b: 紧接 s_a 之后
//   每个 stage 内 k_step=0 列偏移 0, k_step=1 列偏移 +kMmaK(=16)
//
// ★ 寄存器双缓冲时间线（每 k 迭代内）:
//   Step 1: G→S cp.async 预取 stage(k+kStages-1) 全部 k_step（条件）
//   Step 2: MMA k_step=0（用 reg[load], 已在上轮 post-wait 中 ldmatrix）
//   Step 3: for k_step=1..kValTileK-1: ldmatrix(列偏移 k_step*kMmaK)→reg[store], flip, MMA→reg[load]
//   Step 4: adaptive wait + __syncthreads
//   Step 5: S→R ldmatrix 预加载 stage(k+1) k_step=0 → reg[store]（条件）
//
// 参考: LeetCUDA/kernels/hgemm/mma/swizzle/hgemm_mma_stage_tn_swizzle_x2.cu
// Grid:  ((N+127)/128/S, (M+127)/128, S), S=(N+2047)/2048, 3D block swizzle
// Block: (256, 1, 1), 8 warps
template <const int kMmaM = 16,             // MMA atom M dim (m16n8k16)
          const int kMmaN = 8,              // MMA atom N dim
          const int kMmaK = 16,             // MMA atom K dim
          const int kMmaTileM = 2,          // warps along M, 2 → warp tile M = 32
          const int kMmaTileN = 4,          // warps along N, 4 → warp tile N = 32
          const int kValTileM = 4,          // value-repeat along M, BM = 16*2*4 = 128
          const int kValTileN = 4,          // value-repeat along N, BN = 8*4*4 = 128
          const int kValTileK = 2,          // MMA_K slices per BK tile, BK = kMmaK*2 = 32
          const int kStages = 3,            // cp.async pipeline depth
          const int kBlockSwizzle = 0>      // 1 enables 3D grid swizzle for L2 locality
__global__ void __launch_bounds__(256)
    hgemm_mma_stages_tn_swizzle(half *A, half *B, half *C, int M, int N, int K) {
  static_assert(kValTileK == 2, "Only support kValTileK=2 for register double buffering");
  static_assert(kBlockSwizzle == 0 || kBlockSwizzle == 1, "kBlockSwizzle must be 0 or 1");
  static_assert(kStages >= 2, "kStages must be >= 2 for cp.async pipeline");
  // Block Swizzle: 在 grid x 维度做 swizzle, 改善 L2 cache 局部性
  const int bx = ((int)kBlockSwizzle) * blockIdx.z * gridDim.x + blockIdx.x;
  const int by = blockIdx.y;
  constexpr int BM = kMmaM * kMmaTileM * kValTileM; // 16*2*4=128
  constexpr int BN = kMmaN * kMmaTileN * kValTileN; // 8*4*4=128
  constexpr int BK = kMmaK * kValTileK;             // 16*2=32

  // kStages stages, each with BM×BK for A, BN×BK for B
  // smem: kValTileK 个 kMmaK slice 连续存放在同一个 BK=32 宽 tile 中
  extern __shared__ half smem[];
  half *s_a = smem;
  half *s_b = smem + kStages * BM * BK;
  constexpr int s_a_stage_offset = BM * BK;
  constexpr int s_b_stage_offset = BN * BK;
```

```
1827
1828    const int tid = threadIdx.y * blockDim.x + threadIdx.x;
1829    const int warp_id = tid / kWarpSize;
1830    const int lane_id = tid % kWarpSize;
1831    const int warp_m = warp_id % kMmaTileM; // 0,1 (M 方向 2 个 warp) kMmaTileM = 2
1832    const int warp_n = warp_id / kMmaTileM; // 0,1,2,3 (N 方向 4 个 warp)
1833
1834    // 线程到 global memory 的映射 (用于加载 A 和 B) 共 256 个线程
1835    // TN 布局关键: A[m*K+k] 是 row-major, B^T[n*K+k] 是 row-major (内维连续的是 K)
1836    // 注意: smem_a_k 和 smem_b_k 依然使用 0/8, 虽然BK=16*2=32, 在后续的kValTileK循环中
1837    // 会加上 k_step*kMmaK=0/16, 最终得到 smem 中的列偏移 0/8/16/24, 正好覆盖 BK=32
1838    int load_smem_a_m = tid / 2;                 // 0~127
1839    int load_smem_a_k = (tid % 2 == 0) ? 0 : 8;  // 0, 8
1840    int load_smem_b_n = tid / 2;                 // 0~127 → B^T 的 N 方向 (row-major 的行)
1841    int load_smem_b_k = (tid % 2 == 0) ? 0 : 8;  // 0, 8  → B^T 的 K 方向 (row-major 的列)
1842    int load_gmem_a_m = by * BM + load_smem_a_m; // C/A 全局行号 = M 方向的 tile 起始 + 线程偏移
1843    int load_gmem_b_n = bx * BN + load_smem_b_n; // C/B 全局列号 = N 方向的 tile 起始 + 线程偏移
1844    if (load_gmem_a_m >= M || load_gmem_b_n >= N)
1845      return;
1846
1847    // 8个Warps(MMAs)的排布是M方向2个, N方向4个, 则MMA Atom一次性能处理的tile大小是[2*16,4*8]=[32,32].
1848    // CUDA中每个block能放的线程数是有上限的 (warp数量有限), 一般为4或者8个warps。为了能处理更大的C tile,
1849    // 需要把MMA Atom的tile再M方向和N方向各自重复4次, 得到[128,128]的C block tile, 这就是Value Tile的作用。
1850    // MMA Tile对应到cutlass cute中的TiledMMA的概念, Value Tile对应到cutlass cute中的PermuteMNK的概念。
1851    uint32_t RC[kValTileM][kValTileN][2] = {0}; // 初始化为 0
1852
1853    uint32_t smem_a_base_ptr = __cvta_generic_to_shared(s_a);
1854    uint32_t smem_b_base_ptr = __cvta_generic_to_shared(s_b);
1855
1856    // Prefetch (kStages-1) stages: load all k_step slices for each early stage
1857  #pragma unroll
1858    for (int k = 0; k < (kStages - 1); ++k) {
1859  #pragma unroll
1860      for (int k_step = 0; k_step < kValTileK; ++k_step) {
1861        int load_gmem_a_k = k * BK + (k_step * kMmaK) + load_smem_a_k;
1862        int load_gmem_a_addr = load_gmem_a_m * K + load_gmem_a_k;
1863        int load_gmem_b_k = k * BK + (k_step * kMmaK) + load_smem_b_k;
1864        int load_gmem_b_addr = load_gmem_b_n * K + load_gmem_b_k;
1865
1866        uint32_t load_smem_a_ptr =
1867            (smem_a_base_ptr +
1868             (k * s_a_stage_offset + load_smem_a_m * BK +
1869              k_step * kMmaK +
1870              swizzle_A<kMmaK>(load_smem_a_m, load_smem_a_k)) *
1871                 sizeof(half));
1872        CP_ASYNC_CG(load_smem_a_ptr, &A[load_gmem_a_addr], 16);
1873
1874        uint32_t load_smem_b_ptr =
1875            (smem_b_base_ptr +
1876             (k * s_b_stage_offset + load_smem_b_n * BK +
1877              k_step * kMmaK +
1878              swizzle_B<kMmaK>(load_smem_b_n, load_smem_b_k)) *
1879                 sizeof(half));
1880        CP_ASYNC_CG(load_smem_b_ptr, &B[load_gmem_b_addr], 16);
1881      }
1882      CP_ASYNC_COMMIT_GROUP();
1883    }
1884
1885    CP_ASYNC_WAIT_GROUP(kStages - 2);
1886    __syncthreads();
1887
1888    // Register double buffers: RA[0/1] for k_step=0/1 A data, RB[0/1] for B data
1889    uint32_t RA[2][kValTileM][4];
```

```
  uint32_t RB[2][kValTileN][2];
  int reg_st_idx = 0; // write target for ldmatrix
  int reg_ld_idx = 1; // read source for MMA

// Initial ldmatrix: load stage 0, S -> R, k_step=0 (0~15列) → reg[0]
// 此时，reg_st_idx = 0，对0位置的寄存器buffer做初始化。
#pragma unroll
  for (int i = 0; i < kValTileM; ++i) {
    int warp_smem_a_m = warp_m * (kMmaM * kValTileM) + i * kMmaM;
    int lane_smem_a_m = warp_smem_a_m + lane_id % 16;
    int lane_smem_a_k = (lane_id / 16) * 8;
    uint32_t lane_smem_a_ptr =
        (smem_a_base_ptr +
         (0 * s_a_stage_offset + lane_smem_a_m * BK +
          swizzle_A<kMmaK>(lane_smem_a_m, lane_smem_a_k)) *
             sizeof(half));
    LDMATRIX_X4(RA[reg_st_idx][i][0], RA[reg_st_idx][i][1],
                RA[reg_st_idx][i][2], RA[reg_st_idx][i][3],
                lane_smem_a_ptr);
  }
#pragma unroll
  for (int j = 0; j < kValTileN; ++j) {
    int warp_smem_b_n = warp_n * (kMmaN * kValTileN) + j * kMmaN;
    int lane_smem_b_n = warp_smem_b_n + lane_id % 8;
    int lane_smem_b_k = ((lane_id / 8) % 2) * 8;
    uint32_t lane_smem_b_ptr =
        (smem_b_base_ptr +
         (0 * s_b_stage_offset + lane_smem_b_n * BK +
          swizzle_B<kMmaK>(lane_smem_b_n, lane_smem_b_k)) *
             sizeof(half));
    LDMATRIX_X2(RB[reg_st_idx][j][0], RB[reg_st_idx][j][1],
                lane_smem_b_ptr);
  }

  // 统一循环: k 从 0 开始，条件 G→S / S→R / wait 处理所有边界情况
  // BK = kMmaK * kValTileK = 32，每个 k 迭代覆盖 BK=32 个 K 元素
  const int NUM_K_TILES = div_ceil(K, BK);
  for (int k = 0; k < NUM_K_TILES; ++k) {
    int smem_sel = k % kStages;
    int smem_sel_next = (k + kStages - 1) % kStages;

    // G→S: 条件预取 stage(k+kStages-1) 全部 k_step slice
    if (k + kStages - 1 < NUM_K_TILES) {
#pragma unroll
      for (int k_step = 0; k_step < kValTileK; ++k_step) {
        int load_gmem_a_k =
            (k + kStages - 1) * BK + (k_step * kMmaK) + load_smem_a_k;
        int load_gmem_a_addr = load_gmem_a_m * K + load_gmem_a_k;
        int load_gmem_b_k =
            (k + kStages - 1) * BK + (k_step * kMmaK) + load_smem_b_k;
        int load_gmem_b_addr = load_gmem_b_n * K + load_gmem_b_k;

        uint32_t load_smem_a_ptr =
            (smem_a_base_ptr +
             (smem_sel_next * s_a_stage_offset + load_smem_a_m * BK +
              k_step * kMmaK +
              swizzle_A<kMmaK>(load_smem_a_m, load_smem_a_k)) *
                 sizeof(half));
        CP_ASYNC_CG(load_smem_a_ptr, &A[load_gmem_a_addr], 16);

        uint32_t load_smem_b_ptr =
            (smem_b_base_ptr +
             (smem_sel_next * s_b_stage_offset + load_smem_b_n * BK +
```

```
              k_step * kMmaK +
              swizzle_B<kMmaK>(load_smem_b_n, load_smem_b_k)) *
                 sizeof(half));
        CP_ASYNC_CG(load_smem_b_ptr, &B[load_gmem_b_addr], 16);
      }
      CP_ASYNC_COMMIT_GROUP();
    }

    // 内层 k_step 循环: flip → ldmatrix(k_step+1) → MMA, 乒乓交替
    // 初始: st=0, ld=1, reg[0]=preload k_step=0; reg[1]=未初始化
    // 每轮: flip→ldmatrix next k_step→reg[st], MMA reg[ld] (k_step+1 的 ldmatrix 条件化)

#pragma unroll
    for (int k_step = 0; k_step < kValTileK; ++k_step) {
      reg_st_idx ^= 1; // 0 -> 1; 1 -> 0
      reg_ld_idx ^= 1; // 1 -> 0; 0 -> 1

      if (k_step + 1 < kValTileK) {
        int smem_k_offset = (k_step + 1) * kMmaK;
#pragma unroll
        for (int i = 0; i < kValTileM; ++i) {
          int warp_smem_a_m = warp_m * (kMmaM * kValTileM) + i * kMmaM;
          int lane_smem_a_m = warp_smem_a_m + lane_id % 16;
          int lane_smem_a_k = (lane_id / 16) * 8;
          uint32_t lane_smem_a_ptr =
              (smem_a_base_ptr +
              (smem_sel * s_a_stage_offset + lane_smem_a_m * BK +
                smem_k_offset +
                swizzle_A<kMmaK>(lane_smem_a_m, lane_smem_a_k)) *
                  sizeof(half));
          LDMATRIX_X4(RA[reg_st_idx][i][0], RA[reg_st_idx][i][1],
                      RA[reg_st_idx][i][2], RA[reg_st_idx][i][3],
                      lane_smem_a_ptr);
        }
  #pragma unroll
        for (int j = 0; j < kValTileN; ++j) {
          int warp_smem_b_n = warp_n * (kMmaN * kValTileN) + j * kMmaN;
          int lane_smem_b_n = warp_smem_b_n + lane_id % 8;
          int lane_smem_b_k = ((lane_id / 8) % 2) * 8;
          uint32_t lane_smem_b_ptr =
              (smem_b_base_ptr +
              (smem_sel * s_b_stage_offset + lane_smem_b_n * BK +
                smem_k_offset +
                swizzle_B<kMmaK>(lane_smem_b_n, lane_smem_b_k)) *
                  sizeof(half));
          LDMATRIX_X2(RB[reg_st_idx][j][0], RB[reg_st_idx][j][1],
                      lane_smem_b_ptr);
        }
      }

#pragma unroll
      for (int i = 0; i < kValTileM; ++i) {
#pragma unroll
        for (int j = 0; j < kValTileN; ++j) {
          HMMA16816(RC[i][j][0], RC[i][j][1],
                    RA[reg_ld_idx][i][0], RA[reg_ld_idx][i][1],
                    RA[reg_ld_idx][i][2], RA[reg_ld_idx][i][3],
                    RB[reg_ld_idx][j][0], RB[reg_ld_idx][j][1],
                    RC[i][j][0], RC[i][j][1]);
        }
      }
    }

```

```
    // 自适应等待：满载期用 kStages-2，尾部排空用 0
    if (k + kStages - 1 < NUM_K_TILES) {
      CP_ASYNC_WAIT_GROUP(kStages - 2);
    } else {
      CP_ASYNC_WAIT_GROUP(0);
    }
    __syncthreads();

    // 从下一阶段加载 k_step=0 数据，供下一轮 k 迭代使用，此时 reg_st_idx=0
    if (k + 1 < NUM_K_TILES) {
      smem_sel = (k + 1) % kStages; // old smem_sel + 1
#pragma unroll
      for (int i = 0; i < kValTileM; ++i) {
        int warp_smem_a_m = warp_m * (kMmaM * kValTileM) + i * kMmaM;
        int lane_smem_a_m = warp_smem_a_m + lane_id % 16;
        int lane_smem_a_k = (lane_id / 16) * 8;
        uint32_t lane_smem_a_ptr =
            (smem_a_base_ptr +
             (smem_sel * s_a_stage_offset + lane_smem_a_m * BK +
              swizzle_A<kMmaK>(lane_smem_a_m, lane_smem_a_k)) *
                 sizeof(half));
        LDMATRIX_X4(RA[reg_st_idx][i][0], RA[reg_st_idx][i][1],
                    RA[reg_st_idx][i][2], RA[reg_st_idx][i][3],
                    lane_smem_a_ptr);
      }
#pragma unroll
      for (int j = 0; j < kValTileN; ++j) {
        int warp_smem_b_n = warp_n * (kMmaN * kValTileN) + j * kMmaN;
        int lane_smem_b_n = warp_smem_b_n + lane_id % 8;
        int lane_smem_b_k = ((lane_id / 8) % 2) * 8;
        uint32_t lane_smem_b_ptr =
            (smem_b_base_ptr +
             (smem_sel * s_b_stage_offset + lane_smem_b_n * BK +
              swizzle_B<kMmaK>(lane_smem_b_n, lane_smem_b_k)) *
                 sizeof(half));
        LDMATRIX_X2(RB[reg_st_idx][j][0], RB[reg_st_idx][j][1],
                    lane_smem_b_ptr);
      }
    }
  }

  // Epilogue: 复用 RA[2][4][4] 寄存器做 warp shuffle → 128-bit collective store
  // 做RC的warp shuffle正好需要[2][4][4]的寄存器空间，RA[2][4][4]正好可以复用
  for (int i = 0; i < kValTileM; ++i) {
#pragma unroll
    for (int j = 0; j < kValTileN; ++j) {
      RA[0][j][0] = RC[i][j][0];
      RA[1][j][0] = RC[i][j][1];
      RA[0][j][1] = __shfl_sync(0xffffffff, RC[i][j][0], lane_id + 1);
      RA[0][j][2] = __shfl_sync(0xffffffff, RC[i][j][0], lane_id + 2);
      RA[0][j][3] = __shfl_sync(0xffffffff, RC[i][j][0], lane_id + 3);
      RA[1][j][1] = __shfl_sync(0xffffffff, RC[i][j][1], lane_id + 1);
      RA[1][j][2] = __shfl_sync(0xffffffff, RC[i][j][1], lane_id + 2);
      RA[1][j][3] = __shfl_sync(0xffffffff, RC[i][j][1], lane_id + 3);
    }
    if (lane_id % 4 == 0) {
      int store_warp_smem_c_m = warp_m * (kMmaM * kValTileM) + i * kMmaM;
      int store_lane_gmem_c_m = by * BM + store_warp_smem_c_m + lane_id / 4;
#pragma unroll
      for (int j = 0; j < kValTileN; ++j) {
        int store_warp_smem_c_n = warp_n * (kMmaN * kValTileN) + j * kMmaN;
        int store_lane_gmem_c_n = bx * BN + store_warp_smem_c_n;
        int store_gmem_c_addr_0 =
```

```
            store_lane_gmem_c_m * N + store_lane_gmem_c_n;
        int store_gmem_c_addr_1 =
            (store_lane_gmem_c_m + 8) * N + store_lane_gmem_c_n;
        *reinterpret_cast<float4 *>(&C[store_gmem_c_addr_0]) =
            *reinterpret_cast<float4 *>(&RA[0][j][0]);
        *reinterpret_cast<float4 *>(&C[store_gmem_c_addr_1]) =
            *reinterpret_cast<float4 *>(&RA[1][j][0]);
      }
    }
  }
}

// ============================================================================
// Phase 7c: HGEMM CuTe — CUTLASS CuTe DSL 实现 (SM80+, Tensor Core 全自动调度)
// ============================================================================
// 面试要点 (CuTe vs 手写 MMA PTX):
//   - CuTe (CUTLASS Templates) 是 NVIDIA CUTLASS 3.x 的核心 DSL，提供编译期
//     Tensor 抽象，自动推导 MMA、Copy、Swizzle 的线程-数据映射
//   - 与手写 MMA PTX (Phase 7b) 的对比:
//     - 手写 MMA: 手动计算 smem 地址、手动 ldmatrix/mma PTX、手动 swizzle、手动 epilogue shuffle
//     - CuTe: 声明 TiledMMA / TiledCopy / SmemLayout，编译器自动推导线程-数据映射，
//       cute::copy / cute::gemm 自动展开为高效指令序列
//   - 核心抽象 ("CuTe 五要素"):
//     1. Tensor: 全局/共享/寄存器数据的逻辑视图 = (ptr, shape, stride)
//     2. TiledMMA: 描述 MMA 指令 + warp/warpgroup 排布 + value tile 的 compile-time type
//     3. TiledCopy: 描述数据搬运 (G→S, S→R, R→S, S→G) 的线程-元素映射
//     4. Layout + Swizzle: 描述 smem 的数据排布和 bank conflict 消除
//     5. Pipeline: cp.async + multistage, 由 cute::copy 自动管理
//
//   CuTe kernel 的参数推导链 (launch wrapper 负责实例化，kernel 只接收实例化后的类型):
//     MMA Atom (SM80_16x8x16_F16F16F16F16_TN)
//       → TiledMMA (EURepeat=MxNxK, ValTile=MxNxK)
//       → TiledCopy (G2S/S2R/R2S/S2G, 每种有 ThrLayout + ValLayout)
//       → SmemLayout (Atom + Swizzle + tile_to_shape + Stage)
//
//   调参口诀 (面试速记):
//     BM/BN/BK 越大 → smem 越大 → occupancy 越低，但 K 循环更少 → 指令开销更低
//     Swizzle<B,M,S> 的核心不是改变逻辑 Tensor 的 shape，而是把一维 offset
//     重新解释为一个二维逻辑空间，再把二维坐标映射回 bank-conflict-free 的物理 offset:
//       1. 连续的 2^M 个元素组成二维空间中的一个基本元素; (元素宽度)
//       2. 连续的 2^S 个基本元素组成一行; (列数)
//       3. 二维空间包含 2^B 行; (行数)
//       4. 对二维坐标做列置换: icol' = irow ^ icol;
//       5. 保留基本元素内部的低 M 位，再将置换后的二维坐标编码回 offset。
//     因此 M 描述基本元素宽度，S 描述二维空间的列数，B 描述二维空间的行数。
//     CuTe 的位级实现等价于:
//       offset' = offset ^ ((offset & YYY) >> S),
//       YYY = ((1 << B) - 1) << (M + S)。
//     对 Swizzle<3,3,3>: 每个基本元素有 2^3=8 个值，每行有 2^3=8 个基本元素，
//     二维空间有 2^3=8 行; 元素地址位 [6:8] XOR 到 [3:5]，低 3 位保持不变。
//     一个完整 swizzle 周期覆盖 2^(M+S+B)=2^9=512 个元素 (FP16 下为 1024B)。
//     kStage: 2=最低延迟, 3/4=更好隐藏延迟 → 权衡 smem 占用
//     EURepeat: 在 M/N/K 方向重复 MMA atom 的执行单元布局，决定 TiledMMA 的线程排布
//       与逻辑 tile 组织; MMA atom越多，需要的线程数就越多
//
// ★ 与手写 MMA Swizzle (Phase 7b-3) 的本质区别:
//   - 手写 swizzle: 在 smem 地址计算时手动 XOR 列索引
//   - CuTe Swizzle: SmemLayout 声明式指定地址位的 XOR pattern, cute::copy 自动应用
//   - CuTe 优势: 类型安全，编译器可在编译期推导映射; 布局变更通常只需修改类型定义
//
// ★ 本实现的 Tile 配置:
//   - gmem/smem tile: BM=128, BN=256, BK=32, kStage=2; 这是 A/B tile 的目标 shape。
//   - MMA atom: SM80_16x8x16_F16F16F16F16_TN; EURepeat=2x2x1, MMA_P_T=32x32x16。
```

```
//   - TiledMMA 的逻辑 MMA tile 是 32×32×16，G2S/S2R copy 再把它映射到更大的 BM×BN tile;
//     BN=256 不是由 "8×2×16" 直接推导出的 MMA tile，而是本 wrapper 选择的 B tile 尺寸。
//   - Smem Swizzle<3,3,3>: 512-element address period 的 XOR 重排，针对 ldmatrix
//     的访问模式降低 bank conflict; 512 是 swizzle 周期，不等于 A atom 的逻辑元素数。
//   - Threads: size(MMA{}) = 128 (4 warps × 2×2 EU repeat) 。
//   - Dynamic smem: Stage=2 时 A[128×32] + B[256×32] 共 49,152 bytes (约 48 KiB) 。
//
// 参考: LeetCUDA/kernels/hgemm/cutlass/hgemm_mma_stage_tn_cute.cu
// 参考: https://zhuanlan.zhihu.com/p/671419093 (CuTe Swizzle 详解)
// ============================================================================

#if defined(NOTES_V2_ENABLE_CUTE)

#include <cute/tensor.hpp>

// ============================================================================
// Phase 7c-1: CuTe HGEMM Kernel — TN 布局 + Smem Swizzle + Multistage Pipeline
// ============================================================================
// TN 布局: C[M×N] = A[M×K] × B^T[N×K]
//   - A[M][K] row-major, B^T[N][K] row-major (等价 B col-major [K×N])
//   - CuTe 中通过 make_tensor(make_gmem_ptr(ptr), shape, stride) 描述
//
// Pipeline 流程 (stage 是 K tile 的循环缓冲槽，不是一条完整计算链) :
//   1. PREFETCH: 预加载 kStage-1 个 K tile (G→S via cp.async)，填满 stage。
//   2. 主循环 over K tiles:
//      a. 从当前 stage 做 S→R: cute::copy(...) → ldmatrix;
//      b. 对当前 K slice 做 MMA: cute::gemm(...) → mma.sync;
//      c. 在处理当前 tile 的首个 MMA slice 时，异步提交下一个 tile 的 G→S;
//      d. 当前 tile 的最后一个 K slice 完成后等待并切换 smem stage。
//   3. Epilogue: R→S (UniversalCopy 写入 C scratchpad) → S→G (128-bit store) 。
//
// 面试对比 — 手写 MMA vs CuTe 代码量:
//   手写: ldmatrix PTX 地址计算 + swizzle XOR + mma PTX + epilogue shuffle (~200行)
//   CuTe: partition + copy + gemm (~80行), 其余由 launch wrapper 的类型推导完成
template <typename T,                       // element type (half)
          int BM, int BN, int BK,           // block tile: BM=128, BN=256, BK=32
          int kStage,                       // cp.async pipeline depth (2)
          typename TiledMMA,                // MMA: SM80_16x8x16_F16F16F16F16_TN, EURepeat=2x2x1
          typename G2SCopyA,                // G→S A: SM80_CP_ASYNC_CACHEGLOBAL<uint128_t>
          typename G2SCopyB,                // G→S B: same as G2SCopyA
          typename SmemLayoutA,             // smem A: Swizzle<3,3,3> + 8×BK atom → (BM,BK,kStage)
          typename SmemLayoutB,             // smem B: same atom → (BN,BK,kStage)
          typename SmemLayoutC,             // C scratchpad: Swizzle<3,3,3> + 32×32 atom, pipe=4
          typename S2RCopyAtomA,            // S→R A: SM75_U32x4_LDSM_N (ldmatrix.x4)
          typename S2RCopyAtomB,            // S→R B: same as S2RCopyAtomA
          typename R2SCopyAtomC,            // R→S C: UniversalCopy<int> (32-bit store)
          typename S2GCopyAtomC,            // S→G C: UniversalCopy<uint128_t> (128-bit wide store)
          typename S2GCopyC,                // S→G TiledCopy: ThrLayout{32,4} × ValLayout{1,8}
          const int BlockSwizzle = 0>       // 1 enables 3D grid swizzle for L2 locality
__global__ void hgemm_mma_stages_tn_cute(T *Aptr, T *Bptr, T *Dptr, int m,
                                         int n, int k) {
  using namespace cute;
  static_assert(BlockSwizzle == 0 || BlockSwizzle == 1, "BlockSwizzle must be 0 or 1");
  static_assert(kStage >= 2, "kStage must be >= 2 for cp.async pipeline");

  // 动态 shared memory: KStage 个 A/B tile; C epilogue 复用当前 A stage 的空间。
  extern __shared__ T shm_data[];
  T *Ashm = shm_data;
  T *Bshm = shm_data + cute::cosize(SmemLayoutA{});

  int idx = threadIdx.x;
  // BlockSwizzle: 0/1 控制是否启用 thread block swizzle，改善 L2 cache 局部性
  int ix = ((int)BlockSwizzle) * blockIdx.z * gridDim.x + blockIdx.x;
```

```
  int iy = blockIdx.y; // M 方向 block 索引
  // 当前 kernel 只有 CTA 级边界判断，没有 tile 内的逐元素 predication；调用方需保证
  // M % BM == 0、N % BN == 0、K % BK == 0，才能避免边界 tile 的越界访问或未写回。
  if (iy * BM >= m || ix * BN >= n) return;

  // CuTe Tensor 抽象: (ptr, shape, stride) 三元组描述数据布局
  // make_stride(leading_dim, Int<1>{})  → row-major: 同行相邻元素步长=1, 跨行步长=leading_dim
  Tensor A = make_tensor(make_gmem_ptr(Aptr), make_shape(m, k),
                         make_stride(k, Int<1>{}));
  Tensor B = make_tensor(make_gmem_ptr(Bptr), make_shape(n, k),
                         make_stride(k, Int<1>{}));
  Tensor D = make_tensor(make_gmem_ptr(Dptr), make_shape(m, n),
                         make_stride(n, Int<1>{}));

  // local_tile: 按 tiler 切出当前 CTA 的逻辑 tile；返回 Tensor 仍保留未切出的 remainder mode。
  // make_coord(iy, _): 固定 M/N 方向的 CTA 坐标，_ 保留 K 方向的 tile remainder:
  //   gA: (BM, BK, num_tile_k), gB: (BN, BK, num_tile_k), gD: (BM, BN)。
  Tensor gA = local_tile(A, make_tile(Int<BM>{}, Int<BK>{}), make_coord(iy, _));
  Tensor gB = local_tile(B, make_tile(Int<BN>{}, Int<BK>{}), make_coord(ix, _));
  Tensor gD = local_tile(D, make_tile(Int<BM>{}, Int<BN>{}), make_coord(iy, ix));

  // Shared memory Tensor: 按 SmemLayout 解读 smem 区域
  auto sA = make_tensor(make_smem_ptr(Ashm), SmemLayoutA{}); // (BM, BK, kStage)
  auto sB = make_tensor(make_smem_ptr(Bshm), SmemLayoutB{}); // (BN, BK, kStage)

  // TiledMMA partition: 将 MMA 的 A/B/C tile 映射到各线程的寄存器 fragment
  TiledMMA tiled_mma;
  auto thr_mma = tiled_mma.get_slice(idx);
  auto tCrA = thr_mma.partition_fragment_A(gA(_, _, 0)); // (MMA, MMA_M, MMA_K)
  auto tCrB = thr_mma.partition_fragment_B(gB(_, _, 0)); // (MMA, MMA_N, MMA_K)
  auto tCrD = thr_mma.partition_fragment_C(gD);          // (MMA, MMA_M, MMA_N)
  clear(tCrD); // 累加器清零

  // G2S TiledCopy: 描述 global → shared memory 的数据搬运（128-bit cp.async）。
  G2SCopyA g2s_tiled_copy_a;
  auto g2s_thr_copy_a = g2s_tiled_copy_a.get_slice(idx);
  auto tAgA_copy = g2s_thr_copy_a.partition_S(gA); // (CPY, CPY_M, CPY_K, num_tile_k)
  auto tAsA_copy = g2s_thr_copy_a.partition_D(sA); // (CPY, CPY_M, CPY_K, kStage)

  G2SCopyB g2s_tiled_copy_b;
  auto g2s_thr_copy_b = g2s_tiled_copy_b.get_slice(idx);
  auto tBgB_copy = g2s_thr_copy_b.partition_S(gB); // (CPY, CPY_N, CPY_K, num_tile_k)
  auto tBsB_copy = g2s_thr_copy_b.partition_D(sB);

  // S2R TiledCopy: 描述 shared → register 的数据搬运（使用 ldmatrix）
  // make_tiled_copy_A/B: 根据 TiledMMA 自动推导 S2R copy 的线程-数据映射
  auto s2r_tiled_copy_a = make_tiled_copy_A(S2RCopyAtomA{}, tiled_mma);
  auto s2r_thr_copy_a = s2r_tiled_copy_a.get_slice(idx);
  auto tAsA = s2r_thr_copy_a.partition_S(sA);     // (CPY, CPY_M, CPY_K, kStage)
  auto tCrA_view = s2r_thr_copy_a.retile_D(tCrA); // (CPY, CPY_M, CPY_K) — 与 rA 寄存器布局对齐

  auto s2r_tiled_copy_b = make_tiled_copy_B(S2RCopyAtomB{}, tiled_mma);
  auto s2r_thr_copy_b = s2r_tiled_copy_b.get_slice(idx);
  auto tBsB = s2r_thr_copy_b.partition_S(sB);
  auto tCrB_view = s2r_thr_copy_b.retile_D(tCrB);

  // PREFETCH: 预加载前 (kStage - 1) 个 K tile，填满循环缓冲；每次 copy 提交一个异步 G→S 操作。
  int itile_to_read = 0, ismem_read = 0, ismem_write = 0;
#pragma unroll
  for (int istage = 0; istage < kStage - 1; ++istage) {
    cute::copy(g2s_tiled_copy_a, tAgA_copy(_, _, _, istage),
               tAsA_copy(_, _, _, istage));
    cute::copy(g2s_tiled_copy_b, tBgB_copy(_, _, _, istage),
```

```
                tBsB_copy(_, _, _, istage));
    cp_async_fence();
    ++itile_to_read;
    ++ismem_write;
  }
  cp_async_wait<kStage - 2>();
  __syncthreads();

  // K 维第一层循环 — 按 BK 大小将 K 切分为 num_k_tiles 个 tile。
  // 外层 k_tile 遍历这些 BK-tile，内层 k_step 遍历 tile 内的 MMA_K slice。
  // 当前实现使用整除，要求 K % BK == 0；没有为尾部 K tile 做 predication/padding。
  int num_k_tiles = k / BK;
  // 循环前预加载首轮数据：将第一个 K tile 的第 0 个 K slice 从 smem 加载到寄存器。
  cute::copy(s2r_tiled_copy_a, tAsA(_, _, 0, ismem_read), tCrA_view(_, _, 0));
  cute::copy(s2r_tiled_copy_b, tBsB(_, _, 0, ismem_read), tCrB_view(_, _, 0));

#pragma unroll 1
  for (int k_tile = 0; k_tile < num_k_tiles; ++k_tile) {
    // num_k_steps: BK tile 内 K 方向的 MMA 迭代次数，取自 fragment 的第三 mode（K-mode）。
    //
    // 为什么只显式展开 K？MMA_M 和 MMA_N 去哪了？
    //   tCrA=(MMA, MMA_M, MMA_K), tCrB=(MMA, MMA_N, MMA_K), tCrD=(MMA, MMA_M, MMA_N)
    //   - K 方向：每个 k_step 需要从 smem 加载**不同的** A/B 数据（ldmatrix），
    //     所以 K 循环必须显式写，每次迭代做 S→R copy + cute::gemm。
    //   - M/N 方向：同一个 k_step 内，所有 M、N 位置的 MMA atom 共享
    //     同一批寄存器数据。cute::gemm 根据 tiled_mma 的类型信息（EURepeat=2×2）
    //     在编译期自动展开为覆盖全部 (MMA_M, MMA_N) 对的 mma.sync 指令序列，
    //     无需手动写 M/N 循环。——这等价于手写 MMA 中的：
    //       for (i=0; i<kValTileM; ++i)
    //         for (j=0; j<kValTileN; ++j)
    //           HMMA16816(RC[i][j][0], RC[i][j][1], ...);
    //
    // CUTLASS 推导链（mma_atom.hpp）：
    //   make_tiled_mma 将 MMA_P_T::<2>（即 kMmaPK）存入 PermutationMNK
    //   → TiledMMA::permutation_mnk<2>() 返回 kMmaPK
    //   → thrfrg_A 对 A(M,K) 按 (permutation_mnk<0>(), permutation_mnk<2>())
    //     做 logical_divide，即按 (kMmaPM,kMmaPK) tile 化 K 维：
    //        K 被分为 BK/kMmaPK 个 perm-K slice
    //     随后按 AtomShape_MNK<2>（MMA_K_atom）做 zipped_divide
    //     每个 perm-K 含 kMmaEURepeatK 个 atom-K slice
    //   → partition_A 选当前线程后，K-mode size = BK / kMmaPK = 32 / 16 = 2
    // 本配置：kMmaPK=16（MMA_K_atom=16 × kMmaEURepeatK=1），BK=32 → num_k_steps=2
    int num_k_steps = size<2>(tCrA);

#pragma unroll
    for (int k_step = 0; k_step < num_k_steps; ++k_step) {
      int k_step_next = (k_step + 1) % num_k_steps;

      if (k_step == num_k_steps - 1) {
          // 等待下一个 K tile 的 smem 数据就绪，然后切换到下一个 stage。
        cp_async_wait<kStage - 2>();
        __syncthreads();
        ismem_read = (ismem_read + 1) % kStage;
      }

      // S→R：加载下一组 A/B fragment 到寄存器。
      //
      // cute::copy 原生支持多 mode——理论上可以一次加载全部 K slice：
      //   cute::copy(s2r_tiled_copy_a, tAsA(_, _, _, ismem_read), tCrA_view);
      // 但这里刻意逐 k_step_next 加载，目的是做**寄存器双缓冲**：
      //   当前 k_step 做 MMA 计算时，ldmatrix 预加载 k_step_next 的数据，
      //   让 S→R 延迟与 MMA 计算重叠。
      // 如果一次性加载全部 K slice，S→R 和 MMA 就会彻底串行。
```

```
      cute::copy(s2r_tiled_copy_a, tAsA(_, _, k_step_next, ismem_read),
                 tCrA_view(_, _, k_step_next));
      cute::copy(s2r_tiled_copy_b, tBsB(_, _, k_step_next, ismem_read),
                 tCrB_view(_, _, k_step_next));

      if (k_step == 0) {
        // G→S: 提交下一个 K tile（整个 BK）的异步拷贝。
        if (itile_to_read < num_k_tiles) {
          cute::copy(g2s_tiled_copy_a, tAgA_copy(_, _, _, itile_to_read),
                     tAsA_copy(_, _, _, ismem_write));
          cute::copy(g2s_tiled_copy_b, tBgB_copy(_, _, _, itile_to_read),
                     tBsB_copy(_, _, _, ismem_write));
          ++itile_to_read;
          ismem_write = (ismem_write + 1) % kStage;
        }
        cp_async_fence();
      }

      // MMA: cute::gemm 内部根据 tiled_mma 的类型信息，自动展开为覆盖
      // 全部 (MMA_M, MMA_N) 对的 mma.sync 指令序列，累加到 tCrD。
      cute::gemm(tiled_mma, tCrD, tCrA(_, _, k_step), tCrB(_, _, k_step), tCrD);
    }
  }

  // Epilogue: D 寄存器 → shared memory → global memory。
  // 使用当前 A stage 作为 C scratchpad，避免为 epilogue 单独分配完整的 C shared memory。
  // 这里是“复用 storage”，不是把 A Tensor 转换成 C Tensor：
  //   sA 的逻辑 shape 是 (BM, BK, kStage)=(128,32,2)，而 sA(_,_,ismem_read)
  //   只选出其中一个 A-stage 的 storage 起点；sC 随后以完全独立的 SmemLayoutC
  //   解释这同一段地址。当前 wrapper 的固定配置下：
  //     一个 A stage: 128 x 32 = 4096 个 half；
  //     C scratchpad: 32 x 32 x 4 = 4096 个 half。
  //   两者容量刚好相等，但 logical shape 和 layout 不是同一个：A 的 layout atom
  //   是 8x32，C 的 layout atom 是 32x32，二者都含 Swizzle<3,3,3>，却不能把
  //   C 数据再按 SmemLayoutA 读取。launch wrapper 的 static_assert 用当前配置
  //   检查 C scratchpad 能放进一个 A pipe；此处只别名该单个 stage，不是整块双缓冲 sA。
  //
  // mainloop 已结束，不会再通过 sA 的 A/B load view 消费这个 pipe 的旧 A 数据。
  // 从这一行开始，对这块地址的所有访问都通过 SmemLayoutC 派生的 sC 完成：R2S
  // 用它写 C，S2G 也用它读 C。因此 C 的 swizzle/address mapping 在写和读两侧一致。
  auto sC = make_tensor(sA(_, _, ismem_read).data(), SmemLayoutC{});

  // R2S TiledCopy: 寄存器 → shared memory（使用 UniversalCopy 做类型转换）
  // R2SCopyAtomC = Copy_Atom<UniversalCopy<int>, T>; 以 32-bit 粒度搬运 T 数据
  auto r2s_tiled_copy_c = make_tiled_copy_C(R2SCopyAtomC{}, tiled_mma);
  auto r2s_thr_copy_c = r2s_tiled_copy_c.get_slice(idx);
  // tCrD 是 TiledMMA 产生的 accumulator fragment，逻辑 shape 为
  // (MMA, MMA_M, MMA_N)。它的寄存器元素已经就位，但该 layout 是为 MMA
  // accumulator 服务的，并不直接按 R2S copy atom 的 source-value 顺序表达。
  //
  // retile_S 中的 S 指“这个 TiledCopy 的 source side”，不是 shared memory。
  // 它基于 r2s_tiled_copy_c 的 reference layout 创建一个共享 tCrD.data() 的
  // zero-copy layout view，将逻辑坐标表示为 (CPY, CPY_M, CPY_N)。这一步不读取
  // 或写入寄存器/共享内存，不进行 dtype conversion，也不交换 lane 间数据；真正的
  // R2S 数据搬运发生在下面的 cute::copy(r2s_tiled_copy_c, ..., tCsC_r2s(...))。
  // 可将它与主循环的 s2r_thr_copy_a.retile_D(tCrA) 对照：二者都是为了让已有
  // fragment 的 per-thread value ordering 匹配某个 TiledCopy 的 source/destination
  // 角色，而不是另一次 memory transfer。
  // tCrD: (MMA, MMA_M, MMA_N) -> retile -> tCrC_r2s: (CPY, CPY_M, CPY_N)
  auto tCrC_r2s = r2s_thr_copy_c.retile_S(tCrD); // (CPY, CPY_M, CPY_N)
  // get_slice(idx) 已固定 CTA 内当前 logical copy thread；partition_D 再按
  // R2S TiledCopy 的 destination mapping 切分 sC。推导链：
  //   MMA_P_T = Tile<kMmaPM, kMmaPN, kMmaPK> = Tile<32, 32, 16>
```

```
//     → TiledMMA::tile_size<0>(mma) = 32, tile_size<1>(mma) = 32
//     → make_tiled_copy_C 的 tiler = (32, 32)
//     → SmemLayoutC 的逻辑 M×N = (kMmaPM, kMmaPN) = (32, 32) 恰好等于该 tiler
//     → partition_D: zipped_divide(sC, tiler=(32,32)) 后 M/N 维无 remainder
//     → 结果 shape 的 M/N remainder = _1, _1
// 最后一维 pipe=4 来自 SmemLayoutC 的第三维 kSmemLayoutCBatch，而不是
// “thread-level tensor 自动只剩 CPY”这一条规则。_1 表示该逻辑 mode 的 extent
// 是 1，并非该 mode 被删除或 C tile 没有 M/N 覆盖。
auto tCsC_r2s = r2s_thr_copy_c.partition_D(sC); // (CPY, _1, _1, pipe)

// S2G TiledCopy: shared memory → global memory (128-bit store)
// S2GCopyAtomC(Copy_Atom<UniversalCopy<cute::uint128_t>, T>) -> S2GCopyC
S2GCopyC s2g_tiled_copy_c;
auto s2g_thr_copy_c = s2g_tiled_copy_c.get_slice(idx);
// tCsC_s2g 与 tCsC_r2s 都从同一个 sC 产生，因而保留相同的 pipe=4；前者是
// S2G source mapping，后者是 R2S destination mapping。tCgC_s2g 则面向完整
// gD=(BM,BN)=(128,256)，相对于 S2G copy tiler 仍有非平凡的 M/N repetition，
// 所以它保留 CPY_M、CPY_N。这里的 CPY/CPY_M/CPY_N 都是编译期 copy-partition
// 的逻辑 mode，不应直接理解为固定的 lane 数、warp 数或物理连续维度。
auto tCsC_s2g = s2g_thr_copy_c.partition_S(sC); // (CPY, _1, _1, pipe)
auto tCgC_s2g = s2g_thr_copy_c.partition_D(gD); // (CPY, CPY_M, CPY_N)

// group_modes<B,E> 将 layout 的半开区间 [B,E) 组合成一个嵌套 mode：
//   group_modes<1,3>(Tensor(CPY, CPY_M, CPY_N))
//       -> Tensor(CPY, (CPY_M, CPY_N))
// 它只重组 Tensor 的 layout，不分配内存、不移动 data()，也不改变元素访问的物理地址。
// 第二个 mode 仍然保存原来 CPY_M/CPY_N 的层次关系，只是现在可以用一个坐标
// i+j 访问这个组合 mode；因此这里的 group_modes 更准确地说是“合并 mode”，
// 而不是把底层数据拷贝或无条件 flatten 成普通一维数组。
// 从 type/shape 结构看，结果确实是 Tensor(CPY, (CPY_M, CPY_N))；但 grouped
// mode 的 cardinality 是两个子 mode 的乘积，因此 size<1>(...) 可作为一个标量
// 范围遍历其全部 logical coordinate。于是 (_, i+j) 中的 i+j 是该 nested mode
// 的线性 logical coordinate，不是在 C++ 中对二维 tuple 做加法，更不是 raw
// pointer offset；最终物理地址仍由各自 Tensor 的 grouped layout 计算。
//
// 这里为什么要对 tCrC_r2s 和 tCgC_s2g 同时 group?
//   - tCrC_r2s: 每个线程持有的寄存器 C fragment，原始形状为 (CPY, CPY_M, CPY_N);
//   - tCgC_s2g: 该线程对应的 global-memory C 输出位置，形状与源 Tensor 对齐;
//   - 两者使用相同的 [1,3) 合并后，源/目的 Tensor 仍保持相同的坐标空间，
//     cute::copy 可以按相同的 (CPY, i+j) 坐标完成寄存器到 global 的对应搬运。
//
// tCsC_r2s/tCsC_s2g 没有 group 第 3 个 pipe mode，因为它们还需要显式保留
// shared-memory C scratchpad 的 pipeline 维度；step=size<3>(tCsC_r2s) 表示
// 每一轮可以复用的 C shared-memory pipeline 深度。外层 i 在合并后的 C 元素空间中
// 按 step 前进，内层 j 选择当前 pipeline slot：寄存器先写入 sC(_,0,0,j)，
// 再从同一个 slot 写回 global memory。
// 当前 kSmemLayoutCBatch=4，因此 step=4。代码未对 i+j 做尾部 predication，
// 这个循环依赖当前静态 layout 中 grouped extent 能被 pipe depth 整除；不应将
// “每轮正好处理 4 个 fragment”误认为任意 tile/copy 配置都自动成立的通用规则。
auto tCgC_s2gx = group_modes<1, 3>(tCgC_s2g);
auto tCrC_r2sx = group_modes<1, 3>(tCrC_r2s);

int step = size<3>(tCsC_r2s); // C scratchpad 的 pipe 数（由 kSmemLayoutCBatch=4 指定）
// 双层循环的语义（注意 step 与 CPY_N 无关）：
//   group_modes<1,3> 之后，tCrC_r2sx 的 shape 为 (CPY, (CPY_M, CPY_N))。
//   size<1>(tCrC_r2sx) = CPY_M * CPY_N，即该线程需处理的 C fragment 总数。
//   外循环以 step=4 为步长，内循环 j=0..3 每次处理 1 个 fragment，将其写入
//   第 j 号 C scratchpad pipe slot，再读出写回 global。
//
//   这里 step=4 是 scratchpad pipeline 深度，不是 CPY_N。CPY_N 的值取决于
//   TiledMMA 的 C-layout 如何将 (128,256) 的 gD tile 分配到 128 个线程上，
//   通常 ≠ 4。循环之所以成立，是因为 grouped mode 用线性坐标 i+j 遍历整个
//   CPY_M * CPY_N 的扁平空间——它不再区分 M 和 N 方向，也不要求 CPY_N == step。
```

```
  //   唯一前提是 CPY_M * CPY_N 能被 step 整除（当前静态 layout 编译期保证）。
  //
  //   第 j 号 pipe slot 在 sC 上的物理地址由 R2S partition_D / S2G partition_S
  //   各自推导；因为二者都从同一个 sC（SmemLayoutC）出发，pipe mode 保持一致，
  //   写入和读取命中同一段 shared memory。
#pragma unroll
  for (int i = 0; i < size<1>(tCrC_r2sx); i += step) {
    // 每轮处理 step 个 fragment，内层 j 选 pipe slot。
#pragma unroll
    for (int j = 0; j < step; ++j) {
      // 这两行是 R2R staging：make_tensor_like<T> 分配当前线程私有、拥有 storage
      // 的 register Tensor；普通 cute::copy 将 accumulator fragment materialize
      // 成输出元素类型 T 的临时 payload。
      //
      // cute::copy（无 copy-atom 的普通版）内部按元素做
      //   dst(i) = static_cast<T>(static_cast<SrcType>(src(i)))
      // 因此当 accumulator 与 T 类型不同时，它自动完成 dtype 转换；当二者相同
      // （如本 kernel 的 f16 accumulator + T=half），cast 退化为 no-op。
      //
      // 即使类型一致，t 还有一个不可省略的作用：layout 归一化。
      // tCrC_r2sx(_, i+j) 的 layout 是 retile_S → group_modes → slice 层层
      // 叠加的 composed layout，而 make_tensor_like<T> + cute::copy 把它
      // 物化到一个拥有简单 strides 的 owning register Tensor 中。后续
      // r2s_tiled_copy_c 的 copy_unpack 需要按 copy-atom 的 val-layout
      // 拆分 source 元素，简单 owning layout 比复杂的 composed layout 更容易
      // 被编译器推导内联。上游同源实现将其概括为"cope with accumulator and
      // output data type difference"，实际承担了类型适配和 layout 归一化两个职责。
      //
      // 这不是 retile_S 的一部分，也不是 warp shuffle：源/目的都在当前线程的
      // register 中，代码没有显式 __shfl_sync。不能仅根据这段 C++ 断言最终 SASS
      // 绝不会含 shuffle；但语义上的跨线程 regrouping 不由此 R2R copy 承担，而是
      // 由随后的 R2S -> shared memory -> S2G 路径完成。若特定 accumulator/output
      // type 与 R2S copy-atom contract 已直接兼容，可以设计直接 R2S 的变体；本例
      // 保持同源实现的通用 staging 写法。
      auto t = make_tensor_like<T>(tCrC_r2sx(_, i + j));
      cute::copy(tCrC_r2sx(_, i + j), t);
      // R2S 的 copy atom 才在此处将 thread-local payload 写入 j 号 C scratchpad
      // slot；SmemLayoutC 定义写入后的跨线程地址重组。
      //
      // cute::copy(r2s_tiled_copy_c, src, dst) 的调用链路：
      //   r2s_tiled_copy_c 是 TiledCopy，但继承自 Copy_Atom<UniversalCopy<int>,T>
      //   → 匹配 copy(Copy_Atom<...> const&, src, dst) 重载 [copy.hpp:~L190]
      //   → src(t) 与 dst(tCsC_r2s(_,0,0,j)) 都是 rank-1 → 直接 copy_atom.call
      //   → copy_atom.call 内部：若 size(src)==NumValSrc（atom 编译期 val-count），
      //     走 copy_unpack；否则递归剥 mode，最终都落到 UniversalCopy<int>::copy
      //     → 硬件层面就是逐 uint32_t 的 register-to-smem store [arch/copy.hpp:~L46]
      //
      //   这里没有任何运行时"查找表"：t 的第 i 个元素之所以对应 dst 的第 i 个
      //   shared memory offset，是因为 pre-partition 阶段已经把映射烧进 layout 了：
      //     - t 的 layout 来自 retile_S → group_modes → slice → make_tensor_like
      //       → 元素顺序已按 R2S atom 的 source-side value ordering 排好
      //     - tCsC_r2s 的 layout 来自 r2s_thr_copy_c.partition_D(sC)
      //       → TiledCopy::tidfrg_D 用 LayoutCopy_TV + Tiler_MN + ValLayoutDst
      //         计算了当前线程每个 val 在 smem 中的物理 offset
      //   两者共享同一个 copy atom 的 ValLayoutSrc/ValLayoutDst 定义，
      //   因此逐元素 copy 天然把正确的数据写到了正确的地址。
      cute::copy(r2s_tiled_copy_c, t, tCsC_r2s(_, 0, 0, j));
    }
    // 这是 CTA 范围的 rendezvous：所有线程完成当前批 R2S 写入后，S2G 才能从
    // sC 的重组布局读取完整数据。它承担了手写版中显式跨 lane 拼接所需的协作边界。
    __syncthreads();

    // S→G: 从同一个 pipe slot 读回 global memory，保持源/目的坐标一一对应。
```

```
#pragma unroll
    for (int j = 0; j < step; ++j) {
      // S2GCopyC 的 atom 为 UniversalCopy<uint128_t>: 与 R2S 的 UniversalCopy<int>
      //  (32-bit) 不同，它一次搬运 128-bit (8 个 half)，映射为一条 wide store
      //  (如 st.global.v4.f32 或 st.global.b128)。
      //
      // cute::copy(s2g_tiled_copy_c, src, dst) 的调用链路:
      //   s2g_tiled_copy_c 是 TiledCopy，继承自 Copy_Atom<UniversalCopy<uint128_t>,T>
      //   → 匹配 copy(Copy_Atom<...> const&, src, dst) 重载 [copy.hpp:~L190]
      //   → src=tCsC_s2g(_,0,0,j) 与 dst=tCgC_s2gx(_,i+j) 都是 rank-1
      //   → 直接 copy_atom.call(src, dst)
      //   → copy_atom.call 内部: size(src)==NumValSrc → copy_unpack
      //     → 逐 128-bit chunk 调用 UniversalCopy<uint128_t>::copy
      //     → 硬件层面就是 shared-memory-to-global wide store [arch/copy.hpp:~L46]
      //
      // S2G 的 pre-partition 与 R2S 对称，但 source/destination 角色互换:
      //   - tCsC_s2g 来自 s2g_thr_copy_c.partition_S(sC): TiledCopy::tidfrg_S
      //     用 LayoutCopy_TV + Tiler_MN + ValLayoutSrc 计算了当前线程每个 val
      //     在 sC (shared memory) 中的物理 offset。
      //     S2GCopyC 的 tiler = product(ThrLayout{32,4}, ValLayout{1,8})
      //                       = (32×1, 4×8) = (32, 32) — 恰好与 R2S tiler 相同，
      //     因此 tCsC_s2g 也是 (CPY, _1, _1, pipe)，_1,_1 来自 sC=(32,32,4) 无
      //     M/N remainder。
      //   - tCgC_s2gx(_, i+j) 来自 s2g_thr_copy_c.partition_D(gD) → group_modes
      //     → slice。gD=(128,256) 相对于 tiler (32,32) 有 4×8 个 tile repetition，
      //     因此 tCgC_s2g 为 (CPY, CPY_M, CPY_N)，group_modes<1,3> 合并 M/N
      //     repetition 后得到 (CPY, (CPY_M,CPY_N))，再用线性坐标 i+j 切片。
      //   两者共享同一个 copy atom 的 ValLayoutSrc/ValLayoutDst 定义，
      //   因此逐 chunk copy 天然从正确的 smem 地址读到正确的 gmem 地址。
      cute::copy(s2g_tiled_copy_c, tCsC_s2g(_, 0, 0, j), tCgC_s2gx(_, i + j));
    }
    // 所有线程都完成当前 pipe slots 的 S2G 读取后，下一轮 R2S 才能覆盖这 4 个
    // scratchpad slots，避免一部分线程仍在读取旧数据时被其他线程提前重写。
    __syncthreads();
  }

  // 与 hgemm_mma_stages_tn 的手写 epilogue 对照: 手写版必须显式处理 MMA fragment
  // 的物理分布，使用 RC0/RC1 暂存，再用 __shfl_sync 从同一 warp 的相邻 lane 收齐
  // 8 个 half，并由 lane_id % 4 == 0 的线程做 float4 (128-bit) global store。
  // CuTe 版没有把这些 lane 映射写死在 kernel 源码中: retile_S 表达 accumulator
  // fragment 与 R2S copy 的对应，R2S/sC/S2G 通过 shared-memory scratchpad 完成
  // CTA 范围的数据重组，S2GCopyC 以 uint128_t 表达 wide store。两者的共同目标都是
  // 将分散在计算线程寄存器中的 C fragment 变为适合连续 global-memory 写回的布局;
  // 区别是手写版直接编排 shuffle/store，CuTe 版将映射交给 TiledMMA/TiledCopy 类型推导。
}

// ============================================================================
// Phase 7c-2: CuTe HGEMM Launch Wrapper — 类型实例化 + Grid/Block 配置
// ============================================================================
// CuTe 的核心设计理念: "类型即配置" (Type-level configuration)
// 所有的 MMA atom、TiledCopy、SmemLayout、Swizzle 都是 **编译期类型**，
// kernel 接收这些类型的实例 (通常为空 struct)，在编译期完成全部映射推导。
//
// 以下每个类型定义后面都标注了它在 kernel body 中的对应变量/用法，形成 1:1 对照。
//
// 面试常问: 「CuTe 为什么比手写 PTX 简洁? 」
// 答: 手写需要:
//   1) 手动计算每线程的 smem 地址偏移
//   2) 手动计算 ldmatrix 的 lane 映射
//   3) 手动插入 swizzle XOR 到每个地址计算点
//   4) 手动做 epilogue 的 warp shuffle 编排
// CuTe 的 partition + copy + gemm 通过 TiledCopy/TiledMMA 的类型信息
// 在编译期自动完成上述全部推导，生成与手写等价的 PTX 指令序列。
```

```cpp
// ========================================================================
template <typename T, const int Stages = 2, const int BlockSwizzle = 0>
void launch_hgemm_mma_stages_tn_cute(T *a, T *b, T *c, int M, int N, int K) {
  using namespace cute;

  // ── Tile 尺寸（kernel 模板参数 BM/BN/BK/kStage 的值）──
  //
  // kernel 用法对照:
  //   BM=128:   local_tile(A, make_tile(Int<BM>{}, Int<BK>{}), ...) → gA=(BM,BK,num_k_tiles)
  //             local_tile(D, make_tile(Int<BM>{}, Int<BN>{}), ...) → gD=(BM,BN)
  //   BN=256:   local_tile(B, make_tile(Int<BN>{}, Int<BK>{}), ...) → gB=(BN,BK,num_k_tiles)
  //             local_tile(D, ...) → gD 的列方向
  //   BK=32:    num_k_tiles = k / BK; 每个 BK tile 内 num_k_steps = BK/kMmaPK = 2 个 MMA_K slice
  //   kStage=2: sA/sB 的第三维 = (BM,BK,kStage); cp_async_wait<kStage-2>() 流水线同步
  //   kSmemLayoutCBatch=4: step = size<3>(tCsC_r2s) → C scratchpad pipe 深度 = 4
  auto BM = Int<128>{};
  auto BN = Int<256>{};
  auto BK = Int<32>{};
  auto KStage = Int<Stages>{};
  auto kSmemLayoutCBatch = Int<4>{};

  // ── SmemLayoutA / SmemLayoutB ──
  // kernel 用法:
  // auto sA = make_tensor(make_smem_ptr(Ashm), SmemLayoutA{}); // (BM, BK, kStage)
  // auto sB = make_tensor(make_smem_ptr(Bshm), SmemLayoutB{}); // (BN, BK, kStage)
  // SmemLayout 由三部分组成: Swizzle<3,3,3> + atom layout(8×BK) + tile_to_shape 扩展到完整 tile+stage。
  //
  // 对当前 base layout shape=(8,32), stride=(32,1), T=half 的例子:
  //   - M=3: 连续 2^3=8 个 half 组成一个基本元素，即 16 bytes;
  //   - S=3: 连续 2^3=8 个基本元素组成一行，即 128 bytes;
  //   - B=3: 共有 2^3=8 行。
  // 于是一个 8×32 的逻辑 tile 正好对应 8 行 × 128B，覆盖全部 32 个 shared-memory bank。
  // Swizzle 对每个基本元素的列坐标执行 icol' = irow ^ icol,
  // 再将 (irow, icol') 和基本元素内部的 3 个低位编码回物理 offset。
  // 其位级形式为 offset' = offset ^ ((offset & (0b111 << 6)) >> 3):
  // 地址位 [6:8] XOR 到 [3:5]，地址位 [0:2] 不参与置换。
  // composition(Swizzle, base_layout) 的语义是 R(c) = Swizzle(base_layout(c)):
  // 逻辑坐标仍由 base_layout 给出，只有最终的 shared-memory offset 被重排。
  // tile_to_shape: 通过 blocked product 重复这个带 swizzle 的 block layout,
  // 使结果 shape 匹配 BM×BK×kStage; 默认按目标 shape 的 mode order 重复各维。
  using SmemLayoutAtom = decltype(composition(
      Swizzle<3, 3, 3>{},
      make_layout(make_shape(Int<8>{}, Int<BK>{}),
                  make_stride(Int<BK>{}, Int<1>{}))));
  using SmemLayoutA = decltype(tile_to_shape(
      SmemLayoutAtom{}, make_shape(Int<BM>{}, Int<BK>{}, Int<KStage>{})));
  using SmemLayoutB = decltype(tile_to_shape(
      SmemLayoutAtom{}, make_shape(Int<BN>{}, Int<BK>{}, Int<KStage>{})));

  // ── MMA (TiledMMA) ──
  // kernel 用法:
  // TiledMMA tiled_mma;
  // auto thr_mma = tiled_mma.get_slice(threadIdx.x);  // 每线程的 MMA slice
  // auto tCrA = thr_mma.partition_fragment_A(gA(_,_,0)); // (MMA, MMA_M, MMA_K)
  // auto tCrB = thr_mma.partition_fragment_B(gB(_,_,0)); // (MMA, MMA_N, MMA_K)
  // auto tCrD = thr_mma.partition_fragment_C(gD);        // (MMA, MMA_M, MMA_N)
  // cute::gemm(tiled_mma, tCrD, tCrA(_,_,k_step), tCrB(_,_,k_step), tCrD);
  // MMA 还决定 S2R/R2S copy 的 tiler: make_tiled_copy_A/B/C 都依赖 tiled_mma 的
  // tile_size<0/1>(mma) = (32,32) 来推导线程→数据映射。
  //
  // 推导链: SM80_16x8x16_F16F16F16F16_TN → MMA_Atom
  //   → make_tiled_mma(atom, EURepeat{2,2,1}, ValTile{32,32,16})
  //   → TiledMMA: 128 threads = 4 warps × (2×2 EU slices), 逻辑 MMA tile = 32×32×16
```

```
  // TN = A row-major, B col-major; 因此传给本 kernel 的 B 指针实际指向
  // B^T[N,K] 的 row-major 存储（等价于 GEMM 语义中的 B[K,N] col-major）。
  using mma_op = SM80_16x8x16_F16F16F16F16_TN;
  using mma_traits = MMA_Traits<mma_op>;
  using mma_atom = MMA_Atom<mma_traits>;
  using mma_atom_shape = mma_traits::Shape_MNK; // (Int<16>, Int<8>, Int<16>)

  static constexpr int kMmaEURepeatM = 2;
  static constexpr int kMmaEURepeatN = 2;
  static constexpr int kMmaEURepeatK = 1;
  static constexpr int kMmaPM = 1 * kMmaEURepeatM * get<0>(mma_atom_shape{}); // 32
  static constexpr int kMmaPN = 2 * kMmaEURepeatN * get<1>(mma_atom_shape{}); // 32
  static constexpr int kMmaPK = 1 * kMmaEURepeatK * get<2>(mma_atom_shape{}); // 16
  // kMmaPK=16 → kernel 中 num_k_steps = size<2>(tCrA) = BK/kMmaPK = 32/16 = 2

  using MMA_EU_RepeatT = decltype(make_layout(make_shape(
      Int<kMmaEURepeatM>{}, Int<kMmaEURepeatN>{}, Int<kMmaEURepeatK>{})));
  using MMA_P_T = Tile<Int<kMmaPM>, Int<kMmaPN>, Int<kMmaPK>>;
  using MMA = decltype(make_tiled_mma(mma_atom{}, MMA_EU_RepeatT{}, MMA_P_T{}));

  // ── G2SCopyA / G2SCopyB ──
  // kernel 用法:
  // G2SCopyA g2s_tiled_copy_a; G2SCopyB g2s_tiled_copy_b;
  // cute::copy(g2s_tiled_copy_a, tAgA_copy(_,_,_,istage), tAsA_copy(_,_,_,istage));
  // cute::copy(g2s_tiled_copy_b, tBgB_copy(_,_,_,istage), tBsB_copy(_,_,_,istage));
  // G→S: 128-bit cp.async。ThrLayout{32,4} × ValLayout{1,8}:
  //   128 个线程，每线程搬运 1×8=8 个 half = 128 bits。
  using g2s_copy_op = SM80_CP_ASYNC_CACHEGLOBAL<cute::uint128_t>;
  using g2s_copy_traits = Copy_Traits<g2s_copy_op>;
  using g2s_copy_atom = Copy_Atom<g2s_copy_traits, T>;
  using G2SCopyA = decltype(make_tiled_copy(
      g2s_copy_atom{},
      make_layout(make_shape(Int<32>{}, Int<4>{}),
                  make_stride(Int<4>{}, Int<1>{})),
      make_layout(make_shape(Int<1>{}, Int<8>{}))));
  using G2SCopyB = G2SCopyA;

  // ── S2RCopyAtomA / S2RCopyAtomB ──
  // kernel 用法:
  // auto s2r_tiled_copy_a = make_tiled_copy_A(S2RCopyAtomA{}, tiled_mma);
  // auto s2r_tiled_copy_b = make_tiled_copy_B(S2RCopyAtomB{}, tiled_mma);
  // cute::copy(s2r_tiled_copy_a, tAsA(_,_,k_step_next,ismem_read), tCrA_view(_,_,k_step_next));
  // cute::copy(s2r_tiled_copy_b, tBsB(_,_,k_step_next,ismem_read), tCrB_view(_,_,k_step_next));
  // S→R: ldmatrix (SM75_U32x4_LDSM_N)。make_tiled_copy_A/B 根据 TiledMMA 自动推导
  // 与 MMA fragment 对齐的 shared→register 映射，无需手动指定 ThrLayout/ValLayout。
  using s2r_copy_op = SM75_U32x4_LDSM_N;
  using s2r_copy_traits = Copy_Traits<s2r_copy_op>;
  using s2r_copy_atom = Copy_Atom<s2r_copy_traits, T>;
  using S2RCopyAtomA = s2r_copy_atom;
  using S2RCopyAtomB = s2r_copy_atom;

  // ── SmemLayoutC ──
  // kernel 用法:
  // auto sC = make_tensor(sA(_,_,ismem_read).data(), SmemLayoutC{});
  // 复用当前 A stage 的空间作为 C scratchpad
  // sC shape = (kMmaPM, kMmaPN, kSmemLayoutCBatch) = (32, 32, 4)
  // pipe 数 4 由 kSmemLayoutCBatch 指定 → step = size<3>(tCsC_r2s) = 4
  // 与 A/B 相同的 Swizzle<3,3,3> 规则，但 base layout 是 32×32（MMA tile 大小），
  // 再扩展为 4 个 epilogue scratchpad pipe slots。这 4 个 slots 不等同于主循环
  // 的 kStage K-tile pipeline。
  using SmemLayoutAtomC = decltype(composition(
      Swizzle<3, 3, 3>{},
      make_layout(make_shape(Int<kMmaPM>{}, Int<kMmaPN>{}),
```

```
                  make_stride(Int<kMmaPN>{}, Int<1>{})));
    using SmemLayoutC = decltype(tile_to_shape(
        SmemLayoutAtomC{},
        make_shape(Int<kMmaPM>{}, Int<kMmaPN>{}, Int<kSmemLayoutCBatch>{})));

    // ── R2SCopyAtomC ──
    // kernel 用法:
    // auto r2s_tiled_copy_c = make_tiled_copy_C(R2SCopyAtomC{}, tiled_mma);
    // cute::copy(r2s_tiled_copy_c, t, tCsC_r2s(_, 0, 0, j));
    // R→S: UniversalCopy<int> 以 32-bit 粒度将寄存器 payload 写入 C scratchpad。
    // make_tiled_copy_C 从 TiledMMA 的 tile_size<0/1>(mma)=(32,32) 推导 tiler,
    // 因此 R2S copy 的 tiler = (32,32), 与 SmemLayoutC 的 (32,32) 恰好匹配。
    using R2SCopyAtomC = Copy_Atom<UniversalCopy<int>, T>;

    // ── S2GCopyC ──
    // kernel 用法:
    // S2GCopyC s2g_tiled_copy_c;
    // cute::copy(s2g_tiled_copy_c, tCsC_s2g(_, 0, 0, j), tCgC_s2gx(_, i+j));
    // S→G: UniversalCopy<uint128_t> 以 128-bit 粒度做 shared→global wide store。
    // ThrLayout{32,4} × ValLayout{1,8} → tiler = product((32,4),(1,8)) = (32,32),
    // 与 R2S tiler 相同, 因此 partition_S(sC) 得到的 pipe 维度一致。
    using S2GCopyAtomC = Copy_Atom<UniversalCopy<cute::uint128_t>, T>;
    using S2GCopyC = decltype(make_tiled_copy(
        S2GCopyAtomC{},
        make_layout(make_shape(Int<32>{}, Int<4>{}),
                    make_stride(Int<4>{}, Int<1>{})),
        make_layout(make_shape(Int<1>{}, Int<8>{}))));

    // ── Grid/Block 配置 ──
    // kernel 用法:
    // int idx = threadIdx.x;         // 0..127
    // int ix = ((int)BlockSwizzle) * blockIdx.z * gridDim.x + blockIdx.x;
    // int iy = blockIdx.y;           // M 方向 CTA 坐标
    // if (iy * BM >= m || ix * BN >= n) return;  // CTA 级边界保护
    // 由于 kernel 没有 tile 内的逐元素 predication, 调用方需保证
    // M % BM == 0、N % BN == 0、K % BK == 0。
    // 当不启用 BlockSwizzle 时, BZ=1 退化为纯 2D grid, 行为不变。
    constexpr int kSwizzleStride = 2048;
    int BX = (N + BN - 1) / BN;
    int BY = (M + BM - 1) / BM;
    int BZ = BlockSwizzle ? (N + kSwizzleStride - 1) / kSwizzleStride : 1;
    BX = BlockSwizzle ? (BX + BZ - 1) / BZ : BX;
    dim3 block(size(MMA{}));   // 128 threads (= size(TiledMMA))
    dim3 grid(BX, BY, BZ);

    // ── Dynamic Shared Memory 大小 ──
    // kernel 用法:
    // extern __shared__ T shm_data[];
    // T *Ashm = shm_data;
    // T *Bshm = shm_data + cute::cosize(SmemLayoutA{});
    // cosize(SmemLayoutA) = BM*BK*kStage = 128*32*2 = 8192 个 T
    // cosize(SmemLayoutB) = BN*BK*kStage = 256*32*2 = 16384 个 T
    // 合计 A+B = 24576 个 T; C epilogue 复用一个 A stage (128*32 = 4096 个 T) 。
    // static_assert 保证 C scratchpad (kMmaPM*kMmaPN*kSmemLayoutCBatch = 32*32*4 = 4096)
    // 能放进一个 A stage (128*32 = 4096)。
    static constexpr int shm_size_AB =
        cute::cosize(SmemLayoutA{}) + cute::cosize(SmemLayoutB{});
    static constexpr int shm_size_C = cute::cosize(SmemLayoutC{});
    static_assert(size<0>(SmemLayoutA{}) * size<1>(SmemLayoutA{}) >= size(SmemLayoutC{}),
                  "C shared memory must fit within one A pipe");
    static constexpr int kShmSize =
        cute::max(shm_size_AB, shm_size_C) * sizeof(T);

```

```
  cudaFuncSetAttribute(
      hgemm_mma_stages_tn_cute<T, BM, BN, BK, KStage, MMA, G2SCopyA, G2SCopyB,
                               SmemLayoutA, SmemLayoutB, SmemLayoutC,
                               S2RCopyAtomA, S2RCopyAtomB, R2SCopyAtomC,
                               S2GCopyAtomC, S2GCopyC, BlockSwizzle>,
      cudaFuncAttributeMaxDynamicSharedMemorySize, kShmSize);

  hgemm_mma_stages_tn_cute<T, BM, BN, BK, KStage, MMA, G2SCopyA, G2SCopyB,
                           SmemLayoutA, SmemLayoutB, SmemLayoutC, S2RCopyAtomA,
                           S2RCopyAtomB, R2SCopyAtomC, S2GCopyAtomC, S2GCopyC,
                           BlockSwizzle>
      <<<grid, block, kShmSize>>>(a, b, c, M, N, K);
}

#endif /* NOTES_V2_ENABLE_CUTE */

#if defined(NOTES_V2_ENABLE_WGMMA) || defined(NOTES_V2_ENABLE_TMA_MMA_WS)
// CUDA 13.2 promotes these TMA operations to cuda::ptx. Keep the older
// experimental wrappers so the Hopper path remains buildable on prior toolkits.
__device__ __forceinline__ void tma_fence_proxy_async_shared_cta() {
#if CUDART_VERSION >= 13020
  cuda::ptx::fence_proxy_async(cuda::ptx::space_shared);
#else
  cuda::device::experimental::fence_proxy_async_shared_cta();
#endif
}

__device__ __forceinline__ void tma_load_2d(
    void *dst, const CUtensorMap *tensor_map, int minor_coord, int major_coord,
    cuda::barrier<cuda::thread_scope_block> &barrier) {
#if CUDART_VERSION >= 13020
  const int32_t coords[]{minor_coord, major_coord};
  auto *barrier_handle = cuda::device::barrier_native_handle(barrier);
  cuda::ptx::cp_async_bulk_tensor(cuda::ptx::space_cluster,
                                  cuda::ptx::space_global, dst, tensor_map,
                                  coords, barrier_handle);
#else
  cuda::device::experimental::cp_async_bulk_tensor_2d_global_to_shared(
      dst, tensor_map, minor_coord, major_coord, barrier);
#endif
}

__device__ __forceinline__ void tma_arrive_expect_tx(
    cuda::barrier<cuda::thread_scope_block> &barrier, uint32_t bytes) {
#if CUDART_VERSION >= 13020
  auto *barrier_handle = cuda::device::barrier_native_handle(barrier);
  [[maybe_unused]] auto token = cuda::ptx::mbarrier_arrive_expect_tx(
      cuda::ptx::sem_release, cuda::ptx::scope_cta, cuda::ptx::space_shared,
      barrier_handle, bytes);
#else
  [[maybe_unused]] auto token =
      cuda::device::barrier_arrive_tx(barrier, 1, bytes);
#endif
}
#endif

#if defined(NOTES_V2_ENABLE_WGMMA)
// ============================================================================
// Phase 7d: HGEMM WGMMA — m64n128k16 + TMA + Warp Specialization (Hopper)
// ============================================================================
// 面试要点（WGMMA vs MMA 对比）：
//   - MMA: warp 级（32 threads），同步执行
//   - WGMMA: warpgroup 级（128 threads = 4 warps），异步执行（fire-and-forget）
```

```
//   - m64n128k16: M=64, N=128, K=16 → 一次处理 64×128×16=131K 个乘加（MMA m16n8k16的 4x16=64倍）
//   - TMA (Tensor Memory Accelerator): 硬件 DMA，2D 寻址，零寄存器开销
//   - Warp Specialization: Producer 做 TMA，Consumer 做 WGMMA，通过 barrier 同步
//   - 128B swizzle: shared memory 的 128B swizzle 模式，避免 bank conflict

// ---- WGMMA 辅助函数 ----
// wgmma.commit_group / wait_group 语义（PTX ISA §9.7.15.7）：
//   - commit_group: 将此前所有未提交的 wgmma.mma_async 归入一个新的 wgmma-group
//     （per-warpgroup，故需 .sync.aligned 确保所有线程同步执行）。
//   - wait_group N: 阻塞直到最多 N 个 wgmma-group 尚未完成（pending ≤ N）。
//     N=0 → 等所有 group 完成。★ 与 cp.async.wait_group 语义一致（PTX ISA §9.7.9.25.3.3），
//     都是 "wait until only N or fewer groups are pending"。
#define WGMMA_FENCE() asm volatile("wgmma.fence.sync.aligned;\n" ::: "memory")
#define WGMMA_COMMIT_GROUP()                                                \
  asm volatile("wgmma.commit_group.sync.aligned;\n" ::: "memory")
#define WGMMA_WAIT_GROUP(n)                                                 \
  asm volatile("wgmma.wait_group.sync.aligned %0;\n" ::"n"(n) : "memory")

// SMEM_DESC_ENCODE: 将 byte 偏移编码为 WGMMA descriptor 位域中的 14-bit 值。
// 编码规则（PTX ISA §9.7.15.5.1.2.2）：
//   encoded = (x & 0x3FFFF) >> 4
// 其中 0x3FFFF = 2^18 - 1 = 262143，只保留低 18 bit。
// >>4 是因为 shared memory 地址/偏移必须 16B 对齐（2^4），低 4 bit 恒为 0，
// 可省略以扩大可表示范围。
// 解码：decoded = encoded << 4（回到原始 byte 值）。
#define SMEM_DESC_ENCODE(x) ((((uint64_t)(x)) & 0x3FFFF) >> 0x4)

// make_smem_desc: 构造 WGMMA（warpgroup MMA）的 64-bit shared memory 矩阵描述符。
// 硬件 WGMMA（如 wgmma.mma_async.m64n128k16）需要 descA/descB 两个 64-bit 描述符，
// 来定位 shared memory 中的 A/B tile 并告知 swizzle 模式。
//
// 64-bit descriptor 位域布局（参见 PTX ISA §9.7.15.5.1.2.2 & CUTLASS GmmaDescriptor）：
//   ------------------------------------------------------------------------
//   | 位域                | 范围      | 大小 | 含义
//   |---------------------|-----------|------|-----------------------------
//   | start_address       | [0, 14)   | 14   | smem 基址编码
//   | (unused)            | [14, 16)  |  2   | -
//   | leading_byte_offset | [16, 30)  | 14   | 主要维度（K）的字节步长；swizzle 下硬件未使用（assumed=1）
//   | (unused)            | [30, 32)  |  2   | -
//   | stride_byte_offset  | [32, 46)  | 14   | 跨越维度（M/N）的字节步长：K-Major 下为 8-row stripe 间距
//   | (unused)            | [46, 49)  |  3   | -
//   | base_offset         | [49, 52)  |  3   | swizzle pattern 偏移
//   | (unused)            | [52, 62)  | 10   | -
//   | layout_type         | [62, 64)  |  2   | swizzle 模式
//   ------------------------------------------------------------------------
//   layout_type: 0=None, 1=128B swizzle, 2=64B swizzle, 3=32B swizzle
//
// K-Major + 128B swizzle + half(16-bit) 的几何推导（PTX ISA §9.7.15.5.1.2）：
//   - 128B swizzle atom 在 128-bit 单位下为 8×8
//   - 归一化到 half(2B) 后：atom 覆盖 8×(128/16)=64 个 K 方向元素 × 8 个 M 方向元素
//   - 即每个 atom = 64(K) × 8(M) 个 half = 64×8×2 = 1024 bytes
//   - 这是 stride_byte_offset=1024 的来源
//
//   - leading_byte_offset 在 K-Major swizzle 下 unused（hardware assumes 1），
//     此处填 16 仅为占位，编码后为 1。
//
//   - base_offset=0（bits [49,52) 未显式置位），表示 smem ptr 已 1024B 对齐。
//     若未对齐需计算 (pattern_start_addr >> 7) & 0x7。
//
// 参考：CUTLASS GmmaDescriptor (cute/arch/mma_sm90_desc.hpp)
// 注：descA/descB 共用此函数；对 B 而言物理存储为 [BN, BK]，详见 WgmmaSMem 注释。
__device__ inline uint64_t make_smem_desc(half *ptr) {
  // __cvta_generic_to_shared: 将通用地址空间中的指针转换为 shared memory 地址
```

```
  //  (一个 32-bit 的 smem byte 偏移量，相对于当前 CTA 的 shared memory 基址)。
  uint32_t addr = static_cast<uint32_t>(__cvta_generic_to_shared(ptr));

  // 从全零开始：所有位域初始化为 0。
  uint64_t desc = 0x0000000000000000;

  // ── bits [0, 14): start_address ──
  // SMEM_DESC_ENCODE(addr) 将 addr 右移 4 位（丢弃低 4 个 0 bit），
  // 只保留 14 位。硬件解码时左移 4 位恢复完整的 16B-aligned 地址。
  // 可寻址范围：2^(14+4) = 2^18 = 256K 个 half = 512 KB 共享内存。
  desc |= SMEM_DESC_ENCODE(addr);

  // ── bits [16, 30): leading_byte_offset ──
  // 原始值 16（字节），编码后为 (16 & 0x3FFFF) >> 4 = 1。
  // << 16 将编码值放到 bits [16, 30)。
  // K-Major + 128B swizzle 下该字段硬件未使用（assumed to be 1，参见 PTX ISA §9.7.15.5.1.2.1.1），
  // 此处填 16 仅为占位，使编码值为 1 满足硬件预期。
  // 若为 MN-Major 或 INTERLEAVE 布局，LBO 有实际含义：
  //   对 INTERLEAVE：同一 8×2 brick 内第一列到第二列的字节偏移。
  //   对 MN-Major swizzle：偏移从首 (swizzle-byte-size/16) 行到下一组行。
  desc |= SMEM_DESC_ENCODE((uint64_t)16) << 16;

  // ── bits [32, 46): stride_byte_offset ──
  // 原始值 1024（字节），编码后为 (1024 & 0x3FFFF) >> 4 = 64。
  // << 32 将编码值放到 bits [32, 46)。
  // K-Major 下定义为"从首 8 行到次 8 行的字节偏移"（PTX ISA §9.7.15.5.1.2.1.2）。
  // 1024 的来源（K-Major + 128B swizzle + half）：
  //   一个 128B swizzle atom 覆盖 64(K) × 8(M) 个 half。
  //   从 1 个 8-row stripe 到下一个 stripe 需要跨越 8 × 64 × 2 = 1024 bytes。
  // 若为 MN-Major: SBO 是从首列到下一列的偏移（8 列一组）。
  desc |= SMEM_DESC_ENCODE((uint64_t)1024) << 32;

  // ── bits [62, 64): layout_type (swizzle mode) ──
  // 1llu << 62  = 二进制 01 在 bits [62, 64) = layout_type = 1。
  // 1 = 128B swizzle mode。
  // 其他取值：0=None, 2=64B swizzle, 3=32B swizzle。
  // 128B swizzle 将 smem 中连续的行按 128B 粒度进行 XOR 重映射，
  // 确保同一 warp 内各线程访问不同 bank 的同一偏移时不会产生 bank conflict。
  desc |= 1llu << 62;

  return desc;
}

// ---- WGMMA PTX 指令宏 ----
// wgmma.mma_async.sync.aligned.m64n128k16.f16.f16.f16
// m64n128k16: M=64, N=128, K=16。一次 WGMMA 计算 D[64×128] += A[64×16] × B[16×128]。
// f16.f16.f16: A=f16, B=f16, D=f16（累加器也是 f16 精度）。
// 每线程 32 个 uint32 输出：D=64×128=8192 half，warpgroup 128 线程分担，
// 每线程 64 half = 32 uint32。
// descA/descB: shared memory 描述符（由 make_smem_desc 生成，64-bit 编码）。
// ScaleD: 0=清零累加器再写入，1=与累加器中的旧值累加（K 维迭代必须用 1）。
// ScaleA/ScaleB: 1=正常符号，-1=翻转符号（此处始终用 1）。
// TransA/TransB: 0=K-major（K 维连续），1=MN-major（M/N 维连续）。本实现始终用 0。
// 参考：PTX ISA §9.7.15.4 (wgmma.mma_async)
#define WGMMA_M64N128K16_F16F16F16(d, sA, sB, ScaleD, ScaleA, ScaleB, TransA,  \
                                   TransB)                                     \
  {                                                                            \
    uint64_t desc_a = make_smem_desc(&(sA)[0]);                                \
    uint64_t desc_b = make_smem_desc(&(sB)[0]);                                \
    asm volatile(                                                              \
        "{\n"                                                                  \
        "wgmma.mma_async.sync.aligned.m64n128k16.f16.f16.f16 "                 \
        "{%0,   %1,   %2,   %3,   %4,   %5,   %6,   %7,   "                    \
```

```
        " %8,   %9,   %10,  %11,  %12,  %13,  %14,  %15,  "                    \
        " %16,  %17,  %18,  %19,  %20,  %21,  %22,  %23,  "                    \
        " %24,  %25,  %26,  %27,  %28,  %29,  %30,  %31},"                     \
        " %32,"                                                                \
        " %33,"                                                                \
        " %34, %35, %36, %37, %38;\n"                                          \
        "}\n"                                                                  \
        : "+r"((d)[0][0]), "+r"((d)[0][1]), "+r"((d)[0][2]), "+r"((d)[0][3]),  \
          "+r"((d)[1][0]), "+r"((d)[1][1]), "+r"((d)[1][2]), "+r"((d)[1][3]),  \
          "+r"((d)[2][0]), "+r"((d)[2][1]), "+r"((d)[2][2]), "+r"((d)[2][3]),  \
          "+r"((d)[3][0]), "+r"((d)[3][1]), "+r"((d)[3][2]), "+r"((d)[3][3]),  \
          "+r"((d)[4][0]), "+r"((d)[4][1]), "+r"((d)[4][2]), "+r"((d)[4][3]),  \
          "+r"((d)[5][0]), "+r"((d)[5][1]), "+r"((d)[5][2]), "+r"((d)[5][3]),  \
          "+r"((d)[6][0]), "+r"((d)[6][1]), "+r"((d)[6][2]), "+r"((d)[6][3]),  \
          "+r"((d)[7][0]), "+r"((d)[7][1]), "+r"((d)[7][2]), "+r"((d)[7][3])   \
        : "l"(desc_a), "l"(desc_b), "n"(int32_t(ScaleD)),                      \
          "n"(int32_t(ScaleA)), "n"(int32_t(ScaleB)), "n"(int32_t(TransA)),    \
          "n"(int32_t(TransB)));                                               \
  }

// ---- TMA Shared Memory Layout ----
// Multi-stage pipeline: kStages 个 stage，每个 stage 存储 A[BM×BK] + B[BK×BN]
//
// 每个 stage 包含两块 smem:
//   A tile: [BM, BK] row-major → BK×BM 个 half，地址连续
//   B tile: TMA 按 [BN, BK] 物理写入（详见下方 ★），BN×BK 个 half
//
// ★ 关键区别 — WGMMA 的 B 物理存储 vs 逻辑视图:
//
//   上层数据: 源矩阵 B^T [N, K] row-major（TN 布局，B 的列以行优先形式存储）。
//   物理存储: TMA 将 B^T 的 tile 写入 smem，物理布局为 [BN, BK] row-major
//             （BN=128 行，BK=64 列，每行 64 个连续的 half）。
//            TMA 的 smem_box_shape = (BK=64, BN=128) 定义了每个 tile 的 shape，
//            TMA 按行写入，行内 BK 个 half 连续 → 物理上就是 [BN][BK]。
//   逻辑视图: WGMMA 指令 (wgmma.mma_async.m64n128k16, PTX ISA §9.7.15.4)
//            通过 imm-trans-b=0 指定 B 为 K-major，即逻辑上按 [BK, BN] 寻址。
//            实际 [BN, BK] → 逻辑 [BK, BN] 的"转置"是通过 descB 中的 128B
//            swizzle (layout_type=1) 在硬件地址重映射层完成的，无需软件转置。
//
//   stride_byte_offset=1024 的来源（K-major + 128B swizzle + f16, PTX ISA §9.7.15.5.1.2.1.2）:
//     128B swizzle atom = 8(N) × 8(K) × 128-bit = 8 rows × 64 个 half。
//     stride_byte_offset = 从当前 8-row stripe 到下一个 8-row stripe 的字节偏移
//                        = 8 rows × (BK=64) halfs × 2 bytes = 1024。
//     这个值恰好等于物理 [BN, BK] 布局中连续 8 行的跨度 ← 进一步证实物理存储是 [BN, BK]。
//
//   与 MMA(TN) 的对比:
//     MMA (TN):  smem 存 B^T [N, K] row-major，ldmatrix 按列解出 col-major B fragment。
//     WGMMA:     smem 物理存 [BN, BK]（TMA 直写），WGMMA 通过 swizzle 硬件以 K-major [BK, BN] 逻辑视图读取。
//   简言之: swizzle 桥接了 "物理 row-major [BN, BK]" 和 "逻辑 K-major [BK, BN]" 的差异。
template <int BM, int BN, int BK, int QSIZE> struct WgmmaSMem {
  alignas(128) half A[BM * BK * QSIZE]; // A tile: row-major [BM, BK]
  // B tile 注记:
  //   - 物理存储: TMA 从 B^T[N,K] row-major 搬入，物理上是 [BN, BK] row-major（BN 行，BK 列）
  //   - 逻辑视图: WGMMA 通过 descB 的 128B swizzle（PTX ISA §9.7.15.5.1.2）将地址重映射，
  //     以 K-major [BK, BN] 逻辑视图供 wgmma.mma_async（imm-trans-b=0, PTX ISA §9.7.15.4）读取
  //   - B[BN * BK * QSIZE] 与 B[BK * BN * QSIZE] 数值等价（BN×BK = BK×BN），
  //     但写 BN*BK 更能体现物理存储形态
  alignas(128) half B[BN * BK * QSIZE];
};

// ---- WGMMA Kernel: Warp Specialization + TMA ----
// 面试重点 — Warp Specialization（Hopper 最核心的编程模型变化）:
//
```

```cpp
// 背景：传统 GEMM kernel 中，所有线程同步地"加载→计算→存回"，
// 数据搬运和计算无法重叠。Hopper 的 TMA + WGMMA 支持将工作拆分为两个
// warpgroup，让数据搬运和矩阵乘完全异步执行：
//
//   WG0 (128 threads, Producer): 仅 thread 0 提交 TMA 2D 拷贝指令，
//     将 A/B tile 从 HBM 搬到 shared memory。
//   WG1 (128 threads, Consumer): 所有 128 个线程参与 WGMMA 矩阵乘。
//
// Producer 和 Consumer 通过 cuda::barrier（CTA 级别）同步：
//   - full[stage]:  Producer 发信号表示 stage stage 的数据已就绪，可被使用
//   - empty[stage]: Consumer 发信号表示 stage stage 的使用完毕，可以被覆盖
//
// 本节重点理解：
//   1) 为什么 Producer 只需要 thread 0？TMA 是硬件 DMA 指令，一次提交
//      即可搬运整个 2D tile，无需所有线程参与。
//   2) barrier 的 arrive count = 129：128 个 Consumer 线程 + 1 个 Producer 提交线程。
//   3) 多 stage pipeline 使 Consumer 计算 stage stage 的同时，
//      Producer 可以搬运 stage (stage+1)，隐藏 HBM→SMEM 延迟。
//
// Tile Hierarchy（与 MMA m16n8k16 kernel 对比）：
//   WGMMA Atom:       m64n128k16（一次处理 64×128×16，是 MMA 的 64 倍）
//   K Tile:           BK=64，每个 K tile 包含 BK/kWgmmaK=4 个 WGMMA atom
//   M Tile:           BM=128，每个 M tile 包含 BM/kWgmmaM=2 个 WGMMA atom
//   N Tile:           BN=128，单个 kWgmmaN=128 即可覆盖，无需在 N 方向分块
//   Block Tile:       C[128,128] = A[128,64] × B[64,128]
//   Threads:          256 = 2 warpgroups × 128 threads/warpgroup
//     Producer(WG0):  128 threads（仅 thread 0 做 TMA 提交）
//     Consumer(WG1):  128 threads = 4 warps（全部参与 WGMMA 和写回）
//
// kStages=3: 3 级流水线。Consumer 滞后 Producer 最多 2 步，确保 HBM→SMEM
// 的延迟被计算完全掩盖。
//
// Grid:  ((N+127)/128/S, (M+127)/128, S), S=(N+2047)/2048, 3D block swizzle
//   - grid.z = S 个 swizzle 分区，将连续 block 打散到不同 N 区域改善 L2 命中
// Block: (256, 1, 1), 2 warpgroups
// source: LeetCUDA/kernels/hgemm/wgmma/hgemm_wgmma_fp16acc_stages_tn.cu
template <const int kWgmmaM = 64,           // WGMMA atom M dim (m64n128k16)
          const int kWgmmaN = 128,          // WGMMA atom N dim
          const int kWgmmaK = 16,           // WGMMA atom K dim
          const int BM = 128,               // block tile M, 2 × kWgmmaM
          const int BN = 128,               // block tile N, 1 × kWgmmaN
          const int BK = 64,                // block tile K, 4 × kWgmmaK
          const int kNumThreads = 256,      // 2 wargroups × 128 threads
          const int kStages = 3,            // TMA pipeline depth (full/empty barriers)
          const int kBlockSwizzle = 0>      // 1 enables 3D grid swizzle for L2 locality
__global__ void __launch_bounds__(kNumThreads)
    hgemm_wgmma_stages_tn(
        int M, int N, int K, half *C,
        const CUtensorMap *__restrict__ tensorMapA,
        const CUtensorMap *__restrict__ tensorMapB) {
  static_assert(kBlockSwizzle == 0 || kBlockSwizzle == 1, "kBlockSwizzle must be 0 or 1");
  // 注意：tensorMapA/tensorMapB 需要由 host 侧按当前 tile 布局预先创建；
  // 对 row-major [M, K] 矩阵，TMA shape 参数写的是 (K, M) 而不是 (M, K)，
  // 也就是TMA descriptor中把连续的维度写在最内层，非连续的维度写在最外层。
  // 这是 TMA descriptor 最容易背错的地方之一。notes 这里只保留 kernel 主体，
  // 不展开宿主侧 create_tensor_map 细节。

  // Block Swizzle: 在 grid x 维度做 swizzle，改善 L2 cache 局部性
  // bx = blockIdx.z * gridDim.x + blockIdx.x，将相邻 block 打散到不同 N 区域
  const int bx = ((int)kBlockSwizzle) * blockIdx.z * gridDim.x + blockIdx.x;
  const int by = blockIdx.y;
  constexpr int kConsumerThreads = kNumThreads / 2; // 128 threads = 1 warpgroup
  // kNumConsumers = (kNumThreads / 128) - 1 = 2 - 1 = 1（1 个 consumer WG）
```

```
  // 这里的 -1 是因为 2 个 warpgroup 中 1 个是 producer，剩余都是 consumer
  constexpr int kNumConsumers = (kNumThreads / kConsumerThreads) - 1; // 1 consumer WG
  // kWarpgroupM = BM / kNumConsumers: 每个 consumer warpgroup 负责的 M 行数
  // 当只有一个 consumer 时，它负责全部 BM=128 行
  constexpr int kWarpgroupM = BM / kNumConsumers; // 128

  // 边界检查: 确保当前 block 不超出 M/N 范围
  if (bx >= div_ceil(N, BN) || by >= div_ceil(M, BM))
    return;

  // ---- Shared Memory 分配 ----
  // 动态 shared memory（由 host 侧通过 kernel launch 的 smem 参数指定大小）
  // TMA + WGMMA 仅需 __align__(128)，而非 __align__(1024)。原因:
  //   make_smem_desc() 构造 WGMMA descriptor 时，start_address 字段
  //   完整编码了 __cvta_generic_to_shared 的绝对 32-bit 偏移量，
  //   硬件根据该绝对地址自动推导并补偿 swizzle phase 偏移（等价于
  //   descriptor 内置的 base_offset 机制）。因此即使 smem 基址未落到
  //   1024B 边界，WGMMA 硬件也能正确读取 TMA 写入的数据。
  // 对比 hgemm_tma_mma_ws_tn: 消费者使用 ldmatrix + 手写 swizzle
  //   函数 tma_swizzle_128B()，该函数无法感知 smem 绝对地址，硬编码
  //   了 phase=0 的假设，因此必须 __align__(1024) 来保证 phase 确实为 0。
  //   从健壮性角度看本 kernel 也应该用 __align__(1024); 这里保留 128 是
  //   为了突出两种消费者的特点与对比。
  extern __shared__ __align__(128) uint8_t smem_tma_wgmma_ws[];
  WgmmaSMem<BM, BN, BK, kStages> &s =
      *reinterpret_cast<WgmmaSMem<BM, BN, BK, kStages> *>(smem_tma_wgmma_ws);
  half *s_a = s.A;
  half *s_b = s.B;

  // ---- cuda::barrier 初始化 ----
  //
  // ★ Barrier 机制速查（arrive / wait 核心语义）:
  //
  // cuda::barrier 是一个 CTA 级同步原语，基于 **phase（奇偶交替）** 工作:
  //
  //   init(bar, N): 设置 arrive_count = N。
  //     含义: 每 phase 需要 N 个线程各调用一次 arrive()，barrier 才翻转 phase。
  //
  //   arrive(): 线程声明"我已到达此 barrier 点"。
  //     - 非阻塞，立即返回一个 token（包含当前 phase 值）。
  //     - 不关心"谁"到达，只关心"到达次数"是否达到 arrive_count。
  //
  //   wait(token): 线程阻塞，直到 barrier 的当前 phase ≠ token 中的 phase。
  //     - 即: 等待 phase 翻转。
  //     - Phase 翻转条件: (1) arrive 次数达到 arrive_count
  //                       (2) 若使用了 barrier_arrive_tx，还需 TMA 字节全部写完
  //
  //   典型用法: b.wait(b.arrive())
  //     - arrive() 注册自己到达，拿到 token（当前 phase = P）
  //     - wait(token) 阻塞直到 phase 翻转（P → P+1）
  //     - 如果自己是第 N 个到达者 → phase 立即翻转 → wait 立即返回（不阻塞）
  //     - 如果自己不是最后一个 → wait 阻塞，直到第 N 个到达者触发翻转
  //
  // ★ 本 kernel 的 Pipeline 同步协议（kStages=3, arrive_count=129）:
  //
  //   Producer (1 thread)              Consumer (128 threads)
  //   ──────────────────               ──────────────────────
  //                                     C0: arrive(empty[*]) ×128  [init: 标记所有 stage 为空]
  //   ┌─ for each k_tile: ─┐           ┌─ for each k_tile: ─┐
  //   │ P1: arrive+wait(empty[q])      │ C1: arrive+wait(full[q])
  //   │     ↓ 等 stage q 被消费完      │     ↓ 等 TMA 数据就绪
  //   │ P2: TMA(A[q]) + TMA(B[q])      │ C2: WGMMA 计算
  //   │     ↓ 异步拷贝                 │ C3: wait WGMMA 完成
```

```
  //   | P3: arrive_tx(full[q])          | C4: arrive(empty[q])
  //   |     ↑ 通知: stage q 数据就绪     |     ↑ 通知: stage q 已消费完
  //   └─────────────────────┘           └─────────────────────┘
  //
  //   关键不变式 (invariant):
  //     - Producer 不会覆盖 Consumer 正在读的 stage
  //     - Consumer 不会读 Producer 还没写完的 stage
  //     - 通过 full/empty 两个 barrier 的 phase 交替来保证
  //
  //   每个 stage 有两个 barrier:
  //      full[stage]: TMA 数据就绪信号。Producer 发 (arrive_tx), Consumer 等 (wait)。
  //     empty[stage]: Stage 空闲信号。 Consumer 发 (arrive),    Producer 等 (wait)。
  //
  //   arrive_count = 128 (consumer) + 1 (producer) = 129:
  //     每 phase, 128 个 consumer 线程 + 1 个 producer 线程都要 arrive 一次。
#pragma nv_diag_suppress static_var_with_dynamic_init
  __shared__ cuda::barrier<cuda::thread_scope_block> full[kStages];
  __shared__ cuda::barrier<cuda::thread_scope_block> empty[kStages];
#pragma nv_diag_default static_var_with_dynamic_init

  // K 方向总 tile 数。要求 K 能被 BK 整除, 否则尾 tile 被丢弃。
  const int NUM_K_TILES = div_ceil(K, BK);
  const int wg_idx = threadIdx.x / kConsumerThreads; // 0=Producer, 1=Consumer
  const int wg_tid = threadIdx.x % kConsumerThreads; // 0~127 within warpgroup

  // 初始化 barriers (仅 thread 0 执行)
  if (threadIdx.x == 0) {
    for (int i = 0; i < kStages; ++i) {
      // arrive_count = 129: 128 consumer + 1 producer
      // init 是 CUDA barrier 的 hidden friend 函数 (定义在 cuda::barrier 类体内),
      // 只能通过 ADL (Argument-Dependent Lookup) 调用。因为第一个参数 &full[i] 的类型是
      // cuda::barrier<cuda::thread_scope_block>*, 编译器通过 ADL 在 cuda 命名空间中自动找到它。
      init(&full[i], kConsumerThreads + 1);  // 128 consumer + 1 producer
      init(&empty[i], kConsumerThreads + 1); // same
    }
    // fence_proxy_async_shared_cta: 确保 barrier 在 smem 中的初始化
    // 对 async proxy (TMA/WGMMA) 可见。
    tma_fence_proxy_async_shared_cta();
  }
  __syncthreads();

  // ======================================================================
  // ★ Warp Specialization 并发模型 — 为什么只有 if/else 没有 while?
  // ======================================================================
  //
  // 256 个线程在同一个 SM 上**并发执行**。if/else 只是分工, 不是先后顺序:
  //   - WG0 (threadIdx.x 0~127):   全部走 if 分支, 做 Producer。
  //   - WG1 (threadIdx.x 128~255): 全部走 else 分支, 做 Consumer。
  //
  // SM 的 warp scheduler 会交替调度来自 WG0 和 WG1 的 warp, 两者**同时**推进:
  //   Producer: for (k_iter = 0 .. NUM_K_TILES) { P1→P2→P3 }  ← 自己的循环
  //   Consumer: for (k_iter = 0 .. NUM_K_TILES) { C1→C2→C3→C4 } ← 自己的循环
  //
  // 两个 warpgroups 各自独立迭代 K-tiles, 通过 full[] / empty[] barrier 同步:
  //   - Producer 领先 Consumer 太多 → wait(empty[q]) 阻塞, 等 Consumer 消费完
  //   - Consumer 领先 Producer 太多 → wait(full[q]) 阻塞, 等 TMA 搬运完
  //
  // 这是生产者-消费者模型的 GPU 实现: 不需要共享循环变量, barrier 的 phase
  // 交替天然保证了流水线串行化 (at most kStages-1 steps ahead)。
  // ======================================================================
  // Producer Warpgroup (WG0, threadIdx.x 0~127)
  // 职责: 提交 TMA 2D 拷贝, 将 A/B tile 从 HBM 异步搬运到 SMEM。
  // 只有 wg_tid==0 执行实际拷贝提交, 其余 127 个线程空闲。
```

```
  // ================================================================
  if (wg_idx == 0) {
    if (wg_tid == 0) {
      // stage: 当前操作的 stage 索引 (round-robin 0 -> 1 -> 2 -> 0 -> ...)
      int stage = 0;
      for (int k = 0; k < NUM_K_TILES; ++k, ++stage) {
        if (stage == kStages)
          stage = 0;

        // Step P1: 等待 stage stage 变为"空" (可被覆盖写入)
        //
        // 模式 empty[stage].wait(empty[stage].arrive()):
        //   - arrive(): Producer (1 个线程) 在 empty barrier 上注册到达,
        //     获得当前 phase 的 token。如果 Consumer 已经在 C4 步骤积累了
        //     128 次 arrive (来自上一轮迭代或 C0 初始化), 则加上这次共 129 次
        //     → phase 立即翻转。
        //   - wait(token): 阻塞直到 barrier phase ≠ token 中的 phase。
        //     由于 arrive() 是第 129 次到达, phase 在 arrive 时已翻转,
        //     wait 看到 phase 已变 → 立即返回 (首轮不阻塞)。
        //
        // 语义: Producer 说"我准备好写 stage stage 了, Consumer 用完没有? "
        //       如果 Consumer 还没用完 → phase 未翻转 → wait 阻塞等待。
        //       如果 Consumer 已用完 → phase 已翻转 → wait 立即返回。
        empty[stage].wait(empty[stage].arrive());

        // Step P2: 提交 TMA 2D 拷贝指令
        // cp_async_bulk_tensor_2d_global_to_shared:
        //   参数1(dst): smem 目标地址 (当前 stage 的 A/B tile 起始位置)
        //   参数2(tensorMap): Host 预创建的 TMA descriptor (描述源矩阵的 shape/stride/dtype)
        //   参数3/4(coords): (k_offset, m_offset) 或 (k_offset, n_offset) 的全局坐标
        //   参数5(barrier): 拷贝完成后自动 arrive 到此 barrier
        //
        // TMA 是硬件 DMA 引擎: 一次指令提交即可搬运整个 2D tile,
        // 无需线程逐元素搬运, 零寄存器开销。
        // TMA 2D 加载 A tile: coords = (k_offset, m_offset)
        tma_load_2d(&s_a[stage * BK * BM], tensorMapA, k * BK, by * BM,
                    full[stage]);

        // TMA 2D 加载 B tile: coords = (k_offset, n_offset)
        tma_load_2d(&s_b[stage * BK * BN], tensorMapB, k * BK, bx * BN,
                    full[stage]);

        // Step P3: 通知 Consumer: stage stage 的 TMA 数据已就绪
        //
        // barrier_arrive_tx(bar, arrive_count_update, byte_count):
        //   - 在 bar 上注册 1 次到达 (arrive_count_update=1)
        //   - 同时声明预期有 byte_count 字节将通过 async copy (TMA) 写入 smem
        //   - Phase 翻转条件:
        //       (a) 总 arrive 次数达到 129 (128 consumer + 1 producer)
        //       (b) 所有声明的 async 字节已写入 smem
        //     两个条件都满足后 phase 才翻转, Consumer 的 wait() 才返回。
        //   - 即: Consumer 不会在 TMA 数据完整到达前就开始读 smem。
        tma_arrive_expect_tx(full[stage], (BK * BN + BK * BM) * sizeof(half));
      }
    }
  }
  // ================================================================
  // Consumer Warpgroup (WG1, threadIdx.x 128~255)
  // 职责: 等待 TMA 数据就绪 -> 发射 WGMMA 做矩阵乘 -> 积累结果 -> 写回 C
  // 所有 128 个线程 (4 warps) 全部参与。
  // ================================================================
  else {
    // Step C0: Consumer 初始化 — 标记所有 stage 为"空" (可被 Producer 写入)
```

```
3276      //
3277      // 所有 128 个 Consumer 线程对每个 stage 的 empty barrier 调用 arrive()。
3278      // 这是 Pipeline 的"预热"步骤——没有它，Producer 的 empty[stage].wait()
3279      // 在第一轮会永远阻塞（因为 Producer 的 1 次 arrive 不足以凑够 129）。
3280      //
3281      // 注意：此时每个 empty[i] 只有 128 次 arrive，未达 129，phase 不翻转。
3282      // Producer 后续的 empty[stage].arrive() 作为第 129 次，触发 phase 翻转。
3283      for (int i = 0; i < kStages; ++i) {
3284        [[maybe_unused]] auto token = empty[i].arrive();
3285      }
3286
3287      // 累加器寄存器声明
3288      // d[kWarpgroupM / kWgmmaM][kWgmmaN / 16][4]:
3289      //   - d[0][*][*]: M 方向第 1 个 WGMMA atom（rows 0~63）
3290      //   - d[1][*][*]: M 方向第 2 个 WGMMA atom（rows 64~127）
3291      //   - d[*][g][*]: N 方向第 g 组 16 列
3292      //   - d[*][*][0..3]: 4 条 uint32 寄存器，共 8 个 half（覆盖 16×16 子块）
3293      // 每个线程总共 2 * 8 * 4 = 2 * 32 = 64 uint32 = 128 half（每次WGMMA Atom 32 uint32 输出）
3294      // 128 线程 * 128 half = 16384 half = 128 * 128 = BM*BN（刚好覆盖整个 C tile）
3295      uint32_t d[kWarpgroupM / kWgmmaM][kWgmmaN / 16][4] = {};
3296
3297      int stage = 0;
3298      // K 维外循环：沿 K tile 迭代（BK=64，每个 K tile 做 4 次 WGMMA 累加）
3299      for (int k = 0; k < NUM_K_TILES; ++k, ++stage) {
3300        if (stage == kStages)
3301          stage = 0;
3302
3303        // Step C1: 等待 TMA 数据就绪（full 信号）
3304        //
3305        // 模式 full[stage].wait(full[stage].arrive()):
3306        //   - 128 个 Consumer 线程各调用 arrive()，共 128 次到达。
3307        //   - 当前 phase 累积：128/129，phase 尚未翻转。
3308        //   - wait(token) 阻塞，直到 Producer 的 barrier_arrive_tx（Step P3）
3309        //     贡献第 129 次到达 + TMA 字节全部写完 → phase 翻转 → wait 返回。
3310        //
3311        // 语义：Consumer 说"我准备好读 stage stage 了，数据到了没有？"
3312        full[stage].wait(full[stage].arrive());
3313
3314        // Step C2: 发射 WGMMA 指令序列
3315        //
3316        // WGMMA 是异步指令（fire-and-forget），发射后立即返回，不等待计算完成。
3317        // 标准流程：FENCE → 发射 WGMMA → COMMIT → WAIT。
3318        //
3319        // WGMMA_FENCE（wgmma.fence.sync.aligned）：
3320        //   - 确保 TMA 写入 smem 的数据对 async proxy（WGMMA）可见
3321        //   - 确保累加器寄存器 d[] 已准备好接收 WGMMA 输出
3322        //   - 本质是 proxy 间的内存序（memory ordering），不是传统 __syncthreads
3323        //
3324        // 每个 WGMMA_M64N128K16_F16F16F16 指令：
3325        //   D[64×128] += A[64×16] × B[16×128]
3326        //   A/B 均为 K-major（imm-trans=0），由 descA/descB 描述 smem 中的布局。
3327        //   ScaleD=1: 累加。K 维有 BK/kWgmmaK=4 次迭代，每次都用 ScaleD=1。
3328        WGMMA_FENCE();
3329
3330        // M 维迭代：BM/kWgmmaM = 128/64 = 2 个 WGMMA atom
3331        // 每个 atom 处理 64 行 M 方向数据。
3332  #pragma unroll
3333        for (int m = 0; m < kWarpgroupM / kWgmmaM; ++m) {
3334          // wgmma_sA 指向当前 stage 中 A tile 的第 m 个 64*BK 子块
3335          // s_a 布局：[kStages][BM][BK] -> stage * BK*BM + BK * (m*64)
3336          half *wgmma_sA = s_a + stage * BK * BM + BK * m * kWgmmaM;
3337
3338          // K 维迭代：BK/kWgmmaK = 64/16 = 4 次 WGMMA（累加）
```

```
        // 每次处理 K=16 维的矩阵乘，4 次累加后覆盖完整的 BK=64。
#pragma unroll
        for (int k_step = 0; k_step < BK / kWgmmaK; ++k_step) {
          // 第 k_step 次 WGMMA:
          //   A: wgmma_sA + k_step * kWgmmaK（A 的 K 维起始位置）
          //   B: s_b + stage * BK * BN + k_step * kWgmmaK（B 的 K 维起始位置）
          //   注意 B 的 smem 布局是 [BK, BN] row-major，K-major 读取时从
          //   第 k_it*16 行开始取 16 行，每行 BN=128 列。
          //   ScaleD=1: 累加（K 维迭代需要累积结果）
          WGMMA_M64N128K16_F16F16F16(d[m], wgmma_sA + k_step * kWgmmaK,
                                     s_b + stage * BK * BN + k_step * kWgmmaK,
                                     1, // ScaleD=1: accumulate（不清零）
                                     1, 1, 0, 0);
        }
      }

      // Step C3: 提交并等待 WGMMA 完成
      //
      // WGMMA_COMMIT_GROUP (wgmma.commit_group.sync.aligned):
      //   将自上一次 COMMIT 以来发射的所有 WGMMA 归为一组，提交到 async proxy 执行。
      //   类比 cp.async.commit_group，但 scope 是 warpgroup 而非 per-thread。
      //
      // WGMMA_WAIT_GROUP(0) (wgmma.wait_group.sync.aligned 0):
      //   阻塞直到最多 N 个 group 尚未完成（pending ≤ N）。N=0 即等待所有 group。
      //   ★ 语义与 cp.async.wait_group 完全一致（PTX ISA §9.7.15.7.3 vs §9.7.9.25.3.3）：
      //     两者都是 "wait until only N or fewer of the most recent groups are pending"。
      //   此处用 0 是因为 Consumer 在本次迭代中只 commit 了 1 个 group，
      //   必须等它全部完成才能进入下一步（读下一 stage 的数据）。
      WGMMA_COMMIT_GROUP();
      WGMMA_WAIT_GROUP(0);

      // Step C4: 释放 stage stage — 通知 Producer 可以覆盖写入
      //
      // 128 个 Consumer 线程在 empty[stage] 上 arrive()，为 **下一 phase** 积累到达次数。
      // 这 128 次 arrive 不会立即翻转 phase（还需 Producer 在下一轮的 1 次 arrive）。
      //
      // 语义：Consumer 说"stage stage 我已经读完了，你可以放心覆盖了。"
      [[maybe_unused]] auto token = empty[stage].arrive();
    }

    // ============================================================
    // Epilogue: 将寄存器中的累加结果写回 global memory C
    // ============================================================
    //
    // WGMMA m64n128k16.f16.f16.f16 的输出寄存器映射：
    //
    // 对 warpgroup 内每个线程，输出 64 个 half = 32 个 uint32。
    // 这些 half 分布在 d[2][8][4] 数组中：
    //   d[m_it][g][0]: (row, col) 和 (row, col+2) 位置的 2 个 half
    //   d[m_it][g][1]: (row+8, col) 和 (row+8, col+2) 位置的 2 个 half
    //   d[m_it][g][2]: (row, col+8) 和 (row, col+10) 位置的 2 个 half
    //   d[m_it][g][3]: (row+8, col+8) 和 (row+8, col+10) 位置的 2 个 half
    //
    // 其中：
    //   row = warp * 16 + lane / 4    (0~63, 4 warps * 16 rows)
    //   col = g * 16 + 2 * (lane % 4)  (0~126, step 2, 8 g's * 16 cols)
    //
    // 每个线程覆盖一个 16x16 子块中的 16 个 half：
    //   +-------+-------+
    //   | reg[0] | reg[2] |  <- rows [row, row+8)
    //   |(col,+2)|(col+8)|
    //   +-------+-------+
    //   | reg[1] | reg[3] |  <- rows [row+8, row+16)
```

```
    //   |(col,+2)|(col+8)|
    //   +-------+-------+
    //   每个 reg 包含 2 个连续 half（col 和 col+2），用 uint32 存储。
    //
    // 三维遍历:
    //   m_it=0: rows 0~63,  m_it=1: rows 64~127
    //   g=0..7: 每个覆盖 16 列，8*16=128 列 ok
    // 每个线程写 4 uint32 * 2 half/uint32 * 8 g * 2 m_it = 128 half。
    // 128 线程 * 128 half = 16384 half = 128 * 128 ok

    const int lane = wg_tid % 32;
    const int warp = wg_tid / 32;
    // row: 当前线程在当前 WGMMA atom 中负责的起始行。
    // warp 0~3 各负责 16 行: rows [warp*16, warp*16+15]。
    // lane/4: 每 4 个连续 lane 负责同一行（因为 WGMMA fragment 中每行有 4 个 uint32，
    //         分别覆盖列对 {c0,c1}、{c2,c3}、{c8,c9}、{c10,c11}，由 lane%4 区分）。
    //         因此 lane/4 给出该行在 16-row block 内的行号（0~7 对应 rows 0~7，
    //         同一 warp 中 lane/4==0 的有 lane 0,1,2,3，都负责 row 0）。
    const int row = warp * 16 + lane / 4;
    // block_C: 当前 C tile 在 global memory 中的起始地址
    // by*BM 是 M 方向偏移，bx*BN 是 N 方向偏移
    half *block_C = C + by * BM * N + bx * BN;

    // 2 * (8 * 4) = 2 * 32 = 64 uint32 = 128 half
    // NOTE: 这里可以考虑通过 R->S, S->G 的方式做一次 128 bits写回。
#pragma unroll
    for (int m = 0; m < kWarpgroupM / kWgmmaM; ++m) {
      int yo = m * kWgmmaM; // M 方向行偏移 (0 或 64)
#pragma unroll
      for (int g = 0; g < kWgmmaN / 16; ++g) {
        int col = g * 16 + 2 * (lane % 4); // N 方向列偏移 (0,2,4,...,14 then 16,18,...)
        // 一次 uint32 store 写入 2 个 half（连续列 col 和 col+2）
        // 左上象限: (row, col)
        *reinterpret_cast<uint32_t *>(&block_C[(row + yo) * N + col]) = d[m][g][0];
        // 左下象限: (row+8, col)
        *reinterpret_cast<uint32_t *>(&block_C[(row + yo + 8) * N + col]) = d[m][g][1];
        // 右上象限: (row, col+8)
        *reinterpret_cast<uint32_t *>(&block_C[(row + yo) * N + col + 8]) = d[m][g][2];
        // 右下象限: (row+8, col+8)
        *reinterpret_cast<uint32_t *>(&block_C[(row + yo + 8) * N + col + 8]) = d[m][g][3];
      }
    }
  }
}

#endif /* NOTES_V2_ENABLE_WGMMA */

#if defined(NOTES_V2_ENABLE_TMA_MMA_WS)
// SM120 不支持 WGMMA，但支持相同的 TMA 生产者协议和 warp 级 mma.sync。
// 保持 128B TMA swizzle 并显式声明物理布局供 ldmatrix 消费者使用。
template <int BM, int BN, int BK, int QSIZE> struct TmaMmaWSSMem {
  static_assert(BK == 64, "The 128B swizzle helper below is specialized for BK=64");
  half A[BM * BK * QSIZE];
  half B[BN * BK * QSIZE];
};

// tma_swizzle_128B: 计算 128B TMA swizzle 下 smem 中 (row, k) 的物理 K 偏移。
//
// TMA descriptor 使用 CU_TENSOR_MAP_SWIZZLE_128B 时，硬件会将写入 smem 的
// 数据按 128B 粒度做 XOR 重映射以消除 bank conflict。消费者在 ldmatrix 前必须
// 对地址施加相同的 swizzle 变换，否则读到的数据与 TMA 写入的物理位置不一致。
//
// 位级公式与 CuTe 对照:
```

```
//   swizzle_k = (((k >> 3) ^ (row & 7)) << 3) | (k & 7)
//
//   等价于 CuTe Swizzle<3,4,3>:
//     M=3 → 基本元素宽 2^3=8 个 half（16 bytes）
//     S=4 → 每行 2^4=16 个基本元素（32 个 half）...不，BK=64...
//
// 实际让我们从函数出发：
//   - k & 7：低 3 位保持不变。即 8 个 half（16 bytes）为一组，组内顺序不变。
//   - k >> 3：K 被划分为 8 个 chunk（BK=64，每 chunk 8 个 half）。
//   - row & 7：取行号的低 3 位（swizzle 周期为 8 行）。
//   - XOR：将 chunk 索引与行号的低 3 位做异或。
//
// ★ swizzle 映射图（BK=64，每 row 8 chunk à 8 half；周期 = 8 rows × 512 half = 1024B）：
//
//   下图表示同一 chunk 索引在各行被映射到的物理 chunk 位置。例如 chunk 0 在
//   row 0 位于物理 chunk 0，在 row 1 被 XOR 到物理 chunk 1，以此类推。
//
//   chunk idx:     0     1     2     3     4     5     6     7
//                ─────────────────────────────────────────────
//   row 0 (0^c):   0     1     2     3     4     5     6     7   ← 恒等
//   row 1 (1^c):   1     0     3     2     5     4     7     6   ← 相邻对交换
//   row 2 (2^c):   2     3     0     1     6     7     4     5
//   row 3 (3^c):   3     2     1     0     7     6     5     4
//   row 4 (4^c):   4     5     6     7     0     1     2     3
//   row 5 (5^c):   5     4     7     6     1     0     3     2
//   row 6 (6^c):   6     7     4     5     2     3     0     1
//   row 7 (7^c):   7     6     5     4     3     2     1     0
//
//   ── 8 行一周期 ──
//   row 8 (0^c): 同 row 0 映射；row 9 同 row 1；依此类推。
//
//   例子——ldmatrix 读取 row 1、逻辑 k=0 处的数据：
//     swizzle_k = (((0 >> 3) ^ (1 & 7)) << 3) | (0 & 7) = ((0 ^ 1) << 3) | 0 = 8
//     即 row 1 的 chunk 0 被 TMA 写到了物理 K=8 的位置，ldmatrix 地址需从 K=8 读。
//
//   例子——ldmatrix 读取 row 3、逻辑 k=16 处的数据：
//     swizzle_k = (((16 >> 3) ^ (3 & 7)) << 3) | (16 & 7)
//               = ((2 ^ 3) << 3) | 0 = (1 << 3) | 0 = 8
//     即 row 3 的 chunk 2 被映射到物理 chunk 1（K=8）。
//
// 关键结论：TMA 和 ldmatrix 两侧必须使用相同的 swizzle；任何一侧不用或用了
// 不同的 pattern，整个 tile 的输出都会被破坏。128B = 128 BYTES = 64 half
// = 8 chunk × 8 half/chunk = 8 rows × 512 half/row。
__device__ __forceinline__ int tma_swizzle_128B(int row, int k) {
  return (((k >> 3) ^ (row & 7)) << 3) | (k & 7);
}

// 消费者 MMA 层级映射参考 hgemm_mma_stages_tn_swizzle。与那个
// 八 warp kernel 不同，本 warp-specialized 消费者仅拥有四个 warp。
template <const int kMmaM = 16,             // MMA atom M dim (m16n8k16)
          const int kMmaN = 8,              // MMA atom N dim
          const int kMmaK = 16,             // MMA atom K dim
          const int kMmaTileM = 2,          // consumer warps along M, warp tile M = 32
          const int kMmaTileN = 2,          // consumer warps along N, warp tile N = 16
          const int kValTileM = 4,          // value-repeat along M, BM = 16*2*4 = 128
          const int kValTileN = 8,          // value-repeat along N, BN = 8*2*8 = 128
          const int kValTileK = 4,          // MMA_K slices per BK tile, BK = 16*4 = 64
          const int kStages = 2,            // TMA full/empty pipeline depth
          const int kNumThreads = 256,      // 128 producer + 128 consumer threads
          const int kBlockSwizzle = 0>      // 1 enables 3D grid swizzle for L2 locality
__global__ void __launch_bounds__(kNumThreads)
    hgemm_tma_mma_ws_tn(
        int M, int N, int K, half *C,
```

```
3528        const CUtensorMap *__restrict__ tensorMapA,
3529        const CUtensorMap *__restrict__ tensorMapB) {
3530
3531  constexpr int BM = kMmaM * kMmaTileM * kValTileM;
3532  constexpr int BN = kMmaN * kMmaTileN * kValTileN;
3533  constexpr int BK = kMmaK * kValTileK;
3534  static_assert(kMmaM == 16 && kMmaN == 8 && kMmaK == 16, "This kernel uses mma.sync.m16n8k16");
3535  static_assert(kMmaTileM * kMmaTileN == 4, "The consumer warpgroup has exactly four warps");
3536  static_assert(BM == 128 && BN == 128 && BK == 64, "TMA desc and 128B swizzle require 128x128x64");
3537  static_assert(kNumThreads == 256, "Use one producer and one consumer warpgroup");
3538  static_assert(kBlockSwizzle == 0 || kBlockSwizzle == 1, "kBlockSwizzle must be 0 or 1");
3539
3540  const int bx = kBlockSwizzle * blockIdx.z * gridDim.x + blockIdx.x;
3541  const int by = blockIdx.y;
3542  constexpr int kConsumerThreads = kNumThreads / 2;
3543  static_assert(kConsumerThreads == kMmaTileM * kMmaTileN * kWarpSize,
3544                "Consumer threads must cover the MMA warp grid");
3545
3546  if (bx >= div_ceil(N, BN) || by >= div_ceil(M, BM))
3547    return;
3548
3549  // TMA CU_TENSOR_MAP_SWIZZLE_128B 要求 smem 基地址 1024 字节对齐，
3550  // 以确保硬件 swizzle phase 从零开始。消费者 tma_swizzle_128B()
3551  // 假设零 phase；其他对齐（如 128）会导致 phase 偏移，产生不匹配的
3552  // 物理地址，破坏整个输出。
3553  //
3554  // 为什么 hgemm_wgmma_stages_tn 只需要 __align__(128)，而本 kernel
3555  // 必须 __align__(1024)？核心区别在于消费者如何感知 swizzle phase：
3556  //
3557  // WGMMA 消费者：通过 make_smem_desc() 构造 64-bit WGMMA descriptor，
3558  //   其中 bits [49,52) 的 base_offset 字段编码了 smem 基址相对于
3559  //   1024B 边界的偏移量（(addr >> 7) & 0x7）。硬件在读取 smem 时会
3560  //   用这个 base_offset 自动补偿 swizzle phase，因此 WGMMA 路径不
3561  //   需要 1024 字节对齐也能得到正确数据。
3562  //
3563  // 本 kernel 消费者：使用 ldmatrix + 手写 tma_swizzle_128B() 来
3564  //   计算 smem 物理地址。这个函数是纯软件公式，没有任何 base_offset
3565  //   补偿机制。它假设 smem 基址恰好落在 1024B 边界上（phase=0）。
3566  //   如果基址只满足 128B 对齐，TMA 硬件会以非零 phase 写入数据，
3567  //   而 ldmatrix 以零 phase 读取 → 物理地址彻底错位 → 全输出错误。
3568  //
3569  // 简言之：WGMMA 用硬件 descriptor base_offset 自动解决 phase 偏移；
3570  // ldmatrix 路径没有等价机制，必须靠 1024B 对齐来保证 phase=0。
3571  extern __shared__ __align__(1024) uint8_t smem_tma_mma_ws[];
3572  TmaMmaWSSMem<BM, BN, BK, kStages> &s =
3573      *reinterpret_cast<TmaMmaWSSMem<BM, BN, BK, kStages> *>(smem_tma_mma_ws);
3574  half *s_a = s.A;
3575  half *s_b = s.B;
3576
3577#pragma nv_diag_suppress static_var_with_dynamic_init
3578  __shared__ cuda::barrier<cuda::thread_scope_block> full[kStages];
3579  __shared__ cuda::barrier<cuda::thread_scope_block> empty[kStages];
3580#pragma nv_diag_default static_var_with_dynamic_init
3581
3582  const int NUM_K_TILES = div_ceil(K, BK);
3583  const int wg_idx = threadIdx.x / kConsumerThreads;
3584  const int wg_tid = threadIdx.x % kConsumerThreads;
3585
3586  if (threadIdx.x == 0) {
3587    for (int stage = 0; stage < kStages; ++stage) {
3588      init(&full[stage], kConsumerThreads + 1);
3589      init(&empty[stage], kConsumerThreads + 1);
3590    }
```

```
    tma_fence_proxy_async_shared_cta();
  }
  __syncthreads();

  if (wg_idx == 0) {
    // 生产者 Warpgroup（wg_idx==0）
    if (wg_tid == 0) {
      int stage = 0;
      for (int k = 0; k < NUM_K_TILES; ++k, ++stage) {
        if (stage == kStages)
          stage = 0;

        empty[stage].wait(empty[stage].arrive());

        tma_load_2d(&s_a[stage * BM * BK], tensorMapA, k * BK, by * BM,
                    full[stage]);
        tma_load_2d(&s_b[stage * BN * BK], tensorMapB, k * BK, bx * BN,
                    full[stage]);
        tma_arrive_expect_tx(full[stage], (BM * BK + BN * BK) * sizeof(half));
      }
    }
  } else {
    // 消费者 Warpgroup（wg_idx==1）
    // 初始化：标记所有 stage 为"空"（可被 Producer 写入）
    for (int stage = 0; stage < kStages; ++stage) {
      [[maybe_unused]] auto token = empty[stage].arrive();
    }

    const int warp_id = wg_tid / kWarpSize;
    const int lane_id = wg_tid % kWarpSize;
    // WG1 内形成 2x2 warp grid，warp_m / warp_n 分别表示 warp 在 M/N 方向的索引。
    const int warp_m = warp_id % kMmaTileM; // kMmaTileM = 2, {0,1}
    const int warp_n = warp_id / kMmaTileM; // kMmaTileN = 2, {0,1}
    uint32_t RA[kValTileM][4];
    uint32_t RB[kValTileN][2];
    uint32_t RC[kValTileM][kValTileN][2] = {0};

    const uint32_t smem_a_base_ptr = __cvta_generic_to_shared(s_a);
    const uint32_t smem_b_base_ptr = __cvta_generic_to_shared(s_b);
    int stage = 0;
    for (int k = 0; k < NUM_K_TILES; ++k, ++stage) {
      if (stage == kStages)
        stage = 0;

      full[stage].wait(full[stage].arrive());
      tma_fence_proxy_async_shared_cta();

      // 与 cp.async pipeline（hgemm_mma_stages_tn 等）不同，这里 ldmatrix +
      // mma.sync 之前/后 不需要 __syncthreads，原因在于 TMA + async proxy 的
      // 同步机制与 warp specialization 的线程分工：
      //
      // 1. 生产者-消费者解耦：TMA 拷贝由 producer（wg_idx==0 的单个线程）
      //    通过 tma_load_2d + tma_arrive_expect_tx 提交，而非所有 256 个线程
      //    各自做 cp.async。cp.async 需要 __syncthreads 确保所有线程的异步
      //    拷贝都完成后再进入计算；而 TMA 路径用 cuda::barrier 的 full/empty
      //    协议完成 CTA 级同步——full[stage].wait() 返回时，producer 已通过
      //    expect_tx 声明了完整的传输字节数，且硬件保证这些字节已全部写入 smem。
      //
      // 2. async proxy fence 替代 __syncthreads：fence_proxy_async(space_shared)
      //    在 async proxy（TMA 写入通道）和后续 smem load（ldmatrix 读取通道）
      //    之间建立内存序。它确保 fence 之前的所有 async proxy 写操作（TMA）
      //    对 fence 之后的所有 smem 读操作（ldmatrix）可见。这是一个轻量级
      //    proxy 间 ordering，不需要阻塞线程，也不会让 warp 等待其他 warp。
```

```
      //
      // 3. 消费者内部 warp 间无依赖：WG1 内的 4 个 warp（2×2 grid）各自独立
      //    读取 smem 中互不重叠的 A/B 区域（warp_m 和 warp_n 分别索引不同的
      //    行区间）。它们之间不需要读写共享数据，因此不需要 __syncthreads 来
      //    对齐 warp 间的执行进度。mma.sync 本身是 warp 级同步指令，保证同一
      //    warp 内 32 个线程的 ldmatrix 和 mma 操作正确协作。

      // 注意：这里已经没有NUM_K_TILES循环了，因为K loop load已经被WG0生产者处理了
#pragma unroll
      for (int k_step = 0; k_step < kValTileK; ++k_step) {

#pragma unroll
        for (int i = 0; i < kValTileM; ++i) { // kValTileM = 4
          // kMmaM * kValTileM = 16*4 = 64，每个消费者 warp 在 M 方向覆盖 64 行。
          const int warp_smem_a_m = warp_m * (kMmaM * kValTileM) + i * kMmaM;
          const int lane_smem_a_m = warp_smem_a_m + lane_id % 16;
          // 与 hgemm_mma_stages_tn_swizzle 不同：k_step * kMmaK 必须放在
          // lane_smem_a_k 中传给 tma_swizzle_128B()，而非像 swizzle kernel
          // 那样作为 smem_k_offset 加在 swizzle 外部。原因：
          //   tma_swizzle_128B 是 128B TMA swizzle，作用于完整 BK=64，
          //   swizzle 周期覆盖全部 64 列，k_step=0 和 k_step=1 的 chunk 会
          //   被 XOR 交叉混合 → k_step 偏移必须在 swizzle 内部参与 chunk 计算。
          //   swizzle_A<kMmaK> 作用于 kMmaK=16，每个 kMmaK slice 独立
          //   swizzle，slice 之间互不跨越 → k_step * kMmaK 可以加在外部。
          const int lane_smem_a_k = (k_step * kMmaK) + (lane_id / 16) * 8;
          const uint32_t lane_smem_a_ptr = smem_a_base_ptr +
              (stage * BM * BK + lane_smem_a_m * BK +
              tma_swizzle_128B(lane_smem_a_m, lane_smem_a_k)) *
                  sizeof(half);
          LDMATRIX_X4(RA[i][0], RA[i][1], RA[i][2], RA[i][3], lane_smem_a_ptr);
        }

#pragma unroll
        for (int j = 0; j < kValTileN; ++j) { // kValTileN = 8
          // kMmaN * kValTileN = 8*8 = 64，每个消费者 warp 在 B^T 的 N 方向覆盖 64 行。
          const int warp_smem_b_n = warp_n * (kMmaN * kValTileN) + j * kMmaN;
          const int lane_smem_b_n = warp_smem_b_n + lane_id % 8;
          const int lane_smem_b_k = (k_step * kMmaK) + ((lane_id / 8) % 2) * 8;
          const uint32_t lane_smem_b_ptr = smem_b_base_ptr +
              (stage * BN * BK + lane_smem_b_n * BK +
              tma_swizzle_128B(lane_smem_b_n, lane_smem_b_k)) *
                  sizeof(half);
          LDMATRIX_X2(RB[j][0], RB[j][1], lane_smem_b_ptr);
        }

#pragma unroll
        for (int i = 0; i < kValTileM; ++i) {
#pragma unroll
          for (int j = 0; j < kValTileN; ++j) {
            HMMA16816(RC[i][j][0], RC[i][j][1],
                      RA[i][0], RA[i][1], RA[i][2], RA[i][3],
                      RB[j][0], RB[j][1],
                      RC[i][j][0], RC[i][j][1]);
          }
        }
      }
      [[maybe_unused]] auto token = empty[stage].arrive();
    }

    // Epilogue：复用 RA[0] 和 RA[1]（已在寄存器中）作为 shuffle 缓冲，
    // 与 hgemm_mma_stages_tn_swizzle 的 epilogue 模式一致。四个相邻
    // lane 各持有两个 uint32 fragment；shuffle 汇聚为每行一个 float4，
    // 由 lane 0 发出对齐的 128-bit store。
```

```cpp
#pragma unroll
    for (int i = 0; i < kValTileM; ++i) {
      const int store_warp_smem_c_m = warp_m * (kMmaM * kValTileM) + i * kMmaM;
      const int store_lane_gmem_c_m = by * BM + store_warp_smem_c_m + lane_id / 4;
#pragma unroll
      for (int j = 0; j < kValTileN; ++j) {
        const int store_warp_smem_c_n = warp_n * (kMmaN * kValTileN) + j * kMmaN;
        RA[0][0] = RC[i][j][0];
        RA[1][0] = RC[i][j][1];
        RA[0][1] = __shfl_sync(0xffffffff, RC[i][j][0], lane_id + 1);
        RA[0][2] = __shfl_sync(0xffffffff, RC[i][j][0], lane_id + 2);
        RA[0][3] = __shfl_sync(0xffffffff, RC[i][j][0], lane_id + 3);
        RA[1][1] = __shfl_sync(0xffffffff, RC[i][j][1], lane_id + 1);
        RA[1][2] = __shfl_sync(0xffffffff, RC[i][j][1], lane_id + 2);
        RA[1][3] = __shfl_sync(0xffffffff, RC[i][j][1], lane_id + 3);
        if (lane_id % 4 == 0) {
          const int store_lane_gmem_c_n = bx * BN + store_warp_smem_c_n;
          const int store_gmem_c_addr_0 =
              store_lane_gmem_c_m * N + store_lane_gmem_c_n;
          const int store_gmem_c_addr_1 =
              (store_lane_gmem_c_m + 8) * N + store_lane_gmem_c_n;
          *reinterpret_cast<float4 *>(&C[store_gmem_c_addr_0]) =
              *reinterpret_cast<float4 *>(&RA[0][0]);
          *reinterpret_cast<float4 *>(&C[store_gmem_c_addr_1]) =
              *reinterpret_cast<float4 *>(&RA[1][0]);
        }
      }
    }
  }
}
#endif /* NOTES_V2_ENABLE_TMA_MMA_WS */

#if defined(NOTES_V2_ENABLE_WGMMA) || defined(NOTES_V2_ENABLE_TMA_MMA_WS)
// ---- Host-side TMA Tensor Map helpers (WGMMA test) ----
// 面试要点（TMA descriptor 创建 — cuTensorMapEncodeTiled）：
//   - TMA descriptor 描述 global memory 中矩阵的 shape/stride/dtype，
//     以及硬件搬运的 box (tile) 大小和 smem swizzle 模式
//   - 关键易错点：TMA shape 参数写的是 (W,H) 而不是 (H,W)！
//     对 row-major [M,K] 矩阵，TMA shape = (K,M)，minor=K，major=M
//   - cuTensorMapEncodeTiled 是 CUDA Driver API，需 #include <cuda.h> 并链接 -lcuda
//   - smem_box 维度顺序也是 (minor, major) = (BK, BM) 或 (BK, BN)
//   - Swizzle 模式通常选 CU_TENSOR_MAP_SWIZZLE_128B 配合 WGMMA 的 128B swizzle
//
// ★ gmem_prob_stride + 1 的含义：
//   语法层面 — 数组名退化 + 指针算术：
//     gmem_prob_stride 类型为 uint64_t[5]，在表达式中退化为 uint64_t*
//     （指向首元素 gmem_prob_stride[0]）。+ 1 偏移 1 个 uint64_t，指向
//     gmem_prob_stride[1]，等价于 &gmem_prob_stride[1]。
//   语义层面 — 跳过隐式的最内维 stride：
//     cuTensorMapEncodeTiled 的 globalStrides 参数只接收 tensorRank-1 个
//     stride 值。最内维（fastest-changing dimension）的 stride 是隐式的，
//    固定等于 sizeof(element_type)，不需要显式传入。
//     对于 tensorRank=2 的 FP16 矩阵：
//       dim 0（内维，K）：stride = sizeof(half)    ← 隐式，API 不需要
//       dim 1（外维，M）：stride = sizeof(half)*K   ← 需要传给 API
//     gmem_prob_stride 数组的布局是内维在前：
//       [0] = sizeof(half)                                ← 内维，不传
//       [1] = sizeof(half) * BlockMinorSize * blocks_width ← 外维，传给 API
//   所以 + 1 的作用是跳过 [0]，让 API 从 [1] 开始读取，正好得到外维 stride。
//
// ★ smem_box_stride 为什么不需要 +1?
//   与 globalStrides 不同，smemBoxStrides 参数接收完整的 tensorRank 个 stride，
//   最内维 stride 也必须显式传入（不隐式）。
```

```
//   smem_box_stride[5] = {1, 1, 1, 1, 1} 表示 smem tile 中所有维度元素
//   都是连续存放的，维度间没有 padding/stride。
//   对于 tensorRank=2: strides[0]=1（内维 stride），strides[1]=1（外维 stride），
//   即 smem 中第 (i,j) 元素的偏移 = i*1 + j*1 = i+j（元素连续排列）。
//   所以这里直接传 smem_box_stride（即 &smem_box_stride[0]），API 会读到全部
//   两个 stride 值，不需要跳过任何一个。
//
// 参考: CUDA Programming Guide §TMA, PTX ISA §9.7.15.5, CUTLASS GmmaDescriptor

template <int BlockMajorSize, int BlockMinorSize>
__host__ static inline void create_tensor_map(CUtensorMap *tma_map,
                                              half *gmem_ptr,
                                              int blocks_height,
                                              int blocks_width) {
  void *gmem_address = (void *)gmem_ptr;
  uint64_t gmem_prob_shape[5] = {(uint64_t)BlockMinorSize * blocks_width,
                                 (uint64_t)BlockMajorSize * blocks_height,
                                 1, 1, 1};
  uint64_t gmem_prob_stride[5] = {
      sizeof(half), sizeof(half) * BlockMinorSize * blocks_width, 0, 0, 0};
  uint32_t smem_box_shape[5] = {uint32_t(BlockMinorSize),
                                uint32_t(BlockMajorSize), 1, 1, 1};
  uint32_t smem_box_stride[5] = {1, 1, 1, 1, 1};
  CUresult result = cuTensorMapEncodeTiled(
      tma_map, CU_TENSOR_MAP_DATA_TYPE_FLOAT16, 2, gmem_address,
      gmem_prob_shape, gmem_prob_stride + 1, smem_box_shape, smem_box_stride,
      CU_TENSOR_MAP_INTERLEAVE_NONE, CU_TENSOR_MAP_SWIZZLE_128B,
      CU_TENSOR_MAP_L2_PROMOTION_NONE, CU_TENSOR_MAP_FLOAT_OOB_FILL_NONE);
  if (result != CUDA_SUCCESS)
    printf("cuTensorMapEncodeTiled failed: %d\n", (int)result);
}

__host__ static inline CUtensorMap *allocate_and_create_tensor_map(
    half *src, int blocks_height, int blocks_width) {
  CUtensorMap *tma_map_d;
  cudaMalloc(&tma_map_d, sizeof(CUtensorMap));
  CUtensorMap tma_map_host;
  create_tensor_map<128, 64>(&tma_map_host, src, blocks_height, blocks_width);
  cudaMemcpy(tma_map_d, &tma_map_host, sizeof(CUtensorMap),
             cudaMemcpyHostToDevice);
  return tma_map_d;
}
#endif /* NOTES_V2_ENABLE_WGMMA || NOTES_V2_ENABLE_TMA_MMA_WS */

// ============================================================================
// Phase 8: FlashAttention-2 (Split-Q + MMA m16n8k16)
// ============================================================================
// 面试要点（FlashAttention 算法）：
//   1. 核心问题: 标准 Attention 的 O(N^2) 中间矩阵 (S=QK^T) 必须写入 HBM,
//      但 HBM 带宽是瓶颈 → FlashAttention 用 tiling + online softmax 避免写回
//   2. FA 三板斧:
//      a) Tiling: Q 分块 [Br,d], K/V 沿 seqlen 分块 [Bc,d]
//      b) Online Softmax: 迭代更新 m(行max) 和 l(行sum), 无需全局同步
//      c) Recomputation(反向): 反向传播时重新计算 S/P, 而非存储中间矩阵
//   3. Split-Q 设计: 所有 warp 共享同一块 K, 各 warp 处理 Q 的不同行片段
//      - warp_KV=0（所有 warp 共享 K），warp_QP=warp_id（各 warp 不同 Q 行）
//      - 优点: 减少 warp 间通信和 shuffle
//   4. Online rescaling 公式（FA 核心，arXiv:2307.08691）：
//      for each K,V tile:
//        S_cur = Q @ K^T                          // 未缩放，存入 R_S
//        m_new = max(m_old, row_max(S_cur * scale))
//        P_cur = exp(S_cur * scale - m_new)       // ← 写回 R_S 寄存器!
//        l_new = exp(m_old - m_new) * l_old + row_sum(P_cur)
```

```
//        O_new = diag(exp(m_old - m_new)) * O_old + P_cur @ V
//      O_final = O_new / l_final
//   5. 为什么 R_S 可以直接用作 P@V 的 A 矩阵?
//      - R_S 经过 softmax 后，存储的是 P = exp(S - m)，数据仍然是 half 精度
//      - 当前实现依赖 m16n8k16 这一路径下约定好的 fragment 布局，使 softmax
//        后的 P 可以继续留在 R_S 中供后面的 P@V 直接消费
//      - 这是此实现的寄存器布局复用技巧，不要背成“所有 MMA A/C fragment
//        都天然同构”的通用结论
//
// 本实现参考: FlashAttention-2 (Dao et al., arXiv:2307.08691)
// 从 LeetCUDA flash-attn/mma/basic/flash_attn_mma_split_q.cu 提取
// Grid:  ((QKV_seqlen + 63) / 64, QKV_batch * QKV_head, 1), Br=64
// Block: (128, 1, 1), kNumThreads=kWarpSize×kMmaTileSeqLenQ×kMmaTileSeqLenK=128
// source: LeetCUDA/kernels/flash-attn/mma/basic/flash_attn_mma_split_q.cu

// ---- 寄存器填充辅助函数 ----
template <typename T, int M, const int N, const int K = 2>
__device__ inline void fill_3D_regs(T (&R)[M][N][K], T val) {
#pragma unroll
  for (int i = 0; i < M; ++i)
#pragma unroll
    for (int j = 0; j < N; ++j)
#pragma unroll
      for (int k = 0; k < K; ++k)
        R[i][j][k] = val;
}

template <typename T, int M, const int N = 2>
__device__ inline void fill_2D_regs(T (&R)[M][N], T val) {
#pragma unroll
  for (int i = 0; i < M; ++i)
#pragma unroll
    for (int j = 0; j < N; ++j)
      R[i][j] = val;
}

// ============================================================================
// FlashAttention-2 Split-Q Kernel（完整实现）
// ============================================================================
// Q,K,V,O: [batch_size, num_heads, seq_len, head_dim], [B,H,N,d]
//
// Tile 设计（以 kHeadDim=64 为例）:
//   Br = kMmaAtomM * kMmaTileSeqLenQ * kValTileSeqLenQ = 16*4*1 = 64
//   Bc = kMmaAtomN * kMmaTileSeqLenK * kValTileSeqLenK = 8*1*8  = 64
//   Warp 布局: 4 warps, warp_QP 0~3 各处理 16 行, warp_KV=0 共享 K
//
// 执行流程:
//   1) 预加载 Q[Br,d] 到 smem（只加载一次，split-Q 的核心优势）
//   2) 外循环: 沿 K seqlen 分块迭代 (Tc = seqlen/Bc)
//   3)   3a: cp.async 加载当前 V + 预加载下一个 K（多 stage pipeline）
//   4)   3b: Q@K^T — 沿 head dim 内循环 → ldmatrix Q/K → HMMA16816
//   5)   3c: Online Safe Softmax — warp reduce max/row → exp(S*scale - max)
//          → 关键: 将 P 写回 R_S 寄存器（替换 S），R_S 现在存储 P matrix
//   6)   3d: P@V — 沿 V_Bc 内循环 → ldmatrix V (transposed) → 直接用 R_S 做 A
//          → 当前实现依赖同一组寄存器布局约定，避免为 P@V 额外重组 P fragment
//   7)   3e: Online rescaling — O_new = exp(m_old-m_new)*O_old + P@V
//   8) 最终 rescale: O_final = (1/l_final) * O_final
//   9) Epilogue: warp shuffle + 128-bit collective store

template <
    const int kHeadDim,          // head dim: 32, 64, 128
    const int kMmaAtomM,         // 16 (MMA instruction M dimension)
    const int kMmaAtomN,         // 8  (MMA instruction N dimension)
```

```
    const int kMmaAtomK,          // 16 (MMA instruction K dimension)
    const int kMmaTileSeqLenQ,    // MMA tiles along Q's M dim, 4 → Br=16*4=64
    const int kMmaTileSeqLenK,    // MMA tiles along K's N dim, 1 → Bc basis=8
    const int kMmaTileSeqLenP,    // MMA tiles for P@V M dim, must equal kMmaTileSeqLenQ
    const int kMmaTileHeadDimV,   // MMA tiles for P@V N dim (head dim direction)
    const int kValTileSeqLenQ,    // value tiles along Q's M, 1 → Br per warp=16
    const int kValTileSeqLenK,    // value tiles along K's N, 8 → Bc_warp=8*8=64
    const int kValTileSeqLenP,    // value tiles for P@V M dim, 1
    const int kValTileHeadDimV,   // value tiles for P@V N dim, kHeadDim/(8*kMmaTileHeadDimV)
    const int kStage,             // pipeline stages for K: >= 1
    const int kPad>               // padding for bank conflict avoidance
__global__ void __launch_bounds__(kWarpSize *kMmaTileSeqLenQ *kMmaTileSeqLenK)
    flash_attn_mma_stages_split_q(half *Q, half *K, half *V, half *O,
                                  int QKV_seqlen, int QKV_head) {
  constexpr int Br = kMmaAtomM * kMmaTileSeqLenQ * kValTileSeqLenQ; // 64
  constexpr int Bc = kMmaAtomN * kMmaTileSeqLenK * kValTileSeqLenK; // 64
  constexpr int kNumThreads = kWarpSize * kMmaTileSeqLenQ * kMmaTileSeqLenK; // 128
  const int Tc = (QKV_seqlen + Bc - 1) / Bc;
  // 原始实现默认 seqlen 与 Bc 对齐；最后一个不完整 tile 需要额外 pad/边界处理。
  // 这里保留 ceil 写法是为了说明 tile 划分方式，不等于当前实现已经完整处理了尾 tile。
  const float scale = 1.0f / sqrtf((float)kHeadDim);

  // Block indexing
  const int QKV_batch_id = blockIdx.y / QKV_head;
  const int QKV_head_id = blockIdx.y % QKV_head;
  const int Q_tile_id = blockIdx.x;
  const int tid = threadIdx.x;
  const int warp_id = tid / kWarpSize;
  const int lane_id = tid % kWarpSize;
  const int warp_QP = warp_id; // Split-Q: 每个 warp 处理不同的 Q 行片段
  const int warp_KV = 0; // 所有 warp 共享 K（减少跨 warp 通信）

  // Global memory base offsets for this (batch, head)
  // 这里默认 Q/K/V 共享同一 per-head 基址布局，对应 self-attention 场景
  const int Q_gmem_offset =
      (QKV_batch_id * QKV_head * QKV_seqlen + QKV_head_id * QKV_seqlen) *
      kHeadDim;
  const int K_gmem_offset = Q_gmem_offset;
  const int V_gmem_offset = Q_gmem_offset;
  const int O_gmem_offset = Q_gmem_offset;

  // Thread-to-smem mapping for cooperative load
  int load_smem_Q_Br = tid / (kNumThreads / Br);
  int load_smem_Q_d =
      (tid % (kNumThreads / Br)) * (kHeadDim / (kNumThreads / Br));
  int load_smem_K_Bc = tid / (kNumThreads / Bc);
  int load_smem_K_d =
      (tid % (kNumThreads / Bc)) * (kHeadDim / (kNumThreads / Bc));
  int load_smem_V_Bc = tid / (kNumThreads / Bc);
  int load_smem_V_d =
      (tid % (kNumThreads / Bc)) * (kHeadDim / (kNumThreads / Bc));

  int load_gmem_Q_Br = Q_tile_id * Br + load_smem_Q_Br;
  if (load_gmem_Q_Br >= QKV_seqlen)
    return;

  // ---- Shared memory layout ----
  extern __shared__ half smem[];
  constexpr int Q_tile_size = Br * (kHeadDim + kPad);  // Q tile: [Br, d+kPad]
  constexpr int KV_tile_size = Bc * (kHeadDim + kPad); // K/V tile: [Bc, d+kPad]
  half *Q_tile_smem = smem;
  half *K_tile_smem = Q_tile_smem + Q_tile_size;
  half *V_tile_smem = K_tile_smem + kStage * KV_tile_size; // kStage copies of K
```

```
  // 原始 kernel 还留了一个优化点：若 kStage=1，K 和 V 在时序上并不重叠，
  // 理论上可以复用同一块 KV shared memory 来进一步压缩 smem 占用。

  uint32_t smem_Q_base_ptr = __cvta_generic_to_shared(Q_tile_smem);
  uint32_t smem_K_base_ptr = __cvta_generic_to_shared(K_tile_smem);
  uint32_t smem_V_base_ptr = __cvta_generic_to_shared(V_tile_smem);

  // ---- Online Softmax persistent state ----
  // lane_block_row_max_old[i][r]: running max for row r of warp tile i
  // lane_block_row_sum_old[i][r]: running denominator l for row r of warp tile i
  float lane_block_row_max_old[kValTileSeqLenQ][2];
  float lane_block_row_sum_old[kValTileSeqLenQ][2];
  fill_2D_regs<float, kValTileSeqLenQ, 2>(lane_block_row_max_old, -INFINITY);
  fill_2D_regs<float, kValTileSeqLenQ, 2>(lane_block_row_sum_old, 0.0f);

  // ---- Register allocation ----
  uint32_t R_Q[kValTileSeqLenQ][4];                   // Q regs
  uint32_t R_K[kValTileSeqLenK][2];                   // K regs
  uint32_t R_V[kValTileHeadDimV][2];                  // V regs
  // R_S / R_O / R_D 都按 mma.sync.aligned.m16n8k16 的 fragment 约定存储。
  // 对单个 m16n8k16 tile 而言：
  //   - reg[0] 持有该 tile 前 8 行里的两个 half 值
  //   - reg[1] 持有该 tile 后 8 行里的两个 half 值
  // 后续 softmax、P@V、online rescale 都直接围绕这组 fragment 布局做寄存器内变换。
  uint32_t R_S[kValTileSeqLenQ][kValTileSeqLenK][2]; // S=Q@K^T / P=softmax(S)
  uint32_t R_O[kValTileSeqLenP][kValTileHeadDimV][2]; // O for current tile
  uint32_t R_D[kValTileSeqLenP][kValTileHeadDimV]
              [2]; // O accumulator (final output)

  fill_3D_regs<uint32_t, kValTileSeqLenQ, kValTileSeqLenK, 2>(R_S, 0);
  fill_3D_regs<uint32_t, kValTileSeqLenP, kValTileHeadDimV, 2>(R_D, 0);

  // ========================================================================
  // Step 1: 加载 Q[Br, d] 到 shared memory（整个外循环只加载一次）
  // ========================================================================
  {
    int load_gmem_Q_addr =
        Q_gmem_offset + load_gmem_Q_Br * kHeadDim + load_smem_Q_d;
    uint32_t load_smem_Q_ptr =
        smem_Q_base_ptr +
        (load_smem_Q_Br * (kHeadDim + kPad) + load_smem_Q_d) * sizeof(half);
#pragma unroll
    for (int i = 0; i < (kHeadDim / (kNumThreads / Br)); i += 8) {
      CP_ASYNC_CG(load_smem_Q_ptr + i * 2, &Q[load_gmem_Q_addr + i], 16);
    }
    CP_ASYNC_COMMIT_GROUP();
  }

  // ========================================================================
  // Step 2: 预加载前 (kStage-1) 个 K tile（多 stage pipeline 预热）
  // 注意：Q 由 blockIdx.x 固定到当前 Q tile；而 K/V 的 seqlen 遍历始终从 tile 0 开始，
  // 后续在外循环里不断递增到 tile 1/2/3/.../Tc-1。
  // ========================================================================
  if constexpr (kStage > 1) {
#pragma unroll
    for (int stage = 0; stage < (kStage - 1); ++stage) {
      int load_gmem_K_Bc = stage * Bc + load_smem_K_Bc;
      int load_gmem_K_addr =
          K_gmem_offset + load_gmem_K_Bc * kHeadDim + load_smem_K_d;
      uint32_t load_smem_K_ptr =
          smem_K_base_ptr +
          (stage * KV_tile_size + load_smem_K_Bc * (kHeadDim + kPad) +
           load_smem_K_d) *
```

```
              sizeof(half);
#pragma unroll
      for (int i = 0; i < (kHeadDim / (kNumThreads / Bc)); i += 8) {
        CP_ASYNC_CG(load_smem_K_ptr + i * 2, &K[load_gmem_K_addr + i], 16);
      }
      CP_ASYNC_COMMIT_GROUP();
    }
    CP_ASYNC_WAIT_GROUP(kStage - 2);
    __syncthreads();
  }

  // ============================================================================
  // Step 3: 外循环 — 沿 K seqlen 迭代 (Tc = ceil(seqlen/Bc))
  //   每次迭代处理一个 K[Bc,d] + V[Bc,d] tile
  // ============================================================================
#pragma unroll 1
  for (int tile_K_seqlen = 0; tile_K_seqlen < Tc; ++tile_K_seqlen) {
    int smem_sel = tile_K_seqlen % kStage;
    int smem_sel_next = (tile_K_seqlen + (kStage - 1)) % kStage;

    // ---- 3a: 异步加载 K/V tile (多 stage pipeline) ----
    if constexpr (kStage > 1) {
      // 只有 kStage>1 才能真正做 K 的 pipeline:
      //   smem_sel 负责“当前正在计算”的 K tile，smem_sel_next 负责“下一轮预取”的 K tile。
      // 若 kStage=1，这两个槽位永远都等于 0，当前 K 还没算完就无法安全覆盖同一块 smem。
      // Load current V tile (no pipeline for V — one stage is enough)
      {
        int load_gmem_V_Bc = tile_K_seqlen * Bc + load_smem_V_Bc;
        int load_gmem_V_addr =
            V_gmem_offset + load_gmem_V_Bc * kHeadDim + load_smem_V_d;
        uint32_t load_smem_V_ptr =
            smem_V_base_ptr +
            (load_smem_V_Bc * (kHeadDim + kPad) + load_smem_V_d) * sizeof(half);
#pragma unroll
        for (int i = 0; i < (kHeadDim / (kNumThreads / Bc)); i += 8) {
          CP_ASYNC_CG(load_smem_V_ptr + i * 2, &V[load_gmem_V_addr + i], 16);
        }
        CP_ASYNC_COMMIT_GROUP();
      }

      // Prefetch next K tile (pipelined)
      if ((tile_K_seqlen + 1) < Tc) {
        int load_gmem_K_Bc = (tile_K_seqlen + 1) * Bc + load_smem_K_Bc;
        int load_gmem_K_addr =
            K_gmem_offset + load_gmem_K_Bc * kHeadDim + load_smem_K_d;
        uint32_t load_smem_K_ptr =
            smem_K_base_ptr +
            (smem_sel_next * KV_tile_size + load_smem_K_Bc * (kHeadDim + kPad) +
             load_smem_K_d) *
                sizeof(half);
#pragma unroll
        for (int i = 0; i < (kHeadDim / (kNumThreads / Bc)); i += 8) {
          CP_ASYNC_CG(load_smem_K_ptr + i * 2, &K[load_gmem_K_addr + i], 16);
        }
        CP_ASYNC_COMMIT_GROUP();
      }
    }

    // ---- 3b: Q@K^T = S[Br, Bc] — 沿 head dim (d/kMmaAtomK=16) 内循环 ----
    fill_3D_regs<uint32_t, kValTileSeqLenQ, kValTileSeqLenK, 2>(R_S, 0);
#pragma unroll
    for (int tile_K_d = 0; tile_K_d < (kHeadDim / kMmaAtomK); ++tile_K_d) {
      // ldmatrix.x4: 加载 Q 的 m16k16 片段到 R_Q
```

```
#pragma unroll
      for (int i = 0; i < kValTileSeqLenQ; ++i) {
        int warp_smem_Q_Br =
            warp_QP * (kMmaAtomM * kValTileSeqLenQ) + i * kMmaAtomM;
        int lane_smem_Q_Br =
            warp_smem_Q_Br + lane_id % 16; // ldmatrix uses 16 lanes
        int lane_smem_Q_d = tile_K_d * kMmaAtomK + (lane_id / 16) * 8; // 0, 8
        uint32_t lane_smem_Q_ptr =
            smem_Q_base_ptr +
            (lane_smem_Q_Br * (kHeadDim + kPad) + lane_smem_Q_d) * sizeof(half);
        LDMATRIX_X4(R_Q[i][0], R_Q[i][1], R_Q[i][2], R_Q[i][3],
                    lane_smem_Q_ptr);
      }

      // ldmatrix.x2: 加载 K 的 k16n8 片段到 R_K
      // K[Bc,d] row-major = K^T[d,Bc] col-major (NT 布局的 B 矩阵)
#pragma unroll
      for (int j = 0; j < kValTileSeqLenK; ++j) {
        int warp_smem_K_Bc =
            warp_KV * (kMmaAtomN * kValTileSeqLenK) + j * kMmaAtomN;
        int lane_smem_K_Bc =
            warp_smem_K_Bc + lane_id % 8; // ldmatrix B uses 8 lanes
        int lane_smem_K_d =
            tile_K_d * kMmaAtomK + ((lane_id / 8) % 2) * 8; // 0, 8
        uint32_t lane_smem_K_ptr =
            smem_K_base_ptr +
            (smem_sel * KV_tile_size + lane_smem_K_Bc * (kHeadDim + kPad) +
             lane_smem_K_d) *
                sizeof(half);
        LDMATRIX_X2(R_K[j][0], R_K[j][1], lane_smem_K_ptr);
      }

      // MMA: S[tile] += Q[tile] @ K^T[tile]
#pragma unroll
      for (int i = 0; i < kValTileSeqLenQ; ++i) {
#pragma unroll
        for (int j = 0; j < kValTileSeqLenK; ++j) {
          HMMA16816(R_S[i][j][0], R_S[i][j][1], R_Q[i][0], R_Q[i][1], R_Q[i][2],
                    R_Q[i][3], R_K[j][0], R_K[j][1], R_S[i][j][0],
                    R_S[i][j][1]);
        }
      }
    } // end loop over d
    __syncthreads();

    // ==========================================================================
    // 3c: Online Safe Softmax — row-wise max + exp + sum, then store P back to R_S
    // ==========================================================================
    // MMA C fragment layout for m16n8k16 (PTX ISA 对应的线程寄存器分布):
    //   - R_S[i][j][0] 对应当前 16x8 tile 的 rows 0~7 里的两个 half 值 {c0,c1}
    //   - R_S[i][j][1] 对应 rows 8~15 里的两个 half 值 {c2,c3}
    //   - lane 0~3 持有 row 0 的片段，lane 4~7 持有 row 1, ..., lane 28~31 持有 row 7
    //   - 对于 rows 8~15 也是同样的 lane 分组，只是读取的是 reg[1]
    // 这就是为什么后面做 row max / row sum 时，warp 内真正参与同一行归约的是
    // {0,4,8,...,28} 这一类 4-lane 子组，而不是整 warp 32 个线程。
    // Each (i, j) pair = one 16x8 MMA tile; there are kValTileSeqLenQ x
    // kValTileSeqLenK tiles.

    float lane_row_max_new[kValTileSeqLenQ][2];
    float lane_row_sum_new[kValTileSeqLenQ][2];
    fill_2D_regs<float, kValTileSeqLenQ, 2>(lane_row_max_new, -INFINITY);
    fill_2D_regs<float, kValTileSeqLenQ, 2>(lane_row_sum_new, 0.0f);

```

```
    // Pass 1: Thread-level reduce max across all columns (kValTileSeqLenK tiles)
#pragma unroll
    for (int i = 0; i < kValTileSeqLenQ; ++i) {
#pragma unroll
      for (int j = 0; j < kValTileSeqLenK; ++j) {
        // Extract half values from R_S registers (C matrix fragment layout)
        float2 t_reg_S_0 =
            __half22float2(HALF2(R_S[i][j][0])); // rows 0~7:  {c0, c1}
        float2 t_reg_S_1 =
            __half22float2(HALF2(R_S[i][j][1])); // rows 8~15: {c2, c3}
        // S = (Q@K^T) * scale
        float tmp_max_0 = max(t_reg_S_0.x, t_reg_S_0.y) * scale;
        float tmp_max_1 = max(t_reg_S_1.x, t_reg_S_1.y) * scale;
        lane_row_max_new[i][0] = max(lane_row_max_new[i][0], tmp_max_0);
        lane_row_max_new[i][1] = max(lane_row_max_new[i][1], tmp_max_1);
      }
      // Warp-level reduce max (kWarpWidth = 4 for Q@K^T — only lanes
      // {0,4,8,...,28} hold valid data)
      lane_row_max_new[i][0] =
          warp_reduce_max<4, float>(lane_row_max_new[i][0]);
      lane_row_max_new[i][1] =
          warp_reduce_max<4, float>(lane_row_max_new[i][1]);
    }

    // Pass 2: Compute P = exp(S*scale - m_new), store back to R_S
    // 面试关键点：这里将 P 写回 R_S 寄存器!
    // 为什么可以? 当前实现依赖 m16n8k16 这一路径下约定好的 fragment 布局,
    // 使 softmax 后的 P 能继续留在 R_S 中供后面的 P@V 直接消费，无需额外重组。
#pragma unroll
    for (int i = 0; i < kValTileSeqLenQ; ++i) {
      // m_new = max(m_old, m_cur)
      float block_row_max_new_0 =
          max(lane_block_row_max_old[i][0], lane_row_max_new[i][0]);
      float block_row_max_new_1 =
          max(lane_block_row_max_old[i][1], lane_row_max_new[i][1]);

#pragma unroll
      for (int j = 0; j < kValTileSeqLenK; ++j) {
        float2 t_reg_S_0 = __half22float2(HALF2(R_S[i][j][0]));
        float2 t_reg_S_1 = __half22float2(HALF2(R_S[i][j][1]));

        // P = exp(S * scale - m_new), 用 fma 保证精度
        t_reg_S_0.x =
            __expf(__fmaf_rn(t_reg_S_0.x, scale, -block_row_max_new_0));
        t_reg_S_0.y =
            __expf(__fmaf_rn(t_reg_S_0.y, scale, -block_row_max_new_0));
        t_reg_S_1.x =
            __expf(__fmaf_rn(t_reg_S_1.x, scale, -block_row_max_new_1));
        t_reg_S_1.y =
            __expf(__fmaf_rn(t_reg_S_1.y, scale, -block_row_max_new_1));

        // Accumulate row sums
        lane_row_sum_new[i][0] += (t_reg_S_0.x + t_reg_S_0.y);
        lane_row_sum_new[i][1] += (t_reg_S_1.x + t_reg_S_1.y);

        // 关键: 将 P 写回 R_S! R_S 现在存储的是 P = softmax(S), 不是 S
        HALF2(R_S[i][j][0]) = __float22half2_rn(t_reg_S_0);
        HALF2(R_S[i][j][1]) = __float22half2_rn(t_reg_S_1);
      }

      // Warp-level reduce sum (kWarpWidth = 4, same as max)
      lane_row_sum_new[i][0] =
          warp_reduce_sum<4, float>(lane_row_sum_new[i][0]);
```

```
      lane_row_sum_new[i][1] =
          warp_reduce_sum<4, float>(lane_row_sum_new[i][1]);
    }
    __syncthreads();

    // ============================================================================
    // 3d: P@V — P[Br,Bc] @ V[Bc,d] = O[Br,d]
    // ============================================================================
    // Wait for V to be ready before computing P@V
    if constexpr (kStage > 1) {
      if ((tile_K_seqlen + 1) < Tc) {
        CP_ASYNC_WAIT_GROUP(1);
      } else {
        CP_ASYNC_WAIT_GROUP(0);
      }
    } else {
      CP_ASYNC_WAIT_GROUP(0);
    }
    __syncthreads();

    fill_3D_regs<uint32_t, kValTileSeqLenP, kValTileHeadDimV, 2>(R_O, 0);

    // tile_V_Bc: iterate over chunks of Bc/K=16 columns in P matrix
    // Bc=kMmaAtomK=16 → 1 iteration for kHeadDim≤64 configurations
#pragma unroll
    for (int tile_V_Bc = 0; tile_V_Bc < (Bc / kMmaAtomK); ++tile_V_Bc) {
      // ldmatrix.x2.trans: load V[Bc,d] with transposition
      // V is row-major [Bc,d], but NN matmul needs B matrix in col-major → use
      // transposed ldmatrix
#pragma unroll
      for (int j = 0; j < kValTileHeadDimV; ++j) {
        int warp_smem_V_d =
            warp_KV * (kMmaAtomN * kValTileHeadDimV) + j * kMmaAtomN;
        int lane_smem_V_Bc = tile_V_Bc * kMmaAtomK + lane_id % 16;
        int lane_smem_V_d = warp_smem_V_d;
        uint32_t lane_smem_V_ptr =
            smem_V_base_ptr +
            (lane_smem_V_Bc * (kHeadDim + kPad) + lane_smem_V_d) * sizeof(half);
        LDMATRIX_X2_T(R_V[j][0], R_V[j][1], lane_smem_V_ptr);
      }

      // P matrix layout in R_S[i][j][2]:
      //   MMA = m16n8k16, Br=16x4=64, Bc=8x8=64, layout: 4 warps
      //   |   64x64   |      warp_KV 0       |
      //   | warp_QP 0 | MMA 0 ... MMA 0 (x8) |
      //   | warp_QP 1 | MMA 1 ... MMA 1 (x8) |
      //   | warp_QP 2 | MMA 2 ... MMA 2 (x8) |
      //   | warp_QP 3 | MMA 3 ... MMA 3 (x8) |
      // tile_V_Bc selects which 16-column slice of P to use:
      //   tile_V_Bc=0 → cols  0:16 → MMA indices 0,1  → w=0
      //   tile_V_Bc=1 → cols 16:32 → MMA indices 2,3  → w=2
      //   tile_V_Bc=2 → cols 32:48 → MMA indices 4,5  → w=4
      //   tile_V_Bc=3 → cols 48:64 → MMA indices 6,7  → w=6
      // 对应的 MMA A fragment 布局可以这样记:
      //   - rows 0~7:  lane 0~3 -> {a0,a1} 与 {a4,a5}，lane 4~7 -> 下一行，依次类推
      //   - rows 8~15: lane 0~3 -> {a2,a3} 与 {a6,a7}，lane 4~7 -> 下一行，依次类推
      // 当前实现正是利用这一路径下 A fragment 与前面生成的 P fragment 可以直接对接，
      // 才能把 R_S 中的 P 直接喂给 HMMA16816 做 P@V；复习时不要把它背成对所有 MMA
      // fragment 都无条件成立的通用结论。layout转换逻辑:
      //   C layout: 8 x (16 x 8) layout -> A layout: 4 x (16 x 16) layout
      int w = tile_V_Bc * 2;
#pragma unroll
      for (int i = 0; i < kValTileSeqLenP; ++i) {
```

```
#pragma unroll
        for (int j = 0; j < kValTileHeadDimV; ++j) {
          HMMA16816(R_O[i][j][0], R_O[i][j][1], // C fragment output
                    R_S[i][w][0], R_S[i][w][1], R_S[i][w + 1][0], R_S[i][w + 1][1],  // A fragment = P
                    R_V[j][0], R_V[j][1], // B fragment = V
                    R_O[i][j][0], R_O[i][j][1]); // C fragment output
        }
      }
    } // end for tile_V_Bc
    __syncthreads();

    // ============================================================================
    // 3e: Online rescaling — O_new = exp(m_old - m_new) * O_old + P@V
    // ============================================================================
    // 公式来源: FA2 paper Eq.(7-8), 使用 exp(m_old - m_new) 做 O 与 l 的 rescale
#pragma unroll
    for (int i = 0; i < kValTileSeqLenP; ++i) {
      float block_row_max_new_0 = lane_row_max_new[i][0];
      float block_row_max_new_1 = lane_row_max_new[i][1];
      float block_row_sum_new_0 = lane_row_sum_new[i][0];
      float block_row_sum_new_1 = lane_row_sum_new[i][1];

      float block_row_max_old_0 = lane_block_row_max_old[i][0];
      float block_row_max_old_1 = lane_block_row_max_old[i][1];

      block_row_max_new_0 = max(block_row_max_old_0, block_row_max_new_0);
      block_row_max_new_1 = max(block_row_max_old_1, block_row_max_new_1);

      // Handle first iteration: m_old = -inf, need to use m_new directly
      block_row_max_old_0 =
          (tile_K_seqlen > 0 ? block_row_max_old_0 : block_row_max_new_0);
      block_row_max_old_1 =
          (tile_K_seqlen > 0 ? block_row_max_old_1 : block_row_max_new_1);

      float rescale_o_factor_0 =
          __expf(block_row_max_old_0 - block_row_max_new_0);
      float rescale_o_factor_1 =
          __expf(block_row_max_old_1 - block_row_max_new_1);

      // Rescale O_old + Add P@V in one fused step
#pragma unroll
      for (int j = 0; j < kValTileHeadDimV; ++j) {
        // R_O / R_D 与前面的 R_S 一样, 都按 MMA C fragment 布局解释:
        //   reg[0] -> rows 0~7 的 {c0,c1}
        //   reg[1] -> rows 8~15 的 {c2,c3}
        float2 t_reg_O_0 = __half22float2(HALF2(R_O[i][j][0]));
        float2 t_reg_O_1 = __half22float2(HALF2(R_O[i][j][1]));
        float2 t_reg_D_0 = __half22float2(HALF2(R_D[i][j][0]));
        float2 t_reg_D_1 = __half22float2(HALF2(R_D[i][j][1]));

        // O_new = exp(m_old - m_new) * O_old + P@V  (fused multiply-add)
        t_reg_D_0.x = __fmaf_rn(rescale_o_factor_0, t_reg_D_0.x, t_reg_O_0.x);
        t_reg_D_0.y = __fmaf_rn(rescale_o_factor_0, t_reg_D_0.y, t_reg_O_0.y);
        t_reg_D_1.x = __fmaf_rn(rescale_o_factor_1, t_reg_D_1.x, t_reg_O_1.x);
        t_reg_D_1.y = __fmaf_rn(rescale_o_factor_1, t_reg_D_1.y, t_reg_O_1.y);

        HALF2(R_D[i][j][0]) = __float22half2_rn(t_reg_D_0);
        HALF2(R_D[i][j][1]) = __float22half2_rn(t_reg_D_1);
      }

      // Update l: l_new = exp(m_old - m_new) * l_old + row_sum(P)
      float block_row_sum_old_0 = lane_block_row_sum_old[i][0];
      float block_row_sum_old_1 = lane_block_row_sum_old[i][1];
```

```
      lane_block_row_sum_old[i][0] = __fmaf_rn(
          rescale_o_factor_0, block_row_sum_old_0, block_row_sum_new_0);
      lane_block_row_sum_old[i][1] = __fmaf_rn(
          rescale_o_factor_1, block_row_sum_old_1, block_row_sum_new_1);

      // Update m
      lane_block_row_max_old[i][0] = block_row_max_new_0;
      lane_block_row_max_old[i][1] = block_row_max_new_1;
    }

    // Wait for next K tile to be ready in smem before next iteration
    if constexpr (kStage > 1) {
      if ((tile_K_seqlen + 1) < Tc) {
        CP_ASYNC_WAIT_GROUP(0);
      }
      __syncthreads();
    }
  } // end outer loop over K seqlen
  __syncthreads();

  // ============================================================================
  // Step 4: 最终 rescale — O_final = (1/l_final) * O_final
  // ============================================================================
#pragma unroll
  for (int i = 0; i < kValTileSeqLenP; ++i) {
    float rescale_factor_0 = __frcp_rn(lane_block_row_sum_old[i][0]);
    float rescale_factor_1 = __frcp_rn(lane_block_row_sum_old[i][1]);
#pragma unroll
    for (int j = 0; j < kValTileHeadDimV; ++j) {
      float2 t_reg_D_0 = __half22float2(HALF2(R_D[i][j][0]));
      float2 t_reg_D_1 = __half22float2(HALF2(R_D[i][j][1]));
      t_reg_D_0.x = rescale_factor_0 * t_reg_D_0.x;
      t_reg_D_0.y = rescale_factor_0 * t_reg_D_0.y;
      t_reg_D_1.x = rescale_factor_1 * t_reg_D_1.x;
      t_reg_D_1.y = rescale_factor_1 * t_reg_D_1.y;
      HALF2(R_D[i][j][0]) = __float22half2_rn(t_reg_D_0);
      HALF2(R_D[i][j][1]) = __float22half2_rn(t_reg_D_1);
    }
  }

  // ============================================================================
  // Step 5: Epilogue — Collective store via warp shuffle + 128-bit store
  // ============================================================================
  // 利用 warp shuffle 将分散在各 lane 的寄存器数据收集到 lane 0~3,
  // 然后用 LDST128BITS (st.global.v4.f32) 一次性写入 16 bytes
#pragma unroll
  for (int i = 0; i < kValTileSeqLenP; ++i) {
#pragma unroll
    for (int j = 0; j < kValTileHeadDimV; ++j) {
      uint32_t R_Z[2][4];
      R_Z[0][0] = R_D[i][j][0];
      R_Z[1][0] = R_D[i][j][1];
      R_Z[0][1] = __shfl_sync(0xffffffff, R_D[i][j][0], lane_id + 1, 4);
      R_Z[0][2] = __shfl_sync(0xffffffff, R_D[i][j][0], lane_id + 2, 4);
      R_Z[0][3] = __shfl_sync(0xffffffff, R_D[i][j][0], lane_id + 3, 4);
      R_Z[1][1] = __shfl_sync(0xffffffff, R_D[i][j][1], lane_id + 1, 4);
      R_Z[1][2] = __shfl_sync(0xffffffff, R_D[i][j][1], lane_id + 2, 4);
      R_Z[1][3] = __shfl_sync(0xffffffff, R_D[i][j][1], lane_id + 3, 4);

      // st.global.v4.f32: 128-bit store, 4 lanes × 32-bit
      if (lane_id % 4 == 0) {
        int store_warp_regs_O_Br =
            warp_QP * (kMmaAtomM * kValTileSeqLenP) + i * kMmaAtomM;
```

```
        int store_lane_gmem_O_Br =
            Q_tile_id * Br + store_warp_regs_O_Br + lane_id / 4;
        int store_warp_regs_O_d =
            warp_KV * (kMmaAtomN * kValTileHeadDimV) + j * kMmaAtomN;
        int store_lane_gmem_O_d = store_warp_regs_O_d;
        int store_gmem_O_addr_0 = O_gmem_offset +
                                  (store_lane_gmem_O_Br + 0) * kHeadDim +
                                  store_lane_gmem_O_d;
        int store_gmem_O_addr_1 = O_gmem_offset +
                                  (store_lane_gmem_O_Br + 8) * kHeadDim +
                                  store_lane_gmem_O_d;
        // LDST128BITS = reinterpret_cast<float4*>
        *reinterpret_cast<float4 *>(&O[store_gmem_O_addr_0]) =
            *reinterpret_cast<float4 *>(&R_Z[0][0]);
        *reinterpret_cast<float4 *>(&O[store_gmem_O_addr_1]) =
            *reinterpret_cast<float4 *>(&R_Z[1][0]);
      }
    }
  }
}

// ============================================================
// 以下是测试代码，验证 Phase 1 - Phase 8 的kernel的正确性，不评估性能。
// ============================================================

static inline void check(cudaError_t err, const char *msg) {
  if (err != cudaSuccess) {
    fprintf(stderr, "[ERROR] %s: %s\n", msg, cudaGetErrorString(err));
    exit(EXIT_FAILURE);
  }
}


static void test_block_reduce(int N) {

  srand(42);
  float *h_a = (float *)malloc((size_t)N * sizeof(float));
  for (int i = 0; i < N; i++)
    h_a[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;

  // CPU reference: sum of all elements
  double ref = 0.0;
  for (int i = 0; i < N; i++) ref += (double)h_a[i];

  float *d_a, *d_y;
  check(cudaMalloc(&d_a, (size_t)N * sizeof(float)), "blockreduce alloc A");
  check(cudaMalloc(&d_y, sizeof(float)), "blockreduce alloc Y");

  check(cudaMemcpy(d_a, h_a, (size_t)N * sizeof(float), cudaMemcpyHostToDevice), "blockreduce H2D A");
  check(cudaMemset(d_y, 0, sizeof(float)), "blockreduce zero Y");

  dim3 block(128);
  dim3 grid((N + 127) / 128);
  block_reduce_all<128><<<grid, block>>>(d_a, d_y, N);
  check(cudaGetLastError(), "blockreduce launch");
  check(cudaDeviceSynchronize(), "blockreduce sync");

  float result;
  check(cudaMemcpy(&result, d_y, sizeof(float), cudaMemcpyDeviceToHost), "blockreduce D2H");

  float err = fabsf(result - (float)ref);
  printf("| %-42s | %.6e | %-4s | %-8s |\n", "BlockReduce", err,
         err < 1e-2f ? "PASS" : "FAIL", "None");
```

```
4473
4474    free(h_a);
4475    cudaFree(d_a); cudaFree(d_y);
4476  }
4477
4478
4479  static void test_dot(int N) {
4480
4481    srand(42);
4482    float *h_a = (float *)malloc((size_t)N * sizeof(float));
4483    float *h_b = (float *)malloc((size_t)N * sizeof(float));
4484    for (int i = 0; i < N; i++) {
4485      h_a[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
4486      h_b[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
4487    }
4488
4489    // CPU reference
4490    double ref = 0.0;
4491    for (int i = 0; i < N; i++) ref += (double)h_a[i] * (double)h_b[i];
4492
4493    float *d_a, *d_b, *d_y;
4494    check(cudaMalloc(&d_a, (size_t)N * sizeof(float)), "dot alloc A");
4495    check(cudaMalloc(&d_b, (size_t)N * sizeof(float)), "dot alloc B");
4496    check(cudaMalloc(&d_y, sizeof(float)), "dot alloc Y");
4497
4498    check(cudaMemcpy(d_a, h_a, (size_t)N * sizeof(float), cudaMemcpyHostToDevice), "dot H2D A");
4499    check(cudaMemcpy(d_b, h_b, (size_t)N * sizeof(float), cudaMemcpyHostToDevice), "dot H2D B");
4500    check(cudaMemset(d_y, 0, sizeof(float)), "dot zero Y");
4501
4502    dim3 block(128);
4503    dim3 grid((N + 127) / 128);
4504    dot<128><<<grid, block>>>(d_a, d_b, d_y, N);
4505    check(cudaGetLastError(), "dot launch");
4506    check(cudaDeviceSynchronize(), "dot sync");
4507
4508    float result;
4509    check(cudaMemcpy(&result, d_y, sizeof(float), cudaMemcpyDeviceToHost), "dot D2H");
4510
4511    float err = fabsf(result - (float)ref);
4512    printf("| %-42s | %.6e | %-4s | %-8s |\n", "Dot", err,
4513           err < 1e-2f ? "PASS" : "FAIL", "None");
4514
4515    // ---- Dot Vec4 ----
4516    check(cudaMemset(d_y, 0, sizeof(float)), "dot_vec4 zero Y");
4517    dim3 block_v4(32);
4518    dot_vec4<32><<<grid, block_v4>>>(d_a, d_b, d_y, N);
4519    check(cudaGetLastError(), "dot_vec4 launch");
4520    check(cudaDeviceSynchronize(), "dot_vec4 sync");
4521
4522    check(cudaMemcpy(&result, d_y, sizeof(float), cudaMemcpyDeviceToHost), "dot_vec4 D2H");
4523    float err_v4 = fabsf(result - (float)ref);
4524    printf("| %-42s | %.6e | %-4s | %-8s |\n", "Dot-Vec4", err_v4, err_v4 < 1e-2f ? "PASS" : "FAIL", "None");
4525
4526    free(h_a); free(h_b);
4527    cudaFree(d_a); cudaFree(d_b); cudaFree(d_y);
4528  }
4529
4530
4531  static void test_relu(int N) {
4532    srand(42);
4533    float *h_x = (float *)malloc((size_t)N * sizeof(float));
4534    float *h_y = (float *)malloc((size_t)N * sizeof(float));
4535    for (int i = 0; i < N; i++)
```

```
    h_x[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;

  float *d_x, *d_y;
  check(cudaMalloc(&d_x, (size_t)N * sizeof(float)), "relu alloc X");
  check(cudaMalloc(&d_y, (size_t)N * sizeof(float)), "relu alloc Y");
  check(cudaMemcpy(d_x, h_x, (size_t)N * sizeof(float), cudaMemcpyHostToDevice), "relu H2D");

  for (int i = 0; i < N; i++) h_y[i] = fmaxf(0.0f, h_x[i]);
  dim3 block256(256);
  dim3 grid256((N + 255) / 256);
  relu<<<grid256, block256>>>(d_x, d_y, N);
  check(cudaGetLastError(), "relu launch");
  check(cudaDeviceSynchronize(), "relu sync");
  check(cudaMemcpy(h_y, d_y, (size_t)N * sizeof(float), cudaMemcpyDeviceToHost), "relu D2H");
  float max_err = 0.0f;
  for (int i = 0; i < N; i++) {
    float expected = fmaxf(0.0f, h_x[i]);
    float err = fabsf(h_y[i] - expected);
    if (err > max_err) max_err = err;
  }
  printf("| %-42s | %.6e | %-4s | %-8s |\n", "ReLU", max_err, max_err < 1e-4f ? "PASS" : "FAIL", "None");

  dim3 block64(64);
  relu_vec4<<<grid256, block64>>>(d_x, d_y, N);
  check(cudaGetLastError(), "relu_vec4 launch");
  check(cudaDeviceSynchronize(), "relu_vec4 sync");
  check(cudaMemcpy(h_y, d_y, (size_t)N * sizeof(float), cudaMemcpyDeviceToHost), "relu_vec4 D2H");
  max_err = 0.0f;
  for (int i = 0; i < N; i++) {
    float expected = fmaxf(0.0f, h_x[i]);
    float err = fabsf(h_y[i] - expected);
    if (err > max_err) max_err = err;
  }
  printf("| %-42s | %.6e | %-4s | %-8s |\n", "ReLU-Vec4", max_err, max_err < 1e-4f ? "PASS" : "FAIL", "None"
    );

  free(h_x); free(h_y);
  cudaFree(d_x); cudaFree(d_y);
}


static void test_elementwise(int N) {
  srand(42);
  float *h_a = (float *)malloc((size_t)N * sizeof(float));
  float *h_b = (float *)malloc((size_t)N * sizeof(float));
  for (int i = 0; i < N; i++) {
    h_a[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
    h_b[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
  }

  float *d_a, *d_b, *d_c;
  check(cudaMalloc(&d_a, (size_t)N * sizeof(float)), "eadd alloc A");
  check(cudaMalloc(&d_b, (size_t)N * sizeof(float)), "eadd alloc B");
  check(cudaMalloc(&d_c, (size_t)N * sizeof(float)), "eadd alloc C");
  check(cudaMemcpy(d_a, h_a, (size_t)N * sizeof(float), cudaMemcpyHostToDevice), "eadd H2D A");
  check(cudaMemcpy(d_b, h_b, (size_t)N * sizeof(float), cudaMemcpyHostToDevice), "eadd H2D B");

  dim3 block256(256);
  dim3 grid256((N + 255) / 256);
  elementwise_add<<<grid256, block256>>>(d_a, d_b, d_c, N);
  check(cudaGetLastError(), "eadd launch");
  check(cudaDeviceSynchronize(), "eadd sync");
  float *h_c = (float *)malloc((size_t)N * sizeof(float));
```

```
  check(cudaMemcpy(h_c, d_c, (size_t)N * sizeof(float), cudaMemcpyDeviceToHost), "eadd D2H");
  float max_err = 0.0f;
  for (int i = 0; i < N; i++) {
    float err = fabsf(h_c[i] - (h_a[i] + h_b[i]));
    if (err > max_err) max_err = err;
  }
  printf("| %-42s | %.6e | %-4s | %-8s |\n", "ElemwiseAdd", max_err, max_err < 1e-4f ? "PASS" : "FAIL", "
    None");

  dim3 block64(64);
  check(cudaMemset(d_c, 0, (size_t)N * sizeof(float)), "eadd_vec4 zero C");
  elementwise_add_vec4<<<grid256, block64>>>(d_a, d_b, d_c, N);
  check(cudaGetLastError(), "eadd_vec4 launch");
  check(cudaDeviceSynchronize(), "eadd_vec4 sync");
  check(cudaMemcpy(h_c, d_c, (size_t)N * sizeof(float), cudaMemcpyDeviceToHost), "eadd_vec4 D2H");
  max_err = 0.0f;
  for (int i = 0; i < N; i++) {
    float err = fabsf(h_c[i] - (h_a[i] + h_b[i]));
    if (err > max_err) max_err = err;
  }
  printf("| %-42s | %.6e | %-4s | %-8s |\n", "ElemwiseAdd-Vec4", max_err, max_err < 1e-4f ? "PASS" : "FAIL",
     "None");

  free(h_a); free(h_b); free(h_c);
  cudaFree(d_a); cudaFree(d_b); cudaFree(d_c);
}


static void test_histogram(int N) {
  srand(42);
  int BINS = 16;
  int *h_hist = (int *)calloc(BINS, sizeof(int));
  int *h_hist_ref = (int *)calloc(BINS, sizeof(int));
  int *h_idx = (int *)malloc((size_t)N * sizeof(int));
  for (int i = 0; i < N; i++) h_idx[i] = rand() % BINS;
  for (int i = 0; i < N; i++) h_hist_ref[h_idx[i]]++;

  int *d_idx, *d_hist;
  check(cudaMalloc(&d_idx, (size_t)N * sizeof(int)), "hist alloc idx");
  check(cudaMalloc(&d_hist, BINS * sizeof(int)), "hist alloc hist");
  check(cudaMemcpy(d_idx, h_idx, (size_t)N * sizeof(int), cudaMemcpyHostToDevice), "hist H2D idx");
  check(cudaMemset(d_hist, 0, BINS * sizeof(int)), "hist zero");

  dim3 block256(256);
  dim3 grid256((N + 255) / 256);
  histogram<<<grid256, block256>>>(d_idx, d_hist, N);
  check(cudaGetLastError(), "histogram launch");
  check(cudaDeviceSynchronize(), "histogram sync");
  check(cudaMemcpy(h_hist, d_hist, BINS * sizeof(int), cudaMemcpyDeviceToHost), "hist D2H");

  float max_err = 0.0f;
  for (int i = 0; i < BINS; i++) {
    float err = fabsf((float)(h_hist[i] - h_hist_ref[i]));
    if (err > max_err) max_err = err;
  }
  printf("| %-42s | %.6e | %-4s | %-8s |\n", "Histogram", max_err, max_err < 1e-4f ? "PASS" : "FAIL", "None"
    );

  free(h_hist); free(h_hist_ref); free(h_idx);
  cudaFree(d_idx); cudaFree(d_hist);
}
```

```
static void test_merge_attn_states(int num_tokens, int num_heads,
                                   int head_size) {

  size_t out_size = (size_t)num_tokens * num_heads * head_size * sizeof(float);
  size_t lse_size = (size_t)num_heads * num_tokens * sizeof(float);

  float *h_prefix_out = (float *)malloc(out_size);
  float *h_suffix_out = (float *)malloc(out_size);
  float *h_prefix_lse = (float *)malloc(lse_size);
  float *h_suffix_lse = (float *)malloc(lse_size);
  float *h_output_ref = (float *)malloc(out_size);

  srand(42);
  for (int i = 0; i < num_tokens * num_heads * head_size; i++) {
    h_prefix_out[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
    h_suffix_out[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
  }
  for (int i = 0; i < num_heads * num_tokens; i++) {
    h_prefix_lse[i] = ((float)rand() / RAND_MAX) * 20.0f - 10.0f;
    h_suffix_lse[i] = ((float)rand() / RAND_MAX) * 20.0f - 10.0f;
  }

  // CPU reference: 与 kernel 完全相同的浮点计算
  for (int t = 0; t < num_tokens; t++) {
    for (int h = 0; h < num_heads; h++) {
      float p_lse = h_prefix_lse[h * num_tokens + t];
      float s_lse = h_suffix_lse[h * num_tokens + t];
      p_lse = isinf(p_lse) ? -INFINITY : p_lse;
      s_lse = isinf(s_lse) ? -INFINITY : s_lse;

      float max_lse = fmaxf(p_lse, s_lse);
      p_lse -= max_lse;
      s_lse -= max_lse;
      float p_se = expf(p_lse);
      float s_se = expf(s_lse);
      float p_scale = p_se / (p_se + s_se);
      float s_scale = s_se / (p_se + s_se);

      int head_off = t * num_heads * head_size + h * head_size;
      for (int d = 0; d < head_size; d++) {
        h_output_ref[head_off + d] =
            h_prefix_out[head_off + d] * p_scale +
            h_suffix_out[head_off + d] * s_scale;
      }
    }
  }

  float *d_prefix_out, *d_suffix_out, *d_prefix_lse, *d_suffix_lse, *d_output;
  check(cudaMalloc(&d_prefix_out, out_size), "merge_attn alloc p_out");
  check(cudaMalloc(&d_suffix_out, out_size), "merge_attn alloc s_out");
  check(cudaMalloc(&d_prefix_lse, lse_size), "merge_attn alloc p_lse");
  check(cudaMalloc(&d_suffix_lse, lse_size), "merge_attn alloc s_lse");
  check(cudaMalloc(&d_output, out_size), "merge_attn alloc output");

  check(cudaMemcpy(d_prefix_out, h_prefix_out, out_size,
                   cudaMemcpyHostToDevice),
        "merge_attn H2D p_out");
  check(cudaMemcpy(d_suffix_out, h_suffix_out, out_size,
                   cudaMemcpyHostToDevice),
        "merge_attn H2D s_out");
  check(cudaMemcpy(d_prefix_lse, h_prefix_lse, lse_size,
                   cudaMemcpyHostToDevice),
        "merge_attn H2D p_lse");
```

```
  check(cudaMemcpy(d_suffix_lse, h_suffix_lse, lse_size,
                   cudaMemcpyHostToDevice),
        "merge_attn H2D s_lse");

  int threads_per_head = head_size / 4;
  int total_threads = num_tokens * num_heads * threads_per_head;
  dim3 block(128);
  dim3 grid((total_threads + 127) / 128);
  merge_attn_states<<<grid, block>>>(
      d_output, d_prefix_out, d_prefix_lse, d_suffix_out, d_suffix_lse,
      num_tokens, num_heads, head_size);
  check(cudaGetLastError(), "merge_attn launch");
  check(cudaDeviceSynchronize(), "merge_attn sync");

  float *h_output = (float *)malloc(out_size);
  check(cudaMemcpy(h_output, d_output, out_size, cudaMemcpyDeviceToHost),
        "merge_attn D2H");

  float max_err = 0.0f;
  for (int i = 0; i < num_tokens * num_heads * head_size; i++) {
    float err = fabsf(h_output[i] - h_output_ref[i]);
    if (err > max_err) max_err = err;
  }
  printf("| %-42s | %.6e | %-4s | %-8s |\n", "MergeAttnStates", max_err,
         max_err < 1e-4f ? "PASS" : "FAIL", "None");

  // 边界测试: +inf LSE → 权重退化为 0（空 attention 段）
  h_prefix_lse[0] = INFINITY; // head 0, token 0 的 LSE = +inf
  h_suffix_lse[0] = 0.0f;
  check(cudaMemcpy(d_prefix_lse, h_prefix_lse, lse_size,
                   cudaMemcpyHostToDevice),
        "merge_attn H2D inf lse");
  merge_attn_states<<<grid, block>>>(
      d_output, d_prefix_out, d_prefix_lse, d_suffix_out, d_suffix_lse,
      num_tokens, num_heads, head_size);
  check(cudaGetLastError(), "merge_attn inf launch");
  check(cudaDeviceSynchronize(), "merge_attn inf sync");
  check(cudaMemcpy(h_output, d_output, out_size, cudaMemcpyDeviceToHost),
        "merge_attn inf D2H");

  // token 0, head 0 的所有元素应等于 suffix_output（prefix 权重 α=0）
  float inf_err = 0.0f;
  for (int d = 0; d < head_size; d++) {
    float err = fabsf(h_output[d] - h_suffix_out[d]);
    if (err > inf_err) inf_err = err;
  }
  printf("| %-42s | %.6e | %-4s | %-8s |\n", "MergeAttnStates-inf", inf_err,
         inf_err < 1e-4f ? "PASS" : "FAIL", "None");

  free(h_prefix_out);
  free(h_suffix_out);
  free(h_prefix_lse);
  free(h_suffix_lse);
  free(h_output_ref);
  free(h_output);
  cudaFree(d_prefix_out);
  cudaFree(d_suffix_out);
  cudaFree(d_prefix_lse);
  cudaFree(d_suffix_lse);
  cudaFree(d_output);
}
```

```cpp
static void test_softmax(int N) {
  // Use N = blockDim.x so one block processes all elements as one token.
  constexpr int kNumThreads = 256;
  if (N != kNumThreads) {
    printf("  OnlineSafeSoftmax: N must be %d (got %d), skipping.\n", kNumThreads, N);
    return;
  }

  srand(42);
  float *h_x = (float *)malloc((size_t)N * sizeof(float));
  float *h_y_ref = (float *)malloc((size_t)N * sizeof(float));
  for (int i = 0; i < N; i++)
    h_x[i] = ((float)rand() / RAND_MAX) * 10.0f - 5.0f;  // [-5, 5]

  // CPU reference: softmax(x_i) = exp(x_i - max) / sum(exp(x_j - max))
  float max_val = -FLT_MAX;
  for (int i = 0; i < N; i++) if (h_x[i] > max_val) max_val = h_x[i];
  double sum_exp = 0.0;
  for (int i = 0; i < N; i++) sum_exp += (double)expf(h_x[i] - max_val);
  for (int i = 0; i < N; i++)
    h_y_ref[i] = expf(h_x[i] - max_val) / (float)sum_exp;

  float *d_x, *d_y;
  check(cudaMalloc(&d_x, (size_t)N * sizeof(float)), "softmax alloc X");
  check(cudaMalloc(&d_y, (size_t)N * sizeof(float)), "softmax alloc Y");

  check(cudaMemcpy(d_x, h_x, (size_t)N * sizeof(float), cudaMemcpyHostToDevice), "softmax H2D X");

  dim3 block(256);
  dim3 grid(1);  // one token covering all N elements
  online_safe_softmax_per_token<<<grid, block>>>(d_x, d_y, N);
  check(cudaGetLastError(), "softmax launch");
  check(cudaDeviceSynchronize(), "softmax sync");

  float *h_y = (float *)malloc((size_t)N * sizeof(float));
  check(cudaMemcpy(h_y, d_y, (size_t)N * sizeof(float), cudaMemcpyDeviceToHost), "softmax D2H");

  float max_err = 0.0f;
  for (int i = 0; i < N; i++) {
    float err = fabsf(h_y[i] - h_y_ref[i]);
    if (err > max_err) max_err = err;
  }
  printf("| %-42s | %.6e | %-4s | %-8s |\n", "OnlineSafeSoftmax", max_err, max_err < 1e-4f ? "PASS" : "FAIL"
    , "None");

  // ---- Safe Softmax ----
  safe_softmax_per_token<<<grid, block>>>(d_x, d_y, N);
  check(cudaGetLastError(), "safe_softmax launch");
  check(cudaDeviceSynchronize(), "safe_softmax sync");
  check(cudaMemcpy(h_y, d_y, (size_t)N * sizeof(float), cudaMemcpyDeviceToHost), "safe_softmax D2H");
  max_err = 0.0f;
  for (int i = 0; i < N; i++) {
    float err = fabsf(h_y[i] - h_y_ref[i]);
    if (err > max_err) max_err = err;
  }
  printf("| %-42s | %.6e | %-4s | %-8s |\n", "SafeSoftmax", max_err, max_err < 1e-4f ? "PASS" : "FAIL", "
    None");

  // ---- Naive Softmax ----
  softmax_per_token<<<grid, block>>>(d_x, d_y, N);
  check(cudaGetLastError(), "naive_softmax launch");
  check(cudaDeviceSynchronize(), "naive_softmax sync");
  check(cudaMemcpy(h_y, d_y, (size_t)N * sizeof(float), cudaMemcpyDeviceToHost), "naive_softmax D2H");
```

```
  max_err = 0.0f;
  for (int i = 0; i < N; i++) {
    float err = fabsf(h_y[i] - h_y_ref[i]);
    if (err > max_err) max_err = err;
  }
  printf("| %-42s | %.6e | %-4s | %-8s |\n", "NaiveSoftmax", max_err, max_err < 1e-4f ? "PASS" : "FAIL", "
    None");

  free(h_x); free(h_y); free(h_y_ref);
  cudaFree(d_x); cudaFree(d_y);
}


static void test_rms_norm(int N, int K) {

  srand(42);
  float *h_x = (float *)malloc((size_t)N * K * sizeof(float));
  float *h_y_ref = (float *)malloc((size_t)N * K * sizeof(float));
  for (int i = 0; i < N * K; i++)
    h_x[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
  float g = 1.5f;  // gain

  // CPU reference: y = (x / rms(x)) * g
  float epsilon = 1e-5f;
  for (int n = 0; n < N; n++) {
    double sum_sq = 0.0;
    for (int k = 0; k < K; k++) sum_sq += (double)h_x[n * K + k] * (double)h_x[n * K + k];
    float rms = sqrtf((float)sum_sq / (float)K + epsilon);
    for (int k = 0; k < K; k++)
      h_y_ref[n * K + k] = (h_x[n * K + k] / rms) * g;
  }

  float *d_x, *d_y;
  check(cudaMalloc(&d_x, (size_t)N * K * sizeof(float)), "rmsnorm alloc X");
  check(cudaMalloc(&d_y, (size_t)N * K * sizeof(float)), "rmsnorm alloc Y");
  check(cudaMemcpy(d_x, h_x, (size_t)N * K * sizeof(float), cudaMemcpyHostToDevice), "rmsnorm H2D X");

  dim3 block(128);
  dim3 grid(N);
  rms_norm<<<grid, block>>>(d_x, d_y, g, N, K);
  check(cudaGetLastError(), "rmsnorm launch");
  check(cudaDeviceSynchronize(), "rmsnorm sync");

  float *h_y = (float *)malloc((size_t)N * K * sizeof(float));
  check(cudaMemcpy(h_y, d_y, (size_t)N * K * sizeof(float), cudaMemcpyDeviceToHost), "rmsnorm D2H");

  float max_err = 0.0f;
  for (int i = 0; i < N * K; i++) {
    float err = fabsf(h_y[i] - h_y_ref[i]);
    if (err > max_err) max_err = err;
  }
  printf("| %-42s | %.6e | %-4s | %-8s |\n", "RMSNorm", max_err, max_err < 1e-4f ? "PASS" : "FAIL", "None");

  // ---- RMS Norm Vec4 ----
  dim3 block_rv4(32);
  rms_norm_vec4<<<grid, block_rv4>>>(d_x, d_y, g, N, K);
  check(cudaGetLastError(), "rmsnorm_vec4 launch");
  check(cudaDeviceSynchronize(), "rmsnorm_vec4 sync");
  check(cudaMemcpy(h_y, d_y, (size_t)N * K * sizeof(float), cudaMemcpyDeviceToHost), "rmsnorm_vec4 D2H");
  max_err = 0.0f;
  for (int i = 0; i < N * K; i++) {
    float err = fabsf(h_y[i] - h_y_ref[i]);
    if (err > max_err) max_err = err;
```

```
  }
  printf("| %-42s | %.6e | %-4s | %-8s |\n", "RMSNorm-Vec4", max_err, max_err < 1e-4f ? "PASS" : "FAIL", "
    None");

  free(h_x); free(h_y); free(h_y_ref);
  cudaFree(d_x); cudaFree(d_y);
}


static void test_layer_norm(int N, int K) {

  srand(42);
  float *h_x = (float *)malloc((size_t)N * K * sizeof(float));
  float *h_y_ref = (float *)malloc((size_t)N * K * sizeof(float));
  for (int i = 0; i < N * K; i++)
    h_x[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
  float g = 1.5f, b = 0.3f;  // gain and bias

  // CPU reference: y = ((x - mean) / std) * g + b
  float epsilon = 1e-5f;
  for (int n = 0; n < N; n++) {
    double sum = 0.0;
    for (int k = 0; k < K; k++) sum += (double)h_x[n * K + k];
    float mean = (float)sum / (float)K;
    double sum_sq = 0.0;
    for (int k = 0; k < K; k++) {
      float diff = h_x[n * K + k] - mean;
      sum_sq += (double)diff * (double)diff;
    }
    float std = sqrtf((float)sum_sq / (float)K + epsilon);
    for (int k = 0; k < K; k++)
      h_y_ref[n * K + k] = ((h_x[n * K + k] - mean) / std) * g + b;
  }

  float *d_x, *d_y;
  check(cudaMalloc(&d_x, (size_t)N * K * sizeof(float)), "layernorm alloc X");
  check(cudaMalloc(&d_y, (size_t)N * K * sizeof(float)), "layernorm alloc Y");
  check(cudaMemcpy(d_x, h_x, (size_t)N * K * sizeof(float), cudaMemcpyHostToDevice), "layernorm H2D X");

  dim3 block(128);
  dim3 grid(N);
  layer_norm<<<grid, block>>>(d_x, d_y, g, b, N, K);
  check(cudaGetLastError(), "layernorm launch");
  check(cudaDeviceSynchronize(), "layernorm sync");

  float *h_y = (float *)malloc((size_t)N * K * sizeof(float));
  check(cudaMemcpy(h_y, d_y, (size_t)N * K * sizeof(float), cudaMemcpyDeviceToHost), "layernorm D2H");

  float max_err = 0.0f;
  for (int i = 0; i < N * K; i++) {
    float err = fabsf(h_y[i] - h_y_ref[i]);
    if (err > max_err) max_err = err;
  }
  printf("| %-42s | %.6e | %-4s | %-8s |\n", "LayerNorm", max_err, max_err < 1e-4f ? "PASS" : "FAIL", "None"
    );

  // ---- Layer Norm Vec4 ----
  dim3 block_lv4(32);
  layer_norm_vec4<<<grid, block_lv4>>>(d_x, d_y, g, b, N, K);
  check(cudaGetLastError(), "layernorm_vec4 launch");
  check(cudaDeviceSynchronize(), "layernorm_vec4 sync");
  check(cudaMemcpy(h_y, d_y, (size_t)N * K * sizeof(float), cudaMemcpyDeviceToHost), "layernorm_vec4 D2H");
  max_err = 0.0f;
```

```
  for (int i = 0; i < N * K; i++) {
    float err = fabsf(h_y[i] - h_y_ref[i]);
    if (err > max_err) max_err = err;
  }
  printf("| %-42s | %.6e | %-4s | %-8s |\n", "LayerNorm-Vec4", max_err, max_err < 1e-4f ? "PASS" : "FAIL", "
    None");

  free(h_x); free(h_y); free(h_y_ref);
  cudaFree(d_x); cudaFree(d_y);
}


static void test_rope(int seq_len, int N) {

  int total_pairs = seq_len * N;
  int total_elems = total_pairs * 2;
  size_t size = (size_t)total_elems * sizeof(float);

  float *h_x = (float *)malloc(size);
  float *h_y_ref = (float *)malloc(size);

  srand(42);
  for (int i = 0; i < total_elems; i++)
    h_x[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;

  // CPU reference: 2D rotation for each pair
  for (int idx = 0; idx < total_pairs; idx++) {
    int token_pos = idx / N;
    int token_idx = idx % N;
    float x1 = h_x[idx * 2];
    float x2 = h_x[idx * 2 + 1];
    float theta = 1.0f / powf(10000.0f, 2.0f * token_idx / (N * 2.0f));
    float angle = (float)token_pos * theta;
    float cos_v = cosf(angle);
    float sin_v = sinf(angle);
    h_y_ref[idx * 2] = x1 * cos_v - x2 * sin_v;
    h_y_ref[idx * 2 + 1] = x1 * sin_v + x2 * cos_v;
  }

  float *d_x, *d_y;
  check(cudaMalloc(&d_x, size), "rope alloc X");
  check(cudaMalloc(&d_y, size), "rope alloc Y");
  check(cudaMemcpy(d_x, h_x, size, cudaMemcpyHostToDevice), "rope H2D");

  dim3 block(256);
  dim3 grid((total_pairs + 255) / 256);
  rope<<<grid, block>>>(d_x, d_y, seq_len, N);
  check(cudaGetLastError(), "rope launch");
  check(cudaDeviceSynchronize(), "rope sync");

  float *h_y = (float *)malloc(size);
  check(cudaMemcpy(h_y, d_y, size, cudaMemcpyDeviceToHost), "rope D2H");

  float max_err = 0.0f;
  for (int i = 0; i < total_elems; i++) {
    float err = fabsf(h_y[i] - h_y_ref[i]);
    if (err > max_err) max_err = err;
  }
  printf("| %-42s | %.6e | %-4s | %-8s |\n", "RoPE", max_err, max_err < 1e-4f ? "PASS" : "FAIL", "None");

  free(h_x);
  free(h_y);
  free(h_y_ref);
```

```
  cudaFree(d_x);
  cudaFree(d_y);
}


static void test_mat_transpose(int row, int col) {

  size_t size_in = (size_t)row * col * sizeof(float);
  size_t size_out = (size_t)col * row * sizeof(float);

  float *h_x = (float *)malloc(size_in);
  float *h_y_ref = (float *)malloc(size_out);

  srand(42);
  for (int i = 0; i < row * col; i++)
    h_x[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;

  // CPU reference: y[j][i] = x[i][j] (row-major)
  for (int i = 0; i < row; i++)
    for (int j = 0; j < col; j++)
      h_y_ref[j * row + i] = h_x[i * col + j];

  float *d_x, *d_y;
  check(cudaMalloc(&d_x, size_in), "mattrans alloc X");
  check(cudaMalloc(&d_y, size_out), "mattrans alloc Y");
  check(cudaMemcpy(d_x, h_x, size_in, cudaMemcpyHostToDevice), "mattrans H2D");

  dim3 block(16, 16);
  dim3 grid((col + 15) / 16, (row + 15) / 16);
  mat_transpose<<<grid, block>>>(d_x, d_y, row, col);
  check(cudaGetLastError(), "mattrans launch");
  check(cudaDeviceSynchronize(), "mattrans sync");

  float *h_y = (float *)malloc(size_out);
  check(cudaMemcpy(h_y, d_y, size_out, cudaMemcpyDeviceToHost), "mattrans D2H");

  float max_err = 0.0f;
  for (int i = 0; i < col * row; i++) {
    float err = fabsf(h_y[i] - h_y_ref[i]);
    if (err > max_err) max_err = err;
  }
  printf("| %-42s | %.6e | %-4s | %-8s |\n", "MatTranspose", max_err, max_err < 1e-6f ? "PASS" : "FAIL", "
    None");

  free(h_x);
  free(h_y);
  free(h_y_ref);
  cudaFree(d_x);
  cudaFree(d_y);
}


static void test_mat_transpose_padded(int row, int col) {

  size_t size_in = (size_t)row * col * sizeof(float);
  size_t size_out = (size_t)col * row * sizeof(float);

  float *h_x = (float *)malloc(size_in);
  float *h_y_ref = (float *)malloc(size_out);

  srand(42);
  for (int i = 0; i < row * col; i++)
    h_x[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
```

```
5092
5093    // CPU reference: y[j][i] = x[i][j] (row-major)
5094    for (int i = 0; i < row; i++)
5095      for (int j = 0; j < col; j++)
5096        h_y_ref[j * row + i] = h_x[i * col + j];
5097
5098    float *d_x, *d_y;
5099    check(cudaMalloc(&d_x, size_in), "mattrans_padded alloc X");
5100    check(cudaMalloc(&d_y, size_out), "mattrans_padded alloc Y");
5101    check(cudaMemcpy(d_x, h_x, size_in, cudaMemcpyHostToDevice), "mattrans_padded H2D");
5102
5103    dim3 block(16, 16);
5104    dim3 grid((col + 15) / 16, (row + 63) / 64);
5105    mat_transpose_padded<<<grid, block>>>(d_x, d_y, row, col);
5106    check(cudaGetLastError(), "mattrans_padded launch");
5107    check(cudaDeviceSynchronize(), "mattrans_padded sync");
5108
5109    float *h_y = (float *)malloc(size_out);
5110    check(cudaMemcpy(h_y, d_y, size_out, cudaMemcpyDeviceToHost), "mattrans_padded D2H");
5111
5112    float max_err = 0.0f;
5113    for (int i = 0; i < col * row; i++) {
5114      float err = fabsf(h_y[i] - h_y_ref[i]);
5115      if (err > max_err) max_err = err;
5116    }
5117    printf("| %-42s | %.6e | %-4s | %-8s |\n", "MatTransposePadded", max_err, max_err < 1e-6f ? "PASS" : "FAIL
       ", "None");
5118
5119    free(h_x);
5120    free(h_y);
5121    free(h_y_ref);
5122    cudaFree(d_x);
5123    cudaFree(d_y);
5124  }
5125
5126
5127  static void test_sgemv(int M, int K) {
5128
5129    srand(42);
5130    float *h_a = (float *)malloc((size_t)M * K * sizeof(float));
5131    float *h_x = (float *)malloc((size_t)K * sizeof(float));
5132    float *h_y_ref = (float *)malloc((size_t)M * sizeof(float));
5133    for (int i = 0; i < M * K; i++) h_a[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
5134    for (int i = 0; i < K; i++) h_x[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
5135
5136    // CPU reference: y[m] = sum_k A[m,k] * x[k]
5137    for (int m = 0; m < M; m++) {
5138      double sum = 0.0;
5139      for (int k = 0; k < K; k++) sum += (double)h_a[m * K + k] * (double)h_x[k];
5140      h_y_ref[m] = (float)sum;
5141    }
5142
5143    float *d_a, *d_x, *d_y;
5144    check(cudaMalloc(&d_a, (size_t)M * K * sizeof(float)), "sgemv alloc A");
5145    check(cudaMalloc(&d_x, (size_t)K * sizeof(float)), "sgemv alloc X");
5146    check(cudaMalloc(&d_y, (size_t)M * sizeof(float)), "sgemv alloc Y");
5147
5148    check(cudaMemcpy(d_a, h_a, (size_t)M * K * sizeof(float), cudaMemcpyHostToDevice), "sgemv H2D A");
5149    check(cudaMemcpy(d_x, h_x, (size_t)K * sizeof(float), cudaMemcpyHostToDevice), "sgemv H2D X");
5150
5151    dim3 block(32, 4);
5152    dim3 grid((M + 3) / 4);
5153    sgemv_k128<<<grid, block>>>(d_a, d_x, d_y, M, K);
```

```
  check(cudaGetLastError(), "sgemv launch");
  check(cudaDeviceSynchronize(), "sgemv sync");

  float *h_y = (float *)malloc((size_t)M * sizeof(float));
  check(cudaMemcpy(h_y, d_y, (size_t)M * sizeof(float), cudaMemcpyDeviceToHost), "sgemv D2H");

  float max_err = 0.0f;
  for (int m = 0; m < M; m++) {
    float err = fabsf(h_y[m] - h_y_ref[m]);
    if (err > max_err) max_err = err;
  }
  printf("| %-42s | %.6e | %-4s | %-8s |\n", "SGEMV-K128", max_err, max_err < 1e-2f ? "PASS" : "FAIL", "None
    ");

  // ---- SGEMV K32 ----
  check(cudaMemset(d_y, 0, M * sizeof(float)), "sgemv_k32 zero Y");
  sgemv_k32<<<grid, block>>>(d_a, d_x, d_y, M, K);
  check(cudaGetLastError(), "sgemv_k32 launch");
  check(cudaDeviceSynchronize(), "sgemv_k32 sync");

  check(cudaMemcpy(h_y, d_y, (size_t)M * sizeof(float), cudaMemcpyDeviceToHost), "sgemv_k32 D2H");

  max_err = 0.0f;
  for (int m = 0; m < M; m++) {
    float err = fabsf(h_y[m] - h_y_ref[m]);
    if (err > max_err) max_err = err;
  }
  printf("| %-42s | %.6e | %-4s | %-8s |\n", "SGEMV-K32", max_err, max_err < 1e-2f ? "PASS" : "FAIL", "None"
    );

  // ---- SGEMV K16 ----
  free(h_a); free(h_x); free(h_y); free(h_y_ref);
  cudaFree(d_a); cudaFree(d_x); cudaFree(d_y);

  int K16 = 16;
  h_a = (float *)malloc((size_t)M * K16 * sizeof(float));
  h_x = (float *)malloc((size_t)K16 * sizeof(float));
  h_y_ref = (float *)malloc((size_t)M * sizeof(float));
  for (int i = 0; i < M * K16; i++) h_a[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
  for (int i = 0; i < K16; i++) h_x[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;

  for (int m = 0; m < M; m++) {
    double sum = 0.0;
    for (int k = 0; k < K16; k++) sum += (double)h_a[m * K16 + k] * (double)h_x[k];
    h_y_ref[m] = (float)sum;
  }

  check(cudaMalloc(&d_a, (size_t)M * K16 * sizeof(float)), "sgemv_k16 alloc A");
  check(cudaMalloc(&d_x, (size_t)K16 * sizeof(float)), "sgemv_k16 alloc X");
  check(cudaMalloc(&d_y, (size_t)M * sizeof(float)), "sgemv_k16 alloc Y");

  check(cudaMemcpy(d_a, h_a, (size_t)M * K16 * sizeof(float), cudaMemcpyHostToDevice), "sgemv_k16 H2D A");
  check(cudaMemcpy(d_x, h_x, (size_t)K16 * sizeof(float), cudaMemcpyHostToDevice), "sgemv_k16 H2D X");

  dim3 grid_k16((M + 7) / 8);
  sgemv_k16<2><<<grid_k16, block>>>(d_a, d_x, d_y, M, K16);
  check(cudaGetLastError(), "sgemv_k16 launch");
  check(cudaDeviceSynchronize(), "sgemv_k16 sync");

  h_y = (float *)malloc((size_t)M * sizeof(float));
  check(cudaMemcpy(h_y, d_y, (size_t)M * sizeof(float), cudaMemcpyDeviceToHost), "sgemv_k16 D2H");

  max_err = 0.0f;
```

```
  for (int m = 0; m < M; m++) {
    float err = fabsf(h_y[m] - h_y_ref[m]);
    if (err > max_err) max_err = err;
  }
  printf("| %-42s | %.6e | %-4s | %-8s |\n", "SGEMV-K16", max_err, max_err < 1e-2f ? "PASS" : "FAIL", "None"
    );

  free(h_a); free(h_x); free(h_y); free(h_y_ref);
  cudaFree(d_a); cudaFree(d_x); cudaFree(d_y);
}


static void test_sgemm(int M, int N, int K) {

  size_t size_a = (size_t)M * K * sizeof(float);
  size_t size_b = (size_t)K * N * sizeof(float);
  size_t size_c = (size_t)M * N * sizeof(float);

  float *h_a = (float *)malloc(size_a);
  float *h_b = (float *)malloc(size_b);
  float *h_c_ref = (float *)malloc(size_c);

  srand(42);
  for (int i = 0; i < M * K; i++) h_a[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
  for (int i = 0; i < K * N; i++) h_b[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;

  float *d_a, *d_b, *d_c;
  check(cudaMalloc(&d_a, size_a), "sgemm alloc A");
  check(cudaMalloc(&d_b, size_b), "sgemm alloc B");
  check(cudaMalloc(&d_c, size_c), "sgemm alloc C");

  check(cudaMemcpy(d_a, h_a, size_a, cudaMemcpyHostToDevice), "sgemm H2D A");
  check(cudaMemcpy(d_b, h_b, size_b, cudaMemcpyHostToDevice), "sgemm H2D B");

  // cuBLAS reference (row-major idiom: swap M/N, swap A/B)
  cublasHandle_t handle;
  cublasCreate(&handle);
  float alpha = 1.0f, beta = 0.0f;
  cublasSgemm(handle, CUBLAS_OP_N, CUBLAS_OP_N, N, M, K,
              &alpha, d_b, N, d_a, K, &beta, d_c, N);
  check(cudaMemcpy(h_c_ref, d_c, size_c, cudaMemcpyDeviceToHost), "sgemm D2H ref");

  // Kernel
  cudaEvent_t start, stop;
  cudaEventCreate(&start);
  cudaEventCreate(&stop);

  dim3 block(32, 32);
  dim3 grid((N + 31) / 32, (M + 31) / 32);
  sgemm<<<grid, block>>>(d_a, d_b, d_c, M, N, K);
  check(cudaGetLastError(), "sgemm launch");
  check(cudaDeviceSynchronize(), "sgemm sync");

  float *h_c = (float *)malloc(size_c);
  check(cudaMemcpy(h_c, d_c, size_c, cudaMemcpyDeviceToHost), "sgemm D2H");

  // Verify
  float max_err = 0.0f;
  for (int i = 0; i < M * N; i++) {
    float err = fabsf(h_c[i] - h_c_ref[i]);
    if (err > max_err) max_err = err;
  }
  printf("| %-42s | %.6e | %-4s | %-8s |\n", "SGEMM", max_err, max_err < 1e-2f ? "PASS" : "FAIL", "None");
```

```
5277
5278    // ---- SGEMM Vec4 (128×128 tile, 4×4 thread tile) ----
5279
5280    dim3 grid_vec4((N + 127) / 128, (M + 127) / 128);
5281    check(cudaMemset(d_c, 0, size_c), "sgemm_vec4 zero C");
5282    sgemm_vec4<<<grid_vec4, block>>>(d_a, d_b, d_c, M, N, K);
5283    check(cudaGetLastError(), "sgemm_vec4 launch");
5284    check(cudaDeviceSynchronize(), "sgemm_vec4 sync");
5285
5286    check(cudaMemcpy(h_c, d_c, size_c, cudaMemcpyDeviceToHost), "sgemm_vec4 D2H");
5287
5288    max_err = 0.0f;
5289    for (int i = 0; i < M * N; i++) {
5290      float err = fabsf(h_c[i] - h_c_ref[i]);
5291      if (err > max_err) max_err = err;
5292    }
5293    printf("| %-42s | %.6e | %-4s | %-8s |\n", "SGEMM-Vec4", max_err, max_err < 1e-2f ? "PASS" : "FAIL", "None
       ");
5294
5295    free(h_a); free(h_b); free(h_c); free(h_c_ref);
5296    cudaFree(d_a); cudaFree(d_b); cudaFree(d_c);
5297    cublasDestroy(handle);
5298    cudaEventDestroy(start);
5299    cudaEventDestroy(stop);
5300  }
5301
5302
5303  static void test_hgemm_mma(int M, int N, int K) {
5304
5305    size_t size_a = (size_t)M * K * sizeof(half);
5306    size_t size_b = (size_t)K * N * sizeof(half);
5307    size_t size_c = (size_t)M * N * sizeof(half);
5308
5309    half *h_a = (half *)malloc(size_a);
5310    half *h_b = (half *)malloc(size_b);
5311    half *h_c_ref = (half *)malloc(size_c);
5312
5313    srand(42);
5314    for (int i = 0; i < M * K; i++) h_a[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
5315    for (int i = 0; i < K * N; i++) h_b[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
5316
5317    // Kernel expects B^T [N×K] row-major layout (TN layout convention).
5318    // We store h_b as B [K×N] row-major for cuBLAS, then create B^T for kernel.
5319    size_t size_b_t = (size_t)N * K * sizeof(half);
5320    half *h_b_t = (half *)malloc(size_b_t);
5321    for (int n = 0; n < N; n++)
5322      for (int k = 0; k < K; k++)
5323        h_b_t[n * K + k] = h_b[k * N + n];
5324
5325    half *d_a, *d_b, *d_b_t, *d_c;
5326    check(cudaMalloc(&d_a, size_a), "hgemm alloc A");
5327    check(cudaMalloc(&d_b, size_b), "hgemm alloc B (cuBLAS)");
5328    check(cudaMalloc(&d_b_t, size_b_t), "hgemm alloc B_t (kernel)");
5329    check(cudaMalloc(&d_c, size_c), "hgemm alloc C");
5330
5331    check(cudaMemcpy(d_a, h_a, size_a, cudaMemcpyHostToDevice), "hgemm H2D A");
5332    check(cudaMemcpy(d_b, h_b, size_b, cudaMemcpyHostToDevice), "hgemm H2D B (cuBLAS)");
5333    check(cudaMemcpy(d_b_t, h_b_t, size_b_t, cudaMemcpyHostToDevice), "hgemm H2D B_t (kernel)");
5334
5335    // cuBLAS FP16 reference (row-major idiom: swap M/N, swap A/B)
5336    // Note: use CUBLAS_COMPUTE_16F to match the kernel's f16.f16.f16.f16 accumulation.
5337    cublasHandle_t handle;
5338    cublasCreate(&handle);
```

```
  half alpha_h = __float2half(1.0f), beta_h = __float2half(0.0f);
  cublasGemmEx(handle, CUBLAS_OP_N, CUBLAS_OP_N, N, M, K,
               &alpha_h, d_b, CUDA_R_16F, N, d_a, CUDA_R_16F, K,
               &beta_h, d_c, CUDA_R_16F, N,
               CUBLAS_COMPUTE_16F, CUBLAS_GEMM_DEFAULT);
  check(cudaMemcpy(h_c_ref, d_c, size_c, cudaMemcpyDeviceToHost), "hgemm D2H ref");

  // MMA kernel (TN layout: A row-major, B^T row-major = B col-major)
  cudaEvent_t start, stop;
  cudaEventCreate(&start);
  cudaEventCreate(&stop);

  constexpr int BM = 128, BN = 128, BK = 16, kStages = 3;
  size_t smem_bytes = kStages * (BM * BK + BN * BK) * sizeof(half); // 24576
  dim3 block(256);
  dim3 grid((N + BN - 1) / BN, (M + BM - 1) / BM);
  hgemm_mma_stages_tn<<<grid, block, smem_bytes>>>(d_a, d_b_t, d_c, M, N, K);
  check(cudaGetLastError(), "hgemm launch");
  check(cudaDeviceSynchronize(), "hgemm sync");

  half *h_c = (half *)malloc(size_c);
  check(cudaMemcpy(h_c, d_c, size_c, cudaMemcpyDeviceToHost), "hgemm D2H");

  // Verify
  float max_err = 0.0f;
  for (int i = 0; i < M * N; i++) {
    float err = fabsf(__half2float(h_c[i]) - __half2float(h_c_ref[i]));
    if (err > max_err) max_err = err;
  }
  printf("| %-42s | %.6e | %-4s | %-8s |\n", "HGEMM MMA", max_err, max_err < 1.0f ? "PASS" : "FAIL", "None")
    ;

  free(h_a); free(h_b); free(h_b_t); free(h_c); free(h_c_ref);
  cudaFree(d_a); cudaFree(d_b); cudaFree(d_b_t); cudaFree(d_c);
  cublasDestroy(handle);
  cudaEventDestroy(start);
  cudaEventDestroy(stop);
}


static void test_hgemm_swizzle(int M, int N, int K) {
  // HGEMM MMA Swizzle — m16n8k16 + multistage pipeline + TN 布局 + XOR swizzle
  //   + Register Double Buffering (kValTileK=2, BK=32)
  // TN layout: C[M×N] = A[M×K] × B^T[N×K]
  // Kernel: hgemm_mma_stages_tn_swizzle with default template params
  //   (kValTileK=2, kStages=3, BK=32)
  // smem: kStages × (BM×BK + BN×BK) halfs

  size_t size_a = (size_t)M * K * sizeof(half);
  size_t size_b = (size_t)K * N * sizeof(half);
  size_t size_c = (size_t)M * N * sizeof(half);

  half *h_a = (half *)malloc(size_a);
  half *h_b = (half *)malloc(size_b);
  half *h_c_ref = (half *)malloc(size_c);

  srand(42);
  for (int i = 0; i < M * K; i++) h_a[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
  for (int i = 0; i < K * N; i++) h_b[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);

  // Kernel expects B^T [N×K] row-major layout (TN layout convention).
  size_t size_b_t = (size_t)N * K * sizeof(half);
  half *h_b_t = (half *)malloc(size_b_t);
```

```
  for (int n = 0; n < N; n++)
    for (int k = 0; k < K; k++)
      h_b_t[n * K + k] = h_b[k * N + n];

  half *d_a, *d_b, *d_b_t, *d_c;
  check(cudaMalloc(&d_a, size_a), "hgemm_swizzle alloc A");
  check(cudaMalloc(&d_b, size_b), "hgemm_swizzle alloc B (cuBLAS)");
  check(cudaMalloc(&d_b_t, size_b_t), "hgemm_swizzle alloc B_t (kernel)");
  check(cudaMalloc(&d_c, size_c), "hgemm_swizzle alloc C");

  check(cudaMemcpy(d_a, h_a, size_a, cudaMemcpyHostToDevice), "hgemm_swizzle H2D A");
  check(cudaMemcpy(d_b, h_b, size_b, cudaMemcpyHostToDevice), "hgemm_swizzle H2D B (cuBLAS)");
  check(cudaMemcpy(d_b_t, h_b_t, size_b_t, cudaMemcpyHostToDevice), "hgemm_swizzle H2D B_t (kernel)");

  // cuBLAS FP16 reference
  cublasHandle_t handle;
  cublasCreate(&handle);
  half alpha_h = __float2half(1.0f), beta_h = __float2half(0.0f);
  cublasGemmEx(handle, CUBLAS_OP_N, CUBLAS_OP_N, N, M, K,
               &alpha_h, d_b, CUDA_R_16F, N, d_a, CUDA_R_16F, K,
               &beta_h, d_c, CUDA_R_16F, N,
               CUBLAS_COMPUTE_16F, CUBLAS_GEMM_DEFAULT);
  check(cudaMemcpy(h_c_ref, d_c, size_c, cudaMemcpyDeviceToHost), "hgemm_swizzle D2H ref");

  // MMA swizzle kernel (default params: kStages=3, kValTileK=2, BK=32)
  constexpr int BM = 128, BN = 128, BK = 32, K_STAGE_S = 3;
  size_t smem_bytes = K_STAGE_S * (BM * BK + BN * BK) * sizeof(half);
  dim3 block(256);
  dim3 grid((N + BN - 1) / BN, (M + BM - 1) / BM);
  hgemm_mma_stages_tn_swizzle<<<grid, block, smem_bytes>>>(d_a, d_b_t, d_c, M, N, K);
  check(cudaGetLastError(), "hgemm_swizzle launch");
  check(cudaDeviceSynchronize(), "hgemm_swizzle sync");

  half *h_c = (half *)malloc(size_c);
  check(cudaMemcpy(h_c, d_c, size_c, cudaMemcpyDeviceToHost), "hgemm_swizzle D2H");

  // Verify
  float max_err = 0.0f;
  for (int i = 0; i < M * N; i++) {
    float err = fabsf(__half2float(h_c[i]) - __half2float(h_c_ref[i]));
    if (err > max_err) max_err = err;
  }
  printf("| %-42s | %.6e | %-4s | %-8s |\n", "HGEMM Swizzle + Reg2x", max_err, max_err < 1.0f ? "PASS" : "
    FAIL", "None");

  free(h_a); free(h_b); free(h_b_t); free(h_c); free(h_c_ref);
  cudaFree(d_a); cudaFree(d_b); cudaFree(d_b_t); cudaFree(d_c);
  cublasDestroy(handle);
}


#if defined(NOTES_V2_ENABLE_CUTE)
static void test_hgemm_cute(int M, int N, int K) {
  // HGEMM CuTe — SM80_16x8x16_F16F16F16F16_TN + Swizzle<3,3,3> + kStage=2
  // TN layout: C[M×N] = A[M×K] × B^T[N×K]
  // Kernel: hgemm_mma_stages_tn_cute via launch_hgemm_mma_stages_tn_cute
  // Tile: BM=128, BN=256, BK=32, 128 threads/block

  size_t size_a = (size_t)M * K * sizeof(half);
  size_t size_b = (size_t)K * N * sizeof(half);
  size_t size_c = (size_t)M * N * sizeof(half);

  half *h_a = (half *)malloc(size_a);
```

```
  half *h_b = (half *)malloc(size_b);
  half *h_c_ref = (half *)malloc(size_c);

  srand(42);
  for (int i = 0; i < M * K; i++)
    h_a[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
  for (int i = 0; i < K * N; i++)
    h_b[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);

  // CuTe kernel expects B^T [N×K] row-major (TN layout).
  size_t size_b_t = (size_t)N * K * sizeof(half);
  half *h_b_t = (half *)malloc(size_b_t);
  for (int n = 0; n < N; n++)
    for (int k = 0; k < K; k++)
      h_b_t[n * K + k] = h_b[k * N + n];

  half *d_a, *d_b, *d_b_t, *d_c;
  check(cudaMalloc(&d_a, size_a), "hgemm_cute alloc A");
  check(cudaMalloc(&d_b, size_b), "hgemm_cute alloc B (cuBLAS)");
  check(cudaMalloc(&d_b_t, size_b_t), "hgemm_cute alloc B_t (kernel)");
  check(cudaMalloc(&d_c, size_c), "hgemm_cute alloc C");

  check(cudaMemcpy(d_a, h_a, size_a, cudaMemcpyHostToDevice), "hgemm_cute H2D A");
  check(cudaMemcpy(d_b, h_b, size_b, cudaMemcpyHostToDevice), "hgemm_cute H2D B (cuBLAS)");
  check(cudaMemcpy(d_b_t, h_b_t, size_b_t, cudaMemcpyHostToDevice), "hgemm_cute H2D B_t (kernel)");

  // cuBLAS FP16 reference (row-major idiom: swap M/N, swap A/B)
  cublasHandle_t handle;
  cublasCreate(&handle);
  half alpha_h = __float2half(1.0f), beta_h = __float2half(0.0f);
  cublasGemmEx(handle, CUBLAS_OP_N, CUBLAS_OP_N, N, M, K, &alpha_h, d_b,
               CUDA_R_16F, N, d_a, CUDA_R_16F, K, &beta_h, d_c, CUDA_R_16F, N,
               CUBLAS_COMPUTE_16F, CUBLAS_GEMM_DEFAULT);
  check(cudaMemcpy(h_c_ref, d_c, size_c, cudaMemcpyDeviceToHost), "hgemm_cute D2H ref");

  // CuTe kernel (Stages=2, TN layout)
  launch_hgemm_mma_stages_tn_cute<half, 2, 0>(d_a, d_b_t, d_c, M, N, K);
  check(cudaGetLastError(), "hgemm_cute launch");
  check(cudaDeviceSynchronize(), "hgemm_cute sync");

  half *h_c = (half *)malloc(size_c);
  check(cudaMemcpy(h_c, d_c, size_c, cudaMemcpyDeviceToHost), "hgemm_cute D2H");

  // Verify
  float max_err = 0.0f;
  for (int i = 0; i < M * N; i++) {
    float err = fabsf(__half2float(h_c[i]) - __half2float(h_c_ref[i]));
    if (err > max_err) max_err = err;
  }
  printf("| %-42s | %.6e | %-4s | %-8s |\n", "HGEMM CuTe Swizzle + Reg2x",
         max_err, max_err < 1.0f ? "PASS" : "FAIL", "None");

  free(h_a); free(h_b); free(h_b_t); free(h_c); free(h_c_ref);
  cudaFree(d_a); cudaFree(d_b); cudaFree(d_b_t); cudaFree(d_c);
  cublasDestroy(handle);
}
#endif /* NOTES_V2_ENABLE_CUTE */


#if defined(NOTES_V2_ENABLE_WGMMA)
static void test_hgemm_wgmma(int M, int N, int K) {
  // HGEMM WGMMA — m64n128k16 + TMA + Warp Specialization (Hopper SM90+)
  // TN layout: C[M×N] = A[M×K] × B^T[N×K]
```

```
  // Kernel: hgemm_wgmma_stages_tn with default template params

  constexpr int BM = 128, BN = 128, BK = 64, kStages = 3, kNumThreads = 256;

  // M, K must be divisible by tile dims
  if (M % BM != 0 || N % BN != 0 || K % BK != 0) {
    printf("| %-42s | %-12s | %-4s | %-8s |\n",
           "HGEMM WGMMA", "SKIP", "SKIP", "None");
    return;
  }

  size_t size_a = (size_t)M * K * sizeof(half);
  size_t size_b = (size_t)K * N * sizeof(half);
  size_t size_c = (size_t)M * N * sizeof(half);

  half *h_a = (half *)malloc(size_a);
  half *h_b = (half *)malloc(size_b);
  half *h_c_ref = (half *)malloc(size_c);

  srand(42);
  for (int i = 0; i < M * K; i++)
    h_a[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
  for (int i = 0; i < K * N; i++)
    h_b[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);

  // B^T [N×K] row-major for TN layout (same as hgemm_mma kernel)
  size_t size_b_t = (size_t)N * K * sizeof(half);
  half *h_b_t = (half *)malloc(size_b_t);
  for (int n = 0; n < N; n++)
    for (int k = 0; k < K; k++)
      h_b_t[n * K + k] = h_b[k * N + n];

  half *d_a, *d_b, *d_b_t, *d_c;
  check(cudaMalloc(&d_a, size_a), "wgmma alloc A");
  check(cudaMalloc(&d_b, size_b), "wgmma alloc B (cuBLAS)");
  check(cudaMalloc(&d_b_t, size_b_t), "wgmma alloc B_t (kernel)");
  check(cudaMalloc(&d_c, size_c), "wgmma alloc C");

  check(cudaMemcpy(d_a, h_a, size_a, cudaMemcpyHostToDevice), "wgmma H2D A");
  check(cudaMemcpy(d_b, h_b, size_b, cudaMemcpyHostToDevice),
        "wgmma H2D B (cuBLAS)");
  check(cudaMemcpy(d_b_t, h_b_t, size_b_t, cudaMemcpyHostToDevice),
        "wgmma H2D B_t (kernel)");

  // cuBLAS FP16 reference (row-major idiom, same as hgemm_mma)
  cublasHandle_t handle;
  cublasCreate(&handle);
  half alpha_h = __float2half(1.0f), beta_h = __float2half(0.0f);
  cublasGemmEx(handle, CUBLAS_OP_N, CUBLAS_OP_N, N, M, K, &alpha_h, d_b,
               CUDA_R_16F, N, d_a, CUDA_R_16F, K, &beta_h, d_c, CUDA_R_16F, N,
               CUBLAS_COMPUTE_16F, CUBLAS_GEMM_DEFAULT);
  check(cudaMemcpy(h_c_ref, d_c, size_c, cudaMemcpyDeviceToHost),
        "wgmma D2H ref");

  // Create TMA tensor maps for A and B^T
  // A[M×K] row-major: TMA box=(BK=64, BM=128), global shape=(K, M)
  // B^T[N×K] row-major: TMA box=(BK=64, BN=128), global shape=(K, N)
  CUtensorMap *tma_a =
      allocate_and_create_tensor_map(d_a, M / BM, K / BK);
  CUtensorMap *tma_b =
      allocate_and_create_tensor_map(d_b_t, N / BN, K / BK);

  // Launch WGMMA kernel
```

```
  // kBlockSwizzle=false → 2D grid, no swizzle
  size_t smem_bytes =
      kStages * (BM * BK + BN * BK) * sizeof(half); // 3*16384*2 = 96KB
  cudaFuncSetAttribute(
      hgemm_wgmma_stages_tn<64, 128, 16, BM, BN, BK, kNumThreads, kStages,
                            false>,
      cudaFuncAttributeMaxDynamicSharedMemorySize, smem_bytes);

  dim3 block(kNumThreads);
  dim3 grid((N + BN - 1) / BN, (M + BM - 1) / BM);
  hgemm_wgmma_stages_tn<64, 128, 16, BM, BN, BK, kNumThreads, kStages, false>
      <<<grid, block, smem_bytes>>>(M, N, K, d_c, tma_a, tma_b);
  check(cudaGetLastError(), "wgmma launch");
  check(cudaDeviceSynchronize(), "wgmma sync");

  half *h_c = (half *)malloc(size_c);
  check(cudaMemcpy(h_c, d_c, size_c, cudaMemcpyDeviceToHost), "wgmma D2H");

  // Verify
  float max_err = 0.0f;
  for (int i = 0; i < M * N; i++) {
    float err = fabsf(__half2float(h_c[i]) - __half2float(h_c_ref[i]));
    if (err > max_err)
      max_err = err;
  }
  printf("| %-42s | %.6e | %-4s | %-8s |\n", "HGEMM TMA WGMMA WS (3-stage)", max_err,
         max_err < 1.0f ? "PASS" : "FAIL", "None");

  free(h_a);
  free(h_b);
  free(h_b_t);
  free(h_c);
  free(h_c_ref);
  cudaFree(d_a);
  cudaFree(d_b);
  cudaFree(d_b_t);
  cudaFree(d_c);
  cudaFree(tma_a);
  cudaFree(tma_b);
  cublasDestroy(handle);
}
#endif /* NOTES_V2_ENABLE_WGMMA */

#if defined(NOTES_V2_ENABLE_TMA_MMA_WS)
template <int kStages, int kBlockSwizzle = 0>
static bool launch_hgemm_tma_mma_ws(int M, int N, int K, half *d_a,
                                    half *d_b_t, half *d_c,
                                    CUtensorMap *tma_a, CUtensorMap *tma_b) {
  constexpr int kMmaM = 16, kMmaN = 8, kMmaK = 16;
  constexpr int kMmaTileM = 2, kMmaTileN = 2;
  constexpr int kValTileM = 4, kValTileN = 8, kValTileK = 4;
  constexpr int BM = kMmaM * kMmaTileM * kValTileM;
  constexpr int BN = kMmaN * kMmaTileN * kValTileN;
  constexpr int BK = kMmaK * kValTileK;
  constexpr int kNumThreads = 256;
  constexpr size_t payload_bytes =
      kStages * (BM * BK + BN * BK) * sizeof(half);
  constexpr size_t smem_bytes = payload_bytes;
  using Kernel = void (*)(int, int, int, half *, const CUtensorMap *,
                          const CUtensorMap *);
  Kernel kernel = hgemm_tma_mma_ws_tn<
      kMmaM, kMmaN, kMmaK, kMmaTileM, kMmaTileN, kValTileM, kValTileN,
      kValTileK, kStages, kNumThreads, kBlockSwizzle>;
```

```
5652
5653    int device = 0;
5654    int max_smem = 0;
5655    cudaFuncAttributes attributes{};
5656    check(cudaGetDevice(&device), "tma_mma_ws get device");
5657    check(cudaDeviceGetAttribute(&max_smem,
5658                                 cudaDevAttrMaxSharedMemoryPerBlockOptin,
5659                                 device),
5660          "tma_mma_ws max shared memory");
5661    check(cudaFuncGetAttributes(&attributes, kernel),
5662          "tma_mma_ws function attributes");
5663    if (smem_bytes + attributes.sharedSizeBytes > size_t(max_smem)) {
5664      printf("| %-42s | %-12s | %-4s | %-8s |\n",
5665             kStages == 2 ? "HGEMM TMA MMA WS (S=2, SW=0)"
5666                          : "HGEMM TMA MMA WS (S=3, SW=0)",
5667             "SMEM SKIP", "SKIP", "None");
5668      return false;
5669    }
5670
5671    check(cudaFuncSetAttribute(kernel, cudaFuncAttributeMaxDynamicSharedMemorySize,
5672                               smem_bytes),
5673          "tma_mma_ws set dynamic shared memory");
5674    dim3 block(kNumThreads);
5675    constexpr int kSwizzleN = 16;
5676    const int n_tiles = N / BN;
5677    const int grid_x = kBlockSwizzle ? div_ceil(n_tiles, kSwizzleN) : n_tiles;
5678    const int grid_z = kBlockSwizzle ? kSwizzleN : 1;
5679    dim3 grid(grid_x, M / BM, grid_z);
5680    kernel<<<grid, block, smem_bytes>>>(M, N, K, d_c, tma_a, tma_b);
5681    check(cudaGetLastError(), "tma_mma_ws launch");
5682    check(cudaDeviceSynchronize(), "tma_mma_ws sync");
5683    return true;
5684  }
5685
5686  static void test_hgemm_tma_mma_ws(int M, int N, int K) {
5687    constexpr int BM = 128, BN = 128, BK = 64;
5688    if (M % BM != 0 || N % BN != 0 || K % BK != 0) {
5689      printf("| %-42s | %-12s | %-4s | %-8s |\n", "HGEMM TMA MMA WS", "SKIP",
5690             "SKIP", "None");
5691      return;
5692    }
5693
5694    const size_t size_a = size_t(M) * K * sizeof(half);
5695    const size_t size_b = size_t(K) * N * sizeof(half);
5696    const size_t size_b_t = size_t(N) * K * sizeof(half);
5697    const size_t size_c = size_t(M) * N * sizeof(half);
5698    half *h_a = static_cast<half *>(malloc(size_a));
5699    half *h_b = static_cast<half *>(malloc(size_b));
5700    half *h_b_t = static_cast<half *>(malloc(size_b_t));
5701    half *h_c = static_cast<half *>(malloc(size_c));
5702    half *h_c_ref = static_cast<half *>(malloc(size_c));
5703    srand(42);
5704    for (int i = 0; i < M * K; ++i)
5705      h_a[i] = __float2half((float(rand()) / RAND_MAX) * 2.0f - 1.0f);
5706    for (int i = 0; i < K * N; ++i)
5707      h_b[i] = __float2half((float(rand()) / RAND_MAX) * 2.0f - 1.0f);
5708    for (int n = 0; n < N; ++n)
5709      for (int k = 0; k < K; ++k)
5710        h_b_t[n * K + k] = h_b[k * N + n];
5711
5712    half *d_a, *d_b, *d_b_t, *d_c;
5713    check(cudaMalloc(&d_a, size_a), "tma_mma_ws alloc A");
5714    check(cudaMalloc(&d_b, size_b), "tma_mma_ws alloc B");
```

```
  check(cudaMalloc(&d_b_t, size_b_t), "tma_mma_ws alloc B_t");
  check(cudaMalloc(&d_c, size_c), "tma_mma_ws alloc C");
  check(cudaMemcpy(d_a, h_a, size_a, cudaMemcpyHostToDevice), "tma_mma_ws H2D A");
  check(cudaMemcpy(d_b, h_b, size_b, cudaMemcpyHostToDevice), "tma_mma_ws H2D B");
  check(cudaMemcpy(d_b_t, h_b_t, size_b_t, cudaMemcpyHostToDevice),
        "tma_mma_ws H2D B_t");

  cublasHandle_t handle;
  cublasCreate(&handle);
  half alpha = __float2half(1.0f), beta = __float2half(0.0f);
  cublasGemmEx(handle, CUBLAS_OP_N, CUBLAS_OP_N, N, M, K, &alpha, d_b,
               CUDA_R_16F, N, d_a, CUDA_R_16F, K, &beta, d_c, CUDA_R_16F, N,
               CUBLAS_COMPUTE_16F, CUBLAS_GEMM_DEFAULT);
  check(cudaMemcpy(h_c_ref, d_c, size_c, cudaMemcpyDeviceToHost),
        "tma_mma_ws D2H reference");

  CUtensorMap *tma_a = allocate_and_create_tensor_map(d_a, M / BM, K / BK);
  CUtensorMap *tma_b = allocate_and_create_tensor_map(d_b_t, N / BN, K / BK);
  for (int block_swizzle : {0, 1}) {
    for (int stages : {2, 3}) {
      const bool launched = stages == 2
          ? (block_swizzle
                ? launch_hgemm_tma_mma_ws<2, 1>(M, N, K, d_a, d_b_t, d_c,
                                                tma_a, tma_b)
                : launch_hgemm_tma_mma_ws<2>(M, N, K, d_a, d_b_t, d_c,
                                             tma_a, tma_b))
          : (block_swizzle
                ? launch_hgemm_tma_mma_ws<3, 1>(M, N, K, d_a, d_b_t, d_c,
                                                tma_a, tma_b)
                : launch_hgemm_tma_mma_ws<3>(M, N, K, d_a, d_b_t, d_c,
                                             tma_a, tma_b));
      if (!launched)
        continue;
      check(cudaMemcpy(h_c, d_c, size_c, cudaMemcpyDeviceToHost),
            "tma_mma_ws D2H");
      float max_err = 0.0f;
      for (int i = 0; i < M * N; ++i)
        max_err = fmaxf(max_err, fabsf(__half2float(h_c[i]) -
                                       __half2float(h_c_ref[i])));
      printf("| %-42s | %.6e | %-4s | %-8s |\n",
             block_swizzle
                 ? (stages == 2 ? "HGEMM TMA MMA WS (S=2, SW=1)"
                                : "HGEMM TMA MMA WS (S=3, SW=1)")
                 : (stages == 2 ? "HGEMM TMA MMA WS (S=2, SW=0)"
                                : "HGEMM TMA MMA WS (S=3, SW=0)"),
             max_err, max_err < 1.0f ? "PASS" : "FAIL", "None");
    }
  }

  free(h_a); free(h_b); free(h_b_t); free(h_c); free(h_c_ref);
  cudaFree(d_a); cudaFree(d_b); cudaFree(d_b_t); cudaFree(d_c);
  cudaFree(tma_a); cudaFree(tma_b);
  cublasDestroy(handle);
}
#endif /* NOTES_V2_ENABLE_TMA_MMA_WS */


static void test_flash_attn(int seqlen, int head_dim) {
  // FlashAttention-2 with split-Q, MMA m16n8k16
  int B = 1, H = 8;

  size_t sz = (size_t)B * H * seqlen * head_dim * sizeof(half);

```

```
  srand(42);
  half *h_q = (half *)malloc(sz);
  half *h_k = (half *)malloc(sz);
  half *h_v = (half *)malloc(sz);
  for (int i = 0; i < B * H * seqlen * head_dim; i++) {
    h_q[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
    h_k[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
    h_v[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
  }

  // CPU reference in FP32: O = softmax(Q @ K^T / sqrt(d)) @ V
  float *ref_q = (float *)malloc(sz * 4 / sizeof(half));  // 4x for float
  float *ref_k = (float *)malloc(sz * 4 / sizeof(half));
  float *ref_v = (float *)malloc(sz * 4 / sizeof(half));
  float *ref_o = (float *)malloc(sz * 4 / sizeof(half));
  int count = B * H * seqlen * head_dim;
  for (int i = 0; i < count; i++) {
    ref_q[i] = __half2float(h_q[i]);
    ref_k[i] = __half2float(h_k[i]);
    ref_v[i] = __half2float(h_v[i]);
  }

  float scale = 1.0f / sqrtf((float)head_dim);
  for (int bi = 0; bi < B * H; bi++) {
    for (int qi = 0; qi < seqlen; qi++) {
      // S[qi, kj] = Q[qi,:] @ K[kj,:]^T * scale
      float smax = -INFINITY;
      float *S = (float *)malloc((size_t)seqlen * sizeof(float));
      for (int kj = 0; kj < seqlen; kj++) {
        float s = 0.0f;
        for (int d = 0; d < head_dim; d++)
          s += ref_q[bi * seqlen * head_dim + qi * head_dim + d] *
               ref_k[bi * seqlen * head_dim + kj * head_dim + d];
        S[kj] = s * scale;
        if (S[kj] > smax) smax = S[kj];
      }
      // softmax
      double sum_exp = 0.0;
      for (int kj = 0; kj < seqlen; kj++) sum_exp += (double)expf(S[kj] - smax);
      float inv_sum = 1.0f / (float)sum_exp;
      // O[qi, :] = sum_kj P[qi, kj] * V[kj, :]
      for (int d = 0; d < head_dim; d++) {
        double o_acc = 0.0;
        for (int kj = 0; kj < seqlen; kj++)
          o_acc += (double)(expf(S[kj] - smax) * inv_sum) *
                   ref_v[bi * seqlen * head_dim + kj * head_dim + d];
        ref_o[bi * seqlen * head_dim + qi * head_dim + d] = (float)o_acc;
      }
      free(S);
    }
  }

  half *d_q, *d_k, *d_v, *d_o;
  check(cudaMalloc(&d_q, sz), "fa alloc Q");
  check(cudaMalloc(&d_k, sz), "fa alloc K");
  check(cudaMalloc(&d_v, sz), "fa alloc V");
  check(cudaMalloc(&d_o, sz), "fa alloc O");
  check(cudaMemcpy(d_q, h_q, sz, cudaMemcpyHostToDevice), "fa H2D Q");
  check(cudaMemcpy(d_k, h_k, sz, cudaMemcpyHostToDevice), "fa H2D K");
  check(cudaMemcpy(d_v, h_v, sz, cudaMemcpyHostToDevice), "fa H2D V");

  // Template params for kHeadDim=64, kStage=2
  constexpr int kHeadDim = 64, kStage = 2, kPad = 8;
```

```
  constexpr int kMmaAtomM = 16, kMmaAtomN = 8, kMmaAtomK = 16;
  constexpr int kMmaTileSeqLenQ = 8;
  constexpr int kMmaTileSeqLenK = 1;
  constexpr int kMmaTileSeqLenP = 8;
  constexpr int kMmaTileHeadDimV = 1;
  constexpr int kValTileSeqLenQ = 1;
  constexpr int kValTileSeqLenK = 8;
  constexpr int kValTileSeqLenP = 1;
  constexpr int kValTileHeadDimV = kHeadDim / (8 * kMmaTileHeadDimV);

  constexpr int Br = kMmaAtomM * kMmaTileSeqLenQ * kValTileSeqLenQ;
  constexpr int Bc = kMmaAtomN * kMmaTileSeqLenK * kValTileSeqLenK;
  size_t smem_bytes = (Br * (kHeadDim + kPad) +
                       kStage * Bc * (kHeadDim + kPad) +
                       Bc * (kHeadDim + kPad)) * sizeof(half);

  dim3 block(256);
  dim3 grid((seqlen + Br - 1) / Br, B * H);

  using FAKernel = void (*)(half *, half *, half *, half *, int, int);
  FAKernel fa_k = flash_attn_mma_stages_split_q<kHeadDim, kMmaAtomM, kMmaAtomN,
      kMmaAtomK, kMmaTileSeqLenQ, kMmaTileSeqLenK, kMmaTileSeqLenP,
      kMmaTileHeadDimV, kValTileSeqLenQ, kValTileSeqLenK, kValTileSeqLenP,
      kValTileHeadDimV, kStage, kPad>;
  cudaFuncSetAttribute(fa_k, cudaFuncAttributeMaxDynamicSharedMemorySize, smem_bytes);
  fa_k<<<grid, block, smem_bytes>>>(d_q, d_k, d_v, d_o, seqlen, head_dim);
  check(cudaGetLastError(), "fa launch");
  check(cudaDeviceSynchronize(), "fa sync");

  half *h_o = (half *)malloc(sz);
  check(cudaMemcpy(h_o, d_o, sz, cudaMemcpyDeviceToHost), "fa D2H");

  float max_err = 0.0f;
  for (int i = 0; i < count; i++) {
    float err = fabsf(__half2float(h_o[i]) - ref_o[i]);
    if (err > max_err) max_err = err;
  }
  printf("| %-42s | %.6e | %-4s | %-8s |\n", "FlashAttn-SplitQ", max_err, max_err < 5e-1f ? "PASS" : "FAIL",
     "None");

  free(h_q); free(h_k); free(h_v); free(h_o);
  free(ref_q); free(ref_k); free(ref_v); free(ref_o);
  cudaFree(d_q); cudaFree(d_k); cudaFree(d_v); cudaFree(d_o);
}


// Bench mode globals and helpers
static bool g_bench_mode = false;
static bool g_bench_fa = false;
static int g_bench_M = 4096, g_bench_N = 4096, g_bench_K = 4096;
static int g_bench_B = 1, g_bench_H = 32, g_bench_Nfa = 4096, g_bench_D = 64;

static float bench_hgemm_tflops(int M, int N, int K, float time_ms) {
  double flops = 2.0 * M * N * K;
  return (float)(flops / (double)time_ms / 1e9);
}

static float bench_fa_tflops(int B, int H, int N, int D, float time_ms) {
  double flops = 4.0 * B * H * N * N * D;
  return (float)(flops / (double)time_ms / 1e9);
}

static bool check_smem_feasible(const void *kernel_func, size_t dyn_smem_bytes) {
```

```
  int device = 0;
  int max_smem = 0;
  cudaFuncAttributes attrs{};
  cudaGetDevice(&device);
  cudaDeviceGetAttribute(&max_smem, cudaDevAttrMaxSharedMemoryPerBlockOptin, device);
  cudaFuncGetAttributes(&attrs, kernel_func);
  return dyn_smem_bytes + attrs.sharedSizeBytes <= (size_t)max_smem;
}

// ============================================================================
// Bench: HGEMM MMA (basic m16n8k16 + multistage pipeline)
// ============================================================================
template <int kStages, int kBlockSwizzle>
static bool launch_timed_hgemm_mma(half *d_a, half *d_b_t, half *d_c, half *h_c,
            half *h_c_ref, int M, int N, int K, size_t size_c,
            cudaEvent_t start, cudaEvent_t stop, float &max_err,
            float &time_ms) {
  constexpr int BM = 128, BN = 128, BK = 16;
  using Kernel = void (*)(half *, half *, half *, int, int, int);
  Kernel k = hgemm_mma_stages_tn<16, 8, 16, 2, 4, 4, 4, kStages, kBlockSwizzle>;
  size_t smem = kStages * (BM * BK + BN * BK) * sizeof(half);
  if (!check_smem_feasible((const void *)k, smem)) return false;
  dim3 block(256);
  const int tiles_n = (N + BN - 1) / BN;
  constexpr int kSwizzleN = 16;
  int gx = kBlockSwizzle ? div_ceil(tiles_n, kSwizzleN) : tiles_n;
  int gz = kBlockSwizzle ? kSwizzleN : 1;
  dim3 grid(gx, (M + BM - 1) / BM, gz);
  cudaEventRecord(start);
  k<<<grid, block, smem>>>(d_a, d_b_t, d_c, M, N, K);
  cudaEventRecord(stop);
  cudaEventSynchronize(stop);
  cudaEventElapsedTime(&time_ms, start, stop);
  check(cudaMemcpy(h_c, d_c, size_c, cudaMemcpyDeviceToHost), "bench mma D2H");
  max_err = 0.0f;
  for (int i = 0; i < M * N; i++) {
    float err = fabsf(__half2float(h_c[i]) - __half2float(h_c_ref[i]));
    if (err > max_err) max_err = err;
  }
  return true;
}

static void bench_hgemm_mma(int M, int N, int K) {
  size_t size_a = (size_t)M * K * sizeof(half);
  size_t size_b = (size_t)K * N * sizeof(half);
  size_t size_c = (size_t)M * N * sizeof(half);
  half *h_a = (half *)malloc(size_a);
  half *h_b = (half *)malloc(size_b);
  half *h_c_ref = (half *)malloc(size_c);
  srand(42);
  for (int i = 0; i < M * K; i++)
    h_a[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
  for (int i = 0; i < K * N; i++)
    h_b[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
  size_t size_b_t = (size_t)N * K * sizeof(half);
  half *h_b_t = (half *)malloc(size_b_t);
  for (int n = 0; n < N; n++)
    for (int k = 0; k < K; k++)
      h_b_t[n * K + k] = h_b[k * N + n];
  half *d_a, *d_b, *d_b_t, *d_c;
  check(cudaMalloc(&d_a, size_a), "bench mma alloc A");
  check(cudaMalloc(&d_b, size_b), "bench mma alloc B");
  check(cudaMalloc(&d_b_t, size_b_t), "bench mma alloc B_t");
```

```
  check(cudaMalloc(&d_c, size_c), "bench mma alloc C");
  check(cudaMemcpy(d_a, h_a, size_a, cudaMemcpyHostToDevice), "bench mma H2D A");
  check(cudaMemcpy(d_b, h_b, size_b, cudaMemcpyHostToDevice), "bench mma H2D B");
  check(cudaMemcpy(d_b_t, h_b_t, size_b_t, cudaMemcpyHostToDevice), "bench mma H2D B_t");
  cublasHandle_t handle;
  cublasCreate(&handle);
  half alpha_h = __float2half(1.0f), beta_h = __float2half(0.0f);
  cublasGemmEx(handle, CUBLAS_OP_N, CUBLAS_OP_N, N, M, K, &alpha_h, d_b, CUDA_R_16F, N,
        d_a, CUDA_R_16F, K, &beta_h, d_c, CUDA_R_16F, N, CUBLAS_COMPUTE_16F,
        CUBLAS_GEMM_DEFAULT);
  check(cudaMemcpy(h_c_ref, d_c, size_c, cudaMemcpyDeviceToHost), "bench mma D2H ref");
  half *h_c = (half *)malloc(size_c);
  cudaEvent_t start, stop;
  cudaEventCreate(&start);
  cudaEventCreate(&stop);
  for (int stages : {2, 3}) {
    for (int swizzle : {0, 1}) {
      float max_err = 0, time_ms = 0;
      bool ok = false;
      if (stages == 2)
        ok = swizzle ? launch_timed_hgemm_mma<2, 1>(d_a, d_b_t, d_c, h_c, h_c_ref, M, N,
            K, size_c, start, stop, max_err, time_ms)
          : launch_timed_hgemm_mma<2, 0>(d_a, d_b_t, d_c, h_c, h_c_ref, M, N,
            K, size_c, start, stop, max_err, time_ms);
      else
        ok = swizzle ? launch_timed_hgemm_mma<3, 1>(d_a, d_b_t, d_c, h_c, h_c_ref, M, N,
            K, size_c, start, stop, max_err, time_ms)
          : launch_timed_hgemm_mma<3, 0>(d_a, d_b_t, d_c, h_c, h_c_ref, M, N,
            K, size_c, start, stop, max_err, time_ms);
      char label[64];
      snprintf(label, sizeof(label), "HGEMM MMA (S=%d, SW=%d)", stages, swizzle);
      if (!ok) {
        printf("| %-42s | %-12s | %-4s | %-8s |\n", label, "SMEM SKIP", "SKIP", "None");
      } else {
        float tflops = bench_hgemm_tflops(M, N, K, time_ms);
        printf("| %-42s | %.6e | %-4s | %-8.1f |\n", label, max_err,
        max_err < 1.0f ? "PASS" : "FAIL", tflops);
      }
    }
  }
  cudaEventDestroy(start);
  cudaEventDestroy(stop);
  free(h_a);
  free(h_b);
  free(h_b_t);
  free(h_c);
  free(h_c_ref);
  cudaFree(d_a);
  cudaFree(d_b);
  cudaFree(d_b_t);
  cudaFree(d_c);
  cublasDestroy(handle);
}

// ============================================================================
// Bench: HGEMM MMA Swizzle + Register Double Buffering (kValTileK=2, BK=32)
// ============================================================================
template <int kStages, int kBlockSwizzle>
static bool launch_timed_hgemm_swizzle(half *d_a, half *d_b_t, half *d_c, half *h_c,
            half *h_c_ref, int M, int N, int K, size_t size_c,
            cudaEvent_t start, cudaEvent_t stop, float &max_err,
            float &time_ms) {
  constexpr int BM = 128, BN = 128, BK = 32;
```

```
  using Kernel = void (*)(half *, half *, half *, int, int, int);
  Kernel k = hgemm_mma_stages_tn_swizzle<16, 8, 16, 2, 4, 4, 4, 2, kStages, kBlockSwizzle>;
  size_t smem = kStages * (BM * BK + BN * BK) * sizeof(half);
  if (!check_smem_feasible((const void *)k, smem)) return false;
  dim3 block(256);
  const int tiles_n = (N + BN - 1) / BN;
  constexpr int kSwizzleN = 16;
  int gx = kBlockSwizzle ? div_ceil(tiles_n, kSwizzleN) : tiles_n;
  int gz = kBlockSwizzle ? kSwizzleN : 1;
  dim3 grid(gx, (M + BM - 1) / BM, gz);
  cudaEventRecord(start);
  k<<<grid, block, smem>>>(d_a, d_b_t, d_c, M, N, K);
  cudaEventRecord(stop);
  cudaEventSynchronize(stop);
  cudaEventElapsedTime(&time_ms, start, stop);
  check(cudaMemcpy(h_c, d_c, size_c, cudaMemcpyDeviceToHost), "bench swizzle D2H");
  max_err = 0.0f;
  for (int i = 0; i < M * N; i++) {
    float err = fabsf(__half2float(h_c[i]) - __half2float(h_c_ref[i]));
    if (err > max_err) max_err = err;
  }
  return true;
}

static void bench_hgemm_swizzle(int M, int N, int K) {
  size_t size_a = (size_t)M * K * sizeof(half);
  size_t size_b = (size_t)K * N * sizeof(half);
  size_t size_c = (size_t)M * N * sizeof(half);
  half *h_a = (half *)malloc(size_a);
  half *h_b = (half *)malloc(size_b);
  half *h_c_ref = (half *)malloc(size_c);
  srand(42);
  for (int i = 0; i < M * K; i++)
    h_a[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
  for (int i = 0; i < K * N; i++)
    h_b[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
  size_t size_b_t = (size_t)N * K * sizeof(half);
  half *h_b_t = (half *)malloc(size_b_t);
  for (int n = 0; n < N; n++)
    for (int k = 0; k < K; k++)
      h_b_t[n * K + k] = h_b[k * N + n];
  half *d_a, *d_b, *d_b_t, *d_c;
  check(cudaMalloc(&d_a, size_a), "bench swizzle alloc A");
  check(cudaMalloc(&d_b, size_b), "bench swizzle alloc B");
  check(cudaMalloc(&d_b_t, size_b_t), "bench swizzle alloc B_t");
  check(cudaMalloc(&d_c, size_c), "bench swizzle alloc C");
  check(cudaMemcpy(d_a, h_a, size_a, cudaMemcpyHostToDevice), "bench swizzle H2D A");
  check(cudaMemcpy(d_b, h_b, size_b, cudaMemcpyHostToDevice), "bench swizzle H2D B");
  check(cudaMemcpy(d_b_t, h_b_t, size_b_t, cudaMemcpyHostToDevice), "bench swizzle H2D B_t");
  cublasHandle_t handle;
  cublasCreate(&handle);
  half alpha_h = __float2half(1.0f), beta_h = __float2half(0.0f);
  cublasGemmEx(handle, CUBLAS_OP_N, CUBLAS_OP_N, N, M, K, &alpha_h, d_b, CUDA_R_16F, N,
        d_a, CUDA_R_16F, K, &beta_h, d_c, CUDA_R_16F, N, CUBLAS_COMPUTE_16F,
        CUBLAS_GEMM_DEFAULT);
  check(cudaMemcpy(h_c_ref, d_c, size_c, cudaMemcpyDeviceToHost), "bench swizzle D2H ref");
  half *h_c = (half *)malloc(size_c);
  cudaEvent_t start, stop;
  cudaEventCreate(&start);
  cudaEventCreate(&stop);
  for (int stages : {2, 3}) {
    for (int swizzle : {0, 1}) {
      float max_err = 0, time_ms = 0;
```

```
      bool ok = false;
      if (stages == 2)
        ok = swizzle ? launch_timed_hgemm_swizzle<2, 1>(d_a, d_b_t, d_c, h_c, h_c_ref, M,
            N, K, size_c, start, stop, max_err, time_ms)
          : launch_timed_hgemm_swizzle<2, 0>(d_a, d_b_t, d_c, h_c, h_c_ref, M,
            N, K, size_c, start, stop, max_err, time_ms);
      else
        ok = swizzle ? launch_timed_hgemm_swizzle<3, 1>(d_a, d_b_t, d_c, h_c, h_c_ref, M,
            N, K, size_c, start, stop, max_err, time_ms)
          : launch_timed_hgemm_swizzle<3, 0>(d_a, d_b_t, d_c, h_c, h_c_ref, M,
            N, K, size_c, start, stop, max_err, time_ms);
      char label[64];
      snprintf(label, sizeof(label), "HGEMM Swizzle+Reg2x (S=%d, SW=%d)", stages, swizzle);
      if (!ok) {
        printf("| %-42s | %-12s | %-4s | %-8s |\n", label, "SMEM SKIP", "SKIP", "None");
      } else {
        float tflops = bench_hgemm_tflops(M, N, K, time_ms);
        printf("| %-42s | %.6e | %-4s | %-8.1f |\n", label, max_err,
        max_err < 1.0f ? "PASS" : "FAIL", tflops);
      }
    }
  }
  cudaEventDestroy(start);
  cudaEventDestroy(stop);
  free(h_a);
  free(h_b);
  free(h_b_t);
  free(h_c);
  free(h_c_ref);
  cudaFree(d_a);
  cudaFree(d_b);
  cudaFree(d_b_t);
  cudaFree(d_c);
  cublasDestroy(handle);
}
#if defined(NOTES_V2_ENABLE_CUTE)
// ============================================================================
// Bench: HGEMM CuTe
// ============================================================================
static void bench_hgemm_cute(int M, int N, int K) {
  size_t size_a = (size_t)M * K * sizeof(half);
  size_t size_b = (size_t)K * N * sizeof(half);
  size_t size_c = (size_t)M * N * sizeof(half);
  half *h_a = (half *)malloc(size_a);
  half *h_b = (half *)malloc(size_b);
  half *h_c_ref = (half *)malloc(size_c);
  srand(42);
  for (int i = 0; i < M * K; i++)
    h_a[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
  for (int i = 0; i < K * N; i++)
    h_b[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
  size_t size_b_t = (size_t)N * K * sizeof(half);
  half *h_b_t = (half *)malloc(size_b_t);
  for (int n = 0; n < N; n++)
    for (int k = 0; k < K; k++)
      h_b_t[n * K + k] = h_b[k * N + n];
  half *d_a, *d_b, *d_b_t, *d_c;
  check(cudaMalloc(&d_a, size_a), "bench cute alloc A");
  check(cudaMalloc(&d_b, size_b), "bench cute alloc B");
  check(cudaMalloc(&d_b_t, size_b_t), "bench cute alloc B_t");
  check(cudaMalloc(&d_c, size_c), "bench cute alloc C");
  check(cudaMemcpy(d_a, h_a, size_a, cudaMemcpyHostToDevice), "bench cute H2D A");
  check(cudaMemcpy(d_b, h_b, size_b, cudaMemcpyHostToDevice), "bench cute H2D B");
```

```
  check(cudaMemcpy(d_b_t, h_b_t, size_b_t, cudaMemcpyHostToDevice), "bench cute H2D B_t");
  cublasHandle_t handle;
  cublasCreate(&handle);
  half alpha_h = __float2half(1.0f), beta_h = __float2half(0.0f);
  cublasGemmEx(handle, CUBLAS_OP_N, CUBLAS_OP_N, N, M, K, &alpha_h, d_b, CUDA_R_16F, N,
        d_a, CUDA_R_16F, K, &beta_h, d_c, CUDA_R_16F, N, CUBLAS_COMPUTE_16F,
        CUBLAS_GEMM_DEFAULT);
  check(cudaMemcpy(h_c_ref, d_c, size_c, cudaMemcpyDeviceToHost), "bench cute D2H ref");
  half *h_c = (half *)malloc(size_c);
  cudaEvent_t start, stop;
  cudaEventCreate(&start);
  cudaEventCreate(&stop);
  for (int stages : {2, 3}) {
    for (int swizzle : {0, 1}) {
      char label[64];
      snprintf(label, sizeof(label), "HGEMM CuTe Swizzle (S=%d, SW=%d)", stages, swizzle);
      bool ok = false;
      float time_ms = 0, max_err = 0;
      if (stages == 2) {
        if (swizzle) {
          cudaEventRecord(start);
          launch_hgemm_mma_stages_tn_cute<half, 2, 1>(d_a, d_b_t, d_c, M, N, K);
          cudaEventRecord(stop);
          cudaEventSynchronize(stop);
          cudaEventElapsedTime(&time_ms, start, stop);
          ok = true;
        } else {
          cudaEventRecord(start);
          launch_hgemm_mma_stages_tn_cute<half, 2, 0>(d_a, d_b_t, d_c, M, N, K);
          cudaEventRecord(stop);
          cudaEventSynchronize(stop);
          cudaEventElapsedTime(&time_ms, start, stop);
          ok = true;
        }
      } else {
        if (swizzle) {
          cudaEventRecord(start);
          launch_hgemm_mma_stages_tn_cute<half, 3, 1>(d_a, d_b_t, d_c, M, N, K);
          cudaEventRecord(stop);
          cudaEventSynchronize(stop);
          cudaEventElapsedTime(&time_ms, start, stop);
          ok = true;
        } else {
          cudaEventRecord(start);
          launch_hgemm_mma_stages_tn_cute<half, 3, 0>(d_a, d_b_t, d_c, M, N, K);
          cudaEventRecord(stop);
          cudaEventSynchronize(stop);
          cudaEventElapsedTime(&time_ms, start, stop);
          ok = true;
        }
      }
      if (ok) {
        check(cudaMemcpy(h_c, d_c, size_c, cudaMemcpyDeviceToHost), "bench cute D2H");
        max_err = 0.0f;
        for (int i = 0; i < M * N; i++) {
          float err = fabsf(__half2float(h_c[i]) - __half2float(h_c_ref[i]));
          if (err > max_err) max_err = err;
        }
        float tflops = bench_hgemm_tflops(M, N, K, time_ms);
        printf("| %-42s | %.6e | %-4s | %-8.1f |\n", label, max_err,
        max_err < 1.0f ? "PASS" : "FAIL", tflops);
      } else {
        printf("| %-42s | %-12s | %-4s | %-8s |\n", label, "LAUNCH ERR", "FAIL", "None");
```

```
      }
    }
  }
  cudaEventDestroy(start);
  cudaEventDestroy(stop);
  free(h_a);
  free(h_b);
  free(h_b_t);
  free(h_c);
  free(h_c_ref);
  cudaFree(d_a);
  cudaFree(d_b);
  cudaFree(d_b_t);
  cudaFree(d_c);
  cublasDestroy(handle);
}
#endif

#if defined(NOTES_V2_ENABLE_WGMMA)
// ============================================================================
// Bench: HGEMM WGMMA (m64n128k16 + TMA + Warp Specialization, SM90+)
// ============================================================================
template <int kStages, int kBlockSwizzle>
static bool launch_timed_hgemm_wgmma(half *d_c, half *h_c, half *h_c_ref,
            CUtensorMap *tma_a, CUtensorMap *tma_b,
            int M, int N, int K, size_t size_c,
            cudaEvent_t start, cudaEvent_t stop,
            float &max_err, float &time_ms) {
  constexpr int BM = 128, BN = 128, BK = 64, kNumThreads = 256;
  using Kernel = void (*)(int, int, int, half *, const CUtensorMap *,
          const CUtensorMap *);
  Kernel k = hgemm_wgmma_stages_tn<64, 128, 16, BM, BN, BK, kNumThreads, kStages,
            kBlockSwizzle>;
  size_t smem = kStages * (BM * BK + BN * BK) * sizeof(half);
  if (!check_smem_feasible((const void *)k, smem)) return false;
  cudaFuncSetAttribute(k, cudaFuncAttributeMaxDynamicSharedMemorySize, smem);
  dim3 block(kNumThreads);
  const int tiles_n = (N + BN - 1) / BN;
  constexpr int kSwizzleN = 16;
  int gx = kBlockSwizzle ? div_ceil(tiles_n, kSwizzleN) : tiles_n;
  int gz = kBlockSwizzle ? kSwizzleN : 1;
  dim3 grid(gx, (M + BM - 1) / BM, gz);
  cudaEventRecord(start);
  k<<<grid, block, smem>>>(M, N, K, d_c, tma_a, tma_b);
  cudaEventRecord(stop);
  cudaEventSynchronize(stop);
  cudaEventElapsedTime(&time_ms, start, stop);
  check(cudaMemcpy(h_c, d_c, size_c, cudaMemcpyDeviceToHost), "bench wgmma D2H");
  max_err = 0.0f;
  for (int i = 0; i < M * N; i++) {
    float err = fabsf(__half2float(h_c[i]) - __half2float(h_c_ref[i]));
    if (err > max_err) max_err = err;
  }
  return true;
}

static void bench_hgemm_wgmma(int M, int N, int K) {
  constexpr int BM = 128, BN = 128, BK = 64;
  if (M % BM != 0 || N % BN != 0 || K % BK != 0) {
    printf("| %-42s | %-12s | %-4s | %-8s |\n", "HGEMM WGMMA (unaligned)", "SKIP", "SKIP",
       "None");
    return;
  }
```

```
  size_t size_a = (size_t)M * K * sizeof(half);
  size_t size_b = (size_t)K * N * sizeof(half);
  size_t size_c = (size_t)M * N * sizeof(half);
  half *h_a = (half *)malloc(size_a);
  half *h_b = (half *)malloc(size_b);
  half *h_c_ref = (half *)malloc(size_c);
  srand(42);
  for (int i = 0; i < M * K; i++)
    h_a[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
  for (int i = 0; i < K * N; i++)
    h_b[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
  size_t size_b_t = (size_t)N * K * sizeof(half);
  half *h_b_t = (half *)malloc(size_b_t);
  for (int n = 0; n < N; n++)
    for (int k = 0; k < K; k++)
      h_b_t[n * K + k] = h_b[k * N + n];
  half *d_a, *d_b, *d_b_t, *d_c;
  check(cudaMalloc(&d_a, size_a), "bench wgmma alloc A");
  check(cudaMalloc(&d_b, size_b), "bench wgmma alloc B");
  check(cudaMalloc(&d_b_t, size_b_t), "bench wgmma alloc B_t");
  check(cudaMalloc(&d_c, size_c), "bench wgmma alloc C");
  check(cudaMemcpy(d_a, h_a, size_a, cudaMemcpyHostToDevice), "bench wgmma H2D A");
  check(cudaMemcpy(d_b, h_b, size_b, cudaMemcpyHostToDevice), "bench wgmma H2D B");
  check(cudaMemcpy(d_b_t, h_b_t, size_b_t, cudaMemcpyHostToDevice), "bench wgmma H2D B_t");
  cublasHandle_t handle;
  cublasCreate(&handle);
  half alpha_h = __float2half(1.0f), beta_h = __float2half(0.0f);
  cublasGemmEx(handle, CUBLAS_OP_N, CUBLAS_OP_N, N, M, K, &alpha_h, d_b, CUDA_R_16F, N,
        d_a, CUDA_R_16F, K, &beta_h, d_c, CUDA_R_16F, N, CUBLAS_COMPUTE_16F,
        CUBLAS_GEMM_DEFAULT);
  check(cudaMemcpy(h_c_ref, d_c, size_c, cudaMemcpyDeviceToHost), "bench wgmma D2H ref");
  CUtensorMap *tma_a = allocate_and_create_tensor_map(d_a, M / BM, K / BK);
  CUtensorMap *tma_b = allocate_and_create_tensor_map(d_b_t, N / BN, K / BK);
  half *h_c = (half *)malloc(size_c);
  cudaEvent_t start, stop;
  cudaEventCreate(&start);
  cudaEventCreate(&stop);
  for (int stages : {1, 2, 3}) {
    for (int swizzle : {0, 1}) {
      float max_err = 0, time_ms = 0;
      bool ok = false;
      if (stages == 2)
        ok = swizzle
          ? launch_timed_hgemm_wgmma<2, 1>(d_c, h_c, h_c_ref, tma_a, tma_b, M, N,
            K, size_c, start, stop, max_err, time_ms)
          : launch_timed_hgemm_wgmma<2, 0>(d_c, h_c, h_c_ref, tma_a, tma_b, M, N, K,
            size_c, start, stop, max_err, time_ms);
      else
        ok = swizzle
          ? launch_timed_hgemm_wgmma<3, 1>(d_c, h_c, h_c_ref, tma_a, tma_b, M, N,
            K, size_c, start, stop, max_err, time_ms)
          : launch_timed_hgemm_wgmma<3, 0>(d_c, h_c, h_c_ref, tma_a, tma_b, M, N, K,
            size_c, start, stop, max_err, time_ms);
      char label[64];
      snprintf(label, sizeof(label), "HGEMM TMA WGMMA WS (S=%d, SW=%d)", stages, swizzle);
      if (!ok) {
        printf("| %-42s | %-12s | %-4s | %-8s |\n", label, "SMEM SKIP", "SKIP", "None");
      } else {
        float tflops = bench_hgemm_tflops(M, N, K, time_ms);
        printf("| %-42s | %.6e | %-4s | %-8.1f |\n", label, max_err,
        max_err < 1.0f ? "PASS" : "FAIL", tflops);
      }
    }
  }
```

```
  }
  cudaEventDestroy(start);
  cudaEventDestroy(stop);
  free(h_a);
  free(h_b);
  free(h_b_t);
  free(h_c);
  free(h_c_ref);
  cudaFree(d_a);
  cudaFree(d_b);
  cudaFree(d_b_t);
  cudaFree(d_c);
  cudaFree(tma_a);
  cudaFree(tma_b);
  cublasDestroy(handle);
}
#endif

#if defined(NOTES_V2_ENABLE_TMA_MMA_WS)
// ============================================================================
// Bench: HGEMM TMA MMA WS (mma.sync + TMA + Warp Specialization, SM120)
// ============================================================================
template <int kStages, int kBlockSwizzle>
static bool bench_launch_tma_mma_ws(int M, int N, int K, half *d_a, half *d_b_t,
            half *d_c, half *h_c, half *h_c_ref, size_t size_c,
            CUtensorMap *tma_a, CUtensorMap *tma_b,
            cudaEvent_t start, cudaEvent_t stop,
            float &max_err, float &time_ms) {
  constexpr int BM = 128, BN = 128, BK = 64, kNumThreads = 256;
  constexpr size_t payload_bytes = kStages * (BM * BK + BN * BK) * sizeof(half);
  using Kernel = void (*)(int, int, int, half *, const CUtensorMap *,
          const CUtensorMap *);
  Kernel kernel = hgemm_tma_mma_ws_tn<16, 8, 16, 2, 2, 4, 8, 4, kStages,
            kNumThreads, kBlockSwizzle>;
  if (!check_smem_feasible((const void *)kernel, payload_bytes)) return false;
  cudaFuncSetAttribute(kernel, cudaFuncAttributeMaxDynamicSharedMemorySize,
          payload_bytes);
  dim3 block(kNumThreads);
  constexpr int kSwizzleN = 16;
  const int n_tiles = N / BN;
  const int grid_x = kBlockSwizzle ? div_ceil(n_tiles, kSwizzleN) : n_tiles;
  const int grid_z = kBlockSwizzle ? kSwizzleN : 1;
  dim3 grid(grid_x, M / BM, grid_z);
  cudaEventRecord(start);
  kernel<<<grid, block, payload_bytes>>>(M, N, K, d_c, tma_a, tma_b);
  cudaEventRecord(stop);
  cudaEventSynchronize(stop);
  cudaEventElapsedTime(&time_ms, start, stop);
  check(cudaMemcpy(h_c, d_c, size_c, cudaMemcpyDeviceToHost), "bench tma mma ws D2H");
  max_err = 0.0f;
  for (int i = 0; i < M * N; ++i)
    max_err = fmaxf(max_err, fabsf(__half2float(h_c[i]) - __half2float(h_c_ref[i])));
  return true;
}

static void bench_hgemm_tma_mma_ws(int M, int N, int K) {
  constexpr int BM = 128, BN = 128, BK = 64;
  if (M % BM != 0 || N % BN != 0 || K % BK != 0) {
    printf("| %-42s | %-12s | %-4s | %-8s |\n", "HGEMM TMA MMA WS (unaligned)", "SKIP",
       "SKIP", "None");
    return;
  }
  const size_t size_a = size_t(M) * K * sizeof(half);
```

```
  const size_t size_b = size_t(K) * N * sizeof(half);
  const size_t size_b_t = size_t(N) * K * sizeof(half);
  const size_t size_c = size_t(M) * N * sizeof(half);
  half *h_a = static_cast<half *>(malloc(size_a));
  half *h_b = static_cast<half *>(malloc(size_b));
  half *h_b_t = static_cast<half *>(malloc(size_b_t));
  half *h_c = static_cast<half *>(malloc(size_c));
  half *h_c_ref = static_cast<half *>(malloc(size_c));
  srand(42);
  for (int i = 0; i < M * K; ++i)
    h_a[i] = __float2half((float(rand()) / RAND_MAX) * 2.0f - 1.0f);
  for (int i = 0; i < K * N; ++i)
    h_b[i] = __float2half((float(rand()) / RAND_MAX) * 2.0f - 1.0f);
  for (int n = 0; n < N; ++n)
    for (int k = 0; k < K; ++k)
      h_b_t[n * K + k] = h_b[k * N + n];
  half *d_a, *d_b, *d_b_t, *d_c;
  check(cudaMalloc(&d_a, size_a), "bench tma mma ws alloc A");
  check(cudaMalloc(&d_b, size_b), "bench tma mma ws alloc B");
  check(cudaMalloc(&d_b_t, size_b_t), "bench tma mma ws alloc B_t");
  check(cudaMalloc(&d_c, size_c), "bench tma mma ws alloc C");
  check(cudaMemcpy(d_a, h_a, size_a, cudaMemcpyHostToDevice), "bench tma mma ws H2D A");
  check(cudaMemcpy(d_b, h_b, size_b, cudaMemcpyHostToDevice), "bench tma mma ws H2D B");
  check(cudaMemcpy(d_b_t, h_b_t, size_b_t, cudaMemcpyHostToDevice),
     "bench tma mma ws H2D B_t");
  cublasHandle_t handle;
  cublasCreate(&handle);
  half alpha = __float2half(1.0f), beta = __float2half(0.0f);
  cublasGemmEx(handle, CUBLAS_OP_N, CUBLAS_OP_N, N, M, K, &alpha, d_b, CUDA_R_16F, N,
        d_a, CUDA_R_16F, K, &beta, d_c, CUDA_R_16F, N, CUBLAS_COMPUTE_16F,
        CUBLAS_GEMM_DEFAULT);
  check(cudaMemcpy(h_c_ref, d_c, size_c, cudaMemcpyDeviceToHost),
     "bench tma mma ws D2H reference");
  CUtensorMap *tma_a = allocate_and_create_tensor_map(d_a, M / BM, K / BK);
  CUtensorMap *tma_b = allocate_and_create_tensor_map(d_b_t, N / BN, K / BK);
  cudaEvent_t start, stop;
  cudaEventCreate(&start);
  cudaEventCreate(&stop);
  for (int stages : {1, 2, 3}) {
    for (int swizzle : {0, 1}) {
      float max_err = 0, time_ms = 0;
      bool ok = false;
      if (stages == 2)
        ok = swizzle ? bench_launch_tma_mma_ws<2, 1>(M, N, K, d_a, d_b_t, d_c, h_c,
            h_c_ref, size_c, tma_a, tma_b,
            start, stop, max_err, time_ms)
          : bench_launch_tma_mma_ws<2, 0>(M, N, K, d_a, d_b_t, d_c, h_c,
            h_c_ref, size_c, tma_a, tma_b,
            start, stop, max_err, time_ms);
      else
        ok = swizzle ? bench_launch_tma_mma_ws<3, 1>(M, N, K, d_a, d_b_t, d_c, h_c,
            h_c_ref, size_c, tma_a, tma_b,
            start, stop, max_err, time_ms)
          : bench_launch_tma_mma_ws<3, 0>(M, N, K, d_a, d_b_t, d_c, h_c,
            h_c_ref, size_c, tma_a, tma_b,
            start, stop, max_err, time_ms);
      char label[64];
      snprintf(label, sizeof(label), "HGEMM TMA MMA WS (S=%d, SW=%d)", stages, swizzle);
      if (!ok) {
        printf("| %-42s | %-12s | %-4s | %-8s |\n", label, "SMEM SKIP", "SKIP", "None");
      } else {
        float tflops = bench_hgemm_tflops(M, N, K, time_ms);
        printf("| %-42s | %.6e | %-4s | %-8.1f |\n", label, max_err,
```

```
        max_err < 1.0f ? "PASS" : "FAIL", tflops);
      }
    }
  }
  cudaEventDestroy(start);
  cudaEventDestroy(stop);
  free(h_a);
  free(h_b);
  free(h_b_t);
  free(h_c);
  free(h_c_ref);
  cudaFree(d_a);
  cudaFree(d_b);
  cudaFree(d_b_t);
  cudaFree(d_c);
  cudaFree(tma_a);
  cudaFree(tma_b);
  cublasDestroy(handle);
}
#endif

// ============================================================================
// Bench: FlashAttention Split-Q (template on kHeadDim for dispatch)
// ============================================================================
template <int kHeadDim>
static void bench_fa_launch(int B, int H, int seqlen, int head_dim,
    half *h_o_ref, float *ref_o, half *d_q, half *d_k, half *d_v, half *d_o) {
  static_assert(kHeadDim == 64 || kHeadDim == 128, "Only D=64 and D=128 are supported");
  constexpr int kStage = 2, kPad = 8;
  constexpr int kMmaAtomM = 16, kMmaAtomN = 8, kMmaAtomK = 16;
  constexpr int kMmaTileSeqLenQ = 8, kMmaTileSeqLenK = 1, kMmaTileSeqLenP = 8;
  constexpr int kMmaTileHeadDimV = 1, kValTileSeqLenQ = 1, kValTileSeqLenK = 8;
  constexpr int kValTileSeqLenP = 1;
  constexpr int kValTileHeadDimV = kHeadDim / (8 * kMmaTileHeadDimV);
  constexpr int Br = kMmaAtomM * kMmaTileSeqLenQ * kValTileSeqLenQ;
  constexpr int Bc = kMmaAtomN * kMmaTileSeqLenK * kValTileSeqLenK;
  size_t smem_bytes =
    (Br * (kHeadDim + kPad) + kStage * Bc * (kHeadDim + kPad) +
    Bc * (kHeadDim + kPad)) *
    sizeof(half);

  dim3 block(256);
  dim3 grid((seqlen + Br - 1) / Br, B * H);

  cudaStream_t timing_stream;
  cudaStreamCreate(&timing_stream);
  cudaEvent_t start, stop;
  cudaEventCreate(&start);
  cudaEventCreate(&stop);

  using FAKernel = void (*)(half *, half *, half *, half *, int, int);
  FAKernel fa_k = flash_attn_mma_stages_split_q<
    kHeadDim, kMmaAtomM, kMmaAtomN, kMmaAtomK, kMmaTileSeqLenQ, kMmaTileSeqLenK,
    kMmaTileSeqLenP, kMmaTileHeadDimV, kValTileSeqLenQ, kValTileSeqLenK,
    kValTileSeqLenP, kValTileHeadDimV, kStage, kPad>;
  bool smem_ok = check_smem_feasible((const void *)fa_k, smem_bytes);
  if (smem_ok) {
    cudaFuncSetAttribute(fa_k, cudaFuncAttributeMaxDynamicSharedMemorySize, smem_bytes);
  }

  cudaEventRecord(start, timing_stream);
  if (smem_ok) {
    fa_k<<<grid, block, smem_bytes, timing_stream>>>(d_q, d_k, d_v, d_o, seqlen, head_dim);
```

```
  }
  cudaEventRecord(stop, timing_stream);
  cudaEventSynchronize(stop);

  float time_ms = 0;
  cudaEventElapsedTime(&time_ms, start, stop);

  size_t sz = (size_t)B * H * seqlen * head_dim * sizeof(half);
  half *h_o = (half *)malloc(sz);
  check(cudaMemcpy(h_o, d_o, sz, cudaMemcpyDeviceToHost), "bench fa D2H");

  int count = B * H * seqlen * head_dim;
  char label[64];
  snprintf(label, sizeof(label), "FlashAttn-SplitQ (D=%d)", kHeadDim);
  float max_err = 0.0f;
  if (smem_ok) {
    for (int i = 0; i < count; i++) {
      float ref_val = h_o_ref ? __half2float(h_o_ref[i]) : ref_o[i];
      float err = fabsf(__half2float(h_o[i]) - ref_val);
      if (err > max_err) max_err = err;
    }
  }
  if (smem_ok) {
    float tflops = bench_fa_tflops(B, H, seqlen, head_dim, time_ms);
    printf("| %-42s | %.6e | %-4s | %-8.1f |\n", label, max_err,
           max_err < 5e-1f ? "PASS" : "FAIL", tflops);
  } else {
    printf("| %-42s | %-12s | %-4s | %-8s |\n", label, "SMEM too large", "SKIP", "None");
  }

  cudaEventDestroy(start);
  cudaEventDestroy(stop);
  cudaStreamDestroy(timing_stream);
  free(h_o);
}

static void bench_flash_attn(int B, int H, int N, int D) {
  int seqlen = N, head_dim = D;

  if (head_dim != 64 && head_dim != 128) {
    printf("| %-42s | %-12s | %-4s | %-8s |\n", "FlashAttn-SplitQ", "unsupported D", "SKIP", "None");
    return;
  }

  size_t sz = (size_t)B * H * seqlen * head_dim * sizeof(half);

  srand(42);
  half *h_q = (half *)malloc(sz);
  half *h_k = (half *)malloc(sz);
  half *h_v = (half *)malloc(sz);
  for (int i = 0; i < B * H * seqlen * head_dim; i++) {
    h_q[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
    h_k[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
    h_v[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
  }

  // Device allocations — shared by kernel and reference
  half *d_q, *d_k, *d_v, *d_o;
  check(cudaMalloc(&d_q, sz), "bench fa alloc Q");
  check(cudaMalloc(&d_k, sz), "bench fa alloc K");
  check(cudaMalloc(&d_v, sz), "bench fa alloc V");
  check(cudaMalloc(&d_o, sz), "bench fa alloc O");
  check(cudaMemcpy(d_q, h_q, sz, cudaMemcpyHostToDevice), "bench fa H2D Q");
```

```
  check(cudaMemcpy(d_k, h_k, sz, cudaMemcpyHostToDevice), "bench fa H2D K");
  check(cudaMemcpy(d_v, h_v, sz, cudaMemcpyHostToDevice), "bench fa H2D V");

  // Reference output: either cuDNN (half) or CPU (float)
  half *h_o_ref = nullptr;
  float *ref_o = nullptr;
  int ref_count = B * H * seqlen * head_dim;

  // (cuDNN / CPU reference code unchanged — same as before) ...

#if defined(NOTES_V2_ENABLE_CUDNN)
  // Try cuDNN SDPA first; fall back to CPU if unsupported on this SM
  bool cudnn_ok = false;
  half *d_o_ref;
  check(cudaMalloc(&d_o_ref, sz), "bench fa alloc O_ref");
  {
    cudnnHandle_t cudnn_handle;
    cudnnCreate(&cudnn_handle);
    auto graph = std::make_shared<fe::graph::Graph>();
    graph->set_io_data_type(fe::DataType_t::HALF)
      .set_intermediate_data_type(fe::DataType_t::FLOAT)
      .set_compute_data_type(fe::DataType_t::FLOAT);

    auto Q = graph->tensor(fe::graph::Tensor_attributes()
      .set_uid(1).set_dim({B, H, seqlen, head_dim})
      .set_stride({H * seqlen * head_dim, seqlen * head_dim, head_dim, 1}));
    auto K = graph->tensor(fe::graph::Tensor_attributes()
      .set_uid(2).set_dim({B, H, seqlen, head_dim})
      .set_stride({H * seqlen * head_dim, seqlen * head_dim, head_dim, 1}));
    auto V = graph->tensor(fe::graph::Tensor_attributes()
      .set_uid(3).set_dim({B, H, seqlen, head_dim})
      .set_stride({H * seqlen * head_dim, seqlen * head_dim, head_dim, 1}));

    auto [O_sdpa, Stats] = graph->sdpa(Q, K, V,
      fe::graph::SDPA_attributes()
        .set_name("sdpa_ref")
        .set_attn_scale(1.0f / sqrtf((float)head_dim)));

    O_sdpa->set_output(true).set_uid(4)
      .set_dim({B, H, seqlen, head_dim})
      .set_stride({H * seqlen * head_dim, seqlen * head_dim, head_dim, 1});

    // Try A first, then fallback (matching Python example: [cudnn.heur_mode.A, cudnn.heur_mode.FALLBACK])
    auto build_status = graph->build(cudnn_handle, {fe::HeurMode_t::A, fe::HeurMode_t::FALLBACK});
    if (build_status.is_good()) {
      std::unordered_map<fe::graph::Tensor_attributes::uid_t, void*> vp = {
        {1, d_q}, {2, d_k}, {3, d_v}, {4, d_o_ref}};
      int64_t ws_size = 0;
      if (graph->get_workspace_size(ws_size).is_good()) {
        int8_t *d_ws = nullptr;
        if (ws_size > 0) check(cudaMalloc(&d_ws, ws_size), "bench fa workspace");
        if (graph->execute(cudnn_handle, vp, d_ws).is_good()) {
          check(cudaDeviceSynchronize(), "bench fa cudnn sync");
          cudnn_ok = true;
        }
        if (d_ws) cudaFree(d_ws);
      }
    }
    cudnnDestroy(cudnn_handle);
  }

  if (cudnn_ok) {
    h_o_ref = (half *)malloc(sz);
```

```
    check(cudaMemcpy(h_o_ref, d_o_ref, sz, cudaMemcpyDeviceToHost), "bench fa D2H ref");
  } else {
    fprintf(stderr, "cudnn SDPA unsupported on this SM, using CPU ref\n");
  }
  cudaFree(d_o_ref);
#endif

  // CPU reference fallback
  if (!h_o_ref) {
    float *ref_q = (float *)malloc(sz * 4 / sizeof(half));
    float *ref_k = (float *)malloc(sz * 4 / sizeof(half));
    float *ref_v = (float *)malloc(sz * 4 / sizeof(half));
    ref_o = (float *)malloc(sz * 4 / sizeof(half));
    for (int i = 0; i < ref_count; i++) {
      ref_q[i] = __half2float(h_q[i]);
      ref_k[i] = __half2float(h_k[i]);
      ref_v[i] = __half2float(h_v[i]);
    }

    float scale = 1.0f / sqrtf((float)head_dim);
    for (int bi = 0; bi < B * H; bi++) {
      for (int qi = 0; qi < seqlen; qi++) {
        float smax = -INFINITY;
        float *S = (float *)malloc((size_t)seqlen * sizeof(float));
        for (int kj = 0; kj < seqlen; kj++) {
          float s = 0.0f;
          for (int d = 0; d < head_dim; d++)
            s += ref_q[bi * seqlen * head_dim + qi * head_dim + d] *
                 ref_k[bi * seqlen * head_dim + kj * head_dim + d];
          S[kj] = s * scale;
          if (S[kj] > smax) smax = S[kj];
        }
        double sum_exp = 0.0;
        for (int kj = 0; kj < seqlen; kj++)
          sum_exp += (double)expf(S[kj] - smax);
        float inv_sum = 1.0f / (float)sum_exp;
        for (int d = 0; d < head_dim; d++) {
          double o_acc = 0.0;
          for (int kj = 0; kj < seqlen; kj++)
            o_acc += (double)(expf(S[kj] - smax) * inv_sum) *
                     ref_v[bi * seqlen * head_dim + kj * head_dim + d];
          ref_o[bi * seqlen * head_dim + qi * head_dim + d] = (float)o_acc;
        }
        free(S);
      }
    }
    free(ref_q);
    free(ref_k);
    free(ref_v);
  }

  if (head_dim == 64)
    bench_fa_launch<64>(B, H, seqlen, head_dim, h_o_ref, ref_o, d_q, d_k, d_v, d_o);
  else
    bench_fa_launch<128>(B, H, seqlen, head_dim, h_o_ref, ref_o, d_q, d_k, d_v, d_o);

  free(h_q);
  free(h_k);
  free(h_v);
  free(h_o_ref);
  free(ref_o);
  cudaFree(d_q);
  cudaFree(d_k);
```

```
  cudaFree(d_v);
  cudaFree(d_o);
}


int main(int argc, char *argv[]) {
#if defined(NOTES_V2_ENABLE_WGMMA) || defined(NOTES_V2_ENABLE_TMA_MMA_WS)
  cuInit(0);
#endif

  for (int i = 1; i < argc; i++) {
    if (strcmp(argv[i], "--bench") == 0) {
      g_bench_mode = true;
    } else if (strcmp(argv[i], "--bench-fa") == 0) {
      g_bench_fa = true;
    } else if (strcmp(argv[i], "--mnk") == 0 && i + 1 < argc) {
      sscanf(argv[++i], "%d,%d,%d", &g_bench_M, &g_bench_N, &g_bench_K);
    } else if (strcmp(argv[i], "--bnhd") == 0 && i + 1 < argc) {
      sscanf(argv[++i], "%d,%d,%d,%d", &g_bench_B, &g_bench_Nfa, &g_bench_H, &g_bench_D);
    }
  }

  if (g_bench_mode) {
    printf("=== notes-v2.cu bench mode ===\n");
    printf("HGEMM: M=%d N=%d K=%d   FA: B=%d H=%d N=%d D=%d\n",
           g_bench_M, g_bench_N, g_bench_K,
           g_bench_B, g_bench_H, g_bench_Nfa, g_bench_D);
    printf("| %-42s | %-12s | %-4s | %-8s |\n", "Kernel", "Max Err", "Pass", "TFLOPS");
    printf("|--------------------------------------------|--------------|------|----------|\n");

    test_block_reduce(g_bench_N);
    test_dot(g_bench_N);
    test_relu(1024);
    test_elementwise(1024);
    test_histogram(1024);
    test_merge_attn_states(512, 16, 128);
    test_softmax(256);
    test_rms_norm(8, 128);
    test_layer_norm(8, 128);
    test_rope(8, 128);
    test_mat_transpose(256, 256);
    test_mat_transpose_padded(256, 256);
    test_sgemv(256, 128);
    test_sgemm(g_bench_M, g_bench_N, g_bench_K);

    bench_hgemm_mma(g_bench_M, g_bench_N, g_bench_K);
    bench_hgemm_swizzle(g_bench_M, g_bench_N, g_bench_K);
#if defined(NOTES_V2_ENABLE_CUTE)
    bench_hgemm_cute(g_bench_M, g_bench_N, g_bench_K);
#endif
#if defined(NOTES_V2_ENABLE_WGMMA)
    bench_hgemm_wgmma(g_bench_M, g_bench_N, g_bench_K);
#endif
#if defined(NOTES_V2_ENABLE_TMA_MMA_WS)
    bench_hgemm_tma_mma_ws(g_bench_M, g_bench_N, g_bench_K);
#endif
    if (g_bench_fa)
      bench_flash_attn(g_bench_B, g_bench_H, g_bench_Nfa, g_bench_D);

    printf("=== Bench done ===\n");
    return 0;
  }

```

```
#if defined(NOTES_V2_ENABLE_TMA_MMA_WS)
  if (argc >= 2 && strcmp(argv[1], "--tma-mma-ws") == 0) {
    int M = 128, N = 128, K = 64;
    if (argc > 4) {
      M = atoi(argv[2]);
      N = atoi(argv[3]);
      K = atoi(argv[4]);
    }
    printf("=== SM120 TMA MMA WS validation ===\n");
    printf("| %-42s | %-12s | %-4s | %-8s |\n", "Kernel", "Max Err", "Pass", "TFLOPS");
    printf("|--------------------------------------------|--------------|------|----------|\n");
    test_hgemm_tma_mma_ws(M, N, K);
    return 0;
  }
#endif
  int M = 1024, N = 1024, K = 1024;
  if (argc > 3) { M = atoi(argv[1]); N = atoi(argv[2]); K = atoi(argv[3]); }

  printf("=== notes-v2.cu verification harness ===\n");
  printf("| %-42s | %-12s | %-4s | %-8s |\n", "Kernel", "Max Err", "Pass", "TFLOPS");
  printf("|--------------------------------------------|--------------|------|----------|\n");

  test_block_reduce(N);
  test_dot(N);
  test_relu(1024);
  test_elementwise(1024);
  test_histogram(1024);
  test_merge_attn_states(512, 16, 128);
  test_softmax(256);
  test_rms_norm(8, 128);
  test_layer_norm(8, 128);
  test_rope(8, 128);
  test_mat_transpose(256, 256);
  test_mat_transpose_padded(256, 256);
  test_sgemv(256, 128);
  test_sgemm(M, N, K);
  test_hgemm_mma(M, N, K);
  test_hgemm_swizzle(M, N, K);
#if (defined(NOTES_V2_ENABLE_CUTE))
  test_hgemm_cute(M, N, K);
#endif
#if (defined(NOTES_V2_ENABLE_WGMMA))
  test_hgemm_wgmma(M, N, K);
#endif
#if defined(NOTES_V2_ENABLE_TMA_MMA_WS)
  test_hgemm_tma_mma_ws(M, N, K);
#endif
  test_flash_attn(1024, 64);

  printf("=== All tests done ===\n");
  return 0;
}

// ============================================================================
// Quick build & run reference
// ============================================================================
// # sm_86 + CuTe (Ampere, RTX 30/40 系列, 无 WGMMA) :
// nvcc -std=c++20 -O2 -arch=sm_86 -DNOTES_V2_ENABLE_CUTE \
//   -I ../../third-party/cutlass/include \
//   -lcublas -lcuda notes-v2.cu -o notes_v2_cute_sm86.bin
//
// # sm_89 单独编译 + 运行:
// nvcc -std=c++20 -O2 -arch=sm_89 -lcublas -lcuda notes-v2.cu -o notes_v2_sm89.bin
```

```
//
// # sm_89 + CuTe (需要 CUTLASS include 路径, 编译 CUDA_HOME 指向的 CUTLASS 或
//    项目内置 third-party/cutlass) :
// nvcc -std=c++20 -O2 -arch=sm_89 -DNOTES_V2_ENABLE_CUTE \
//   -I ../../third-party/cutlass/include \
//   -lcublas -lcuda notes-v2.cu -o notes_v2_cute_sm89.bin
//
// # sm_90a 单独编译 + 运行 (需要 Hopper GPU, H100/H200 均可) :
// nvcc -std=c++20 -O2 -gencode arch=compute_90a,code=sm_90a \
//   -DNOTES_V2_ENABLE_WGMMA -lcublas -lcuda notes-v2.cu -o notes_v2_sm90.bin
//
// # sm_90a + CuTe + WGMMA:
// nvcc -std=c++20 -O2 -gencode arch=compute_90a,code=sm_90a \
//   -DNOTES_V2_ENABLE_CUTE -DNOTES_V2_ENABLE_WGMMA \
//   -I ../../third-party/cutlass/include \
//   -lcublas -lcuda notes-v2.cu -o notes_v2_cute_sm90.bin
//
// # sm_120a TMA + warp-specialized mma.sync (CUDA Toolkit >= 13.2;
//   RTX PRO 5000 / RTX 5090):
// nvcc -std=c++20 -O2 -arch=sm_120a -DNOTES_V2_ENABLE_TMA_MMA_WS \
//   -lcublas -lcuda notes-v2.cu -o notes_v2_tma_mma_ws_sm120.bin
// ./notes_v2_tma_mma_ws_sm120.bin --tma-mma-ws 1024 1024 1024
//
// # sm_120a + CUTE + TMA_MMA_WS + cuDNN SDPA (CUDA Toolkit >= 13.2;
//   requires cudnn-frontend submodule: git submodule update --init):
// nvcc -std=c++20 -O2 -arch=sm_120a \
//   -DNOTES_V2_ENABLE_CUTE -DNOTES_V2_ENABLE_TMA_MMA_WS -DNOTES_V2_ENABLE_CUDNN \
//   -I ../../third-party/cutlass/include -I ../../third-party/cudnn-frontend/include \
//   -L/usr/local/cuda-13.2/targets/x86_64-linux/lib/stubs \
//   -lcublas -lcudnn -lnvrtc -lcuda notes-v2.cu -o notes_v2_cute_ws_sm120a.bin
// # Quick bench: ./notes_v2_cute_ws_sm120a.bin --bench --mnk 512,512,512 --bnhd 1,512,8,64
// # Full bench:  ./notes_v2_cute_ws_sm120a.bin --bench
```